Backend Engineering with Node.js: Zero to Hero

Oct 18, 2026 · @Subhamoy Ghosh

Part 0: How to use this report

This report takes you from "what is an HTTP request" to designing production backends in Node.js, in 12 parts. Every part follows the order you asked for: theory first, then libraries and alternatives, then code.

Every code step is written as a small **build card** so you can write the code yourself instead of pasting a dump:

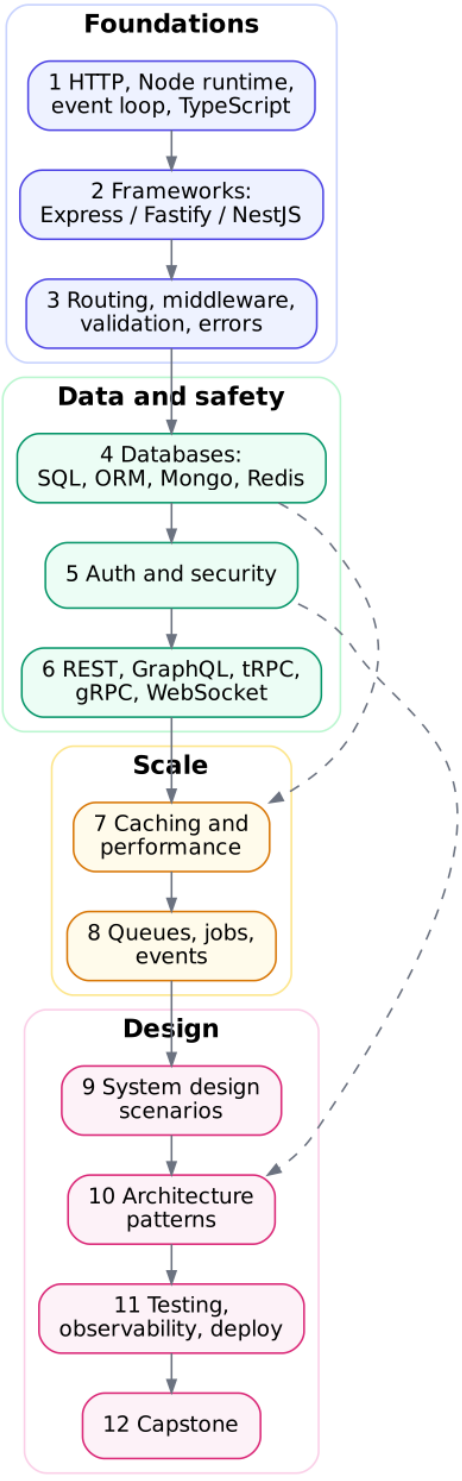
- **Aim:** what you are building in this step.
- **Input / Output:** what goes in and what must come out.
- **Check:** one command or request that proves it works.
- **Reference sketch:** a short snippet to compare against, only after you have tried it.

The parts build on each other, so go in order the first time.

Stage	Parts	You can do this afterwards
Foundations	1, 2, 3	Build a typed, validated REST API with any of the three frameworks
Data and safety	4, 5, 6	Persist data correctly, log users in, resist common attacks
Scale	7, 8	Cache, queue, and serve real-time traffic
Design	9, 10, 11, 12	Pick libraries and architecture for a given scenario, test, ship, and observe it

Assumptions: Node.js 22 LTS, TypeScript, PostgreSQL as the main database, Docker for local services. Examples are TypeScript-first and lean on JavaScript you already know (async/await, ES modules). Library versions move fast, so run `npm view <package> version` before installing and check current docs for exact option names.

Learning roadmap: the diagram below shows the dependency order of the topics.


Foundations
1 HTTP, Node runtime, event loop, TypeScript
2 Frameworks: Express / Fastify / NestJS
3 Routing, middleware, validation, errors
Data and safety
4 Databases: SQL, ORM, Mongo, Redis
5 Auth and security
6 REST, GraphQL, tRPC, gRPC, WebSocket
Scale
7 Caching and performance
8 Queues, jobs, events
Design
9 System design scenarios
10 Architecture patterns
11 Testing, observability, deploy
12 Capstone

Part 1: Foundations (HTTP, Node runtime, TypeScript, tooling)

A backend is a program that waits for requests, applies business rules, reads or writes data, and sends back a response. Everything in later parts is a refinement of that loop.

1.1 Theory: HTTP in 10 minutes

A request has a **method**, a **path**, **headers**, and an optional **body**. A response has a **status code**, headers, and a body (usually JSON).

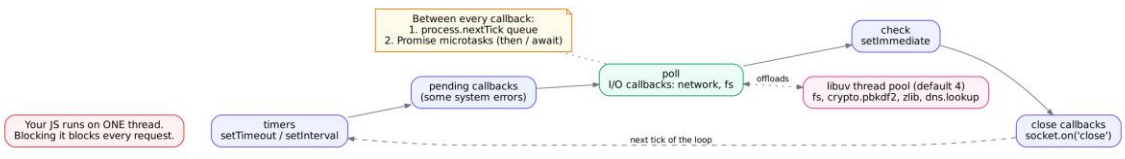
Method	Meaning	Safe (no change)	Idempotent (repeat = same result)
GET	Read a resource	Yes	Yes
POST	Create or trigger an action	No	No
PUT	Replace a resource	No	Yes
PATCH	Partially update	No	Usually not guaranteed
DELETE	Remove	No	Yes

Status range	Meaning	Common codes
2xx	Success	200 OK, 201 Created, 202 Accepted, 204 No Content
3xx	Redirect	301, 302, 304 Not Modified
4xx	Client mistake	400 bad input, 401 not logged in, 403 not allowed, 404 missing, 409 conflict, 422 validation failed, 429 rate limited
5xx	Server mistake	500, 502, 503, 504

Key headers to know now: `Content-Type`, `Authorization`, `Cookie`/`Set-Cookie`, `Cache-Control`, `ETag`, `Origin` (CORS). HTTP is **stateless**: the server does not remember you between requests unless you send something (cookie, token) that lets it look you up.

1.2 Theory: how Node runs your code

Node.js = the V8 JavaScript engine + **libuv** (event loop, networking, thread pool) + built-in modules (`http`, `fs`, `crypto`, `stream`). Your JavaScript runs on **one thread**. That is fast for I/O-heavy work (waiting on databases, networks) because while one request waits, the thread serves another. It is a disaster for CPU-heavy work (big loops, image processing, `JSON.parse` of 200 MB) because nothing else runs until it finishes.



Rules that follow from this diagram:

1. Never block the loop: avoid `*Sync` functions (`fs.readFileSync`) and heavy loops inside request handlers.
2. `process.nextTick` and Promise callbacks run before the loop moves on, so a runaway promise chain can starve I/O.
3. CPU-bound jobs go to `worker_threads`, a child process, or a queue worker (Part 8).
4. To use all CPU cores run several processes: `cluster` module, PM2, or one container per core behind a load balancer.

1.3 Theory: async, streams, and events

- **Promises and async/await** replace callbacks. `await Promise.all([a(), b()])` runs independent work in parallel; sequential `await` lines run one after another.
- **Always handle rejections.** An unhandled rejection crashes modern Node. In Express 4 an `async` handler that throws is *not* caught automatically (Express 5 does catch it).
- **Streams** process data in chunks so a 2 GB file never sits fully in memory: `pipeline(readable, transform, writable)` from `node:stream/promises`.
- **EventEmitter** (`node:events`) is the base of most Node objects: `emitter.on('event', fn)`, `emitter.emit('event', data)`. You will use the idea for domain events in Part 8.
- **AbortController** cancels fetches and timers; use it for request timeouts.

1.4 Libraries and tools for this stage

Need	Pick	Alternatives	Notes
Runtime	Node.js 22 LTS	Bun, Deno	Bun and Deno are faster to start and ship TypeScript, but Node has the widest library and hosting support
Package manager	pnpm	npm, yarn	pnpm is fast and strict about dependencies
TypeScript runner	tsx	ts-node, `node --experimental-strip-types`, SWC	Recent Node 22 releases can strip types natively; tsx is the safe choice everywhere
Compiler / bundler	tsc, tsup	SWC, esbuild	tsc for type checking, tsup for fast builds
Auto restart	`node --watch`	nodemon, tsx watch	
Env variables	`node --env-file=.env`	dotenv	Never commit `.env`
Lint / format	ESLint + Prettier	Biome	Biome is one fast tool for both

Need	Pick	Alternatives	Notes
Dates	date-fns	Luxon, Day.js	Store UTC, convert at the edge

1.5 Build cards

Card 1.A: a bare HTTP server (no framework)

- **Aim:** feel what frameworks do for you.
- **Input:** `GET /health`, `GET /time`, anything else.
- **Output:** `200 {"status":"ok"}`, `200 {"now":"<ISO date>"}`, `404 {"error":"not found"}`, all as JSON with the right `Content-Type`.
- **Check:** `curl -i localhost:3000/health`, then an unknown path.
- **Reference sketch (try yourself first):**

```
import http from 'node:http';

const server = http.createServer((req, res) => {
  const send = (code: number, body: unknown) => {
    res.writeHead(code, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify(body));
  };
  if (req.method === 'GET' && req.url === '/health') return send(200, { status: 'ok'
});
  if (req.method === 'GET' && req.url === '/time') return send(200, { now: new
Date().toISOString() });
  send(404, { error: 'not found' });
});
server.listen(3000);
```

Card 1.B: prove the event loop blocks

- **Aim:** see why one slow handler hurts everyone.
- **Input:** add `GET /block` that runs `while (Date.now() - start < 5000) {}`.
- **Output:** while `/block` runs, `/health` hangs too.
- **Check:** open two terminals; run `curl localhost:3000/block` and immediately `curl localhost:3000/health`. Then move the loop into a `worker_threads` Worker and repeat: `/health` must answer instantly.

Card 1.C: a TypeScript project skeleton

- **Aim:** a repeatable starting point for every project below.
- **Input:** empty folder.
- **Output:** `package.json` with `"type": "module"`, `tsconfig.json` (strict mode), `src/index.ts`, scripts `dev`, `build`, `start`, `typecheck`.
- **Check:** `pnpm dev` restarts when you edit; `pnpm typecheck` fails when you assign a string to a number.

Self-check questions: Why can one Node process serve thousands of waiting requests but not two heavy loops? What is the difference between 401 and 403? Why is PUT idempotent and POST not?

1.6 Deep dive: the event loop, ordering, and the thread pool

The earlier diagram showed the phases. Here is what that means for the order your code runs in.

Order of execution. After the current synchronous code finishes, Node drains the `process.nextTick` queue, then the Promise microtask queue, and only then moves to the next event loop phase (timers, I/O, `setImmediate`). Microtasks also run after *every* individual callback, not only once per loop turn.

```
console.log('A sync');
setTimeout(() => console.log('D timeout 0'), 0);
Promise.resolve().then(() => console.log('C promise'));
process.nextTick(() => console.log('B nextTick'));
console.log('A2 sync end');
// CommonJS (.cjs): A sync, A2 sync end, B nextTick, C promise, D timeout 0
// ES module (.mjs): promise callbacks can run BEFORE nextTick, because the module
itself is evaluated inside a promise job. Run both and compare.
```

`setTimeout(fn, 0)` versus `setImmediate(fn)`. From the main module the order is not guaranteed (it depends on process speed). Inside an I/O callback `setImmediate` always runs first. Never write code that relies on the order of these two.

Starvation. A recursive `process.nextTick` or an endless chain of resolved promises never lets the loop reach the I/O phase, so the server stops answering while CPU sits at 100 percent. If you need to yield to the loop inside a long job, `await setImmediate()` (from `node:timers/promises`) between chunks of work.

The thread pool. libuv keeps a pool of 4 threads (change with `UV_THREADPOOL_SIZE`, maximum 1024, set before the first use). These operations use it: most `fs` calls, `crypto.pbkdf2`/`scrypt`/`randomBytes`, `zlib`, and `dns.lookup`. Network sockets do **not** use it; they use the operating system's non-blocking I/O. Consequence: five parallel `argon2` or `pbkdf2` hashes queue on four threads, and a login burst can delay file reads and DNS lookups for the whole process. Measure with `perf_hooks.monitorEventLoopDelay()` and watch the p99.

Event-loop lag as a health signal. A healthy service has event-loop delay of a few milliseconds. Export it as a metric (Part 11). If it climbs while CPU is high, you are blocking the loop; if it is low while requests are slow, you are waiting on a dependency.

1.7 Deep dive: async patterns and their traps

Combinator	Resolves when	Rejects when	Typical use
`Promise.all([...])`	All succeed (array of results)	The first one fails (others keep running)	Independent calls where any failure ruins the result
`Promise.allSettled([...])`	All finished, each `{status, value/reason}`	Never	Fan out many calls and report partial failures
`Promise.race([...])`	The first settles	The first settles with rejection	Timeouts (prefer `AbortSignal.timeout`)

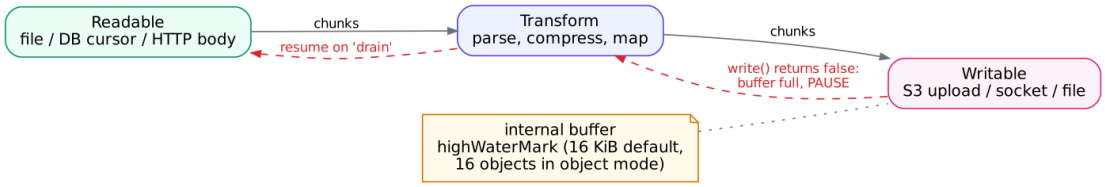
Combinator	Resolves when	Rejects when	Typical use
`Promise.any([...])`	The first **succeeds**	All fail (`AggregateError`)	Ask several mirrors, take the fastest good answer

Traps to learn by heart:

1. **`await` inside `forEach` does nothing.** `items.forEach(async (x) => { await save(x); })` starts all saves and returns immediately. Use `for (const x of items) { await save(x); }` for sequential work, or `await Promise.all(items.map(save))` for parallel.
2. **Unbounded parallelism.** `Promise.all` over 10,000 items opens 10,000 database calls or sockets. Use a limit: `p-limit`, `p-queue`, or process in batches of 50.
3. **Floating promises.** Calling an async function without `await` or `.catch` loses its errors. Enable the ESLint rule `@typescript-eslint/no-floating-promises`.
4. **Unhandled rejection policy.** Since Node 15 an unhandled rejection crashes the process. That is good: log the error, let the orchestrator restart you. Do not swallow it with an empty `process.on('unhandledRejection')` handler.
5. **Lost error context.** Wrap errors with a cause: `throw new Error('could not load user', { cause: err })`. Log `err.cause` too.
6. **Timeouts everywhere.** Every network or database call needs a timeout; without one a single stuck dependency holds a connection forever. `fetch(url, { signal: AbortSignal.timeout(2000) })`; pass the same signal down through your layers.
7. **Async resource cleanup.** Use `try/finally` (or `await using` in recent runtimes) so connections and file handles always close.

1.8 Deep dive: streams and backpressure

A stream moves data in chunks. When the consumer is slower than the producer, data piles up in memory unless the producer is told to wait. That signal is **backpressure**: `writable.write()` returns `false` when its internal buffer is above `highWaterMark`; the producer must pause until the `'drain'` event.



Do not wire this by hand. `pipeline()` from `node:stream/promises` handles backpressure, error propagation, and cleanup (it destroys all streams if one fails). `readable.pipe(writable)` does **not** forward errors and leaks on failure; avoid it in new code.

```
import { pipeline } from 'node:stream/promises';
import { createReadStream, createWriteStream } from 'node:fs';
import { createGzip } from 'node:zlib';
await pipeline(createReadStream('big.log'), createGzip(),
createWriteStream('big.log.gz'));
```

Four kinds: **Readable**, **Writable**, **Duplex** (both, such as a socket), **Transform** (a duplex that changes data). **Object mode** streams carry JavaScript objects rather than bytes (rows from a database cursor). Async generators are the modern way to write a transform: `async function* parse(source) { for await (const chunk of source) yield f(chunk); }` can sit inside `pipeline`. Web streams (`ReadableStream`) are the browser standard and are also in Node; `Readable.toWeb()` and `Readable.fromWeb()` convert between them, and `fetch` response bodies are web streams.

Where streams matter in a backend: file uploads and downloads, exporting a million rows as CSV, proxying a large response, compressing, and streaming an LLM or SSE response to the client.

1.9 Deep dive: parallelism options

Tool	What it is	Shares memory	Use when
`worker_threads`	Extra JavaScript threads in one process	Optional (`SharedArrayBuffer`), messages are copied otherwise	CPU-heavy functions: hashing, image work, parsing, crypto, ML inference
`child_process`	A separate OS process, any program	No, talk over stdio/IPC	Run a CLI (ffmpeg, Python script), isolate untrusted work
`cluster`	N copies of your server sharing one port	No	Use all cores on one machine without a container orchestrator
Containers / pods	N copies managed by Docker, Kubernetes, ECS	No	The normal production answer; one process per container
Queue worker	Separate service that pulls jobs (Part 8)	No	Long, retryable, or bursty work

A worker thread costs roughly tens of megabytes and some startup time, so reuse them from a **pool** (`piscina`) instead of spawning one per request.

1.10 Deep dive: memory, garbage collection, and leaks

V8 keeps objects on a **heap** and frees unreachable ones with a generational garbage collector (a small young generation collected often, an old generation collected rarely). The default old-space limit depends on available memory; raise it only when you have measured a need: `node --max-old-space-size=2048`. In containers set it lower than the container memory limit, or the kernel kills the process (OOM kill) before V8 can react.

Common leak sources in servers:

- A module-level `Map` or array used as a cache with no size or time limit (use `lru-cache`).
- Event listeners added per request and never removed (Node warns about more than 10 listeners; do not ignore the warning).

- Closures that hold large objects, timers that are never cleared, unbounded in-memory queues.
- Per-request data stored on a global.

How to find one: watch `process.memoryUsage().heapUsed` over time (a saw-tooth that trends upward is a leak); start with `node --inspect`, open Chrome DevTools, take two **heap snapshots** a few minutes apart under load, and use the "comparison" view to see which objects keep growing. `Buffer` memory lives outside the V8 heap (`external` and `arrayBuffers` in `memoryUsage()`), so large buffers can exhaust memory while heapUsed looks fine.

1.11 Deep dive: modules, TypeScript, and project hygiene

- **ESM versus CommonJS.** Use ES modules (`"type": "module"`, `import`/`export`) for new code. In ESM relative imports need file extensions (`./util.js`, even when the source is `util.ts`), there is no `__dirname` (use `import.meta.dirname` in current Node), and `require()` of ES modules is possible in recent Node versions. Nest projects often still compile to CommonJS; that is fine.
- **`tsconfig` flags worth enabling:** `strict`, `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`, `noImplicitOverride`, `verbatimModuleSyntax`, `isolatedModules`. Set `module` and `moduleResolution` to `NodeNext` for plain Node projects.
- **Types are erased at runtime.** `req.body as CreateUser` is a lie. Derive the type from a runtime schema: `type CreateUser = z.infer<typeof CreateUserSchema>`.
- **Discriminated unions** model results and states safely: `type Result<T> = { ok: true; value: T } | { ok: false; error: AppError }`. The compiler then forces you to handle both branches. Libraries: `neverthrow`, `effect`, or your own small type.
- **Branded types** stop mixing ids: `type UserId = string & { __brand: 'UserId' }`.
- **Dependency hygiene:** commit the lock file, pin Node in `.nvmrc` or `engines`, run `npm audit`/`pnpm audit`, use Renovate or Dependabot, prefer packages with maintenance activity and few transitive dependencies. Supply-chain attacks target popular packages; review what you add.
- **Graceful shutdown template:** on `SIGTERM` and `SIGINT` call `server.close()`, wait for in-flight requests (with a deadline), close the database pool, flush logs, then `process.exit(0)`.

1.12 More build cards

Card 1.D: predict the output

- **Aim:** build a correct mental model of the loop.
- **Input:** the 5-line ordering snippet above plus two more lines of your own using `setImmediate` inside a `fs.readFile` callback.
- **Output:** write your predicted order on paper first, then run it as `.cjs` and as `.mjs`.
- **Check:** you can explain every difference between your prediction and the real output in one sentence each.

Card 1.E: stream a large file without loading it

- **Aim:** keep memory flat.
- **Input:** generate a 1 GB CSV (a short script writing random rows). Build `GET /export` that reads it, uppercases one column through a Transform (or async generator), gzips, and responds.
- **Output:** the first byte arrives in milliseconds; the process RSS stays under about 150 MB.

- **Check:** `curl -o /dev/null -w '%{time_starttransfer}' ...` and `ps -o rss` while it runs; then rewrite it with `readFile` and watch memory explode.

Card 1.F: CPU offload with a pool

- **Aim:** keep `/health` fast under CPU load.
- **Input:** an endpoint that computes `sha256` of a 50 MB buffer 20 times (or a naive Fibonacci of 40).
- **Output:** version 1 blocks the loop; version 2 runs in a `piscina` pool of 4 workers and `/health` stays under 10 ms.
- **Check:** run autocannon on both endpoints at once and compare the `/health` p99.

Card 1.G: hunt a leak

- **Aim:** use heap snapshots.
- **Input:** a route that pushes `req` data into a module-level array. Start with `--inspect`, hit it 10,000 times.
- **Output:** a heap snapshot comparison showing the growing array; then fix it with an LRU of 1,000 entries and confirm heap stays flat.
- **Check:** a chart of `heapUsed` every 5 seconds before and after.

Card 1.H: lifecycle and failure policy

- **Aim:** shut down cleanly and crash loudly.
- **Input:** handle `SIGTERM`, `uncaughtException`, `unhandledRejection`; add a 5 second shutdown deadline.
- **Output:** `kill -TERM` during an in-flight 3 second request lets that request finish, then exits with code 0; an unhandled rejection logs the error and exits with code 1.
- **Check:** a script that starts a slow request, sends `SIGTERM`, and asserts the response completed.

Card 1.I: a typed `Result`

- **Aim:** make failure part of the type.
- **Input:** `parseAge(input: unknown): Result<number>` using Zod.
- **Output:** callers cannot read `.value` without first narrowing on `.ok`; the compiler rejects the unchecked access.
- **Check:** write one call site that forgets to check and confirm `tsc` fails.

Deep-dive self-check: (1) Why can `await` inside `forEach` run in the wrong order? (2) What is backpressure and which Node function gives it to you for free? (3) Name two operations that use the libuv thread pool and one that does not. (4) Why does a rising `external` memory number not show up in a heap snapshot? (5) When does `Promise.allSettled` beat `Promise.all`?

Part 1 deep dive

1.6 Event loop ordering: predict the output

In a CommonJS file (`.cjs`), what order do these print? Decide before reading on.

```
console.log('A sync start');
setTimeout(() => console.log('E timeout 0'), 0);
setImmediate(() => console.log('F immediate'));
process.nextTick(() => console.log('C nextTick'));
Promise.resolve().then(() => console.log('D promise'));
console.log('B sync end');
```

A and B print first (synchronous code always finishes before anything queued). Then the `nextTick` queue drains, then Promise microtasks, giving C then D. After that the loop starts: whether E or F prints first from the main module is **not guaranteed** (it depends on how fast the process reached the timers phase). Inside an I/O callback the order *is* guaranteed:

```
const fs = require('node:fs');
fs.readFile(__filename, () => {
  setTimeout(() => console.log('timeout'), 0);
  setImmediate(() => console.log('immediate')); // always first here
});
```

Because the callback runs in the poll phase, the very next phase is `check` (setImmediate) and `timers` only comes after the loop wraps around. Two traps to remember: (1) in an ES module the main module itself runs inside a promise job, so there promise callbacks run **before** `nextTick` callbacks, which is a good reason never to depend on this ordering; (2) a function that recursively schedules `process.nextTick` or microtasks forever starves I/O, because the loop never gets to the next phase.

1.7 Modules: ESM versus CommonJS, and a correct TypeScript setup

Topic	CommonJS (CJS)	ES Modules (ESM)
Syntax	`const x = require('x')`, `module.exports`	`import x from 'x'`, `export`
Loading	Synchronous, evaluated on require	Asynchronous, statically analysed
Enable	Default for `.js` without `type`	`"type": "module"` in `package.json`, or `.mjs`
`__dirname`	Built in	`import.meta.dirname` and `import.meta.filename` in current Node 22
Top-level `await`	No	Yes
Interop	`import()` works from CJS	Importing CJS works; recent Node 22 releases also allow `require()` of ESM without a flag (check your exact version)

For new projects choose ESM. With TypeScript and `module: NodeNext`, relative imports must include the **compiled** extension (`import { x } from './x.js'` even though the file is `x.ts`). A solid starting `tsconfig.json`:

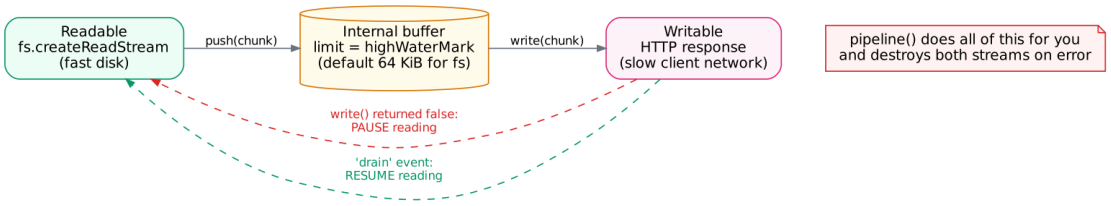
```
{
  "compilerOptions": {
    "target": "ES2023",
```

```
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "verbatimModuleSyntax": true,
    "skipLibCheck": true,
    "sourceMap": true,
    "outDir": "dist",
    "rootDir": "src"
  },
  "include": ["src"]
}
```

`noUncheckedIndexedAccess` makes `arr[0]` typed `T | undefined`, which catches a whole class of crashes. `verbatimModuleSyntax` forces `import type` for type-only imports, keeping runtime imports honest. NestJS projects use decorators, so they add `experimentalDecorators` and `emitDecoratorMetadata` (the Nest CLI scaffolds this).

1.8 Streams and backpressure in depth

Four stream kinds: **Readable** (file, request body), **Writable** (file, response), **Duplex** (socket), **Transform** (gzip, parsers). Each has an internal buffer limited by `highWaterMark`. When a writable's buffer is full, `write()` returns `false`: the producer must stop and wait for `'drain'`. Ignoring this is how a fast disk plus a slow client turns into gigabytes of buffered memory. `pipeline()` implements the whole protocol and also destroys every stream if any one fails, so **always prefer `pipeline` over `.pipe()`**.




```
import { createReadStream, createWriteStream } from 'node:fs';
import { pipeline } from 'node:stream/promises';
import { createGzip } from 'node:zlib';
import { Transform } from 'node:stream';

// 1) compress a file without loading it into memory
await pipeline(createReadStream('big.log'), createGzip(),
createWriteStream('big.log.gz'));

// 2) a custom Transform: uppercase every chunk
const upper = new Transform({
  transform(chunk, _enc, cb) { cb(null, chunk.toString().toUpperCase()); },
});

// 3) async generators make great pipeline stages
async function* numberLines(source: AsyncIterable<Buffer>) {
  let n = 1;
  for await (const chunk of source) {
    for (const line of chunk.toString().split('\n')) yield `${n++}: ${line}\n`;
```

```
  }
}
await pipeline(createReadStream('in.txt'), numberLines,
createWriteStream('out.txt'));

// 4) serving a large download from an HTTP handler (Fastify: reply.send(stream))
//    Express: pipeline(createReadStream(path), res)
```

Note the line splitting in example 3 is naive: a line can straddle two chunks. Real parsers (`split2`, `csv-parse`, `readline`) keep the leftover. Also know **Web Streams** (`ReadableStream`, used by `fetch` bodies) and `Readable.fromWeb` / `Readable.toWeb` for converting between the two worlds.

1.9 Using more than one core

Tool	What it is	Memory	Best for
`cluster` / PM2 cluster mode	Several full Node processes sharing one port	Separate heaps	Using all cores for a stateless HTTP server
`worker_threads`	Threads inside one process, message passing, optional `SharedArrayBuffer`	Separate heaps, can share buffers	CPU work: hashing, parsing, image or PDF processing
`child_process`	Spawn another program or Node script	Fully separate	Running ffmpeg, Python scripts, untrusted work
Containers (one per core)	Orchestrator runs N copies	Fully separate	The modern default: scale with Kubernetes or Cloud Run

The libuv thread pool (default size 4, set with `UV_THREADPOOL_SIZE`) is different: it silently runs `fs`, `zlib`, `crypto.pbkdf2`/`scrypt`, and `dns.lookup`. Five simultaneous `argon2` hashes on a four-thread pool queue up, which looks like random latency spikes on login.

```
// worker.ts
import { parentPort, workerData } from 'node:worker_threads';
function fib(n: number): number { return n < 2 ? n : fib(n - 1) + fib(n - 2); }
parentPort!.postMessage(fib(workerData.n));

// main.ts
import { Worker } from 'node:worker_threads';
export const runFib = (n: number) => new Promise<number>((resolve, reject) => {
  const w = new Worker(new URL('./worker.js', import.meta.url), { workerData: { n }
});
  w.once('message', resolve); w.once('error', reject);
});
```

Creating a worker per request is slow (tens of milliseconds); keep a **pool** (Piscina) and cap it near the core count.

1.10 HTTP server settings that cause real outages

- **Keep-alive mismatch.** Node's default `server.keepAliveTimeout` is 5 seconds. Many load balancers (an AWS ALB defaults to a 60 second idle timeout) reuse connections longer than that, so Node closes a connection the balancer still thinks is open and the user gets a random **502**. Set

Node's `keepAliveTimeout` higher than the balancer's idle timeout (for example 65 000 ms), and keep `headersTimeout` above `keepAliveTimeout`.

- **Slow clients.** `headersTimeout` and `requestTimeout` stop slowloris-style attacks where a client trickles bytes forever. Do not disable them.
- **Body limits.** `express.json({ limit: '100kb' })`; Fastify `bodyLimit`. Unbounded bodies are a memory attack.
- **Outbound timeouts.** `fetch(url, { signal: AbortSignal.timeout(3000) })`. A call with no timeout can hold a request, a connection, and memory forever.
- **Trust proxy.** Behind Nginx or a load balancer, `req.ip` is the proxy unless you set `app.set('trust proxy', ...)` (Express) or `trustProxy` (Fastify), which also breaks IP-based rate limits if forgotten.

1.11 Crashing correctly: process-level errors and shutdown

An `uncaughtException` or `unhandledRejection` means your program is in an unknown state. The safe pattern is **log, then exit, and let the supervisor (Docker, Kubernetes, PM2) restart you**, after finishing in-flight work if possible.

```
import http from 'node:http';
const server = http.createServer(handler);
server.listen(3000);

let shuttingDown = false;
async function shutdown(reason: string, code = 0) {
  if (shuttingDown) return;
  shuttingDown = true;
  console.log(JSON.stringify({ msg: 'shutdown', reason }));
  const force = setTimeout(() => process.exit(1), 10_000); // hard stop
  force.unref();
  server.close(async () => {            // stops new connections, waits for in-flight
    await closeDbPool();                 // your own cleanup
    process.exit(code);
  });
  server.closeIdleConnections?.();       // Node 18.2+: drop idle keep-alive sockets
}
process.on('SIGTERM', () => shutdown('SIGTERM'));
process.on('SIGINT', () => shutdown('SIGINT'));
process.on('unhandledRejection', (e) => { console.error(e);
shutdown('unhandledRejection', 1); });
process.on('uncaughtException', (e) => { console.error(e);
shutdown('uncaughtException', 1); });
```

Distinguish **operational errors** (database down, bad input, timeout: expected, handle them) from **programmer errors** (undefined is not a function: fix the code, crash).

1.12 TypeScript habits for backend code

- Use `unknown` for untrusted data (`JSON.parse`, `req.body`), then **narrow** with a validator. `any` switches the checker off.
- Types vanish at runtime. A TypeScript type on `req.body` validates nothing; derive the type **from** the runtime schema: `type CreateNote = z.infer<typeof createNoteSchema>`.
- **Discriminated unions** model results and states without exceptions for expected failures:

```
type Result<T, E = AppError> = { ok: true; value: T } | { ok: false; error: E };

type PaymentState =
  | { status: 'pending' }
  | { status: 'succeeded'; paidAt: Date }
  | { status: 'failed'; reason: string };

function describe(p: PaymentState) {
  switch (p.status) {
    case 'succeeded': return `paid ${p.paidAt.toISOString()}`;   // paidAt exists
only here
    case 'failed':    return p.reason;
    case 'pending':   return 'waiting';
    default: { const _exhaustive: never = p; return _exhaustive; } // compile error
if a case is missed
  }
}
```

- **Branded ids** stop you passing a `UserId` where an `OrderId` is expected: `type UserId = string & { readonly __brand: 'UserId' }`.
- Prefer string-literal unions or `as const` objects over `enum`. Use `satisfies` to check an object against a type without widening it.

1.13 Memory, garbage collection, and leaks

V8 splits memory into a young generation (short-lived objects, cleaned often and cheaply) and an old generation (cleaned rarely and expensively). A **leak** is memory you keep a reference to by accident. The usual suspects: unbounded `Map` or array used as a cache, event listeners added per request and never removed (watch for `MaxListenersExceededWarning`), timers that are never cleared, closures capturing large objects, and global arrays that collect request data. To investigate: log `process.memoryUsage()`, run with `--inspect` and take two heap snapshots in Chrome DevTools (compare them), or start with `--heapsnapshot-signal=SIGUSR2`. In containers set `--max-old-space-size` a bit below the container limit so Node GC-pressures itself instead of being OOM-killed.

1.14 More build cards

Card 1.D: predict, then verify the event loop

- **Aim:** make the ordering rules automatic.
- **Input:** the two snippets in 1.6, plus a version that adds `queueMicrotask` and a nested `nextTick` inside a Promise callback.
- **Output:** your written prediction for each, then the real output.
- **Check:** run as `.cjs` and again as `.mjs`; explain every difference in one sentence each.

Card 1.E: streaming CSV report

- **Aim:** process a 1 GB file in constant memory.
- **Input:** generate a 1 GB CSV (a script writing 20 million rows); an endpoint `GET /report` that counts rows per country.
- **Output:** correct totals while `process.memoryUsage().rss` stays flat (for example under 150 MB).

- **Check:** call it while watching memory; then rewrite with `fs.readFile` and watch memory explode.

Card 1.F: worker pool

- **Aim:** keep the API responsive under CPU load.
- **Input:** `POST /hash` that runs a CPU-heavy function for about 200 ms.
- **Output:** run it on the main thread and measure `/health` latency under load (it degrades), then through a Piscina pool (it stays flat).
- **Check:** `autocannon -c 20 -d 15` against both endpoints at the same time.

Card 1.G: graceful shutdown

- **Aim:** zero dropped requests on deploy.
- **Input:** a route that sleeps 5 seconds, plus the shutdown code from 1.11.
- **Output:** `kill -TERM` during an in-flight request: the request still completes, new requests are refused, the process exits with code 0.
- **Check:** start the slow request with `curl`, send the signal, observe the response arrive before exit.

1.15 Answer key for the Part 1 self-checks

- **One process, thousands of waiting requests, but not two heavy loops?** Waiting on I/O costs no thread time: Node registers interest and moves on. A heavy loop occupies the single JavaScript thread, so nothing else runs until it ends.
- **401 versus 403?** 401: we do not know who you are (missing or invalid credentials). 403: we know who you are and you are not allowed.
- **Why PUT idempotent, POST not?** PUT replaces the resource at a known URL, so repeating it leaves the same state. POST usually creates a new thing each time, so repeating it creates duplicates (hence idempotency keys in Part 6).

1.6 Deep dive: event loop ordering you can predict

Knowing the order lets you debug "why did this run first?" bugs. Run this file and compare with the expected output.

```
console.log('A sync start');
setTimeout(() => console.log('F timeout 0'), 0);
setImmediate(() => console.log('G immediate'));
Promise.resolve().then(() => console.log('D promise'));
process.nextTick(() => console.log('C nextTick'));
queueMicrotask(() => console.log('E microtask'));
console.log('B sync end');
```

Expected: `A`, `B`, then `C` (the nextTick queue always drains first), then `D` and `E` (promise microtasks, in the order queued). `F` and `G` come after, and their relative order is **not guaranteed** from the main module. Inside an I/O callback (for example inside `fs.readFile`'s callback) `setImmediate` always fires before `setTimeout(0)`, because the loop is in the poll phase and moves to check next.

Practical consequences:

- A recursive `process.nextTick` or an endless promise chain **starves I/O** because microtasks run to exhaustion before the loop continues. Use `setImmediate` to yield to I/O between chunks of heavy work.
- `await` always yields at least one microtask turn, even for an already resolved value.
- `setTimeout` delays are a *minimum*, not a promise: a busy loop delays every timer. Measure loop lag with `monitorEventLoopDelay` (Part 7).

1.7 Deep dive: streams and backpressure

Why streams. `fs.readFile` on a 2 GB file loads 2 GB into memory. A stream reads it in chunks (default 64 KiB for files), so memory stays flat no matter the file size. Four kinds: Readable, Writable, Duplex, Transform.

Backpressure. If a fast reader feeds a slow writer, chunks pile up in memory. `writable.write(chunk)` returns `false` when its internal buffer is full: stop writing until the `drain` event. `pipeline()` does this for you and also destroys every stream when one fails, which prevents leaked file descriptors. Never use bare `.pipe()` in new code.

```
import { createReadStream, createWriteStream } from 'node:fs';
import { createGzip } from 'node:zlib';
import { pipeline } from 'node:stream/promises';
import { Transform } from 'node:stream';

const upper = new Transform({
  transform(chunk, _enc, cb) { cb(null, chunk.toString().toUpperCase()); },
});

await pipeline(
  createReadStream('big.txt'),
  upper,
  createGzip(),
  createWriteStream('big.txt.gz'),
);
```

Use the same idea in HTTP: `res` is a writable stream, so `pipeline(fileStream, res)` serves large downloads in constant memory; reading an upload as a stream lets you forward it to object storage without saving it first. Async iteration (`for await (const chunk of readable)`) is the readable way to consume a stream; `readline.createInterface` reads big text files line by line.

1.8 Deep dive: memory, garbage collection, and leaks

V8 keeps objects on a **heap**. New objects live in a small "young generation" cleaned often and cheaply; survivors move to the "old generation", cleaned by slower mark-and-sweep passes. Big Buffers live outside the JS heap. The default old-space limit depends on the machine; raise it with `node --max-old-space-size=2048 app.js` only after you rule out a leak, and match it to the container memory limit.

Common leak	Why it leaks	Fix
Global object or array used as a cache	Grows forever	Use `lru-cache` with a max size or TTL

Common leak	Why it leaks	Fix
Event listeners added per request	Emitter keeps references	`once`, or remove the listener; watch the MaxListeners warning
Timers never cleared	The timer callback keeps its closure alive	`clearTimeout` / `clearInterval`, use `AbortSignal`
Closures capturing big objects	The closure keeps the whole scope alive	Copy out only the fields needed
Unbounded queues or Maps keyed by user	Memory tied to traffic	Bound sizes, expire entries
Promises that never settle	Pending handlers pile up	Add timeouts

Tools: `process.memoryUsage()` (watch `heapUsed` and `rss` over time), `node --heapsnapshot-signal=SIGUSR2` to write a heap snapshot while running (open in Chrome DevTools Memory tab and compare two snapshots), `node --inspect` plus DevTools, `clinic doctor` and `clinic heapprofiler`. A leak looks like a sawtooth whose floor keeps rising after each garbage collection.

1.9 Deep dive: using more than one core

Tool	What it is	Use for	Sketch
`worker_threads`	Extra JS threads in the same process, message passing, can share memory via `SharedArrayBuffer`	CPU-bound functions (hashing big data, image maths, parsing huge files)	Piscina pool
`cluster` / multiple processes	Several full copies of the app	Serving more requests on all cores	PM2 or one container per core
`child_process`	Spawn another program	Running ffmpeg, Python scripts, shell tools	`spawn` with streamed stdio
Queue workers	Separate worker processes consuming jobs	Anything long or retryable	Part 8

```
// main.ts: run a CPU-heavy function off the main thread
import { Worker } from 'node:worker_threads';

export function fibInWorker(n: number) {
  return new Promise<number>((resolve, reject) => {
    const w = new Worker(new URL('./fib-worker.js', import.meta.url), { workerData: n
});
    w.once('message', resolve);
    w.once('error', reject);
  });
}
```

```
// fib-worker.ts
import { parentPort, workerData } from 'node:worker_threads';
const fib = (n: number): number => (n < 2 ? n : fib(n - 1) + fib(n - 2));
parentPort!.postMessage(fib(workerData as number));
```

Creating a worker per request is slow. Use a **pool** (Piscina) sized to the number of cores minus one. `crypto.pbkdf2`, `scrypt`, `zlib`, and `fs` already use libuv's thread pool (default size 4, change with `UV_THREADPOOL_SIZE`), so password hashing does not block the loop but can saturate the pool.

1.10 Deep dive: modules, ESM versus CommonJS, and TypeScript settings

- **CommonJS** (`require`, `module.exports`) is the old system; **ES modules** (`import`, `export`) is the standard. Set `"type": "module"` in `package.json` to make `.js` files ES modules. ESM has top-level `await`, no `__dirname` (use `import.meta.dirname` in current Node, or `fileURLToPath(import.meta.url)`), and requires file extensions in relative imports (`./util.js`, even for `.ts` sources).
- Prefer the `node:` prefix for built-ins (`node:fs`) so they cannot be shadowed by an npm package.
- NestJS projects often still compile to CommonJS; its ESM move is on the roadmap, so check the current docs before choosing.
- Libraries publish an `exports` map in `package.json`; if an import fails with "ERR_PACKAGE_PATH_NOT_EXPORTED", you are importing a path the author does not expose.

Recommended `tsconfig.json` choices:

Option	Value	Why
`strict`	true	Catches null and any mistakes
`module` and `moduleResolution`	`NodeNext`	Matches how Node resolves files
`target`	`ES2023` or newer	Node 22 supports it natively, no downlevel needed
`noUncheckedIndexedAccess`	true	`arr[0]` is typed possibly undefined
`verbatimModuleSyntax`	true	Clear type-only imports
`skipLibCheck`	true	Faster builds
`outDir`, `rootDir`, `sourceMap`	`dist`, `src`, true	Clean builds with readable stack traces

Run `node --enable-source-maps dist/main.js` so error stacks point at your TypeScript lines.

1.11 Deep dive: errors, process lifecycle, and timeouts

- **Operational errors** (database down, bad input, timeout) are expected: handle them. **Programmer errors** (undefined is not a function) are bugs: log, then let the process restart.
- After an `uncaughtException` the process state is unknown: log it, then exit and let the supervisor (Docker, PM2, Kubernetes) restart it. Do not keep running.
- Wrap causes: `throw new Error('could not load user', { cause: err })` keeps the original stack.

- **Always set timeouts** on outbound calls: `fetch(url, { signal: AbortSignal.timeout(3000) })`. A missing timeout is how one slow partner takes your whole service down.

```
process.on('unhandledRejection', (reason) => { logger.error({ reason },
'unhandledRejection'); process.exit(1); });
process.on('uncaughtException', (err) => { logger.fatal({ err },
'uncaughtException'); process.exit(1); });

for (const sig of ['SIGTERM', 'SIGINT'] as const) {
  process.once(sig, async () => {
    server.close();            // stop accepting new connections
    await db.end();            // close pools
    process.exit(0);
  });
}
```

1.12 Deep dive: HTTP details that cause real bugs

- **HTTP/1.1 keep-alive** reuses a TCP connection for many requests. **HTTP/2** multiplexes many requests on one connection; **HTTP/3** runs over QUIC. In production a reverse proxy (Nginx, Caddy, a cloud load balancer) usually terminates TLS and speaks HTTP/2 to browsers, then plain HTTP/1.1 to your Node process.
- **CORS** is a browser rule, not server security. For "non-simple" requests (custom headers, `PUT`, JSON body) the browser first sends an `OPTIONS` **preflight**; your server must answer with `Access-Control-Allow-Origin`, `-Methods`, and `-Headers`. With cookies, `Allow-Origin` cannot be `*` and you need `Allow-Credentials: true`.
- **Cookies:** `HttpOnly` (JavaScript cannot read it), `Secure` (HTTPS only), `SameSite=Lax` or `Strict` (CSRF protection), `Max-Age`, `Path`, `Domain`.
- **Content negotiation and compression:** `Accept` and `Accept-Encoding` headers; compress at the proxy.
- **Chunked transfer** and streaming let you send a response before it is complete (SSE and large downloads).
- **Request limits:** cap body size (for example 1 MB for JSON), header size, and request time (`server.requestTimeout`, `headersTimeout`) to blunt slow-client attacks.
- **Trust proxy:** behind a load balancer, the client IP is in `X-Forwarded-For`. Enable your framework's `trust proxy` setting only for the proxies you control, otherwise anyone can spoof the IP and bypass rate limits.
- **PUT versus PATCH:** PUT replaces the whole resource; PATCH sends only changed fields (JSON Merge Patch is the simplest format).

1.13 More build cards

Card 1.D: ordering quiz

- **Aim:** predict event loop behaviour.
- **Input:** the snippet in 1.6, plus a second version wrapped inside `fs.readFile`'s callback.
- **Output:** a written prediction of the print order for both versions before you run them.
- **Check:** run both; explain every line you got wrong in one sentence.

Card 1.E: constant-memory file service

- **Aim:** prove streaming keeps memory flat.
- **Input:** a 1 GB generated file; `GET /download` (stream with `pipeline`) and `GET /download-bad` (`readFile` then send).
- **Output:** memory (`rss`) stays near baseline for the first, jumps by about 1 GB for the second.
- **Check:** print `process.memoryUsage().rss` every second while requesting each.

Card 1.F: find a planted leak

- **Aim:** use a heap snapshot.
- **Input:** a handler that pushes each request body into a module-level array.
- **Output:** a heap snapshot comparison naming the growing array.
- **Check:** after the fix, `heapUsed` returns to a stable floor under `autocannon` load.

Card 1.G: worker pool versus blocked loop

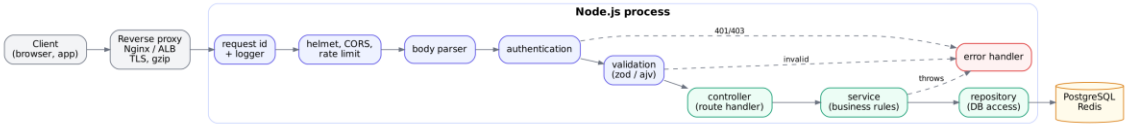
- **Aim:** quantify the gain.
- **Input:** `GET /fib/40` computed inline, then through a Piscina pool.
- **Output:** while 4 requests compute, `GET /health` latency stays under 20 ms with the pool and rises to seconds without it.
- **Check:** run `autocannon` on `/health` in parallel and compare p99.

Part 2: Frameworks (Express, Fastify, NestJS and friends)

A web framework turns the raw `http` server from Card 1.A into something you can organise: routing, middleware, body parsing, error handling, and plugins. They differ mostly in **how much structure they force on you**.

2.1 Theory: the request lifecycle

Every framework passes a request through a chain of steps before your handler runs, and the response back through error handling. Learn this chain once; all frameworks are variations of it.



- **Middleware** is a function that can read or change the request, end the response, or call `next()` to continue the chain. Order matters: logging first, auth before routes, error handler last.
- **Controller / route handler** translates HTTP into a function call and back. It should contain almost no business logic.
- **Service** holds business rules. **Repository** talks to the database. This split is what makes code testable (Part 10).

2.2 The frameworks compared

Framework	Style	Strengths	Weaknesses	Choose it when

Framework	Style	Strengths	Weaknesses	Choose it when
Express 5	Minimal, middleware-based	Largest ecosystem, every tutorial uses it, easy to learn	No built-in validation, structure, or TypeScript help; slower than Fastify	Learning, small APIs, quick prototypes, legacy code
Fastify 5	Minimal but fast, plugin-based	JSON-schema validation, fast serialisation, built-in pino logging, strong TypeScript support, encapsulated plugins	Smaller ecosystem than Express; plugin model needs a learning step	Performance-sensitive APIs, microservices, teams that want schema-first design
NestJS 11	Opinionated, Angular-like (modules, decorators, dependency injection)	Enforced structure, DI makes testing easy, first-class support for GraphQL, WebSockets, queues, microservices, OpenAPI; runs on Express or Fastify underneath	Steeper learning curve, decorator magic, more boilerplate	Large teams, long-lived enterprise apps, many modules
Hono	Tiny, web-standard Request/Response	Runs on Node, Bun, Deno, Cloudflare Workers; very small	Younger ecosystem	Edge/serverless APIs, portable code
Koa	Minimal, async middleware by the Express authors	Clean async model	Small ecosystem, little momentum now	Rare; mostly legacy
AdonisJS	Full-stack, Laravel-like	Batteries included: ORM, auth, validation	Smaller community	You want Rails/Laravel style in Node
tRPC, Elysia (Bun)	Type-safe RPC / Bun-first	End-to-end types	Narrower use cases	TypeScript monorepos (Part 6)

Practical advice: learn Express first because you will meet it everywhere, then build the same API in Fastify and NestJS. After that you will see that the concepts (routing, middleware, validation, DI) are the same and only the spelling changes.

2.3 Express in practice

```
import express from 'express';
const app = express();
app.use(express.json());

app.get('/notes/:id', (req, res) => {
  res.json({ id: req.params.id });
});

// error handler: four arguments, registered last
app.use((err: Error, _req: express.Request, res: express.Response, _next:
express.NextFunction) => {
  res.status(500).json({ error: 'internal error' });
});
app.listen(3000);
```

Common Express companions: `helmet` (security headers), `cors`, `morgan` or `pino-http` (request logs), `compression`, `express-rate-limit`, `multer` (file uploads), `cookie-parser`.

2.4 Fastify in practice

```
import Fastify from 'fastify';
const app = Fastify({ logger: true });

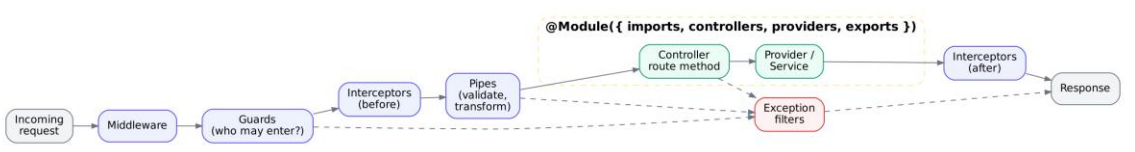
app.get('/notes/:id', {
  schema: {
    params: { type: 'object', properties: { id: { type: 'string' } }, required:
['id'] },
    response: { 200: { type: 'object', properties: { id: { type: 'string' } } } },
  },
}, async (req) => ({ id: (req.params as { id: string }).id }));

await app.listen({ port: 3000 });
```

Fastify ideas to learn: **plugins** (`app.register(plugin, { prefix: '/v1' })`) give encapsulation; **hooks** (`onRequest`, `preHandler`, `onSend`) replace middleware; **decorators** attach shared objects such as a database client. Companions: `@fastify/cors`, `@fastify/helmet`, `@fastify/rate-limit`, `@fastify/jwt`, `@fastify/multipart`, `@fastify/swagger`, and type providers (`@fastify/type-provider-typebox` or `fastify-type-provider-zod`) for end-to-end types.

2.5 NestJS in practice

Nest organises code into **modules**. Each module bundles controllers (HTTP), providers (services, repositories), and what it imports or exports. The framework builds objects for you and injects them (**dependency injection**), so a controller just declares what it needs.




```
@Injectable()
export class NotesService {
```

```
  findOne(id: string) { return { id }; }
}

@Controller('notes')
export class NotesController {
  constructor(private readonly notes: NotesService) {}

  @Get(':id')
  findOne(@Param('id') id: string) { return this.notes.findOne(id); }
}

@Module({ controllers: [NotesController], providers: [NotesService] })
export class NotesModule {}
```

Scaffold with `npx @nestjs/cli new project`. Know these five building blocks: **Module, Controller, Provider, Pipe** (validate/transform input), **Guard** (allow or deny), plus **Interceptor** (wrap handlers: logging, caching, response mapping) and **Exception filter** (shape errors). Switch the HTTP engine to Fastify with `@nestjs/platform-fastify` when you want its speed.

2.6 Build cards

Card 2.A: Notes API in Express

- **Aim:** a CRUD API with an in-memory array, so the framework is the only new thing.
- **Input:** `POST /notes {title, body}`, `GET /notes`, `GET /notes/:id`, `PATCH /notes/:id`, `DELETE /notes/:id`.
- **Output:** `201` with the created note and an `id`; `200` lists; `404` for unknown id; `204` on delete.
- **Check:** write 6 `curl` commands (or a `.http` file for the REST Client extension) that exercise every status code above.

Card 2.B: the same API in Fastify

- **Aim:** compare how schemas replace manual checks.
- **Input / Output:** identical to 2.A, but each route has a JSON schema for body and response.
- **Check:** sending `{ "title": 5 }` returns `400` automatically without you writing an `if`. Note the difference in code size and speed with `npx autocannon -c 50 -d 10 localhost:3000/notes`.

Card 2.C: the same API in NestJS

- **Aim:** learn modules and DI.
- **Input / Output:** identical, split into `NotesModule`, `NotesController`, `NotesService`.
- **Check:** the service has no HTTP types in it; you can call `new NotesService()` in a plain script and it works.

Self-check questions: What does `next()` do in Express? Why does Fastify need schemas for fast responses? What problem does dependency injection solve in a 50-file project?

2.7 Deep dive: Express internals and Express 5

How Express works. `app.use(fn)` and `app.get(path, fn)` push layers onto an ordered stack. For each request Express walks the stack, runs every layer whose method and path match, and moves on when

the layer calls `next()`. Calling `next(err)` (or throwing) skips to the next layer with **four parameters** `(err, req, res, next)`. A layer that neither responds nor calls `next()` leaves the request hanging until the client times out, which is the most common beginner bug.

Routers. `express.Router()` is a mini application with its own stack. Mount it with `app.use('/v1/notes', notesRouter)`. Build one router per feature and attach feature-level middleware (auth, validation) to the router, not globally.

What changed in Express 5 (check the official migration guide for the full list):

- Rejected promises and thrown errors in `async` handlers are forwarded to the error handler automatically. Express 4 needed `express-async-errors` or a wrapper.
- The path syntax moved to a newer `path-to-regexp`: wildcards must be named (`/*splat`), optional parts use braces (`/:file{.:ext}`), and some old regex-style patterns no longer work.
- `req.query` is now a read-only getter, so middleware that rewrote it (for example an old sanitiser) breaks; put validated values on `req.validated` or `res.locals`.
- `req.body` is `undefined` (not `{}`) when no body parser ran.
- Several deprecated methods are removed (`app.del`, `req.param()`, `res.json(status, body)` forms).

Middleware order that works for most APIs: request id, then logger, then `helmet`, then `cors`, then rate limit, then body parsers (with size limits), then authentication, then routers, then a 404 handler, then the error handler.

Body parsing details. `express.json({ limit: '100kb' })` rejects bigger bodies with `413`. Webhooks that need a signature check require the **raw bytes**, so mount `express.raw({ type: 'application/json' })` on that route *before* the global JSON parser, or use the `verify` callback to keep a copy.

Performance habits. Do not do synchronous work in middleware; reuse one validated schema object instead of compiling per request; serve static files from a CDN or Nginx, not Node; set `app.set('trust proxy', 1)` correctly behind a load balancer or rate limiting and `req.ip` will use the proxy's address for everyone.

2.8 Deep dive: Fastify internals

Request lifecycle (hooks run in this order): `onRequest`, `preParsing`, body parsing, `preValidation`, validation, `preHandler`, **your handler**, `preSerialization`, `onSend`, `onResponse`. Errors go to `onError` and the error handler; timeouts to `onTimeout`. Hooks are where Fastify does what Express middleware does, but with a clear phase for each job (auth in `onRequest`, loading data in `preHandler`, response headers in `onSend`).

Why it is fast. JSON schemas are compiled once at startup: **Ajv** builds a validator function for each route, and **fast-json-stringify** builds a serializer for each response schema, so serialising skips the generic `JSON.stringify` path and also drops any field not in the schema (a free guard against leaking `passwordHash`). The router is a radix tree (`find-my-way`), and `pino` logging is asynchronous.

Encapsulation. `app.register(plugin)` creates a **child context**. Hooks, decorators, and routes registered inside the plugin are visible only to that plugin and its children, not to siblings or the parent.

Wrap a plugin with `fastify-plugin` (`fp`) when you *want* it to affect the parent (for example a database plugin that decorates the root instance). This is how Fastify gives you modules without classes.

```
// plugins/db.ts: shared, so wrapped with fp
export default fp(async (app) => {
  const pool = new Pool({ connectionString: app.config.DATABASE_URL });
  app.decorate('db', pool);
  app.addHook('onClose', async () => pool.end());
});

// modules/notes/routes.ts: encapsulated, own prefix and hooks
export default async function notesRoutes(app: FastifyInstance) {
  app.addHook('onRequest', app.authenticate);           // only affects routes in
this plugin
  app.get('/', { schema: listSchema }, async (req) => app.db.query('select * from
notes'));
}
// server.ts
app.register(dbPlugin);
app.register(notesRoutes, { prefix: '/v1/notes' });
```

Type providers. With `@fastify/type-provider-typebox` or `fastify-type-provider-zod`, the route schema also types `req.body`, `req.params`, and the reply, so you write the shape once. **Fastify 5** requires full JSON Schema objects for route schemas (shorthand forms are no longer accepted) and a recent Node version; read its migration guide before upgrading old code.

Fastify habits. Return data from `async` handlers instead of calling `reply.send` twice; never mix `return reply.send()` with a returned value; use `app.inject()` for tests (no network port needed); set `connectionTimeout`, `bodyLimit`, and `requestTimeout`; add `@fastify/under-pressure` to return `503` when event-loop delay or memory is too high.

2.9 Deep dive: NestJS dependency injection and lifecycle

The container. At boot Nest reads each module's `providers`, builds a dependency graph from constructor parameter types, and creates instances in the right order. A class registered as a provider needs `@Injectable()`; a controller asks for providers in its constructor.

Provider forms

Form	Use for	Example
Class (shorthand)	Normal services	`providers: [NotesService]`
`useClass`	Swap implementations by environment	`{ provide: Mailer, useClass: isProd ? SesMailer : ConsoleMailer }`
`useValue`	Constants, test doubles	`{ provide: 'CONFIG', useValue: config }`
`useFactory` (+ `inject`)	Async setup, clients built from config	`{ provide: PG, inject: [ConfigService], useFactory: (c) => new Pool(c.get('DATABASE_URL')) }`

Form	Use for	Example
`useExisting`	Alias to another provider	Expose one instance under two tokens

Inject non-class tokens with `@Inject('CONFIG')` or a typed `InjectionToken` constant. Depend on **abstractions**: define `abstract class NotesRepository` and bind it with `useClass: PrismaNotesRepository`.

Scopes. `DEFAULT` = one singleton for the whole app (use this almost always). `REQUEST` = a new instance per request; it makes every provider that depends on it request-scoped too, which costs performance and complicates lifecycle. `TRANSIENT` = a new instance per consumer. For "current user" or "request id" in a singleton, prefer `AsyncLocalStorage` (the `nestjs-cls` package) over request scope.

Modules. Export what other modules may use; import the module, not the provider. `@Global()` makes a module available everywhere (use for config and logger only). **Dynamic modules** (`forRoot`, `forRootAsync`, `register`) accept options and return a configured module, which is how `ConfigModule.forRoot()`, `TypeOrmModule.forRootAsync()` and `BullModule.registerQueue()` work; write your own when a module needs setup options. Circular imports are a design smell: first extract shared code to a third module; `forwardRef()` is the last resort.

Lifecycle hooks in order: `onModuleInit`, `onApplicationBootstrap`, then on shutdown `onModuleDestroy`, `beforeApplicationShutdown`, `onApplicationShutdown`. Call `app.enableShutdownHooks()` so `SIGTERM` triggers them; close queues, pools, and consumers there.

The five request-pipeline pieces

Piece	Runs	Typical job	Can access the handler's metadata?
Middleware	First, same as Express	Request id, raw-body tweaks, logging	No
Guard	After middleware	Authentication, roles (`CanActivate`)	Yes, through `Reflector` and `@SetMetadata`
Interceptor	Before and after the handler (RxJS)	Timing, caching, response mapping, timeouts	Yes
Pipe	Just before the handler, per parameter	Validate and transform (`ValidationPipe`, `ParseIntPipe`)	Yes
Exception filter	When anything throws	Map errors to a response shape	Yes

Validation the Nest way. Define a DTO class with `class-validator` decorators, then enable `new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true })` globally. `whitelist` strips unknown fields (stops mass assignment), `transform` converts plain objects

into DTO instances and query strings into numbers. If you prefer Zod, use `nestjs-zod` or write a small pipe.

```
@Injectable()
export class TimingInterceptor implements NestInterceptor {
  intercept(ctx: ExecutionContext, next: CallHandler) {
    const start = Date.now();
    return next.handle().pipe(tap(() => Logger.log(`${ctx.getHandler().name}
${Date.now() - start}ms`)));
  }
}

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}
  canActivate(ctx: ExecutionContext) {
    const roles = this.reflector.get<string[]>('roles', ctx.getHandler());
    const user = ctx.switchToHttp().getRequest().user;
    return !roles || roles.includes(user?.role);
  }
}
```

Testing. `Test.createTestingModule({ imports: [NotesModule] }).overrideProvider(NotesRepository).useValue(fakeRepo).compile()` builds the real module with one dependency faked; then call the service directly or create the app and use supertest. This is the main payoff of DI.

Beyond HTTP. The same controllers/providers model drives `@nestjs/graphql`, `@nestjs/websockets`, `@nestjs/microservices` (TCP, Redis, NATS, RabbitMQ, Kafka, gRPC transports), `@nestjs/bullmq`, `@nestjs/schedule`, `@nestjs/cqrs`, and `@nestjs/terminus` (health checks). Nest also runs as a standalone application context with no HTTP server (for scripts and workers) via `NestFactory.createApplicationContext`.

When Nest hurts. Decorator metadata needs `emitDecoratorMetadata`, which some fast compilers (esbuild alone) do not support; use SWC or tsc. Startup time and memory are higher than plain Fastify. Stack traces pass through the framework. Small teams can lose time to ceremony. If your app is a handful of routes, plain Fastify is a better fit.

2.10 Choosing and migrating between frameworks

- Keep business logic **framework-free** (plain classes and functions that take dependencies as arguments). Then moving from Express to Fastify or Nest means rewriting only the thin controller layer.
- Fastify can host Express middleware through `@fastify/express` or `@fastify/middie`, and Nest can run on Fastify with `@nestjs/platform-fastify`; both are useful for gradual migration, though you lose some of the speed.
- Benchmarks in blog posts measure "hello world". Your real bottleneck is almost always the database, so test **your** endpoints with realistic data before choosing on speed.
- Hono and similar web-standard frameworks are worth a look when you may deploy to edge runtimes; they use the standard `Request` and `Response` objects, so handlers port between Node, Bun, Deno, and Workers.

2.11 More build cards

Card 2.D: modular Express with routers and an async error

- **Aim:** organise by feature and confirm Express 5 error forwarding.
- **Input:** `routes/notes.ts` and `routes/users.ts` exporting routers, mounted under `/v1`. One handler is `async` and throws `new NotFoundError()`.
- **Output:** the thrown error reaches your central handler as `404` JSON with no `try/catch` in the route; an unknown path returns your JSON 404, not Express's HTML.
- **Check:** `curl -i` the missing note and a random path; try the same on Express 4 and see the request hang.

Card 2.E: Fastify encapsulation experiment

- **Aim:** see context boundaries.
- **Input:** two plugins, `admin` and `public`; `admin` adds an `onRequest` hook that requires an `x-admin` header and a decorator `app.secret`.
- **Output:** `public` routes work without the header; `app.secret` is `undefined` inside `public`; then wrap the admin plugin with `fp` and watch the hook and decorator leak to the parent.
- **Check:** four `curl` calls with and without the header, before and after `fp`.

Card 2.F: print the hook order

- **Aim:** memorise the lifecycle.
- **Input:** register one hook of every type on a route; each logs its name and a timestamp.
- **Output:** logs in the order `onRequest, preParsing, preValidation, preHandler, handler, preSerialization, onSend, onResponse`.
- **Check:** send an invalid body and note which hooks no longer run; send a request that throws in the handler and find where `onError` appears.

Card 2.G: Nest custom providers and a dynamic module

- **Aim:** master dependency injection.
- **Input:** a `Mailer` abstract class with `ConsoleMailer` and `SmtpMailer`; a `MailModule.forRoot({ driver })` dynamic module that picks one.
- **Output:** switching `driver` changes behaviour with no change in the consuming service.
- **Check:** a unit test overrides `Mailer` with a spy using `overrideProvider` and asserts one call.

Card 2.H: guard, interceptor, filter in one request

- **Aim:** feel the pipeline order.
- **Input:** `@Roles('admin')` decorator with `RolesGuard`, a `TimingInterceptor`, and an exception filter that returns your standard error body.
- **Output:** logs show guard, interceptor (before), pipe, handler, interceptor (after) in that order; a forbidden user gets `403` in your error shape.
- **Check:** add a `console.log` to each piece and read the sequence for one allowed and one denied request.

Card 2.I: same API, three frameworks, one benchmark

- **Aim:** learn to read a benchmark honestly.
- **Input:** your Notes API in Express, Fastify, and Nest (Fastify adapter), all on PostgreSQL.
- **Output:** a table of requests per second and p99 latency for `GET /notes/:id` under `autocannon -c 100 -d 30` for each, plus a second table with the database call replaced by a constant.
- **Check:** write two sentences explaining why the gap between frameworks shrinks when the database is in the path.

Deep-dive self-check: (1) What happens to a request when a middleware neither responds nor calls `next()`? (2) Why is `fastify-plugin` needed for a database plugin? (3) Why does a request-scoped provider slow the whole dependency chain? (4) In what order do guards, interceptors, and pipes run in Nest? (5) Why should business logic not import `express`?

Part 2 deep dive

2.7 Express internals and Express 5 changes

Express is a **stack of layers**. Each `app.use(...)`, `app.get(...)`, or `router.post(...)` pushes a layer holding a path matcher and a handler. For each request Express walks the stack from the top: if a layer matches, it runs; calling `next()` moves to the next layer; `next(err)` skips to the first error-handling layer (a function with **four** parameters). Two consequences: **order is behaviour** (a route registered after the error handler is never protected by it, a body parser registered after a route never parses for it), and a handler that neither responds nor calls `next()` leaves the request hanging until the client times out.

```
import express, { Router } from 'express';
const app = express();

app.use(requestId);          // 1 runs for every request
app.use(express.json());     // 2 parses JSON bodies for everything below

const notes = Router();       // a mini app with its own stack
notes.get('/', listNotes);
notes.get('/:id', getNote);
app.use('/v1/notes', requireAuth, notes);   // path prefix is stripped inside the
router

app.use((_req, res) => res.status(404).json({ error: { code: 'NOT_FOUND' } })); //
fallthrough
app.use(errorHandler);        // LAST: four-argument error handler
```

What changed in Express 5 (check the official migration guide for the full list):

- Rejected promises and thrown errors in `async` handlers are forwarded to the error handler automatically. In Express 4 you needed a wrapper:

```
// Express 4 only: forward async errors to next()
const wrap = (fn: express.RequestHandler): express.RequestHandler =>
  (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
app.get('/notes/:id', wrap(async (req, res) => res.json(await
svc.get(req.params.id))));
```

- Path syntax follows a newer `path-to-regexp`: a bare `*` wildcard must be named (`/files/*splat`), optional segments use braces, and some old regex-style patterns no longer work.

- `req.query` is read through a getter and the default query parser is the simple one (no nested `a[b]=c` objects unless you opt into the extended parser).
- Several long-deprecated methods were removed, and the minimum supported Node version is higher. Plan a migration by running your test suite on 5 and reading deprecation warnings.

Typing Express: describe route generics `Request<Params, ResBody, ReqBody, Query>` or, better, use a validation layer (Zod) that gives you typed data and stop fighting `req.body: any`. To attach `req.user`, use **declaration merging**: `declare global { namespace Express { interface Request { user?: AuthUser } } }`.

2.8 Fastify internals: encapsulation, hooks, schemas

Encapsulation. `app.register(plugin)` creates a child context. Anything you add in a child (decorators, hooks, routes) is visible to that child and *its* children, but not to siblings or the parent. This is Fastify's answer to global middleware soup: each feature is a plugin with its own scope. When a plugin must deliberately share what it adds (a database client every route needs) you wrap it with `fastify-plugin`, which removes that boundary.

```
import fp from 'fastify-plugin';
import type { FastifyInstance } from 'fastify';

export default fp(async (app: FastifyInstance) => {
  const db = await createPool(process.env.DATABASE_URL!);
  app.decorate('db', db);                       // visible to the whole app because
of fp()
  app.addHook('onClose', async () => { await db.end(); });   // closed on app.close()
});
declare module 'fastify' { interface FastifyInstance { db: Pool } }

// notes.routes.ts: a normal plugin, scoped to its own prefix and hooks
export default async function notesRoutes(app: FastifyInstance) {
  app.addHook('preHandler', app.authenticate);  // applies only to routes in this
plugin
  app.get('/', async () => app.db.query('select * from notes'));
}
// server.ts: app.register(dbPlugin); app.register(notesRoutes, { prefix: '/v1/notes'
});
```

Lifecycle (in order): `onRequest`, `preParsing`, body parsing, `preValidation`, validation (schema), `preHandler`, your **handler**, `preSerialization`, serialisation, `onSend`, response written, `onResponse`. `onError` fires on thrown errors and `onTimeout` on timeouts. Put authentication in `onRequest` or `preHandler`, put cleanup or headers in `onSend`.

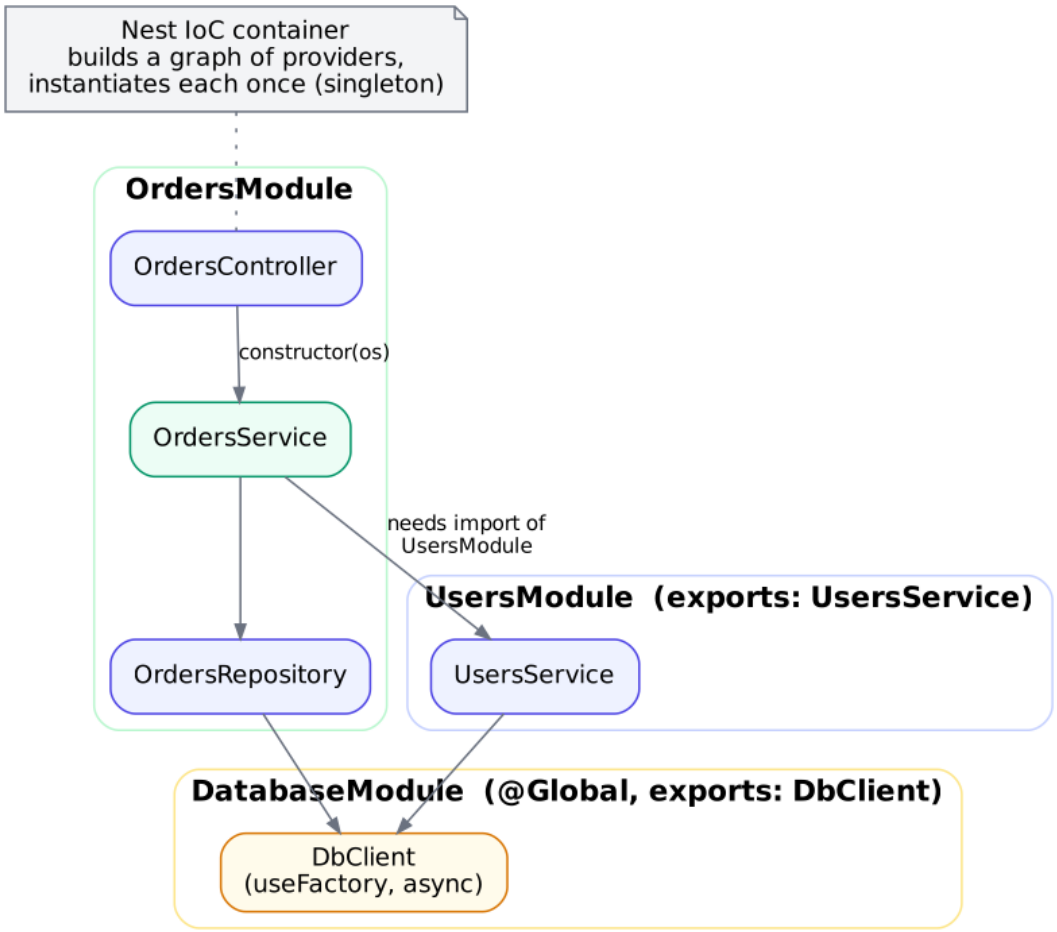
Schemas are performance and safety. Fastify compiles the JSON schemas you give it into validators (Ajv) and into response serialisers (`fast-json-stringify`). The response schema both speeds JSON output and **strips fields not listed**, so a forgotten `passwordHash` never leaves the server. Add type providers (`@fastify/type-provider-typebox`, or `fastify-type-provider-zod`) so one schema gives runtime validation *and* compile-time types.

Gotchas: in an `async` handler return the value (do not also call `reply.send`); calling both causes errors. `@fastify/autoload` registers plugins and routes from a folder tree so you do not hand-wire every file.

`app.inject({ method, url, payload })` runs a request without opening a port, which makes tests fast.

2.9 NestJS internals: dependency injection, scopes, custom providers

Nest has an IoC container: at startup it reads constructor parameter types (via emitted decorator metadata) and builds a **graph of providers**, creating each once.



Rules to internalise: a provider is visible only inside its module unless the module **exports** it; another module must **import** that module to use it; `@Global()` modules (database, config) skip the import step, so use that sparingly. Dependency injection exists so that classes *receive* collaborators rather than constructing them, which is what lets tests swap a real mailer for a fake.

```
@Module({
  imports: [ConfigModule, UsersModule],           // UsersModule must export
UsersService
  providers: [
    OrdersService,                                 // shorthand for { provide:
OrdersService, useClass: OrdersService }
    { provide: 'CLOCK', useValue: { now: () => new Date() } },
```

```
    { provide: MAILER, useClass: process.env.NODE_ENV === 'test' ? FakeMailer :
SmtpMailer },
    { provide: DB, inject: [ConfigService],
      useFactory: async (cfg: ConfigService) =>
createPool(cfg.getOrThrow('DATABASE_URL')) },
  ],
  controllers: [OrdersController],
})
export class OrdersModule {}
// consumer: constructor(@Inject(MAILER) private readonly mailer: Mailer) {}
```

Provider scope	Meaning	Cost
`DEFAULT` (singleton)	One instance for the whole app	Cheapest; use for stateless services
`REQUEST`	New instance per request	Everything depending on it also becomes request-scoped; slower
`TRANSIENT`	New instance per consumer	Rarely needed

Other things to know: **circular dependencies** (A needs B needs A) are solved with `forwardRef`, but treat that as a design smell and extract a third service. **Dynamic modules** (`ConfigModule.forRoot()`, `forRootAsync({ inject, useFactory })`) are how libraries accept configuration. **Lifecycle hooks** (`OnModuleInit`, `OnApplicationBootstrap`, `OnModuleDestroy`, `OnApplicationShutdown`) plus `app.enableShutdownHooks()` give you graceful shutdown for free.

```
// main.ts: global validation + shutdown hooks
app.useGlobalPipes(new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true,
transform: true }));
app.enableShutdownHooks();

// A role guard driven by a custom decorator
export const Roles = (...roles: string[]) => SetMetadata('roles', roles);

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private readonly reflector: Reflector) {}
  canActivate(ctx: ExecutionContext): boolean {
    const required = this.reflector.getAllAndOverride<string[]>('roles',
[ctx.getHandler(), ctx.getClass()]);
    if (!required) return true;
    return required.includes(ctx.switchToHttp().getRequest().user?.role);
  }
}

// An interceptor that times handlers
@Injectable()
export class TimingInterceptor implements NestInterceptor {
  intercept(ctx: ExecutionContext, next: CallHandler) {
    const start = Date.now();
    return next.handle().pipe(tap(() => console.log(`${ctx.getHandler().name}
${Date.now() - start}ms`)));
  }
}
```

```
// usage: @UseGuards(AuthGuard, RolesGuard) @Roles('admin')
@UseInterceptors(TimingInterceptor) on a route
```
`whitelist: true` strips properties not declared on the DTO, and `forbidNonWhitelisted: true` rejects them: this is your **mass-assignment** defence. Testing is where DI pays off: `Test.createTestingModule({ imports: [OrdersModule] }).overrideProvider(MAILER).useValue(fakeMailer).compile()`.

2.10 The same concept in three frameworks

Concept	Express 5	Fastify 5	NestJS 11
Run code before handlers	`app.use(fn)`	Hooks (`onRequest`, `preHandler`)	Middleware, Guards, Interceptors
Validate input	Zod in middleware	JSON schema or type provider	`ValidationPipe` + DTO classes (or Zod pipe)
Share a database client	Module export or `app.locals`	`app.decorate` + `fastify-plugin`	Provider injected by token
Group routes	`Router()`	Plugin with `prefix`	Controller + Module
Central error handling	Four-argument middleware	`setErrorHandler`	Exception filters
Graceful shutdown	Manual (`server.close`)	`app.close()` + `onClose` hooks	`enableShutdownHooks()`
OpenAPI	`zod-to-openapi`, `swagger-jsdoc`	`@fastify/swagger` from schemas	`@nestjs/swagger` decorators
Testing	`supertest(app)`	`app.inject()`	`Test.createTestingModule` + supertest

A word on benchmarks. "Fastify handles roughly two to three times the requests per second of Express" comes from trivial hello-world endpoints. Once a handler waits on a database round trip of a few milliseconds, the framework's share of total time is small and the gap often shrinks to noise. Choose on structure, validation model, and team familiarity first; measure *your* endpoints (Card 2.G) before claiming a speed difference.

Hono in one screen (portable across Node, Bun, Deno, edge runtimes):

```
import { Hono } from 'hono';
import { serve } from '@hono/node-server';
import { zValidator } from '@hono/zod-validator';
import { z } from 'zod';

const app = new Hono();
app.post('/notes', zValidator('json', z.object({ title: z.string().min(1) })), (c) =>
{
  const { title } = c.req.valid('json');            // typed
  return c.json({ id: crypto.randomUUID(), title }, 201);
```

```
});
serve({ fetch: app.fetch, port: 3000 });
```

2.11 More build cards

Card 2.D: Express ordering lab

- **Aim:** prove that middleware order is behaviour.
- **Input:** three versions of one app where you move `express.json()`, a logger, and the error handler around.
- **Output:** a table of observed behaviour (body undefined, error not caught, log missing 404s) for each ordering.
- **Check:** one `curl` script, run against each version.

Card 2.E: Fastify plugin with encapsulation

- **Aim:** feel the scope boundary.
- **Input:** a `db` plugin (with and without `fastify-plugin`) and two route plugins, only one of which registers an auth hook.
- **Output:** `app.db` is visible to routes only with `fp()`; the auth hook protects only its own plugin's routes.
- **Check:** hit routes from both plugins without a token; one returns 401, the other 200.

Card 2.F: Nest guard + interceptor + custom provider

- **Aim:** use the three most common Nest extension points.
- **Input:** a `MAILER` token with `FakeMailer` and `SmtpMailer`, a `RolesGuard`, a `TimingInterceptor`.
- **Output:** an e2e test that overrides `MAILER`, asserts the fake received a message, and gets 403 for a wrong role.
- **Check:** tests pass without any network or SMTP server.

Card 2.G: honest benchmark

- **Aim:** replace opinions with numbers.
- **Input:** the Part 4 Notes API (with PostgreSQL) in Express, Fastify, and Nest, each running one process on the same machine.
- **Output:** requests per second and p99 for `GET /notes/:id` and `POST /notes` from `autocannon -c 50 -d 20`, in a table, with the machine, Node version, and commit recorded.
- **Check:** run each three times and discard the first (warm-up); report the median.

2.12 Answer key for the Part 2 self-checks

- **What does `next()` do?** It hands control to the next matching layer in the stack; `next(err)` jumps to the next error-handling layer; not calling it (and not responding) hangs the request.
- **Why does Fastify need schemas for fast responses?** It compiles the response schema into a specialised serialiser (`fast-json-stringify`) that avoids the generic `JSON.stringify` work and also drops undeclared fields.

- **What problem does DI solve in a 50-file code base?** Classes stop constructing their own dependencies, so you can swap implementations (fake mailer, in-memory repository), test in isolation, and change wiring in one place.

2.7 Deep dive: how Express works inside

An Express app is an ordered **stack of layers**. Each `app.use(fn)` or `app.get(path, fn)` pushes a layer. For each request Express walks the stack top to bottom, runs layers whose path and method match, and moves on when a layer calls `next()`. A layer that sends a response and does not call `next()` ends the chain. `next(err)` jumps straight to the next **error-handling** layer (a function with four parameters).

Consequences you will meet daily:

- **Order is behaviour.** Put `express.json()` before routes that read `req.body`; put the 404 handler after all routes; put the error handler last.
- **Routers** (`express.Router()`) are mini apps: mount `/v1/notes` once, define `/` and `/:id` inside.
- **Async errors.** Express 5 forwards a rejected promise from an async handler to the error handler automatically. In Express 4 you must catch and call `next(err)`, usually with a tiny wrapper (below). Check which major version a tutorial uses.
- **Split app from server.** Export the configured app from `app.ts` and call `listen` only in `server.ts`, so tests can import the app without opening a port.

```
// app.ts
import express from 'express';
import { notesRouter } from './modules/notes/notes.routes.js';
import { errorHandler, notFound } from './shared/errors.js';

export function buildApp() {
  const app = express();
  app.disable('x-powered-by');
  app.use(express.json({ limit: '1mb' }));
  app.use('/v1/notes', notesRouter);
  app.use(notFound);        // after all routes
  app.use(errorHandler);    // very last
  return app;
}

// shared/async-handler.ts: needed on Express 4, harmless on 5
export const ah = (fn: express.RequestHandler): express.RequestHandler =>
  (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
```

Express gives you no built-in typing for `req.body` or `req.user`. Use declaration merging (`declare global { namespace Express { interface Request { user?: AuthUser } } }`) or parse into typed values with Zod inside the handler.

2.8 Deep dive: how Fastify works inside

Plugins and encapsulation. Everything in Fastify is a plugin registered with `app.register(fn, opts)`. A plugin gets its **own scope**: hooks, decorators, and routes added inside are invisible to sibling plugins. This is how you attach auth only to `/admin`. When a plugin must share things with the parent (a database client), wrap it with `fastify-plugin` to break encapsulation on purpose.

Lifecycle hooks, in order: `onRequest`, `preParsing`, `preValidation` (after the body is parsed), `preHandler`, then your handler, then `preSerialization`, `onSend`, `onResponse`; `onError` fires on failure. Put authentication in `onRequest` (cheap, before parsing the body) and authorization in `preHandler`.

Why it is fast. Route schemas are compiled once at startup: **Ajv** builds a validator and **fast-json-stringify** builds a serializer from your response schema. Serialising with a response schema is much faster than `JSON.stringify` and also **strips fields not in the schema**, which prevents leaking `passwordHash` by accident.

```
import Fastify from 'fastify';
import fp from 'fastify-plugin';
import { Pool } from 'pg';

const dbPlugin = fp(async (app) => {
  const pool = new Pool({ connectionString: process.env.DATABASE_URL, max: 10 });
  app.decorate('db', pool);                        // available as app.db everywhere
  app.addHook('onClose', async () => { await pool.end(); });
});

const app = Fastify({ logger: true });
await app.register(dbPlugin);

await app.register(async (admin) => {            // encapsulated scope
  admin.addHook('onRequest', async (req, reply) => {
    if (req.headers['x-admin'] !== 'yes') return reply.code(403).send({ error:
'forbidden' });
  });
  admin.get('/stats', async () => ({ users: 42 }));
}, { prefix: '/admin' });

// tests need no port:
const res = await app.inject({ method: 'GET', url: '/admin/stats' });
```

For end-to-end types use a **type provider** (`@fastify/type-provider-typebox`, or `fastify-type-provider-zod`): the same schema validates input and types `req.body`. Declare custom decorators with TypeScript module augmentation (`declare module 'fastify' { interface FastifyInstance { db: Pool } }`).

2.9 Deep dive: how NestJS works inside

The DI container. At startup Nest reads decorator metadata, builds a graph of providers, and instantiates each as a **singleton** by default. A constructor parameter type (`private readonly notes: NotesService`) is the token Nest uses to find the dependency. Circular dependencies are a design smell; break them with events or `forwardRef` only as a last resort.

Provider scopes: `DEFAULT` (one instance for the app), `REQUEST` (new instance per request; slower, and it makes everything that depends on it request-scoped too), `TRANSIENT` (new instance per consumer). Use request scope only when you truly need per-request state; `AsyncLocalStorage` (`nestjs-cls`) is usually cheaper.

Custom providers let you inject anything under a token:

```
export const DB = Symbol('DB');

@Module({
  providers: [
    { provide: DB, useFactory: (cfg: ConfigService) => new Pool({ connectionString:
cfg.get('DATABASE_URL') }), inject: [ConfigService] },
    { provide: NotesRepository, useClass: PgNotesRepository },   // swap
implementations here
  ],
  exports: [NotesRepository],
})
export class DataModule {}

@Injectable()
export class NotesService {
  constructor(private readonly repo: NotesRepository) {}
}
```

Dynamic modules (`ConfigModule.forRoot()`, `BullModule.forRootAsync({...})`) are modules that take configuration when imported; you will write your own when building reusable pieces.

Request pipeline in practice (the diagram in 2.5):

```
// DTO with validation (class-validator + class-transformer)
export class CreateNoteDto {
  @IsString() @Length(1, 100) title!: string;
  @IsOptional() @IsString() body?: string;
}

// main.ts: global pipe; whitelist strips unknown fields, forbidNonWhitelisted
rejects them
app.useGlobalPipes(new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true,
transform: true }));

// Guard: allow or deny before the handler
@Injectable()
export class AuthGuard implements CanActivate {
  constructor(private readonly jwt: JwtService) {}
  async canActivate(ctx: ExecutionContext) {
    const req = ctx.switchToHttp().getRequest();
    const token = req.headers.authorization?.replace('Bearer ', '');
    try { req.user = await this.jwt.verifyAsync(token); return true; } catch { throw
new UnauthorizedException(); }
  }
}

// Interceptor: wrap the handler (timing, response mapping, caching)
@Injectable()
export class TimingInterceptor implements NestInterceptor {
  intercept(_ctx: ExecutionContext, next: CallHandler) {
    const start = Date.now();
    return next.handle().pipe(tap(() => console.log('took', Date.now() - start,
'ms')));
  }
}
```

```
// Exception filter: one place that shapes errors
@Catch(HttpException)
export class HttpErrorFilter implements ExceptionFilter {
  catch(e: HttpException, host: ArgumentsHost) {
    const res = host.switchToHttp().getResponse();
    res.status(e.getStatus()).json({ error: { message: e.message } });
  }
}
```

Testing in Nest: `Test.createTestingModule({ imports: [NotesModule] }).overrideProvider(NotesRepository).useValue(fakeRepo).compile()` replaces any provider with a fake, which is the payoff of DI.

Honest trade-offs: Nest's decorators and metadata hide control flow, startup is slower, and errors from the DI container can be cryptic. The reward is a uniform structure in big teams and first-party integrations (config, queues, GraphQL, WebSockets, health checks, OpenAPI).

2.10 A glimpse of Hono and Koa

Hono uses the web-standard `Request` and `Response`, so the same code can run on Node (with `@hono/node-server`), Bun, Deno, and edge runtimes. Koa is Express rewritten around async functions with a `ctx` object and an "onion" middleware model (`await next()` runs the downstream, then code after it runs on the way back). Understanding the onion model clarifies Nest interceptors and Fastify hooks too.

```
import { Hono } from 'hono';
import { serve } from '@hono/node-server';
const app = new Hono();
app.get('/notes/:id', (c) => c.json({ id: c.req.param('id') }));
serve({ fetch: app.fetch, port: 3000 });
```

2.11 Choosing with evidence, not hype

Published benchmarks show Fastify serving more requests per second than Express in "hello world" tests. In a real endpoint the database call usually takes 1 to 50 ms while framework overhead is microseconds, so the framework rarely decides your latency. Pick for **team fit, ecosystem, structure, and type safety** first. Benchmark your own endpoint (Card 2.G) before arguing about speed.

Question	Answer that points to...
Is the team new to backend and wants the most tutorials?	Express
Do you want schema-first validation, OpenAPI, and speed without a heavy framework?	Fastify
Will 10 or more developers share the code base for years?	NestJS
Must the code also run on edge or Bun?	Hono
Do you need an admin panel, ORM, and auth out of the	AdonisJS or a full-stack framework

Question	Answer that points to...
box?	

2.12 More build cards

Card 2.D: Express, testable layout

- **Aim:** separate `app` from `server` and mount a router.
- **Input:** the Notes API from 2.A.
- **Output:** `app.ts` exports `buildApp()`; `server.ts` only listens; routes live in `notes.routes.ts`; 404 and error handlers exist.
- **Check:** a supertest test imports `buildApp()` and checks `GET /v1/notes/999` returns `404` JSON without any port being open.

Card 2.E: Fastify plugin scopes

- **Aim:** understand encapsulation.
- **Input:** a `db` plugin wrapped with `fastify-plugin` and an `/admin` plugin with its own `onRequest` hook.
- **Output:** `/admin/stats` returns `403` without the header, `/notes` is unaffected, and `app.db` is available in both.
- **Check:** remove `fastify-plugin` from the `db` plugin and observe `app.db` become undefined in sibling routes; explain why.

Card 2.F: Nest custom provider and override

- **Aim:** use DI to swap an implementation.
- **Input:** `NotesRepository` as an abstract class with `InMemoryNotesRepository` and `PgNotesRepository`.
- **Output:** the app runs on Postgres; the test overrides the provider with the in-memory version.
- **Check:** the service test runs without Docker and in under one second.

Card 2.G: benchmark honestly

- **Aim:** replace opinions with numbers.
- **Input:** the same `GET /notes/:id` (real database query) in Express, Fastify, and Nest on Fastify, identical pool sizes.
- **Output:** a table of requests per second and p99 latency for each, plus a second table for a `GET /ping` with no database.
- **Check:** run `autocannon -c 100 -d 20` three times each and record the median; note how small the gap is on the database route.

Part 3: API building blocks (routing, validation, errors, config, logging)

These five concerns appear in every backend. Getting them right early prevents most production pain.

3.1 Theory

Routing and resource design. Model nouns, not verbs: `/users`, `/users/42`, `/users/42/orders`. Use plural names, nest only one level, and put filters in the query string (`/orders?status=paid&limit=20`). Version in the path (`/v1/...`) or a header. Return the same JSON shape every time.

Validation. Never trust input. Validate **at the edge** (body, params, query, headers) and convert it into typed objects before it reaches your services. Reject with `400` or `422` and a machine-readable message. Validate output too if the shape matters to clients.

Error handling. Separate *expected* errors (not found, conflict, invalid input) from *bugs*. Throw typed errors in services (`NotFoundError`, `ConflictError`), and map them to HTTP codes in **one** central error handler. Never leak stack traces to clients. A consistent error body helps every client:

```
{ "error": { "code": "NOTE_NOT_FOUND", "message": "Note 42 does not exist", "requestId": "b7f3..." } }
```

Configuration. Follow the twelve-factor rule: config lives in environment variables, never in code. Parse and validate them **once at startup** so the app refuses to boot when something is missing.

Logging. Log structured JSON (one object per line) with a request id, not free text. Levels: `error`, `warn`, `info`, `debug`. Never log passwords, tokens, or full card numbers. Redact them in the logger configuration.

3.2 Libraries and alternatives

Concern	Pick	Alternatives	Trade-off
Validation (schema in TypeScript)	Zod	Valibot, ArkType, Yup, Joi, TypeBox, Ajv	Zod has the best ergonomics and types; Valibot is smaller; TypeBox/Ajv are fastest and match Fastify JSON schema
Validation (class-based)	class-validator + class-transformer	Zod pipes	Standard in NestJS; decorators on DTO classes
Env config	Zod parse of `process.env`	envalid, `@nestjs/config`, convict	Fail fast with clear messages
Logging	pino	winston, bunyan, console	pino is the fastest and Fastify's default; pipe through `pino-pretty` in development
HTTP request logs	pino-http	morgan	morgan is simple text, pino-http is structured
Request id	`crypto.randomUUID()` + `AsyncLocalStorage`	cls-hooked, `nestjs-cls`	AsyncLocalStorage carries the id through async calls without passing it by hand

Concern	Pick	Alternatives	Trade-off
Error shape	Own `AppError` classes	`http-errors`, `@hapi/boom`	Small and enough; RFC 9457 "problem details" is the standard format if you want one
Docs	OpenAPI via `@fastify/swagger`, `@nestjs/swagger`, `zod-to-openapi`	Swagger JSDoc	Generate the spec from the same schemas you validate with
IDs	UUID v7 / ULID	auto-increment, nanoid, cuid2	Time-ordered ids index better than random UUID v4

3.3 Build cards

Card 3.A: validation middleware (Express + Zod)

- **Aim:** one reusable `validate(schema)` that checks `body`, `params`, `query`.
- **Input:** a Zod schema `{ title: string (1..100), body: string optional }`.
- **Output:** on success the handler receives typed data; on failure `400` with a list of `{ path, message }`.
- **Check:** post {} and `{"title":""}` and `{"title":"ok"}`: first two fail with readable messages, third passes.
- **Reference sketch:**

```
import { z, ZodTypeAny } from 'zod';
export const validate = (schema: ZodTypeAny) => (req, res, next) => {
  const r = schema.safeParse(req.body);
  if (!r.success) return res.status(400).json({ error: { code: 'VALIDATION', issues:
r.error.issues } });
  req.body = r.data;
  next();
};
```

Card 3.B: central error handler

- **Aim:** services throw typed errors, one handler turns them into responses.
- **Input:** classes `AppError(status, code, message)`, `NotFoundError`, `ConflictError`.
- **Output:** known errors give their status and code; unknown errors give `500` with a generic message and are logged with the stack.
- **Check:** make `GET /notes/999` throw `NotFoundError` (expect 404), and make another route throw a plain `Error` (expect 500, no stack in the body).

Card 3.C: typed config

- **Aim:** crash at startup when config is wrong.
- **Input:** `PORT`, `DATABASE_URL`, `JWT_SECRET`, `NODE_ENV`.
- **Output:** an exported frozen `config` object with correct types (`PORT` is a number).
- **Check:** start the app without `DATABASE_URL`: it must exit with a message naming the missing variable.

Card 3.D: structured logging with request ids

- **Aim:** trace one request through all its log lines.
- **Input:** a middleware that creates `requestId` (or reads `x-request-id`), stores it in `AsyncLocalStorage`, and sets it on the response header.
- **Output:** every log line for that request contains the same `requestId`, and the error body from 3.B includes it.
- **Check:** call an endpoint that logs in two different functions; grep the output for one id and see both lines.

Self-check questions: Why validate at the edge instead of inside the service? Why return `409` for a duplicate email and not `400`? What breaks if you read `process.env.X` all over the code base?

3.4 Deep dive: resource design and updates

Actions that are not CRUD. Not everything is a noun update. Model an action as a sub-resource created with `POST`: `POST /orders/42/cancellation`, `POST /users/7/password-reset`. It keeps verbs out of paths, makes the action auditable (it is a row you can store), and gives you a natural place for an idempotency key.

PUT versus PATCH. `PUT` replaces the whole resource (missing fields become empty), so clients must send everything. `PATCH` changes some fields. Two PATCH formats exist: **JSON Merge Patch** (RFC 7396: send only the changed fields, `null` means delete the field) is simple and what most APIs use; **JSON Patch** (RFC 6902: a list of `{op, path, value}` operations) can express array edits but is harder to validate. Pick Merge Patch unless you really need array operations, and validate it with a *partial* version of your schema (`schema.partial()` in Zod).

Lost updates and optimistic concurrency. Two users load the same record, both edit, the second save silently overwrites the first. Fix it with a version number or `updatedAt`: return an `ETag` with the resource, require `If-Match` on updates, and run `UPDATE notes SET ..., version = version + 1 WHERE id = $1 AND version = $2`. Zero rows changed means someone else won: respond `412 Precondition Failed` (or `409` if you use a body field). No locks are held, so it scales.

Bulk operations and partial failure. For `POST /notes/bulk` either make it all-or-nothing inside one transaction and say so, or return `207 Multi-Status` style per-item results. Never return `200` with a hidden mix of success and failure and no per-item detail. Put hard limits on batch size.

Soft delete versus hard delete. A `deleted_at` column keeps history and allows undo, but every query must filter it (a partial index `WHERE deleted_at IS NULL` and a database view help, and a missing filter is a classic data-leak bug). Regulations such as erasure requests may still require true deletion of personal data. Decide per table.

Field naming and shapes. Choose `camelCase` or `snake_case` once. Dates are ISO 8601 strings in UTC (`2026-10-19T12:00:00Z`). Money is `{ amount: 1999, currency: 'INR' }` in the smallest unit. Use `null` for "known empty" and omit the field for "not requested". Return the created or updated resource from `POST`/`PATCH`, and a `Location` header on `201`.

Content negotiation and limits. Check `Content-Type` on writes (`415 Unsupported Media Type`), set a maximum body size (`413`), and a maximum page size. Add a request timeout so slow clients cannot hold sockets open.

3.5 Deep dive: validation done properly

- **Strict by default.** Reject or strip unknown keys. In Zod `z.object({...}).strict()` rejects them and the default `.strip()` removes them. Pick one deliberately; never spread the raw body into a database call.
- **Coercion belongs at the edge only.** Query strings are always strings, so `z.coerce.number()` is right for `?limit=20` but wrong inside the domain. Use `z.string().transform(...)` or `z.coerce` sparingly and test odd inputs (`""`, `"0x10"`, `"1e3"`, `"NaN"`).
- **Bound everything.** Maximum string length, array size, number range, nesting depth. A 50 MB string inside a valid JSON field is still an attack. `z.string().max(10_000)`, `z.array(...).max(100)`.
- **Cross-field rules** use `refine`/`superRefine` (`endDate` after `startDate`); **discriminated unions** (`z.discriminatedUnion('type', [...])`) validate shapes that depend on a `type` field and give better error messages than plain unions.
- **Normalise, then validate.** Trim and lower-case emails first (`z.string().trim().toLowerCase().email()`), otherwise `A@x.com` and `a@x.com` become two accounts.
- **Regular-expression denial of service (ReDoS).** A badly written regex on user input can take seconds of CPU and block the loop. Avoid nested quantifiers, cap input length before matching, and consider `re2`.
- **Validate uploads by content.** The `Content-Type` header is client-controlled; check magic bytes with `file-type`.
- **Validation versus business rules.** Validation answers "is this shape acceptable?" and returns `400` or `422`. A rule such as "email already registered" needs the database and returns `409`. Do the second in the service, backed by a unique index (checking first and inserting second has a race condition; the index is the real guard).
- **One schema, many uses.** The same Zod/TypeBox schema can give you the request type, the validator, the OpenAPI document (`zod-to-openapi`, `@fastify/swagger`), and a client SDK. Keep schemas in a shared package in a monorepo.

```
export const CreateNote = z.object({
  title: z.string().trim().min(1).max(100),
  body: z.string().max(10_000).optional(),
  tags: z.array(z.string().trim().toLowerCase().max(30)).max(10).default([]),
}).strict();
export type CreateNote = z.infer<typeof CreateNote>;
export const UpdateNote = CreateNote.partial().refine((v) => Object.keys(v).length >
0, 'empty patch');
```

3.6 Deep dive: errors as a system

Two kinds of error. *Operational* errors are expected failures of the outside world (bad input, not found, timeout, database down). Handle and report them. *Programmer* errors are bugs (reading a property of

`undefined`). Log them with the stack, return a generic `500`, and alert. Do not try to continue after a corrupted-state bug; fail the request, and for unexpected process-level errors let the process restart.

A standard error body. RFC 9457 "problem details" is an HTTP standard you can adopt as is, with the content type `application/problem+json`:

```
{
  "type": "https://api.example.com/errors/validation",
  "title": "Request validation failed",
  "status": 422,
  "detail": "2 fields are invalid",
  "instance": "/v1/notes",
  "requestId": "b7f3c2...",
  "errors": [ { "path": "title", "message": "must not be empty" } ]
}
```

Status code choices

Situation	Code	Note
Malformed JSON, wrong types	400	Parser or schema failure
Valid shape, business-invalid	422	Or 400; pick one convention and keep it
Not authenticated	401	Include `WWW-Authenticate` for Basic/Bearer schemes
Authenticated but not allowed	403	For "exists but hidden" resources, 404 avoids leaking existence
Duplicate, version conflict, state conflict	409	Unique violation, stale update
`If-Match` failed	412	Optimistic concurrency
Rate limited	429	Add `Retry-After`
Upstream dependency failed	502, 503, 504	Bad response, unavailable, timeout

Translate database errors. Do not let raw driver messages reach clients. In PostgreSQL the error carries a `code`: `23505` unique violation (return `409`), `23503` foreign-key violation (`409` or `422`), `23502` not-null violation (a bug or `400`), `40001` serialization failure and `40P01` deadlock (retry the transaction a few times with jitter), `57014` statement timeout (`503` or `504`). Map them in the repository so services see typed errors.

Retry only what is safe. Retry reads and idempotent writes on transient failures (network reset, `503`, deadlock). Use exponential backoff with **jitter**, a maximum attempt count, and a total deadline. Never retry a non-idempotent `POST` without an idempotency key.

Process-level safety net. Keep `uncaughtException` and `unhandledRejection` handlers that log and exit non-zero after flushing logs; the supervisor (Kubernetes, PM2, systemd) restarts the process. A handler that logs and continues hides corrupted state.

3.7 Deep dive: configuration and secrets

- **Layers.** Defaults in code, then a committed non-secret config file per environment, then environment variables, then secrets from a secret store. Later layers win. Validate the merged result once.
- **Secrets are not config.** Database passwords, API keys, and signing keys come from a secret manager (AWS Secrets Manager or SSM Parameter Store, Google Secret Manager, HashiCorp Vault, Doppler, 1Password CLI) or the platform's secret injection, and are rotated. For secrets that must live in git, encrypt them (SOPS with age or KMS, Sealed Secrets).
- **Fail fast, with one error message listing every problem**, not the first one.
- **Never log configuration objects whole.** Add a `toJSON` or a redact list.
- **Type the result.** Parse once into a frozen object and import that object; do not read `process.env` anywhere else (lint it with `no-process-env`).
- **Feature flags** are config that can change at run time. For simple cases a table or Redis key is enough; use OpenFeature with Unleash, GrowthBook, or LaunchDarkly when you need targeting and audit.

```
const Env = z.object({
  NODE_ENV: z.enum(['development', 'test', 'production']).default('development'),
  PORT: z.coerce.number().int().min(1).max(65535).default(3000),
  DATABASE_URL: z.string().url(),
  REDIS_URL: z.string().url(),
  JWT_PRIVATE_KEY: z.string().min(1),
  LOG_LEVEL: z.enum(['fatal','error','warn','info','debug','trace']).default('info'),
});
const parsed = Env.safeParse(process.env);
if (!parsed.success) {
  console.error('Invalid environment:', z.flattenError(parsed.error).fieldErrors);
  process.exit(1);
}
export const config = Object.freeze(parsed.data);
```

3.8 Deep dive: logging that you can use at 3 a.m.

- **What to log:** one line per request at `info` (method, route *template* such as `/notes/:id` not the raw URL, status, duration, request id, user id, bytes), warnings for recoverable problems, errors with stack and `cause`, and key business events (order placed, payment failed). **What not to log:** passwords, tokens, full card numbers, session ids, whole request bodies, personal data you do not need.
- **Redaction.** pino supports it natively: `pino({ redact: { paths: ['req.headers.authorization', 'req.headers.cookie', '*.password', '*.token'], censor: '[redacted]' } })`.
- **Child loggers** attach context once: `const log = logger.child({ requestId, userId })`. Combine with `AsyncLocalStorage` so any function can call `getLogger()` and get the request's logger without parameters.
- **Levels.** `error` means someone should look; do not log expected client mistakes (`404`, validation failures) at `error` or your alerts become noise. `debug` is off in production unless you switch it on temporarily for one request id or user.

- **Volume and cost.** Logs cost money to store and search. Sample noisy `info` lines, drop health-check logs, set retention (14 to 30 days is typical), and keep a separate longer-lived **audit log** for security-relevant actions (login, role change, data export) that is append-only.
- **Correlation.** Accept an incoming `x-request-id` or W3C `traceparent` header, create one if missing, return it in the response, and pass it on to every downstream call and queue message. Then one id finds the whole story across services.
- **In production write to stdout as JSON** and let the platform ship it (Docker, Kubernetes, a Loki or CloudWatch agent). Do not manage log files and rotation inside the app. In development use `pino-pretty`.
- **Do not block on logging.** pino writes quickly; heavy transports (to a remote service) should run in a worker thread via `pino.transport`.

3.9 More build cards

Card 3.E: problem-details error handler

- **Aim:** one standard error shape for everything.
- **Input:** your `AppError` classes, Zod errors, and unknown errors.
- **Output:** every error is `application/problem+json` with `type`, `title`, `status`, `requestId`, and for validation an `errors` array; unknown errors show only a generic message in production.
- **Check:** a table-driven test that triggers six different failures (bad JSON, schema error, not found, conflict, thrown `Error`, payload too large) and asserts status and body shape for each.

Card 3.F: map PostgreSQL errors

- **Aim:** keep database details out of the HTTP layer.
- **Input:** a repository that catches driver errors and inspects `err.code`.
- **Output:** inserting a duplicate email throws `ConflictError('EMAIL_TAKEN')` and the API returns `409`; inserting an order with a missing user returns `422`.
- **Check:** fire 20 parallel registrations with the same email; exactly one returns `201`, nineteen return `409`, none return `500`.

Card 3.G: optimistic concurrency with ETag

- **Aim:** prevent lost updates.
- **Input:** a `version` column; `GET /notes/:id` returns `ETag: "3"`; `PATCH` requires `If-Match`.
- **Output:** two clients read version 3; the first update succeeds and returns version 4; the second gets `412`.
- **Check:** a script with two clients and a deliberate interleaving; assert the final row contains only the first client's change plus a clear `412` for the second.

Card 3.H: redaction and request-scoped logging

- **Aim:** safe, traceable logs.
- **Input:** pino with `redact` paths, `AsyncLocalStorage`, and a `getLogger()` helper.
- **Output:** a login request logs the email but never the password or `Authorization` header; every line from the request, including inside the repository, carries the same `requestId`.

- **Check:** `grep -c password app.log` returns 0; `grep <requestId> app.log` shows at least four lines from different files.

Card 3.I: all-errors-at-once config

- **Aim:** friendly startup failure.
- **Input:** the Zod `Env` schema above plus a CI job that starts the app with an empty environment.
- **Output:** the process exits with code 1 and prints every missing or invalid variable in one message.
- **Check:** run with three wrong variables and confirm all three are named.

Card 3.J: contract test from the schema

- **Aim:** make documentation and behaviour agree.
- **Input:** the OpenAPI document generated from your schemas.
- **Output:** an automated test (Schemathesis or `openapi-backend` plus supertest) that sends valid and invalid requests derived from the spec and checks responses against it.
- **Check:** change a response field in code without updating the schema and watch the test fail.

Deep-dive self-check: (1) When do you return `409` versus `422`? (2) Why must the unique index, not an "exists" query, enforce unique emails? (3) How does `If-Match` stop a lost update without locks? (4) Why log the route template instead of the raw URL? (5) What is the difference between a programmer error and an operational error, and how should each be handled?

Part 3 deep dive

3.4 Validation done properly

Validation has four jobs: **reject** bad input, **coerce** strings into real types (everything in a query string or path is a string), **strip or forbid** unknown fields, and **produce a typed value** your code can trust. Validate every input source separately: `params`, `query`, `headers`, `body`.

```
import { z } from 'zod';

export const listQuery = z.object({
  limit: z.coerce.number().int().min(1).max(100).default(20),   // "20" -> 20
  after: z.string().optional(),                                  // opaque cursor
  status: z.enum(['draft', 'published']).optional(),
  sort: z.enum(['createdAt', '-createdAt', 'title']).default('-createdAt'), // allow-
list
});

export const createNote = z.object({
  title: z.string().trim().min(1).max(100),
  body: z.string().max(10_000).optional(),
}).strict();                                    // unknown keys are an error (not
silently dropped)

export const updateNote = createNote.partial()
  .refine((v) => Object.keys(v).length > 0, { message: 'send at least one field' });

export const dateRange = z.object({ from: z.coerce.date(), to: z.coerce.date() })
```

```
  .refine((v) => v.from <= v.to, { message: 'from must be before to', path: ['to'] }); // cross-field rule

export type CreateNote = z.infer<typeof createNote>;   // the type comes FROM the schema
```

A reusable middleware that validates all four sources and attaches the parsed result:

```
type Schemas = { body?: z.ZodTypeAny; query?: z.ZodTypeAny; params?: z.ZodTypeAny };
export const validate = (s: Schemas): express.RequestHandler => (req, _res, next) => {
  const issues: unknown[] = [];
  for (const key of ['params', 'query', 'body'] as const) {
    const schema = s[key];
    if (!schema) continue;
    const r = schema.safeParse(req[key]);
    if (r.success) (req as any).valid = { ...(req as any).valid, [key]: r.data };
    else issues.push(...r.error.issues.map((i) => ({ in: key, path: i.path.join('.'), message: i.message })));
  }
  if (issues.length) return next(new ValidationError(issues));   // handled centrally (3.5)
  next();
};
```

Notes: Zod's API differs a little between major versions (error formatting helpers especially), so confirm against the current docs. Use **discriminated unions** (`z.discriminatedUnion('type', [...])`) for payloads with variants. `400` versus `422` is a style choice: pick one rule (for example `400` for malformed JSON, `422` for well-formed but semantically invalid) and document it. Validation is not authorization: a perfectly valid `userId` in the body is still untrusted (take the user from the token, not from the body).

3.5 Error handling in depth

Build a small hierarchy and one mapping point. The error carries everything the response needs; the handler decides the status and body.

```
export class AppError extends Error {
  constructor(public status: number, public code: string, message: string,
              public details?: unknown, options?: ErrorOptions) {
    super(message, options);                  // `cause` keeps the original error for logs
  }
}
export class NotFoundError extends AppError { constructor(what: string, id: string) { super(404, `${what.toUpperCase()}_NOT_FOUND`, `${what} ${id} does not exist`); } }
export class ConflictError extends AppError { constructor(code: string, msg: string, cause?: unknown) { super(409, code, msg, undefined, { cause }); } }
export class ValidationError extends AppError { constructor(issues: unknown[]) { super(400, 'VALIDATION', 'Invalid request', issues); } }

// Map low-level database errors at the repository boundary, not in controllers.
// PostgreSQL SQLSTATE codes: 23505 unique violation, 23503 foreign key, 40001 serialization failure
export function mapDbError(e: any): never {
  if (e?.code === '23505') throw new ConflictError('DUPLICATE', 'Resource already exists', e);
```

```
  if (e?.code === '23503') throw new ConflictError('REFERENCE', 'Referenced resource
is missing', e);
  throw e;
}

export const errorHandler: express.ErrorRequestHandler = (err, req, res, _next) => {
  const known = err instanceof AppError;
  const status = known ? err.status : 500;
  (status >= 500 ? log().error : log().warn)({ err, route: req.route?.path },
'request failed');
  res.status(status).type('application/problem+json').json({
    type: `https://api.example.com/errors/${known ? err.code : 'INTERNAL'}`,
    title: known ? err.message : 'Internal server error',   // never expose internals
for 500
    status,
    code: known ? err.code : 'INTERNAL',
    details: known ? err.details : undefined,
    requestId: als.getStore()?.requestId,
  });
};
```

The body above follows **RFC 9457 problem details** (`type`, `title`, `status`, `detail`, `instance`, plus your own members). Log levels by class: expected 4xx are `warn` or `info` (not pages at 3 a.m.), 5xx are `error`. Keep a catalogue of stable error `code` strings, because clients branch on codes, never on message text. In **Fastify** use `app.setErrorHandler((err, req, reply) => ...)`; in **Nest** write an `@Catch()` exception filter (`HttpExceptionFilter`) and register it with `APP_FILTER`.

3.6 Configuration in depth

```
const Env = z.object({
  NODE_ENV: z.enum(['development', 'test', 'production']).default('development'),
  PORT: z.coerce.number().int().default(3000),
  DATABASE_URL: z.string().url(),
  REDIS_URL: z.string().url().optional(),
  JWT_SECRET: z.string().min(32),
  LOG_LEVEL: z.enum(['fatal', 'error', 'warn', 'info', 'debug',
'trace']).default('info'),
});
export const config = Object.freeze(Env.parse(process.env)); // throws at startup
with every problem listed
```

- Commit a `.env.example` (names, no secrets). Load `.env` only in development (`node --env-file=.env`); in production inject real environment variables.
- Differentiate **config** (changes per environment: URLs, secrets, limits) from **feature flags** (changes behaviour at run time, ideally without redeploy: OpenFeature, Unleash, GrowthBook) and from **constants** (belong in code).
- Secrets belong in a secrets manager (AWS Secrets Manager, GCP Secret Manager, Vault, Doppler, or your platform's secret store), rotated on a schedule. Never log the config object.
- Keep one module that reads `process.env`; everything else imports `config`. In Nest use `ConfigModule` with a `validate` function that runs the same schema.

3.7 Logging in depth

```
import pino from 'pino';
import { AsyncLocalStorage } from 'node:async_hooks';

export const logger = pino({
  level: config.LOG_LEVEL,
  redact: { paths: ['req.headers.authorization', 'req.headers.cookie', '*.password',
'*.token', '*.refreshToken'], censor: '[redacted]' },
  ...(config.NODE_ENV === 'development' && { transport: { target: 'pino-pretty' } }),
// pretty only locally
});

export const als = new AsyncLocalStorage<{ requestId: string }>();
export const log = () => logger.child({ requestId: als.getStore()?.requestId });

export const requestContext: express.RequestHandler = (req, res, next) => {
  const id = req.get('x-request-id') ?? crypto.randomUUID();   // accept upstream id,
else create one
  res.setHeader('x-request-id', id);
  const start = process.hrtime.bigint();
  res.on('finish', () => log().info({
    method: req.method, route: req.route?.path ?? 'unmatched', status:
res.statusCode,
    ms: Number(process.hrtime.bigint() - start) / 1e6,
  }, 'request completed'));
  als.run({ requestId: id }, next);
};
```

Practices: log the **route template** (`/notes/:id`), not the raw URL, so you can group by endpoint; log one completion line per request with status and duration; pass `x-request-id` (or the W3C `traceparent` header) on every outgoing call so one id follows a request across services; sample or lower the level for noisy debug logs; write to **stdout** and let the platform ship it (never manage log files from the app in containers). Treat **PII** (emails, tokens, addresses) as radioactive: redact by path, log ids not values.

3.8 API design details that separate good from great

- **PATCH semantics.** JSON Merge Patch (`application/merge-patch+json`): send only changed fields; `null` clears a field. JSON Patch (RFC 6902) is an explicit list of operations, more powerful and rarer.
- **Optimistic concurrency.** Two users edit the same record; the second silently overwrites the first. Give each row a `version` (or use `updated_at`/ETag), require `If-Match` on updates, and answer `412 Precondition Failed` (or `409`) on mismatch:

```
UPDATE notes SET title = $1, version = version + 1
WHERE id = $2 AND version = $3 RETURNING *;   -- zero rows returned => someone else
changed it
```

- **Long operations.** Return `202 Accepted` with a `Location: /v1/exports/123` status resource the client polls (or a webhook/SSE completion). Never hold an HTTP request open for minutes.
- **Collections.** Always paginate, always cap `limit`, return a stable default sort (with a unique tie-breaker such as `id`) or pagination will skip or repeat rows. Return `nextCursor` (opaque, base64 of the last sort key) instead of page numbers for large or changing data.

- **Soft delete versus hard delete.** `deleted_at` keeps history and allows undo but leaks into every query (use a view or a default scope) and conflicts with unique constraints (use partial unique indexes). Decide per entity.
- **Formats.** Timestamps in ISO 8601 UTC (`2026-10-19T06:30:00Z`), money as integer minor units plus currency code, ids as opaque strings, consistent casing (pick `camelCase` and never mix).
- **Evolution.** Additive changes are free; for removals, announce with `Deprecation` and `Sunset` headers and keep the old version running for a fixed window.
- **Idempotent methods stay idempotent.** A `DELETE` of something already deleted should still return success (or a consistent `404`), not an inconsistent mix.

3.9 More build cards

Card 3.E: list endpoint with typed query

- **Aim:** one validated, paginated, filterable collection endpoint.
- **Input:** `GET /v1/notes?limit=5&status=published&sort=-createdAt&after=<cursor>` using the `listQuery` schema.
- **Output:** `{ data: [...], nextCursor }`; `limit=1000` is rejected with 400; an unknown `sort` value is rejected; two pages together contain no duplicates when rows are inserted between requests.
- **Check:** a script that pages through 100 seeded rows while another script inserts rows.

Card 3.F: optimistic concurrency

- **Aim:** stop lost updates.
- **Input:** `ETag: "<version>"` on `GET /notes/:id`; `PATCH` requires `If-Match`.
- **Output:** two clients read version 3; the first update succeeds (version 4); the second gets `412`.
- **Check:** run the two clients as parallel `curl` commands.

Card 3.G: database error mapping

- **Aim:** no raw database error ever reaches a client.
- **Input:** a unique index on `users.email`.
- **Output:** registering the same email twice returns `409 DUPLICATE` with your error body (not a 500); the log line contains the original `cause`.
- **Check:** register twice, inspect the response and the log.

Card 3.H: logging safety test

- **Aim:** prove secrets are redacted.
- **Input:** a test that logs a request carrying `Authorization` and a body with `password`.
- **Output:** captured log output (use a pino destination stream in the test) contains `[redacted]` and never the secret strings.
- **Check:** grep the captured output for the secrets in the assertion.

3.10 Answer key for the Part 3 self-checks

- **Why validate at the edge?** Services then receive already-typed, trusted data, so business rules are not cluttered with defensive checks, and invalid input is rejected before it touches the database or other systems.
- **Why 409 for a duplicate email?** The request is well-formed but conflicts with the current state of the resource; 400 means the request itself is malformed.
- **What breaks if `process.env.X` is read everywhere?** Missing or malformed values surface late as runtime bugs, types are lost (all strings), tests must mutate globals, and there is no single place that documents or validates configuration.

3.4 Deep dive: validation with Zod, properly

One schema gives you runtime checking **and** the TypeScript type, so they cannot drift apart. Keep schemas next to the module (`notes.schemas.ts`) and reuse them for the controller, the service input type, and the OpenAPI document.

```
import { z } from 'zod';

export const CreateNote = z.object({
  title: z.string().trim().min(1).max(100),
  body: z.string().max(10_000).optional(),
  tags: z.array(z.string().min(1)).max(10).default([]),
});
export type CreateNoteInput = z.infer<typeof CreateNote>;

export const UpdateNote = CreateNote.partial();            // PATCH: every field
optional

export const ListQuery = z.object({
  limit: z.coerce.number().int().min(1).max(100).default(20),   // query values
arrive as strings
  after: z.string().optional(),
  sort: z.enum(['createdAt', '-createdAt']).default('-createdAt'),
});
```

Things that trip people up:

- **Query strings and path params are strings.** Use `z.coerce.number()` for numbers. Do **not** use `z.coerce.boolean()` for env flags: any non-empty string, including `"false"`, becomes `true`. Parse booleans explicitly (`z.enum(['true','false']).transform(v => v === 'true')`).
- **Unknown keys.** By default Zod object schemas *strip* unknown keys silently. Reject them instead when you want strict APIs (a `.strict()` object in older Zod, `z.strictObject` in Zod 4; check the docs for your version). Stripping is a useful defence against mass assignment, because the DB layer only ever sees known fields.
- **Refinements** express cross-field rules: `.refine(d => d.end > d.start, { path: ['end'], message: 'end must be after start' })`.
- **Transforms** convert after validating (`z.string().transform(s => s.toLowerCase())` for emails). Remember the input type and the output type then differ (`z.input` versus `z.output`).
- **Discriminated unions** model variants cleanly: `z.discriminatedUnion('type', [...])`.

- **Validate responses for secrets.** Define a `PublicUser` schema (without `passwordHash`) and always return `PublicUser.parse(row)`. In Fastify, a response schema does this for you.
- **Two models per resource.** Keep the *input* shape, the *database row* shape, and the *public output* shape separate. Returning DB rows directly is how internal columns leak.

Alternatives: TypeBox and Ajv compile JSON Schema (fastest, native to Fastify, good for generating OpenAPI); Valibot has a smaller bundle; class-validator fits Nest's decorator style; ArkType offers very fast checks with TypeScript-like syntax.

3.5 Deep dive: configuration done right

Parse the environment **once**, fail on the first problem, and export a typed frozen object. Nothing else in the code base reads `process.env`.

```
// config.ts
import { z } from 'zod';

const Env = z.object({
  NODE_ENV: z.enum(['development', 'test', 'production']).default('development'),
  PORT: z.coerce.number().int().default(3000),
  LOG_LEVEL: z.enum(['fatal', 'error', 'warn', 'info', 'debug',
'trace']).default('info'),
  DATABASE_URL: z.string().url(),
  REDIS_URL: z.string().url().default('redis://localhost:6379'),
  JWT_SECRET: z.string().min(32),
});

const parsed = Env.safeParse(process.env);
if (!parsed.success) {
  console.error('Invalid environment:', parsed.error.flatten().fieldErrors);
  process.exit(1);
}
export const config = Object.freeze(parsed.data);
```

- Commit a `.env.example` with every key and a harmless value; never commit `.env`. Add `.env` to `.gitignore` and `.dockerignore`.
- Use a secrets manager in production (AWS Secrets Manager, GCP Secret Manager, HashiCorp Vault, Doppler, or your platform's secret store), injected as environment variables at start.
- Distinguish **config** (changes per environment) from **feature flags** (change at runtime without a deploy) and from **constants** (never change).
- Pass config into modules through parameters (or Nest's `ConfigService`) rather than importing a global everywhere; it keeps tests simple.

3.6 Deep dive: a complete error model

Design errors like an API: stable machine-readable **codes**, human **messages**, and the right **HTTP status**. Services throw; one handler translates.

```
export class AppError extends Error {
  constructor(public status: number, public code: string, message: string, public
details?: unknown) { super(message); }
}
export class NotFoundError extends AppError { constructor(what: string) { super(404,
```

```
'NOT_FOUND', `${what} not found`); } }
export class ConflictError extends AppError { constructor(msg: string) { super(409,
'CONFLICT', msg); } }
export class ForbiddenError extends AppError { constructor() { super(403,
'FORBIDDEN', 'You may not do this'); } }

export function errorHandler(err: unknown, req: Request, res: Response, _next:
NextFunction) {
  if (err instanceof ZodError)
    return res.status(400).json({ error: { code: 'VALIDATION', issues: err.issues,
requestId: req.id } });
  if (err instanceof AppError)
    return res.status(err.status).json({ error: { code: err.code, message:
err.message, requestId: req.id } });
  if ((err as { code?: string }).code === '23505')          // PostgreSQL unique
violation
    return res.status(409).json({ error: { code: 'CONFLICT', message: 'Already
exists', requestId: req.id } });
  req.log.error({ err }, 'unhandled error');                // full detail goes to
logs only
  res.status(500).json({ error: { code: 'INTERNAL', message: 'Something went wrong',
requestId: req.id } });
}
```

Situation	Status	Note
Malformed JSON, wrong types	400	Syntax or type problem
Valid shape but breaks a business rule	422	Many teams use 400 for both; pick one and document it
Not logged in	401	Add `WWW-Authenticate` for Basic or Bearer
Logged in but not allowed	403	
Resource the caller must not even know exists	404	Avoids confirming existence
Duplicate or version conflict	409	Also for optimistic-lock failures
Payload too large	413	Set body limits
Too many requests	429	Include `Retry-After`
Upstream service failed	502 or 503	504 for upstream timeout

Translate low-level errors at the **boundary**: database unique violation (`23505`), foreign key violation (`23503`), serialization failure (`40001`, retry the transaction), timeouts. Optionally use the **RFC 9457 problem details** format (`application/problem+json` with `type`, `title`, `status`, `detail`, `instance`) so generic clients understand your errors. For **optimistic concurrency**, send an `ETag` or a `version` field and update with `WHERE id = :id AND version = :version`; zero rows updated means `409`.

3.7 Deep dive: logging that helps at 3 a.m.

```
import pino from 'pino';
import { AsyncLocalStorage } from 'node:async_hooks';
import { randomUUID } from 'node:crypto';

export const logger = pino({
  level: config.LOG_LEVEL,
  redact: { paths: ['req.headers.authorization', 'req.headers.cookie', '*.password',
'*.token'], censor: '[redacted]' },
  ...(config.NODE_ENV === 'development' && { transport: { target: 'pino-pretty' } }),
});

export const als = new AsyncLocalStorage<{ requestId: string }>();
export const log = () => logger.child({ requestId: als.getStore()?.requestId });

// middleware
app.use((req, res, next) => {
  const id = (req.headers['x-request-id'] as string) ?? randomUUID();
  res.setHeader('x-request-id', id);
  als.run({ requestId: id }, next);
});
```

- **Level discipline:** `error` means someone should look; `warn` is unusual but handled; `info` is business events (order created); `debug` is for development. Too many `error` logs train people to ignore them.
- **Log objects, not strings:** `log().info({ orderId, total }, 'order created')` is searchable by field.
- **Never log** passwords, tokens, full card numbers, or personal data you do not need. Use `redact` and review it.
- **Propagate** the request id to outbound calls (`x-request-id`, or the W3C `traceparent` header when using OpenTelemetry) so one user action can be followed across services.
- **Performance:** pino writes JSON synchronously to stdout by default, which is fast; ship logs from stdout with the platform (Docker, Kubernetes, the cloud agent) instead of writing files from the app.
- **Sampling and volume:** debug logs in production can cost more than the servers; enable them per request id or per user when investigating.

3.8 Deep dive: route and resource design checklist

- Nouns and plurals: `/v1/orders`, `/v1/orders/42`, `/v1/orders/42/items`. Avoid `/getOrder`.
- **Actions that are not CRUD** become sub-resources or verbs on a resource: `POST /v1/orders/42/cancel`, `POST /v1/users/7/password-reset`.
- **Bulk operations:** `POST /v1/notes:batch` or `POST /v1/notes/bulk` returns per-item results; define whether failures are all-or-nothing.
- **Soft delete** (`deleted_at`) when you need undo or audit, but remember every query must filter it (a database view or a partial index helps).
- **Consistent list envelope:**

```
{ "data": [ { "id": "n_1", "title": "..." } ], "page": { "nextCursor":
"eyJpZCI6MTAwfQ", "limit": 20 } }
```

- **Return created resources** with `201` and a `Location` header. Return `204` with an empty body for deletes.
- **Do not expose database ids** if they are guessable and ownership checks are weak; use UUIDs or prefixed ids, and still check ownership.
- **Dates** in ISO 8601 UTC (`2026-10-19T08:30:00Z`); **money** as integer minor units plus currency code.

Middleware order recipe

Order	Middleware	Reason
1	Request id and logger	Everything after can log with the id
2	Security headers, CORS	Applies to every response, including errors
3	Rate limit	Reject abuse before spending CPU
4	Body parser with size limit	Needed by validation and auth that read bodies
5	Authentication	Identifies the caller
6	Routes with validation and authorization	Business entry points
7	404 handler	Nothing matched
8	Error handler	Last, catches everything

3.9 More build cards

Card 3.E: schemas for a list endpoint

- **Aim:** coerce and bound query parameters.
- **Input:** `ListQuery` above applied to `GET /v1/notes`.
- **Output:** `?limit=abc` gives `400` with a clear issue; `?limit=1000` gives `400`; no params uses `limit=20` and `-createdAt`.
- **Check:** four `curl` cases in a script with expected statuses.

Card 3.F: translate database errors

- **Aim:** no raw database errors leak.
- **Input:** a unique index on `users(email)`; `POST /users` twice with the same email.
- **Output:** the second call returns `409 CONFLICT` with a requestId, and the log contains the original database error.
- **Check:** a test that asserts the status, the body shape, and that the body contains no SQL text.

Card 3.G: redaction test

- **Aim:** prove secrets never reach logs.
- **Input:** a login request with `password` and an `Authorization` header, with the logger writing to an in-memory stream.

- **Output:** the captured log lines contain `[redacted]` and no trace of the real values.
- **Check:** the test fails if you remove a path from `redact`.

Card 3.H: optimistic concurrency

- **Aim:** stop lost updates.
- **Input:** `PATCH /v1/notes/:id` with `If-Match: <version>`.
- **Output:** two simultaneous edits: one succeeds and bumps `version`, the other gets `409` (or `412`).
- **Check:** fire both requests in parallel with `Promise.all` in a script and assert exactly one `200`.

Part 4: Databases (SQL, ORMs, MongoDB, Redis)

Most backend bugs and slowdowns live in the data layer, so this part is long on purpose.

4.1 Theory: relational databases (PostgreSQL)

- **Tables, rows, keys.** A primary key identifies a row; a foreign key links rows (`orders.user_id` references `users.id`). Relationships: one-to-many, many-to-many (via a join table).
- **Normalisation.** Store each fact once. Duplicate data only on purpose, for speed, and know why.
- **Indexes.** A B-tree index turns a full table scan into a lookup. Index columns you filter, join, or sort on. Every index slows writes and uses disk, so add them based on slow queries, checked with `EXPLAIN ANALYZE`. A composite index `(user_id, created_at)` serves queries that filter by `user_id` and sort by `created_at`.
- **Transactions (ACID).** A group of statements that all succeed or all roll back. Needed whenever two writes must stay consistent (debit one account, credit another). Isolation levels (read committed, repeatable read, serializable) trade safety for concurrency; PostgreSQL defaults to read committed.
- **Constraints.** `NOT NULL`, `UNIQUE`, `CHECK`, foreign keys put your rules in the database, where no bug in the app can bypass them.
- **Connection pooling.** Opening a connection is expensive and PostgreSQL has a hard connection limit. Keep a pool (typically 10 to 20 per app instance) and use PgBouncer or a managed pooler when you have many instances.
- **The N+1 problem.** Loading 100 posts and then querying the author 100 times. Fix with a join, an `IN` batch, or a loader (DataLoader in GraphQL).
- **Pagination.** `OFFSET` gets slower as pages go deeper. **Keyset (cursor) pagination** (`WHERE id < :last ORDER BY id DESC LIMIT 20`) stays fast.
- **Migrations.** Schema changes are versioned files applied in order. Never edit the production schema by hand. For zero downtime: add column nullable, deploy code that writes both, backfill, then enforce and remove the old column.

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1 AND balance >= 100;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;   -- or ROLLBACK on any error

CREATE INDEX CONCURRENTLY idx_orders_user_created ON orders (user_id, created_at
DESC);
```

4.2 Theory: NoSQL and when to choose it

Type	Example	Good for	Weak at
Relational	PostgreSQL, MySQL	Most business data, reporting, strong consistency, joins	Extremely wide horizontal write scaling needs extra work
Document	MongoDB	Flexible, nested data that is read as one unit (CMS content, catalogues)	Multi-document relations and transactions are clumsier
Key-value	Redis, DynamoDB	Caching, sessions, counters, leaderboards, rate limits	Ad hoc queries
Search	Elasticsearch, OpenSearch, Meilisearch, Typesense	Full-text search, faceting, autocomplete	Source of truth (keep it a secondary copy)
Wide column / time series	Cassandra, ClickHouse, TimescaleDB	Huge write volume, analytics, metrics	Transactions, joins
Graph	Neo4j	Deep relationship traversal	General CRUD

Default advice: start with PostgreSQL. It also handles JSON (`jsonb`), full-text search, queues (`SKIP LOCKED`), and geospatial (PostGIS), so you can postpone extra systems until a measured need appears.

4.3 Libraries and alternatives

Level	Pick	Alternatives	Trade-off
Raw driver	`pg` (node-postgres)	postgres.js, mysql2, better-sqlite3	Full control, you write SQL and map rows
Query builder	Kysely	Knex	Type-safe SQL, close to the metal; Knex is older, less type-safe
ORM (schema-first, generated client)	Prisma		Great developer experience and types; own schema language; historically heavier runtime; check current docs for the query engine details
ORM (SQL-like, TypeScript schema)	Drizzle		Thin, close to SQL, lightweight, fast-growing
ORM (decorator,	TypeORM,	Sequelize	TypeORM is common in Nest; MikroORM has a cleaner unit-of-work model; Sequelize is

Level	Pick	Alternatives	Trade-off
Active Record / Data Mapper)	MikroORM		legacy-heavy
MongoDB	Mongoose	Official `mongodb` driver, Typegoose	Mongoose adds schemas and validation
Redis	ioredis	`redis` (node-redis)	Both are fine; ioredis is popular with BullMQ
Migrations	Prisma Migrate / drizzle-kit	node-pg-migrate, Knex migrations, Flyway, Atlas	Use the tool that matches your ORM
Testing DB	Testcontainers	docker-compose, pg-mem	Real PostgreSQL in a throwaway container

Choosing: want fastest start and great types, use Prisma. Want SQL control with types, use Drizzle or Kysely. Writing complex reports, drop to raw SQL for those queries; no ORM does everything well.

```
// Prisma: transaction + include (avoids N+1)
const order = await prisma.$transaction(async (tx) => {
  const o = await tx.order.create({ data: { userId, items: { create: items } } });
  await tx.stock.updateMany({ where: { sku: { in: skus } }, data: { qty: { decrement:
1 } } });
  return o;
});
const posts = await prisma.post.findMany({ take: 20, include: { author: true } });
```

4.4 Redis in 5 minutes

Redis is an in-memory data structure server. Core types: **string** (cache values, counters with `INCR`), **hash** (an object), **list** (simple queue), **set**, **sorted set** (leaderboards, delayed jobs), **stream** (event log), plus **pub/sub**. Every key can expire (`SET key value EX 60`). Data is lost on restart unless persistence (RDB/AOF) is configured, so treat it as a cache or a fast helper store unless you deliberately design otherwise.

4.5 Build cards

Card 4.A: PostgreSQL in Docker plus a repository

- **Aim:** replace the in-memory array of Card 2.A with a real table.
- **Input:** a `docker-compose.yml` with `postgres:16` (or newer), a migration creating `notes(id, title, body, created_at)`.
- **Output:** a `NotesRepository` with `create`, `findById`, `list`, `update`, `delete`; the HTTP layer only calls the repository.

- **Check:** restart the server and your notes are still there. `psql` shows the rows.

Card 4.B: relations and N+1

- **Aim:** see N+1 and fix it.
- **Input:** tables `users` and `posts` (one user has many posts); an endpoint `GET /posts` returning each post with its author name.
- **Output:** first version runs 1 + N queries (log them); second version runs 1 or 2.
- **Check:** turn on query logging and count statements for 50 posts before and after.

Card 4.C: transaction with a rollback

- **Aim:** all-or-nothing writes.
- **Input:** `POST /transfer {from, to, amount}` using two accounts.
- **Output:** insufficient balance gives `409` and **no** row changes; success changes both rows.
- **Check:** fire 20 parallel transfers with a script and confirm the total balance never changes.

Card 4.D: index and keyset pagination

- **Aim:** measure what an index does.
- **Input:** seed 1 million rows (use `generate_series`); query one user's latest 20 rows.
- **Output:** `EXPLAIN ANALYZE` before (sequential scan) and after (index scan) with the timing difference; an endpoint using a `cursor` query parameter.
- **Check:** record both timings in your notes.

Self-check questions: What does a transaction guarantee that two separate queries do not? Why can an index make writes slower? When would you pick MongoDB over PostgreSQL, honestly?

4.6 Deep dive: transactions, isolation, and locking in PostgreSQL

MVCC in one paragraph. PostgreSQL never overwrites a row in place. An update writes a new row version and marks the old one dead; each transaction sees a consistent snapshot, so readers do not block writers. Dead versions are cleaned by **autovacuum**. Consequences: long-running transactions hold back cleanup and bloat tables; never leave a transaction open while you call an external API.

Isolation levels and what each prevents

Level	Dirty read	Non-repeatable read	Phantom read	Write skew	Notes
Read committed (default)	No	Possible	Possible	Possible	Each statement sees data committed before it started
Repeatable read	No	No	No (snapshot)	Possible	Whole transaction uses one snapshot; may fail with `40001`, so retry
Serializable	No	No	No	No	Detects conflicts and

Level	Dirty read	Non-repeatable read	Phantom read	Write skew	Notes
					aborts one transaction with `40001`; always wrap in a retry loop

Write skew is the subtle one: two transactions each read a set of rows, make a decision, and write *different* rows, and together they break a rule (two on-call doctors both go off duty because each saw the other still on duty). Read committed and repeatable read allow it. Solutions: serializable isolation, a locking read (`SELECT ... FOR UPDATE` on the rows the rule depends on), or a constraint that makes the rule a database fact.

Explicit locking tools

Tool	What it does	Typical use
`SELECT ... FOR UPDATE`	Locks selected rows until commit	Read a balance, then write it based on the value
`FOR UPDATE SKIP LOCKED`	Skips rows already locked by others	Job queues and outbox relays: many workers, each takes different rows
`FOR UPDATE NOWAIT`	Fails immediately if locked	"Someone else is editing"
Optimistic version column	No lock; check version on update	Web forms and APIs (Part 3)
Advisory locks (`pg_advisory_xact_lock(key)`)	Application-defined mutex held by the database	One cron runner at a time, per-tenant critical sections
Atomic SQL (`UPDATE ... SET qty = qty - 1 WHERE qty > 0`)	Single statement is atomic	Counters and stock; often the best option

Deadlocks. Two transactions each hold a lock the other needs; PostgreSQL aborts one (`40P01`). Prevent by always locking rows in the **same order** (sort ids before updating many rows) and keeping transactions short. Retry on `40P01` and `40001`.

Transaction hygiene. Open late, close early; no network calls inside; set `statement_timeout`, `lock_timeout`, and `idle_in_transaction_session_timeout` so a stuck client cannot hold locks forever.

```
-- job queue claim: many workers, no double processing
WITH next AS (
  SELECT id FROM jobs WHERE status = 'queued' AND run_at <= now()
  ORDER BY id LIMIT 10 FOR UPDATE SKIP LOCKED
)
```

```
UPDATE jobs SET status = 'running', started_at = now()
FROM next WHERE jobs.id = next.id RETURNING jobs.*;
```

4.7 Deep dive: indexes and reading query plans

Index types

Type	Good for	Notes
B-tree (default)	Equality, ranges, sorting, `LIKE 'abc%'` (with the right operator class)	Covers most needs
Composite (`a, b, c`)	Filters on `a`, or `a,b`, or `a,b,c`, plus ordering	Column order matters: equality columns first, then range or sort
Partial `WHERE cond`	Index only the rows you query	`WHERE deleted_at IS NULL`, `WHERE status = 'queued'`
Covering `INCLUDE (cols)`	Index-only scans without table visits	Adds size; use for hot read paths
Expression (`lower(email)`)	Case-insensitive lookups	Query must use the same expression
GIN	`jsonb` containment, arrays, full-text (`tsvector`), trigrams (`pg_trgm`)	Slower writes
GiST / SP-GiST	Geometry (PostGIS), ranges, nearest neighbour	
BRIN	Huge, naturally ordered tables (time series by insert time)	Tiny index, coarse
Hash	Equality only	Rarely better than B-tree
Vector (pgvector HNSW/IVFFlat)	Embedding similarity search	Useful for semantic search and ranking pipelines

Reading `EXPLAIN (ANALYZE, BUFFERS)`. Read from the innermost node out. Look for: `Seq Scan` on a large table with a selective filter (missing index); estimated rows far from actual rows (stale statistics, run `ANALYZE`); `Sort` spilling to disk; `Nested Loop` with a huge inner side; many `Buffers: read` (cold cache). An index is ignored when the query wraps the column in a function, casts types, uses a leading wildcard, or the planner decides a scan is cheaper because most rows match.

Rules of thumb. Index every foreign key you join or delete by (PostgreSQL does not do it for you). Prefer one good composite index over several single-column ones. Drop unused indexes (`pg_stat_user_indexes`). Create indexes in production with `CREATE INDEX CONCURRENTLY`. Use `pg_stat_statements` to find the queries that cost the most total time; fix those first.

4.8 Deep dive: schema and query craft

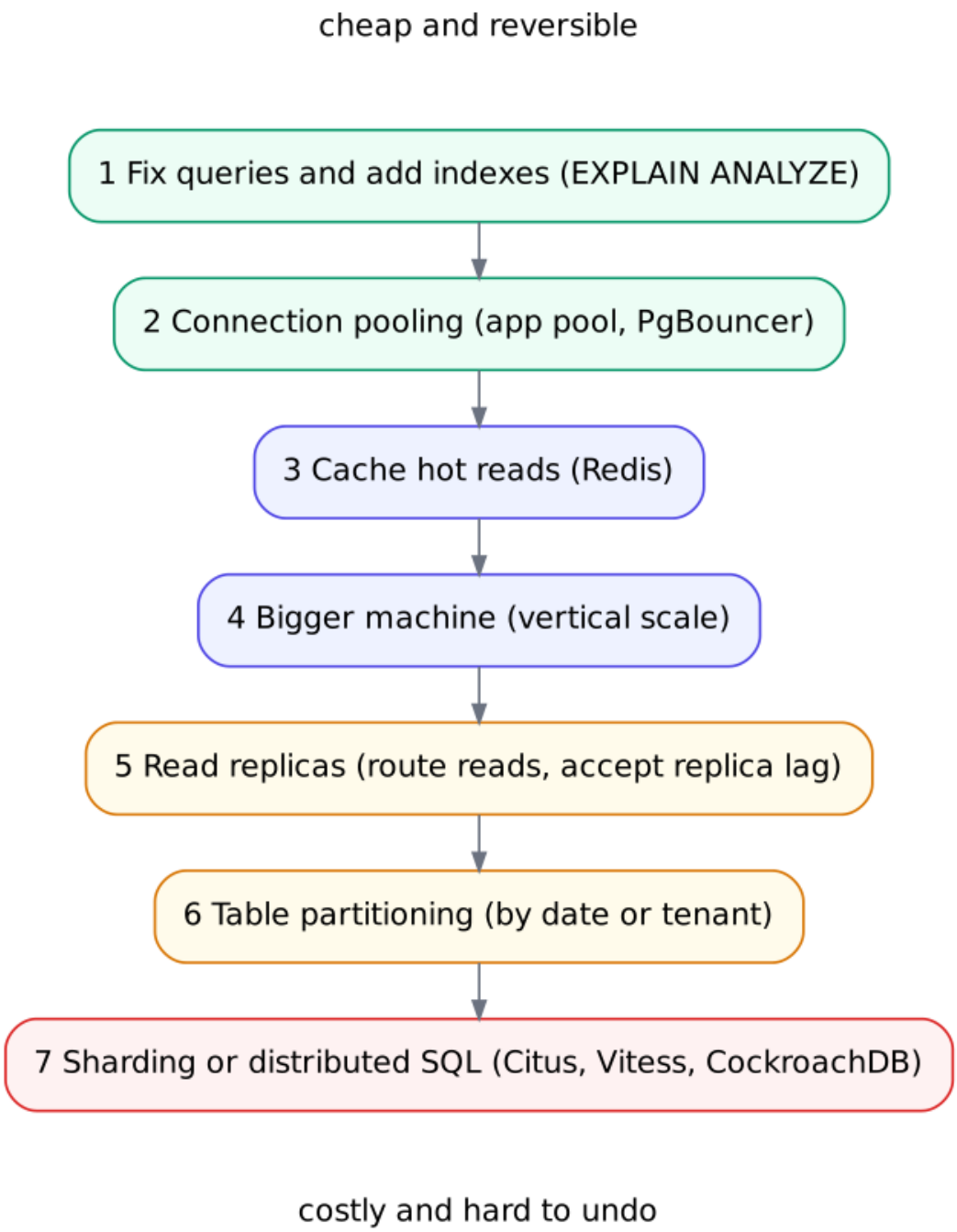
- **Types.** `timestamptz` for instants (never plain `timestamp` for events), `numeric(12,2)` or integer minor units for money, `text` with a `CHECK` rather than `varchar(n)` guesses, `uuid` (time-ordered v7 if available) or `bigint identity` for keys, `jsonb` for genuinely schemaless parts only. Enums are fine for stable sets; a lookup table is easier to change.
- **Constraints are free correctness.** `NOT NULL`, `CHECK (qty >= 0)`, `UNIQUE (tenant_id, slug)`, `FOREIGN KEY ... ON DELETE` with a deliberate choice (`RESTRICT`, `CASCADE`, `SET NULL`), and exclusion constraints for non-overlapping bookings.
- **Upsert.** `INSERT ... ON CONFLICT (email) DO UPDATE SET ... RETURNING *` is atomic and race-free; it is how you make writes idempotent.
- **Batch work.** Insert many rows with one multi-row `INSERT` or `COPY` (via `pg-copy-streams`), not a loop of single inserts. Update in chunks of a few thousand ids with short transactions.
- **Avoid `SELECT *`** in hot paths; select needed columns. Avoid `OFFSET` for deep pages; use keyset.
- **CTEs and window functions** (`ROW_NUMBER() OVER (PARTITION BY ...)`, `LAG`, running totals) solve many reporting queries in SQL instead of in Node loops.
- **Full-text search and fuzzy matching** work in the database: `to_tsvector`, `websearch_to_tsquery`, `ts_rank`, and `pg_trgm` for typo-tolerant `ILIKE`.
- **Row-level security (RLS).** `ALTER TABLE notes ENABLE ROW LEVEL SECURITY; CREATE POLICY tenant ON notes USING (tenant_id = current_setting('app.tenant')::uuid);` then `SET LOCAL app.tenant = ...` inside each transaction. It is a strong safety net for multi-tenancy.
- **Migrations: expand and contract.** (1) *Expand*: add the new nullable column or table, deploy code that writes both. (2) *Backfill* in small batches. (3) *Switch* reads. (4) *Contract*: drop the old column in a later release. Never rename or drop a column in the same release as the code change. Adding a column with a volatile default or a `NOT NULL` without default can lock a big table; check the lock level before running it.
- **Backups.** A backup you have not restored is a hope. Use managed point-in-time recovery or `pgBackRest`/WAL archiving, and rehearse a restore on a schedule.

4.9 Deep dive: pools, replicas, and the scaling ladder

Pool sizing. Each PostgreSQL connection is an operating-system process with real memory cost; default `max_connections` is around 100. If you run 10 app instances with a pool of 20 you need 200 and will fail. Budget: `instances x pool size` plus workers, migrations, and admins must stay under the limit. A small pool (10 to 20 per instance) usually gives better throughput than a huge one, because the database can only run so many queries in parallel. Set `connectionTimeoutMillis` so requests fail fast when the pool is exhausted rather than queueing forever.

PgBouncer (or a managed pooler like RDS Proxy or Supabase's pooler) multiplexes many client connections onto few server connections. In *transaction* mode a server connection is borrowed only for one transaction, so session features (`SET` without `LOCAL`, session advisory locks, `LISTEN/NOTIFY`, and in older versions prepared statements) do not work; check your pooler version and ORM settings (Prisma needs a flag such as `pgbouncer=true`). Serverless functions almost always need a pooler.

Read replicas. A replica copies the primary asynchronously, so a read right after a write can return old data (**replica lag**). Route only tolerant reads (feeds, reports) to replicas; keep "read your own write" reads on the primary, for example for a few seconds after that user writes.



Partitioning splits one big table into parts by range (month), list (tenant), or hash, so queries that filter by the key touch one partition and old data can be dropped instantly (`DROP` a partition instead of a huge `DELETE`). **Sharding** spreads data over several servers; it brings cross-shard queries, resharding, and distributed transactions, so delay it. Alternatives that do it for you: Citus, Vitess (MySQL), CockroachDB, YugabyteDB, or a managed service such as Aurora or Spanner.

4.10 Deep dive: ORMs and query builders in practice

- **Keep SQL visible.** Turn on query logging in development; if you cannot see the SQL you will ship N+1 queries. Prisma: `log: ['query']`; Drizzle: `logger: true`; TypeORM: `logging: true`.
- **Know the escape hatch.** Complex reports, window functions, upserts with conditions, and bulk updates are often clearer in raw SQL (`prisma.$queryRaw`, Drizzle `sql` template, Kysely). Always use the parameterised form; never string-concatenate input.
- **Interactive transactions** (`prisma.$transaction(async tx => ...)`) hold a connection for their whole duration and have a timeout; keep them short and do not call APIs inside them.
- **Generated types versus domain types.** Do not leak ORM models through your API or into the domain layer; map to plain objects in the repository, so a schema change does not ripple everywhere (Part 10).
- **Migrations in CI.** Generate migrations in development, review the SQL by eye, run them as a separate deploy step. Watch for **schema drift** (someone changed production by hand).
- **Soft delete and multi-tenancy filters** are easy to forget in an ORM; centralise them in the repository or an ORM extension, and back them with RLS.
- **Identity maps and lazy loading** (TypeORM, MikroORM) can trigger queries unexpectedly when you access a relation; prefer explicit loading.

4.11 Deep dive: MongoDB

- **Model for your queries.** *Embed* data that is read together and bounded in size (an order with its line items). *Reference* data that is shared or grows without bound (a user with all their events). An ever-growing array inside one document is a classic mistake; documents are limited to 16 MB and large documents are rewritten on update.
- **Indexes.** The same ideas apply. For compound indexes follow **ESR**: Equality fields first, then Sort fields, then Range fields. Check with `.explain('executionStats')` and watch `totalDocsExamined` against `nReturned`.
- **Schema validation.** MongoDB is flexible, not schema-free in a healthy app. Use Mongoose schemas or server-side JSON Schema validation, and a `schemaVersion` field for migrations.
- **Transactions** exist (they need a replica set) but cost performance; if you need them everywhere you probably want a relational database.
- **Aggregation pipeline** (`$match`, `$group`, `$lookup`, `$facet`) is powerful; put `$match` and `$project` early so indexes are used. Heavy `$lookup` joins are a hint to reconsider the model.
- **Mongoose tips.** Use `.lean()` for read-only queries (plain objects, much faster), define indexes in the schema but build them via a migration in production, and set `strictQuery` deliberately.

4.12 Deep dive: Redis patterns and pitfalls

Need	Structure and commands	Notes

Need	Structure and commands	Notes
Cache an object	`SET key json EX 60` or a hash `HSET`	Keep values small; compress large ones
Atomic counter, rate limit	`INCR` + `EXPIRE` (or a Lua script / `rate-limiter-flexible`)	Do the pair atomically, for example with `SET key 0 NX EX 60` then `INCR`, or a Lua script
Leaderboard	Sorted set `ZADD`, `ZRANGE ... REV`, `ZRANK`	Range queries are cheap
Session store	String or hash with TTL	Refresh the TTL on activity
Delayed jobs	Sorted set scored by run time	This is how BullMQ does it
Unique visitors	`PFADD` (HyperLogLog), set	Approximate counts in tiny memory
Event log / consumer groups	Streams `XADD`, `XREADGROUP`, `XACK`	A lightweight alternative to Kafka
Fan-out messages	Pub/Sub `PUBLISH`/`SUBSCRIBE`	Not persistent; subscribers who are offline miss messages
Lock	`SET lock token NX PX 30000`, release with a Lua compare-and-delete	Safe for efficiency locks, not for correctness-critical locking; use fencing tokens or database locks for that

Pitfalls. Never run `KEYS *` in production (it blocks the single command thread); use `SCAN`. Avoid **big keys** (a 50 MB hash) and **hot keys** (one key taking all traffic). Set a **maxmemory** and an **eviction policy**: the default `noeviction` makes writes fail when full; `allkeys-lru` suits pure caches; `volatile-ttl` or `volatile-lru` evicts only keys with a TTL. **Persistence:** RDB snapshots (fast restart, can lose minutes) and AOF (append log, loses about a second with `everysec`); a cache can run without either, but queues and sessions need one. **High availability:** Sentinel (automatic failover for one primary) or Redis Cluster (data split over 16,384 hash slots; multi-key commands need keys in the same slot, which you force with hash tags like `{user:42}:cart`). Always set connection timeouts and an `error` listener on the client; an unhandled `error` event crashes the process.

4.13 More build cards

Card 4.E: observe isolation anomalies

- **Aim:** see a non-repeatable read and write skew happen.
- **Input:** two `psql` sessions, a table `doctors(id, on_call bool)` with two on-call rows.
- **Output:** at `READ COMMITTED` both sessions can go off duty and leave zero on call; at `SERIALIZABLE` one commit fails with `40001`.
- **Check:** write the exact interleaving steps down, run it at both levels, and add a retry loop in Node that makes the serializable version succeed.

Card 4.F: job queue in PostgreSQL

- **Aim:** use `SKIP LOCKED`.
- **Input:** the `jobs` table and the claim query above; 3 worker processes; 1,000 queued jobs.
- **Output:** every job is processed exactly once, and the 3 workers share the load roughly evenly.
- **Check:** each worker writes its id into the row; `SELECT worker, count(*) GROUP BY worker` shows an even split and no duplicates.

Card 4.G: plan reading drill

- **Aim:** diagnose three slow queries.
- **Input:** a 2-million-row `orders` table; queries: filter by `user_id` and `created_at` range, case-insensitive email search, `OFFSET 100000`.
- **Output:** for each query, the `EXPLAIN (ANALYZE, BUFFERS)` before, the fix (composite index, expression index, keyset), and the plan after with timings.
- **Check:** a short table of before/after milliseconds and a sentence naming why the planner chose each plan.

Card 4.H: expand and contract migration

- **Aim:** rename a column with zero downtime.
- **Input:** `users.fullname` to `users.display_name` while a load test runs.
- **Output:** four releases (add column + dual write, backfill in batches of 5,000, read new column, drop old) with no failed requests.
- **Check:** autocannon runs through all four steps; the error count stays 0.

Card 4.I: row-level security for tenants

- **Aim:** make a missed `WHERE` harmless.
- **Input:** table with `tenant_id`, an RLS policy, and a middleware that runs `SET LOCAL app.tenant`.
- **Output:** a deliberately unfiltered `SELECT * FROM notes` inside tenant A's request returns only A's rows.
- **Check:** a test that logs in as tenants A and B and tries to read each other's rows by id; expect 404 for every attempt.

Card 4.J: pool exhaustion drill

- **Aim:** see what happens at the limit.
- **Input:** pool size 5, an endpoint that holds a connection for 2 seconds, 50 concurrent requests.
- **Output:** requests queue, then time out with a clear `503` after `connectionTimeoutMillis`; `/health` (no DB) still responds fast.
- **Check:** graph pool `waiting` count during the test; then raise the pool to 50 and note what happens to database CPU.

Card 4.K: Redis cache plus lock plus leaderboard

- **Aim:** three Redis patterns in one service.

- **Input:** a product cache with TTL, a `SET NX PX` lock around a nightly job, and a sorted-set leaderboard endpoint.
- **Output:** two job instances started together: only one runs; the leaderboard returns the top 10 in under 5 ms with 100,000 members.
- **Check:** start both instances with a script; read the logs; benchmark the endpoint with autocannon.

Deep-dive self-check: (1) What is write skew and which isolation level stops it? (2) Why does the column order of a composite index matter? (3) Why can a bigger pool make a database slower? (4) What does replica lag break? (5) Why is `KEYS *` dangerous? (6) What is the safe order of steps to rename a column in production?

Part 4 deep dive

4.6 SQL you must be fluent in

An ORM hides SQL until the day it generates a slow query; then only SQL knowledge saves you. Five constructs cover most real work:

```
-- JOINs: INNER keeps matches only, LEFT keeps every left row (NULLs when no match)
SELECT u.id, u.email, count(o.id) AS orders, coalesce(sum(o.total_cents), 0) AS spent
FROM users u LEFT JOIN orders o ON o.user_id = u.id AND o.status = 'paid'
GROUP BY u.id HAVING count(o.id) > 0 ORDER BY spent DESC LIMIT 20;

-- EXISTS is often better than IN or a JOIN for "has at least one"
SELECT * FROM users u WHERE EXISTS (SELECT 1 FROM orders o WHERE o.user_id = u.id);

-- CTE: name a step to keep a query readable
WITH recent AS (SELECT * FROM orders WHERE created_at > now() - interval '30 days')
SELECT user_id, count(*) FROM recent GROUP BY user_id;

-- Window functions: compute over related rows WITHOUT collapsing them
SELECT id, user_id, total_cents,
       row_number() OVER (PARTITION BY user_id ORDER BY created_at DESC) AS rn,   --
latest per user: WHERE rn = 1
       sum(total_cents) OVER (PARTITION BY user_id ORDER BY created_at) AS
running_total
FROM orders;

-- UPSERT: insert or update atomically (race-free, unlike select-then-insert)
INSERT INTO counters (key, n) VALUES ('visits', 1)
ON CONFLICT (key) DO UPDATE SET n = counters.n + 1 RETURNING n;
```

Always pass values as **parameters** (`$1, $2` in pg), never string concatenation. Parameters are the injection defence *and* let PostgreSQL reuse plans.

4.7 Indexes: how they work and how to read a plan

A B-tree index keeps keys sorted so PostgreSQL can jump to a value in about log(n) steps and read ranges in order. Know these index types and tricks:

Index	Use it for

Index	Use it for
B-tree (default)	Equality, ranges, `ORDER BY`, `LIKE 'abc%'` (with the right collation)
Composite `(a, b, c)`	Queries filtering on `a`, or `a` and `b`, or all three (**leftmost prefix** rule: an index on `(a, b)` does not help a query filtering only on `b`)
Covering (`INCLUDE (col)`)	Answer a query from the index alone (index-only scan)
Partial (`WHERE deleted_at IS NULL`)	Small index over the rows you actually query; also enables "unique among active rows"
Expression (`lower(email)`)	Case-insensitive lookups that match the query expression exactly
GIN	`jsonb` containment, arrays, full-text (`tsvector`), `pg_trgm` fuzzy matching
BRIN	Huge append-only tables ordered by time; tiny, coarse

Column order in a composite index: **equality columns first, then the range or sort column**. For `WHERE user_id = $1 ORDER BY created_at DESC LIMIT 20`, the index `(user_id, created_at DESC)` returns the rows already ordered with no sort step.

Reading `EXPLAIN (ANALYZE, BUFFERS)`: look at the node types (`Seq Scan` reads the whole table, `Index Scan` and `Index Only Scan` use an index, `Bitmap Heap Scan` is in between), compare **estimated rows with actual rows** (a large mismatch means stale statistics: run `ANALYZE`), read `actual time` at the slowest node, and `Buffers: shared read` for disk reads. The planner will correctly ignore an index when the query touches a large fraction of the table or the table is tiny. Remember: PostgreSQL does **not** automatically index foreign key columns, so index them yourself or joins and cascading deletes crawl. Create indexes on live tables with `CREATE INDEX CONCURRENTLY` (it cannot run inside a transaction block).

4.8 Transactions, isolation levels, and concurrency bugs

The classic anomalies and which PostgreSQL isolation level stops them:

Anomaly	Meaning	READ COMMITTED (default)	REPEATABLE READ	SERIALIZABLE
Dirty read	Read another transaction's uncommitted data	Prevented	Prevented	Prevented
Non-repeatable read	Same row read twice gives different values	Possible	Prevented	Prevented

Anomaly	Meaning	READ COMMITTED (default)	REPEATABLE READ	SERIALIZABLE
Phantom read	Same query returns new rows second time	Possible	Prevented in PostgreSQL	Prevented
Lost update	Two read-modify-write cycles overwrite each other	Possible	Detected (one fails)	Detected (one fails)
Write skew	Two transactions each check a rule then write different rows, jointly breaking the rule (two doctors both go off call)	Possible	Possible	Prevented

Tools to avoid these bugs, from lightest to heaviest:

1. **Atomic statements.** `UPDATE accounts SET balance = balance - 100 WHERE id = 1 AND balance >= 100` does the check and write in one step; check the affected row count.
2. **Optimistic locking.** A `version` column (Part 3.8).
3. **Pessimistic row locks.** `SELECT ... FOR UPDATE` locks the rows you read until commit; `FOR UPDATE SKIP LOCKED` lets workers take different rows (the basis of Postgres-backed queues).
4. **SERIALIZABLE with retry.** The database detects dangerous patterns and aborts one transaction with SQLSTATE `40001`; **your code must retry**.
5. **Advisory locks.** `pg_advisory_xact_lock(key)` for locks not tied to a row (for example "only one cron instance runs").

Deadlocks happen when two transactions lock the same rows in opposite order (SQLSTATE `40P01`). Prevent them by always locking in a consistent order (sort ids), keep transactions short, and retry on the error.

```
import type { Pool, PoolClient } from 'pg';

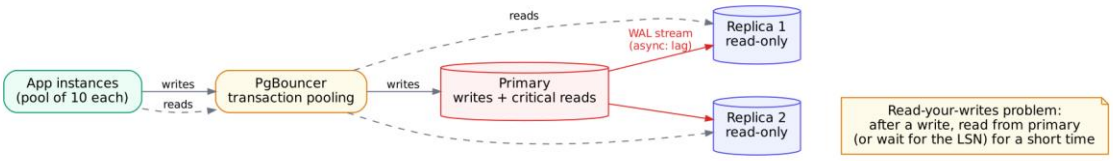
export async function withTx<T>(pool: Pool, fn: (c: PoolClient) => Promise<T>,
maxAttempts = 3): Promise<T> {
  for (let attempt = 1; ; attempt++) {
    const client = await pool.connect();
    try {
      await client.query('BEGIN ISOLATION LEVEL SERIALIZABLE');
      const result = await fn(client);
      await client.query('COMMIT');
      return result;
    } catch (e: any) {
      await client.query('ROLLBACK').catch(() => {});
      const retryable = e?.code === '40001' || e?.code === '40P01';   //
serialization failure, deadlock
      if (retryable && attempt < maxAttempts) continue;                // loop again
(finally still releases)
      throw e;
    } finally {
```

```
      client.release();                                                    // ALWAYS
release or the pool drains
    }
  }
}
```

Do **not** call slow external services (HTTP, email) inside a transaction: you hold locks and a pooled connection while waiting. Do the external call before or after, or use the outbox (Part 8).

4.9 Connection pooling, timeouts, and replicas

- **Pool arithmetic.** Total connections = instances x pool size. This must stay below the server's `max_connections` (default 100) with headroom for admins and migrations. More connections do not mean more throughput: a database with a handful of cores is saturated by a few dozen *active* queries; extra connections only add context switching. Start small (5 to 20 per instance) and measure.
- **`pg.Pool` options:** `max`, `idleTimeoutMillis`, `connectionTimeoutMillis` (fail fast instead of waiting forever for a free connection). A growing wait for pool checkout is the first sign of leaked clients or slow queries.
- **Server-side guards:** `statement_timeout` (kill runaway queries), `idle_in_transaction_session_timeout` (kill forgotten open transactions), `lock_timeout` (do not wait forever for a lock, vital during migrations).
- **PgBouncer** multiplexes many app connections onto few server connections. In *transaction pooling* mode (the common choice) session state does not persist between statements: avoid session-level `SET`, advisory locks held across transactions, `LISTEN/NOTIFY`, and be careful with prepared statements (check your driver and PgBouncer versions for support).
- **Read replicas** scale reads but replicate asynchronously, so a read right after a write may not see it (**replication lag**). Route reads that must be fresh (right after the user's own write) to the primary, or accept staleness for dashboards and feeds.



4.10 Schema design checklist

- **Keys.** `bigint generated always as identity` is compact and fast; UUID v7 is time-ordered (good index locality) and safe to expose; UUID v4 is random (index fragmentation, but unguessable). Never expose sequential ids if enumeration matters.
- **Types.** `timestamptz` for instants (never `timestamp without time zone`), `text` over `varchar(n)` unless a limit is a real rule, `numeric` or integer cents for money, `boolean` not 0/1, `jsonb` for genuinely schemaless extras (not as a way to avoid designing tables).
- **Constraints do the policing:** `NOT NULL` by default, `CHECK (price_cents >= 0)`, `UNIQUE (tenant_id, email)`, foreign keys with deliberate `ON DELETE` behaviour.
- **Many-to-many** needs a join table with a composite primary key `(user_id, role_id)`.
- **Audit columns:** `created_at`, `updated_at` (trigger or ORM), maybe `created_by`.

- **Soft delete** with a partial unique index: `CREATE UNIQUE INDEX ON users (email) WHERE deleted_at IS NULL`.
- **Denormalise only with evidence** (a measured slow read), and keep the copy consistent in the same transaction or via events.

4.11 Safe migrations (expand and contract)

Deploys overlap: for a while the old code and new code both run against the same database, so every migration must work for **both**. To rename `users.name` to `full_name`:

1. **Expand:** add `full_name` (nullable). Deploy code that writes both columns and reads `full_name` falling back to `name`.
2. **Backfill** in small batches (not one giant `UPDATE`, which locks and bloats).
3. **Contract:** after all code uses `full_name`, add `NOT NULL` and drop `name` in a later release.

Pitfalls: adding a column with a *constant* default is fast in modern PostgreSQL, but a volatile default or type change can rewrite the table; adding a foreign key or check constraint locks, so add it `NOT VALID` first then `VALIDATE CONSTRAINT` separately; set `lock_timeout` so a migration that cannot get its lock fails fast instead of queueing and blocking all traffic behind it. Tools: Prisma Migrate, drizzle-kit, `node-pg-migrate`, Atlas, Flyway. Keep migrations in version control, forward-only, and tested against a copy of production-sized data.

4.12 ORM craft: same query, three ways

```
// Raw SQL with pg: full control, you map rows
const { rows } = await pool.query(
  'SELECT id, title FROM notes WHERE user_id = $1 AND created_at < $2 ORDER BY
created_at DESC LIMIT $3',
  [userId, cursor, 20]);

// Prisma: generated, fully typed client
const notes = await prisma.note.findMany({
  where: { userId, createdAt: { lt: cursor } },
  orderBy: { createdAt: 'desc' }, take: 20, select: { id: true, title: true },
});

// Drizzle: SQL-shaped TypeScript
const notes2 = await db.select({ id: n.id, title: n.title }).from(n)
  .where(and(eq(n.userId, userId), lt(n.createdAt, cursor)))
  .orderBy(desc(n.createdAt)).limit(20);
```

Habits that keep ORMs healthy: **select only needed columns**; use `include`/joins or a batch `IN` query rather than a query per row; turn on query logging in development and count queries per request; use interactive transactions for multi-step writes; drop to raw SQL (`$queryRaw`, `sql` template) for reports, bulk inserts, window functions, and anything the ORM expresses awkwardly. Do not let ORM models leak into your API responses (map to DTOs), or a new column silently becomes public.

4.13 MongoDB: modelling and pitfalls

The main design question is **embed or reference**. Embed data that is read together and bounded in size (an order's line items). Reference data that is shared or grows without bound (a user's activity feed): documents have a 16 MB limit, and an ever-growing embedded array is a classic failure. Index the fields

you query and sort on (compound index order follows the same equality-then-range rule), use the **aggregation pipeline** (`$match`, `$group`, `$lookup`, `$project`) for reporting, and remember multi-document transactions need a replica set and cost performance, so model to keep each business operation inside one document where you can. In Mongoose: use `.lean()` for read-only queries (plain objects, much faster), define indexes in schemas but build them deliberately in production, and validate input with your edge validator, not only the Mongoose schema.

4.14 Redis in depth

Need	Structure and commands
Cache an object	`SET user:42 <json> EX 300`
Counter / rate limit	`INCR rate:ip:1.2.3.4` then `EXPIRE` on first hit (do both atomically with a Lua script or `MULTI`)
Distributed lock (simple)	`SET lock:job NX PX 30000 <token>`; release only if the token matches (Lua compare-and-delete)
Leaderboard	Sorted set: `ZADD`, `ZREVRANGE 0 9 WITHSCORES`
Unique visitors	Set (`SADD`), or HyperLogLog (`PFADD`) for approximate counts in tiny memory
Object with fields	Hash: `HSET`, `HGETALL`
Delayed jobs	Sorted set scored by run time
Event log with consumer groups	Stream: `XADD`, `XREADGROUP`, `XACK`

Operational rules: namespace keys (`app:entity:id`); give almost every key a TTL (add random jitter to avoid synchronized expiry); never run `KEYS *` in production (it blocks the single-threaded server), use `SCAN`; avoid huge keys and large values; choose an eviction policy on purpose (`allkeys-lru` for a pure cache, `noeviction` when Redis holds data you cannot lose, such as BullMQ queues); understand persistence (RDB snapshots, AOF log) and that replication and Sentinel/Cluster provide availability and sharding, not backup. Commands run one at a time, so single commands are atomic; multi-step logic needs `MULTI/EXEC` or a Lua script.

```
import Redis from 'ioredis';
const redis = new Redis(config.REDIS_URL!);

// atomic fixed-window rate limit: INCR and set expiry on the first hit, in one round
trip
const script = `local n = redis.call('INCR', KEYS[1])
if n == 1 then redis.call('EXPIRE', KEYS[1], ARGV[1]) end
return n`;
export async function hit(key: string, windowSec: number) {
  return (await redis.eval(script, 1, key, windowSec)) as number;
}
```

4.15 More build cards

Card 4.E: window-function report

- **Aim:** use SQL, not application loops, for analytics.
- **Input:** `orders(user_id, total_cents, created_at)` with 100k seeded rows.
- **Output:** an endpoint returning each user's latest order and running total in one query (`row_number()` and `sum() OVER`).
- **Check:** compare the response time and code size with fetching all orders and reducing in JavaScript.

Card 4.F: reproduce and fix a lost update

- **Aim:** see concurrency bugs happen.
- **Input:** `POST /counter/increment` implemented as `SELECT` then `UPDATE n = <js value + 1>`.
- **Output:** 100 parallel calls leave the counter well below 100; then fix it three ways (atomic `UPDATE n = n + 1`, `FOR UPDATE`, version check with retry) and reach exactly 100 each time.
- **Check:** a script firing 100 concurrent requests, run against each version.

Card 4.G: write skew with two psql sessions

- **Aim:** understand why SERIALIZABLE exists.
- **Input:** table `on_call(doctor, is_on_call)` with two doctors on call and a rule "at least one must stay on call".
- **Output:** under REPEATABLE READ both sessions can set themselves off call and break the rule; under SERIALIZABLE one fails with `40001` and your `withTx` retry resolves it.
- **Check:** run the two sessions step by step in two terminals.

Card 4.H: zero-downtime rename

- **Aim:** practice expand and contract.
- **Input:** rename a column while `autocannon` hammers the read endpoint.
- **Output:** three deploys (expand, switch, contract) with zero failed requests during each.
- **Check:** error count from autocannon is 0 throughout.

Card 4.I: read a plan

- **Aim:** make `EXPLAIN` familiar.
- **Input:** the query `WHERE user_id = $1 ORDER BY created_at DESC LIMIT 20` on 1 million rows, tested with no index, `(created_at)`, `(user_id)`, and `(user_id, created_at DESC)`.
- **Output:** a table of plan node, execution time, and rows examined for each index choice.
- **Check:** explain in two sentences why the last index wins.

4.16 Answer key for the Part 4 self-checks

- **What does a transaction guarantee that two queries do not?** Atomicity (all or nothing), plus isolation from concurrent transactions; two separate queries can fail halfway or see inconsistent data.
- **Why can an index slow writes?** Every insert, update of an indexed column, and delete must also update each index (and may split pages), and indexes consume memory and disk.
- **When MongoDB over PostgreSQL, honestly?** When data is naturally document-shaped and read as a whole, schemas vary widely, and you do not need rich cross-document joins or

transactions; for most business apps PostgreSQL (with `jsonb` for the flexible parts) is the safer default.

4.6 Deep dive: SQL you will use every week

ORMs hide SQL until the day a query is slow. Learn these patterns so you can read what your ORM generates and write the hard queries yourself.

```
-- Upsert: insert or update atomically (no race between SELECT and INSERT)
INSERT INTO page_views (page_id, day, views) VALUES ($1, current_date, 1)
ON CONFLICT (page_id, day) DO UPDATE SET views = page_views.views + 1
RETURNING views;

-- Keyset pagination on a composite sort key (stable, fast)
SELECT id, title, created_at FROM notes
WHERE (created_at, id) < ($1, $2)
ORDER BY created_at DESC, id DESC
LIMIT 20;

-- Window function: top 3 orders per customer without a self join
SELECT * FROM (
  SELECT o.*, row_number() OVER (PARTITION BY customer_id ORDER BY total DESC) AS rn
  FROM orders o
) t WHERE rn <= 3;

-- CTE for readability, aggregate for reports
WITH paid AS (SELECT * FROM orders WHERE status = 'paid')
SELECT date_trunc('day', created_at) AS day, count(*), sum(total) FROM paid GROUP BY
1 ORDER BY 1;
```

Join types: `INNER` (rows that match on both sides), `LEFT` (all left rows, nulls when no match), plus `EXISTS` subqueries for "has at least one" checks (often faster than joining and de-duplicating).

Reading `EXPLAIN (ANALYZE, BUFFERS)`:

You see	Meaning	What to do
`Seq Scan` on a big table with a filter	Whole table read	Add an index matching the filter, or accept it if most rows match anyway
`Index Scan` / `Index Only Scan`	Index used (index-only needs no table visit)	Good
`Bitmap Heap Scan`	Index finds many rows, then fetches pages in order	Normal for medium selectivity
`rows=10 ... actual rows=100000`	Planner estimate is far off	Run `ANALYZE`, check statistics, rewrite the query
`Sort` spilling to disk (`Sort Method: external merge`)	Not enough `work_mem` or missing index for `ORDER BY`	Add an index that supplies the order, or raise `work_mem` carefully

You see	Meaning	What to do
`Nested Loop` with a huge outer side	Join order problem or missing index on the join column	Index the foreign key

Index types: B-tree (default: equality, range, sorting); **GIN** (`jsonb`, arrays, full-text `tsvector`); **GiST** (geometry, ranges, fuzzy search); **BRIN** (huge append-only tables ordered by time, tiny index); **hash** (rare). Composite index order matters: `(a, b)` serves filters on `a` or on `a` and `b`, not on `b` alone. **Partial indexes** (`WHERE deleted_at IS NULL`, `WHERE status = 'pending'`) keep indexes small. `LIKE 'abc%'` can use a B-tree (with the right operator class); `LIKE '%abc%'` needs a trigram GIN index (`pg_trgm`). Foreign keys are **not** indexed automatically in PostgreSQL; index the referencing column.

4.7 Deep dive: transactions and concurrency

What can go wrong without enough isolation:

Anomaly	Example	Prevented by
Dirty read	Read another transaction's uncommitted change	PostgreSQL never allows it
Non-repeatable read	Same row read twice, value changed in between	Repeatable read or higher
Phantom read	Same query returns new rows the second time	Repeatable read (in PostgreSQL) or higher
Lost update	Two transactions read a balance, both write `balance + 10`	Row lock, atomic `UPDATE ... SET x = x + 10`, or optimistic version check
Write skew	Two doctors each go off call because the other is still on	Serializable isolation, or lock the shared rows

Tools:

- **Atomic updates:** `UPDATE accounts SET balance = balance - 100 WHERE id = 1 AND balance >= 100` and check the affected row count. Simplest and fastest.
- **Pessimistic locking:** `SELECT ... FOR UPDATE` locks the rows until commit. Keep transactions short and lock rows in a consistent order (for example by ascending id) to avoid **deadlocks**.
- **`FOR UPDATE SKIP LOCKED`:** lets many workers claim different rows without waiting; the basis of a Postgres job queue and the outbox relay.
- **Optimistic locking:** a `version` column checked in the `WHERE` clause (Card 3.H).
- **Advisory locks:** `pg_advisory_xact_lock(key)` serialises work on an application-defined key (one cron instance, one import per tenant).
- **Serializable isolation** gives the strongest guarantee, but you must **retry** transactions that fail with `40001` (serialization failure) or `40P01` (deadlock).

```
import { Pool, PoolClient } from 'pg';
```

```
export async function withTx<T>(pool: Pool, fn: (c: PoolClient) => Promise<T>):
Promise<T> {
  const c = await pool.connect();
  try {
    await c.query('BEGIN');
    const result = await fn(c);
    await c.query('COMMIT');
    return result;
  } catch (e) {
    await c.query('ROLLBACK');
    throw e;
  } finally {
    c.release();                        // always return the client to the pool
  }
}

export async function withRetry<T>(fn: () => Promise<T>, tries = 3): Promise<T> {
  for (let i = 1; ; i++) {
    try { return await fn(); }
    catch (e) {
      const code = (e as { code?: string }).code;
      if (i >= tries || (code !== '40001' && code !== '40P01')) throw e;
    }
  }
}
```

Rules: never `await` slow external calls (HTTP, email) while holding a transaction open; use the **same client** for every statement in the transaction (with `pg` a `pool.query` inside a transaction is a different connection and is **not** part of it); keep transactions as short as possible.

4.8 Deep dive: connection pooling and capacity

Each PostgreSQL connection is a separate server process with real memory cost, and the server has a `max_connections` limit (often 100 by default). Total connections = app instances times pool size, plus workers, migrations, and admin tools. The arithmetic must fit.

- A common rule of thumb: a pool of roughly `cores * 2` to a few dozen connections per database is the useful range; **more connections do not mean more throughput**, they mean more contention. Size by measurement.
- `pg` Pool options to set: `max`, `idleTimeoutMillis`, `connectionTimeoutMillis` (fail fast instead of hanging when the pool is exhausted), and server-side `statement_timeout` and `idle_in_transaction_session_timeout` so a stuck query or forgotten transaction cannot hold resources forever.
- **PgBouncer** (or a managed pooler such as RDS Proxy, Supabase pooler, Neon pooler) multiplexes many app connections onto few database connections. In *transaction pooling* mode session features (session-level prepared statements, `SET`, advisory session locks, `LISTEN`) do not work as expected; check your driver and ORM docs.
- **Serverless** (Lambda, edge) creates many short-lived instances: use a pooler or an HTTP/data-API driver, and keep one pool per runtime instance outside the handler.
- Monitor: pool size in use versus waiting, `pg_stat_activity`, slow query log (`log_min_duration_statement`), and `pg_stat_statements` for the most expensive queries.

4.9 Deep dive: schema design and safe migrations

Topic	Recommendation
Time	`timestamptz` (UTC) for every timestamp; add `created_at` and `updated_at`
Ids	`bigint generated always as identity` for internal tables, or UUID v7 for ids exposed to clients; avoid random UUID v4 as a clustered-like hot index on huge tables
Money	Integer minor units (`amount_minor bigint`) plus `currency char(3)`; or `numeric(12,2)`; never `float`
Text	`text` with a `CHECK` on length when needed; PostgreSQL treats `text` and `varchar(n)` alike for speed
Enumerations	A lookup table or a `CHECK (status IN (...))`; native enums are harder to change
Flexible data	`jsonb` for genuinely variable attributes, with a GIN index if you query inside it; keep core fields as columns
Constraints	`NOT NULL`, `UNIQUE`, `CHECK`, foreign keys with deliberate `ON DELETE` behaviour
Soft delete	`deleted_at timestamptz`; add partial unique indexes (`WHERE deleted_at IS NULL`)

Zero-downtime migration pattern (expand and contract):

1. **Expand:** add the new nullable column or table (cheap, no rewrite). Deploy code that writes to both old and new.
2. **Backfill** in small batches (1,000 to 10,000 rows per transaction) with pauses, so you do not hold long locks or flood replication.
3. **Switch reads** to the new column. Verify.
4. **Contract:** add the `NOT NULL` or constraint (using `NOT VALID` then `VALIDATE CONSTRAINT` to avoid long locks), then drop the old column in a **later** release.

Other rules: `CREATE INDEX CONCURRENTLY` on busy tables (it cannot run inside a transaction block); set `lock_timeout` for migrations so they fail fast instead of queueing behind a long query and blocking everyone; never rename or drop a column that the running version still reads; treat migration files as immutable once applied.

4.10 Deep dive: ORMs and query builders in practice

The same "latest 20 notes with author name" in three styles:

```
// Prisma
const rows = await prisma.note.findMany({
  take: 20, orderBy: { createdAt: 'desc' },
  select: { id: true, title: true, author: { select: { name: true } } },
});
```

```
// Drizzle (SQL-like, fully typed)
const rows = await db.select({ id: notes.id, title: notes.title, author: users.name
})
  .from(notes).innerJoin(users, eq(notes.authorId, users.id))
  .orderBy(desc(notes.createdAt)).limit(20);

// Raw SQL with parameters (safe: values are never concatenated)
const { rows } = await pool.query(
  'SELECT n.id, n.title, u.name AS author FROM notes n JOIN users u ON u.id =
n.author_id ORDER BY n.created_at DESC LIMIT $1',
  [20],
);
```

Practical rules:

- **Select only what you need** (`select`), especially when rows have large text or `jsonb` columns.
- **Watch generated SQL.** Turn on query logging in development (Prisma `log: ['query']`, Drizzle `logger: true`) and count queries per request.
- **Raw SQL is allowed.** Use the ORM's safe tagged template (`prisma.$queryRaw` with a template literal) for reports and complex queries; never build SQL strings with `+` or template interpolation outside these safe helpers.
- **Hide the ORM behind a repository** so services and tests do not depend on ORM types (Part 10), and so a switch is a local change.
- **Transactions:** pass the transaction client down explicitly (`tx`), and never mix the global client with `tx` inside the same unit of work.
- **Migrations:** generate them from the schema, then **read and edit** them before applying; generated SQL can rewrite a table or drop data.
- **Money, time, and IDs** follow the schema rules above, whatever the ORM defaults say.

4.11 Deep dive: MongoDB and Redis patterns

MongoDB design rules

- **Embed** data that is read together and changes together (order lines inside an order). **Reference** data that is shared, large, or changes independently (a product referenced by many orders).
- Documents have a 16 MB limit; unbounded arrays (every comment on a post inside the post) are the classic mistake. Use a separate collection.
- Create indexes for every query shape (compound indexes follow the same left-prefix rule as SQL); check with `explain('executionStats')`.
- The **aggregation pipeline** (`$match`, `$group`, `$lookup`, `$project`) does server-side analytics; `$lookup` is a join and is not free.
- Multi-document transactions exist on replica sets, but if you need them everywhere, PostgreSQL is probably the better fit.
- Mongoose tips: use `.lean()` for read-only queries (plain objects, far less overhead); define indexes in the schema but build them deliberately in production; set `maxTimeMS` on queries.

Redis recipes (shown with `ioredis`)

```
import Redis from 'ioredis';
const redis = new Redis(process.env.REDIS_URL!);
```

```
// Cache with expiry
await redis.set('app:product:42', JSON.stringify(product), 'EX', 60);

// Distributed lock: acquire (NX = only if absent, PX = ms expiry) and release only
if you still own it
const token = crypto.randomUUID();
const ok = await redis.set('lock:report', token, 'PX', 30_000, 'NX');
const release = `if redis.call('get', KEYS[1]) == ARGV[1] then return
redis.call('del', KEYS[1]) else return 0 end`;
if (ok) { try { /* work */ } finally { await redis.eval(release, 1, 'lock:report',
token); } }

// Leaderboard with a sorted set
await redis.zincrby('app:scores', 10, 'user:7');
const top = await redis.zrevrange('app:scores', 0, 9, 'WITHSCORES');

// Pipeline: many commands, one network round trip
await redis.pipeline().incr('hits').expire('hits', 60).exec();
```

- **Key names:** `app:entity:id` with a clear prefix; always set a TTL on cache keys.
- **Never run `KEYS *` in production** (it blocks the server); use `SCAN` with a cursor.
- **Avoid big keys** (a 50 MB value or a list with millions of items) because Redis processes commands on one thread; split them.
- **Eviction:** when memory is full, `maxmemory-policy allkeys-lru` suits a pure cache; a store that also holds queues or sessions needs `noeviction` or separate instances so a cache fill never evicts jobs.
- **Persistence:** RDB snapshots and AOF logging reduce data loss but do not make Redis a system of record by themselves.
- **High availability:** Redis Sentinel or a managed service (ElastiCache, Upstash, Redis Cloud, Memorystore); clustering shards keys across nodes, and multi-key operations then need keys in the same slot (hash tags).
- A simple counter `INCR` then `EXPIRE` has a small gap between the two commands; make it atomic with a Lua script or `SET key 0 EX 60 NX` first, then `INCR`.

4.12 Deep dive: replicas, backups, and scaling the data layer

- **Backups are only real if you have restored them.** Use automated base backups plus WAL archiving for point-in-time recovery, store them in another region or account, and rehearse a restore on a schedule. Managed databases do this for you; still test the restore and know your recovery time and recovery point.
- **Read replicas** copy data asynchronously, so a read right after a write may miss it (**replication lag**). Send reads that must see the user's own write to the primary ("read your writes"), and everything tolerant of slight staleness to replicas.
- **Order of scaling:** fix queries and indexes, add caching, add a pooler, add read replicas, partition big tables (declarative partitioning by time), and only then shard across databases. Each step costs complexity.
- **Archiving and retention:** decide how long data lives; move old rows to cheap storage; partitions make dropping old data instant.

- **CAP and PACELC in one paragraph:** during a network partition a distributed store must choose consistency or availability; even without partitions it trades latency against consistency. A single PostgreSQL primary is consistent but has one write point; many NoSQL stores relax consistency for availability and scale. State which your feature needs.

4.13 More build cards

Card 4.E: read a query plan

- **Aim:** use `EXPLAIN ANALYZE` fluently.
- **Input:** 1 million `orders` rows; query `WHERE customer_id = $1 ORDER BY created_at DESC LIMIT 20`.
- **Output:** plans for (a) no index, (b) index on `(customer_id)`, (c) index on `(customer_id, created_at DESC)`; a note on which step disappears in (c).
- **Check:** the sort node vanishes in (c) and execution time drops by an order of magnitude.

Card 4.F: survive a deadlock

- **Aim:** reproduce and fix one.
- **Input:** a transfer endpoint that locks `from` then `to`; run `A to B` and `B to A` in parallel 100 times.
- **Output:** errors with code `40P01`; after you lock rows in ascending id order (and add `withRetry`) there are none.
- **Check:** run the same stress script before and after; the error count goes from non-zero to zero.

Card 4.G: Postgres job claim with `SKIP LOCKED`

- **Aim:** a queue without extra infrastructure.
- **Input:** a `jobs` table and 4 worker processes running `SELECT ... FOR UPDATE SKIP LOCKED LIMIT 10`.
- **Output:** 10,000 jobs processed exactly once in total, with no worker waiting on another.
- **Check:** sum of processed counts equals 10,000; no duplicate job ids in the results log.

Card 4.H: expand and contract migration

- **Aim:** rename a column without downtime.
- **Input:** rename `users.name` to `users.full_name` while a load test runs.
- **Output:** four migration and release steps, with zero failed requests during the test.
- **Check:** the k6 or autocannon error count stays at 0 through every step.

Card 4.I: pool exhaustion drill

- **Aim:** see what happens when the pool runs dry.
- **Input:** pool `max` of 5, 50 parallel requests each holding a connection for 500 ms, `connectionTimeoutMillis` of 2000.
- **Output:** requests queue, some time out with a clear error mapped to `503`, and a metric reports waiting clients.
- **Check:** raising the pool to 20 changes the latency curve; explain the trade-off against database capacity.

Card 4.J: Redis lock and counter correctness

- **Aim:** build the two most-used Redis helpers.
- **Input:** a `withLock(key, ttl, fn)` using `SET NX PX` and token-checked release; an atomic counter with TTL.
- **Output:** only one of 10 parallel callers runs the critical section; the counter never exceeds its limit under 1,000 parallel increments.
- **Check:** a stress script asserts both results; kill the lock holder and confirm the lock expires.

Part 5: Authentication, authorization, and security

Authentication answers "who are you?". Authorization answers "what may you do?". Confusing the two is the root of many breaches.

5.1 Theory: passwords

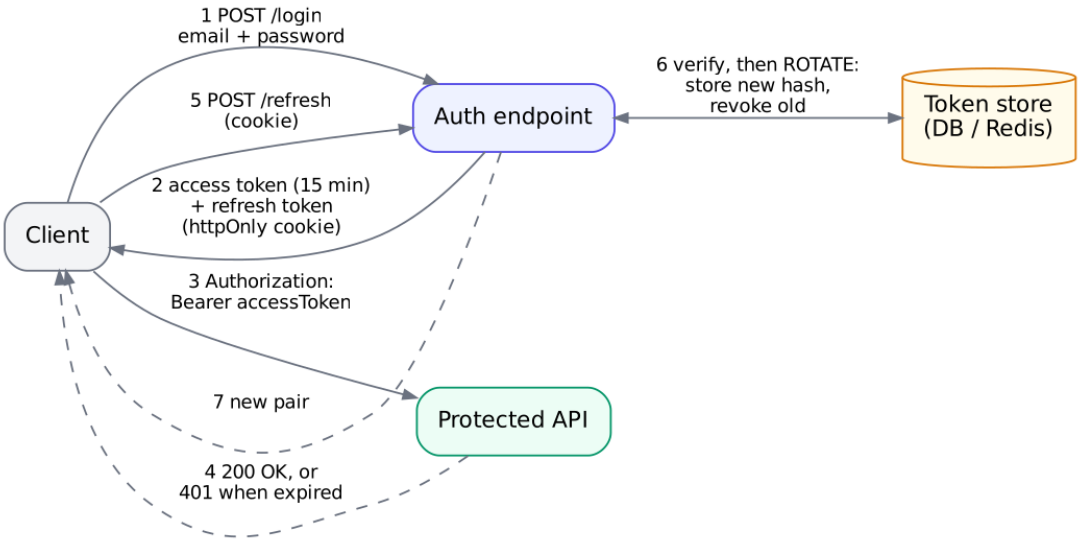
Store only a **slow, salted hash**, never the password or a fast hash such as SHA-256. Use **argon2id** (preferred) or bcrypt. Enforce a minimum length, check against breached-password lists if you can, and rate-limit login attempts. Return the same error for "unknown email" and "wrong password" so attackers cannot enumerate accounts.

5.2 Theory: sessions versus tokens

	Server session	JWT (stateless token)
What the client holds	Random session id in a cookie	Signed token containing claims (user id, role, expiry)
Where state lives	Server store (Redis or DB)	In the token itself
Revoke instantly	Yes, delete the session	Hard; need a deny list or short expiry
Scales across servers	Needs shared store	Any server can verify with the key
Best for	Browser apps with your own backend	Mobile apps, service-to-service, third-party APIs

Honest advice: for a normal web app with a single backend, server sessions in an `httpOnly`, `Secure`, `SameSite` cookie are simpler and safer than JWT. Use JWT when you really need stateless verification across services.

The common JWT design is a **short-lived access token** (about 15 minutes) plus a **long-lived refresh token** that is stored hashed in the database, sent in an `httpOnly` cookie, and **rotated** on each use (the old one is revoked; reuse of a revoked token means theft and kills the whole family).



JWT rules: verify the signature **and** `exp`, `iss`, `aud`; pin the algorithm (never accept `none`); keep secrets or private keys out of code; do not put sensitive data in the payload (it is only encoded, not encrypted).

5.3 Theory: OAuth 2.0 and OpenID Connect

Use OAuth 2.0 / OIDC for "Sign in with Google/GitHub" and for protecting APIs with an identity provider. Today's browser and mobile flow is the **Authorization Code flow with PKCE**. OAuth gives *access* to resources; **OIDC** adds an *identity* layer (the ID token). Do not invent your own protocol; use a vetted library or provider.

5.4 Theory: authorization

- **RBAC** (role-based): `admin`, `editor`, `viewer`. Simple, covers most apps.
- **ABAC / policy-based**: rules on attributes ("editors may edit documents in their own team").
- **Ownership checks**: always check that the resource belongs to the caller (`WHERE id = :id AND owner_id = :userId`). Missing this is the "broken access control" bug, number one on the OWASP Top 10.
- Enforce on the **server**. Hiding a button in the UI is not security.

5.5 Theory: the attacks you must know

Attack	Defence
SQL / NoSQL injection	Parameterised queries or an ORM; never concatenate input into queries; validate types
XSS	Escape output, set a Content-Security-Policy (helmet), `httpOnly` cookies
CSRF	`SameSite` cookies, CSRF tokens for cookie-authenticated forms

Attack	Defence
Brute force / credential stuffing	Rate limit by IP and account, lockout or backoff, MFA
SSRF (server fetches attacker URL)	Allow-list hosts, block private IP ranges, no redirects
Mass assignment	Whitelist fields in your DTO/schema, never spread `req.body` into a DB update
Insecure dependencies	`npm audit`, Dependabot or Renovate, lock files, minimal dependencies
Secrets leaks	Environment variables or a secrets manager, never commit them, rotate on leak
Missing TLS	HTTPS everywhere, HSTS header
Path traversal / unsafe upload	Never use user filenames as paths, check MIME and size, store outside the web root

5.6 Libraries and alternatives

Need	Pick	Alternatives
Password hashing	`argon2`	`bcrypt`, `bcryptjs` (pure JS, slower), Node `crypto.scrypt`
JWT	`jose`	`jsonwebtoken`, `@fastify/jwt`, `@nestjs/jwt`
Sessions	`express-session` + `connect-redis`	`@fastify/session`, `iron-session`, `cookie-session`
Strategy framework	Passport.js (`passport-local`, `passport-jwt`, `passport-google-oauth20`)	`@nestjs/passport` wraps it; Arctic + Oslo for OAuth helpers
Auth library (full)	Better Auth	Auth.js (NextAuth), Lucia-style DIY with sessions
Hosted identity	Auth0, Clerk, Supabase Auth, Firebase Auth, AWS Cognito	Self-host Keycloak or Ory
Authorization	CASL, Casbin	Hand-rolled guards
Security headers	`helmet` / `@fastify/helmet`	Set manually
CORS	`cors` / `@fastify/cors`	Nest `enableCors`
Rate limiting	`express-rate-limit`, `@fastify/rate-limit`, `rate-`	Nginx, API gateway, Cloudflare

Need	Pick	Alternatives
	`limiter-flexible`	
MFA	`otplib` (TOTP), WebAuthn via `@simplewebauthn/server`	Provider MFA

Build or buy? Authentication is hard to get fully right. For a product, strongly consider a hosted provider or a maintained library. Build it yourself once for learning, then choose deliberately.

5.7 Build cards

Card 5.A: register and login with argon2

- **Aim:** safe password storage.
- **Input:** `POST /auth/register {email, password}`, `POST /auth/login {email, password}`.
- **Output:** register returns `201` without the hash; login returns tokens or sets a cookie; wrong credentials return an identical `401`.
- **Check:** look in the database: the `password_hash` column starts with `$argon2id$`; two users with the same password have different hashes.

Card 5.B: access token plus rotating refresh token

- **Aim:** implement the diagram above.
- **Input:** login result, `POST /auth/refresh`, `POST /auth/logout`.
- **Output:** access token expires in 15 minutes; refresh stored as a hash with `user_id`, `family_id`, `expires_at`, `revoked_at`; reusing an old refresh token revokes the family.
- **Check:** call refresh twice with the same token: the second call fails and the newest token also stops working.

Card 5.C: auth guard and role check

- **Aim:** protect routes in one place.
- **Input:** middleware `requireAuth` (verifies token, sets `req.user`) and `requireRole('admin')`.
- **Output:** `401` without token, `403` with the wrong role, `200` otherwise.
- **Check:** a table of 3 users by 3 routes with the expected status, run as a script.

Card 5.D: hardening pass

- **Aim:** apply the attack table.
- **Input:** add helmet, CORS with an explicit origin list, rate limit of 5 login attempts per minute per IP, body size limit.
- **Output:** the 6th login attempt returns `429`; response headers include `Strict-Transport-Security` and `X-Content-Type-Options`.
- **Check:** `curl -i` and a quick loop; then run `npm audit`.

Self-check questions: Why is SHA-256 wrong for passwords? Why rotate refresh tokens? Write the SQL that prevents user A from reading user B's order.

5.8 Deep dive: passwords and account lifecycle

Hashing parameters. argon2id has three knobs: memory cost, time cost (iterations), and parallelism. Tune them so one hash takes roughly 50 to 250 ms on your production hardware, and re-tune every year or two. The OWASP password storage cheat sheet lists current minimum settings; check it instead of copying numbers from old tutorials. Store the full encoded string (it contains algorithm, parameters, and salt), and on each successful login re-hash if the stored parameters are weaker than today's (**rehash on login**). A server-side secret **pepper** (kept in a secret manager, not the database) adds protection if only the database leaks, but complicates rotation; it is optional.

The hashing cost is also an attack surface. Each login burns CPU on the libuv thread pool (Part 1). Rate limit login and registration per IP *and* per account, and cap password length (for example 128 characters) so a 10 MB "password" cannot waste CPU.

Password policy that works. Minimum length of 8 to 12 or more, no composition rules (they push people to `Password1!`), allow long passphrases and password managers (do not block paste), check against a breached-password list (the Have I Been Pwned range API uses k-anonymity, so you send only a 5-character hash prefix), and never force periodic changes without evidence of compromise.

Account flows that are easy to get wrong

Flow	Safe design
Email verification	Random 32-byte token, store only its hash, expiry 24 hours, single use; do not grant sensitive access until verified
Password reset	Same token rules, expiry 15 to 60 minutes; the response is identical whether or not the email exists; after reset revoke all sessions and refresh tokens
Change email or password	Require the current password (re-authentication); notify the old address
Login errors	Same message and similar timing for unknown user and wrong password (hash a dummy value when the user is missing to even out timing)
Lockout	Prefer throttling with backoff and CAPTCHA over hard lockout (hard lockout lets attackers lock out real users)
Account deletion	Revoke sessions, delete or anonymise personal data, keep audit rows without personal data

Multi-factor authentication. TOTP (RFC 6238, `otplib`) is the baseline: store the secret encrypted, accept a one-step clock window, prevent code reuse, and issue one-time recovery codes (hashed). **Passkeys / WebAuthn** (`@simplewebauthn/server`) are phishing-resistant and are the best modern choice; offer them as the primary factor or second factor. Avoid SMS codes for high-value accounts (SIM-swap attacks), though they are better than nothing.

Random values. Use `crypto.randomBytes` / `crypto.randomUUID`, never `Math.random`, for tokens, ids that must be unguessable, and secrets. Compare secrets with `crypto.timingSafeEqual`.

5.9 Deep dive: sessions, cookies, and JWT in detail

Cookie flags (set all of them deliberately):

Flag	Effect
`HttpOnly`	JavaScript cannot read the cookie, so an XSS bug cannot steal it
`Secure`	Sent only over HTTPS
`SameSite=Lax` (or `Strict`)	Not sent on most cross-site requests, which blocks most CSRF; use `None; Secure` only when a cross-site embed truly needs it
`Path`, `Domain`	Narrow scope; omit `Domain` so it is host-only
`Max-Age`/`Expires`	Session lifetime; use both an idle timeout and an absolute timeout
`__Host-` name prefix	Forces `Secure`, `Path=/`, no `Domain`: the strictest form

Server sessions done right. Generate a 128-bit or longer random id, store session data in Redis (or the database) keyed by a **hash** of the id, regenerate the id on login (prevents session fixation), set idle and absolute expiry, and delete the session on logout. A "log out everywhere" feature is just deleting all of a user's session keys (keep a set of session ids per user).

Where to keep tokens in a browser app. The safest common design is cookies with `HttpOnly` + `Secure` + `SameSite`; keeping tokens in `localStorage` exposes them to any XSS bug. If a single-page app calls your API from another site, use the **backend for frontend (BFF)** pattern: a small server owns the tokens and the browser only holds a session cookie.

CSRF. If you authenticate with cookies, a malicious site can make the browser send them. Defences: `SameSite` cookies, checking the `Origin` header on state-changing requests, and CSRF tokens (double-submit or synchroniser pattern: `csrf-csrf` or the framework plugin) for form-based apps. APIs that use a `Bearer` header only are not subject to CSRF but are subject to token theft.

JWT details that matter

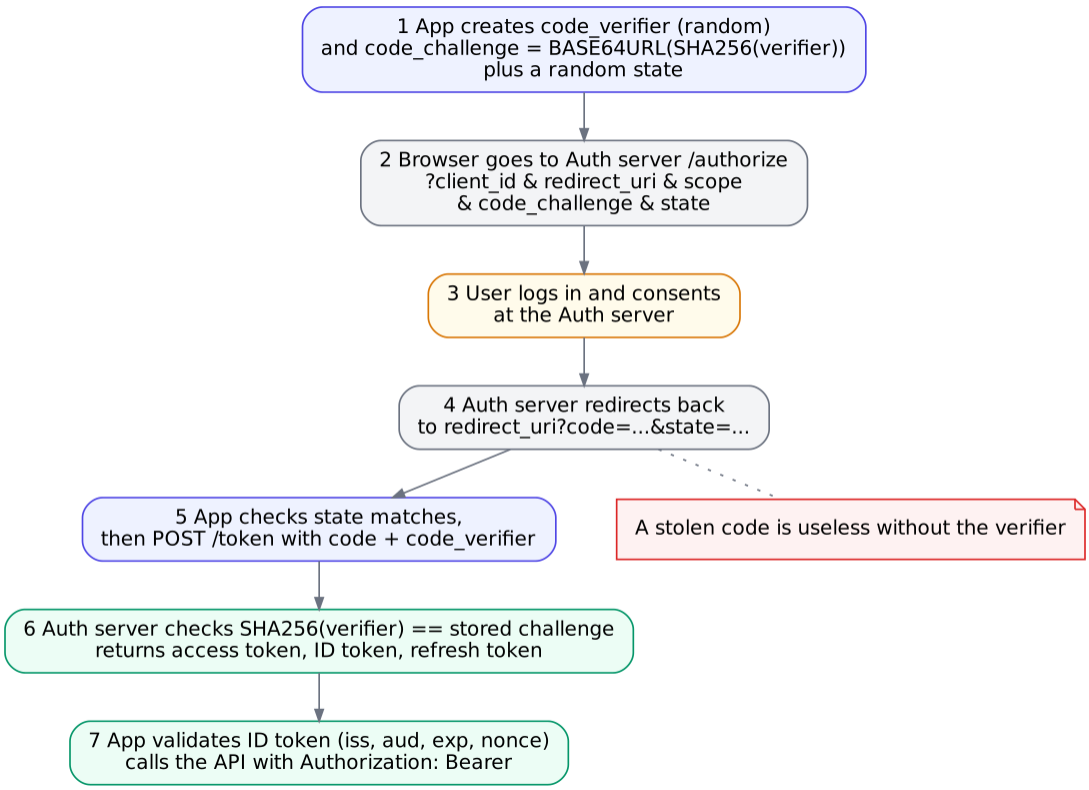
- **Algorithms.** Prefer asymmetric signing (EdDSA or `RS256`/`ES256`) when several services verify tokens: only the auth service holds the private key, others use the public key (published at a `/.well-known/jwks.json` endpoint with a `kid` header so you can rotate keys). `HS256` with a shared secret is fine inside one service.
- **Verification checklist:** signature, algorithm allow-list, `exp`, `nbf`, `iss`, `aud`. With `jose`: `jwtVerify(token, key, { issuer, audience, algorithms: ['ES256'] })`.
- **Claims:** keep them small (`sub`, `role`, `exp`, `iat`, `jti`). A role in the token is stale until expiry; for instant effect check a database or cache for sensitive operations.

- **Revocation.** Short expiry is the main tool. For immediate logout keep a deny list of `jti` values in Redis until their expiry, or give each user a `tokenVersion` that is checked and bumped to invalidate all tokens.
- **Refresh tokens** are opaque random strings, stored hashed, bound to a device/session id, rotated on use, and revoked as a family on reuse (diagram in 5.2).
- **JWE (encrypted JWT)** exists if the payload itself must be secret; usually it is better not to put secrets in tokens at all.
- **Do not** use JWTs as a drop-in replacement for sessions without a revocation plan; many security incidents come from long-lived, unrevocable tokens.

API keys and machine-to-machine access. Generate a random key with a recognisable prefix (`sk_live_...`), show it once, store only its hash plus a short visible identifier, support multiple keys per account with scopes and expiry, and let users rotate and revoke. For service-to-service calls use short-lived tokens (OAuth client credentials grant) or mutual TLS inside a private network.

5.10 Deep dive: OAuth 2.0 and OpenID Connect step by step

Roles: the **resource owner** (the user), the **client** (your app), the **authorization server** (Google, Auth0, Keycloak), and the **resource server** (the API). Grants you will meet: **authorization code + PKCE** (users, browser and mobile apps), **client credentials** (machine to machine), **device code** (TVs and CLIs). Avoid the old implicit and password grants.



Why `state` and PKCE: `state` ties the response to the request and blocks login CSRF; PKCE binds the code to the client that started the flow so an intercepted code is useless. For OIDC also send and check a `nonce` inside the ID token. **Scopes** list what access is requested (`openid email profile`); the API checks scopes on every call. **Access tokens** are for the API and should be treated as opaque by the client; the **ID token** is for your app and proves who the user is. Validate the ID token (signature via the provider's JWKS, `iss`, `aud`, `exp`, `nonce`) before trusting it, and map the provider's stable `sub` claim to your user row (do not use the email as the identity key: emails change and can be reused).

Account linking. If a user signs in with Google using an email that already has a password account, only link them automatically when the provider says the email is **verified**; otherwise you create an account-takeover hole.

Build or buy decision. Use a hosted provider or a maintained server (Keycloak, Ory, Zitadel, Authentik) when you need SSO (SAML/OIDC for business customers), SCIM provisioning, organisations, audit, or compliance. Build on a library (Better Auth, Auth.js, Passport) for a simpler product where you want data in your own database.

5.11 Deep dive: authorization patterns

- **Check at the data layer, not just the route.** Put the owner or tenant filter into the repository method (`findByIdForUser(id, userId)`), so forgetting a check in a controller cannot expose data. IDOR (insecure direct object reference) bugs happen when code fetches `/orders/:id` without checking ownership.
- **Policy functions.** `can(user, 'update', note)` in one module, used by controllers and tests. CASL expresses rules such as "editors may update notes they authored" and can also generate database query filters.
- **Relationship-based access** (Google Zanzibar style): "user is a member of a team that can view the folder that contains the doc". Tools: dedicated engines such as **OpenFGA**, **SpiceDB**, **Permify**, **Oso**. Use them when sharing and nesting get complex; otherwise a join table is enough.
- **Least privilege everywhere.** Database users per service with only needed grants; separate read-only credentials for reporting; short-lived cloud credentials via role assumption instead of long-lived keys.
- **Admin and impersonation.** Log every admin action with actor and target; impersonation sessions must be clearly marked, time-limited, and audited.
- **Test authorization as a matrix:** every role by every endpoint with the expected status. It is the cheapest security test you can write.

5.12 Deep dive: web and API security checklist

Transport and headers. HTTPS only with HSTS (`Strict-Transport-Security: max-age=31536000; includeSubDomains`), TLS terminated at the proxy with modern settings, `helmet` defaults (`X-Content-Type-Options: nosniff`, `Referrer-Policy`, `Cross-Origin-Opener-Policy`), and a **Content-Security-Policy** if you serve HTML (an API that returns JSON mostly needs `nosniff` and correct `Content-Type`).

CORS properly. CORS is a browser rule, not a firewall: it does not stop curl or other servers. Use an explicit allow-list of origins (never reflect the incoming `Origin`, never `*` with credentials), restrict methods and headers, and set `Access-Control-Allow-Credentials: true` only if you use cookies.

Cache preflights with `Access-Control-Max-Age`. A wrong CORS setup plus cookie auth is a common cause of data theft.

Input and output. Validate and bound input (Part 3). Use parameterised queries always. Encode output for its context (HTML, URL, SQL, shell): an API returning JSON is safe from XSS only while clients render it as text. Never pass user input to `child_process.exec` (use `execFile` with an argument array), `eval`, `new Function`, `vm`, dynamic `require`, or template engines in unsafe modes. Prevent **prototype pollution** (`__proto__`, `constructor` keys in merged objects): use `Object.create(null)` maps or schema validation that strips unknown keys, avoid deep-merge libraries on user input, and keep dependencies updated.

SSRF. Any feature that fetches a user-supplied URL (webhooks, image import, link previews) can be turned toward internal services (cloud metadata endpoints like `169.254.169.254`, your database admin port). Allow-list schemes (`https`), resolve DNS yourself and block private, loopback, and link-local ranges *after* resolution, disable or re-validate redirects, set short timeouts and size limits, and run the fetcher in a network segment with no internal access.

File handling. Normalise and reject paths containing `..`; generate your own filenames; serve user uploads from a different origin or storage domain with `Content-Disposition: attachment` for risky types; scan archives and limit decompressed size (zip bombs).

Rate limiting and abuse. Layer limits: edge (CDN or WAF), per IP, per authenticated user or API key, and per sensitive action (login, password reset, OTP verify, expensive search). Return `429` with `Retry-After`. Use **bot and abuse controls** (Turnstile, reCAPTCHA, proof of work) on public forms. Cap pagination and query cost; in GraphQL add depth and complexity limits.

Dependency and supply-chain security. Lock files committed, `npm ci` in CI, automated updates, `npm audit signatures`/provenance where available, no postinstall scripts from unknown packages (`pnpm` blocks them by default in recent versions; check), Socket or Snyk scanning, and minimal dependencies for security-critical code. Pin GitHub Actions to commit SHAs. Run containers as non-root with a read-only filesystem and drop capabilities.

Secrets. Scan commits with gitleaks or GitHub secret scanning; rotate any secret that ever reached a repository; use separate secrets per environment; make rotation a routine, not an emergency.

Cryptography basics. Do not invent algorithms. Use AES-256-GCM (via `crypto.createCipheriv`) or libsodium (`libsodium-wrappers`) for encrypting fields, keep keys in a KMS and use envelope encryption (a data key encrypted by a master key) so you can rotate, add a key id to each ciphertext, and hash (not encrypt) anything you only need to compare.

Logging and detection. Audit-log authentication events and permission changes; alert on spikes in `401`, `403`, `429`, and on logins from new countries; keep a documented incident process and be ready to revoke all sessions and rotate keys.

Threat modelling in 15 minutes. For a new feature list: what are we protecting, who might attack it, how would they (STRIDE: spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege), and what stops them? Then write one test per mitigation.

OWASP Top 10 mapped to this report. Broken access control (5.11), cryptographic failures (5.12), injection (Parts 3 and 4), insecure design (Part 10 and threat modelling), security misconfiguration (headers, CORS, defaults), vulnerable components (supply chain), identification and authentication failures (5.8 to 5.10), software and data integrity failures (CI hardening, signed artifacts, webhook signatures), logging and monitoring failures (Part 11), SSRF (above). The OWASP API Security Top 10 adds broken object-level authorization, excessive data exposure, and unrestricted resource consumption; read it alongside.

5.13 More build cards

Card 5.E: timing-safe login and rehash

- **Aim:** remove user enumeration and upgrade old hashes.
- **Input:** a login handler, a user table with a `password_hash`, and a function `needsRehash(hash)`.
- **Output:** unknown email and wrong password return the same body and similar latency (measure 200 samples each); a user whose hash uses weaker settings gets a fresh hash saved after login.
- **Check:** a script that prints mean and p95 latency for both cases and a database query showing the new hash prefix.

Card 5.F: password reset and email verification tokens

- **Aim:** safe one-time tokens.
- **Input:** a `tokens(user_id, purpose, token_hash, expires_at, used_at)` table; endpoints to request and redeem.
- **Output:** tokens are random, stored hashed, expire, work once; redeeming a reset token revokes all sessions.
- **Check:** try to redeem twice, redeem after expiry, and redeem with a token from another purpose; expect failure each time.

Card 5.G: server sessions in Redis

- **Aim:** a complete cookie session.
- **Input:** login that creates a session (random id, hashed key in Redis, idle and absolute TTL) and sets a `__Host-` cookie; logout; logout-everywhere.
- **Output:** the session id changes at login; deleting the Redis key logs the user out on the next request.
- **Check:** inspect the `Set-Cookie` header flags with `curl -i`; try replaying an old cookie after logout.

Card 5.H: JWKS and key rotation

- **Aim:** verify tokens without sharing a secret.
- **Input:** an auth service that signs with ES256 and publishes `/.well-known/jwks.json`; a second API that verifies using `jose` and a remote JWKS.
- **Output:** you rotate to a new key (new `kid`) and old tokens keep working until expiry while new ones use the new key, with no restart of the API.
- **Check:** issue a token, rotate, issue another, and call the API with both.

Card 5.I: OAuth login with Google or GitHub

- **Aim:** implement code flow with PKCE.
- **Input:** an OAuth client registered with the provider; `GET /auth/login` and `GET /auth/callback`; a library such as Arctic or `openid-client`.
- **Output:** `state` and PKCE verifier are stored server-side per attempt; the callback rejects a wrong `state`; the user row is keyed on the provider's `sub`.
- **Check:** tamper with `state` and confirm a rejection; log in twice and confirm one user row.

Card 5.J: authorization matrix test

- **Aim:** prove access control.
- **Input:** roles `admin`, `editor`, `viewer`, plus two tenants; a list of 8 endpoints.
- **Output:** a generated test table `role x endpoint x tenant -> expected status`, including cross-tenant access to another tenant's id (expect `404`).
- **Check:** the suite fails if you remove an ownership filter from any repository method.

Card 5.K: SSRF-safe fetcher

- **Aim:** make "fetch this URL" safe.
- **Input:** `POST /link-preview {url}`.
- **Output:** requests to `http://127.0.0.1`, `http://169.254.169.254`, a hostname that resolves to `10.x.x.x`, and a redirect to localhost are all refused; a normal `https` page works with a 3 second timeout and 1 MB limit.
- **Check:** a test list of 10 hostile URLs with the expected result for each.

Deep-dive self-check: (1) Why hash a dummy password when the user does not exist? (2) What do `HttpOnly`, `Secure`, and `SameSite` each prevent? (3) Why is the ID token not an API credential? (4) What does PKCE protect against? (5) Give one SSRF defence that works after DNS resolution. (6) Why should an authorization check live in the repository as well as the controller?

Part 5 deep dive

5.8 Password hashing in practice

Argon2id is memory-hard: an attacker with GPUs cannot cheaply run millions of guesses because each guess needs real RAM. You tune three numbers: memory cost, time cost (iterations), and parallelism. OWASP's password-storage guidance lists configurations such as 19 MiB memory, 2 iterations, parallelism 1 as a minimum; confirm the current numbers in the OWASP cheat sheet and then tune upward until hashing takes a few hundred milliseconds on your production hardware (more than that hurts your login throughput).

```
import argon2 from 'argon2';
const opts = { type: argon2.argon2id, memoryCost: 19456, timeCost: 2, parallelism: 1
}; // memoryCost is in KiB

export const hashPassword = (pw: string) => argon2.hash(pw, opts);

// A real hash of a throwaway password, computed once at startup.
const DUMMY_HASH = await argon2.hash('not-a-real-password', opts);
```

```
export async function login(email: string, password: string) {
  const user = await users.findByEmail(email);
  // Verify against a dummy hash when the user does not exist, so response time
  // does not reveal which emails are registered.
  const ok = await argon2.verify(user?.passwordHash ?? DUMMY_HASH, password);
  if (!user || !ok) throw new AppError(401, 'INVALID_CREDENTIALS', 'Email or password
is wrong');
  if (argon2.needsRehash(user.passwordHash, opts)) {          // upgrade old
parameters transparently
    await users.setPasswordHash(user.id, await hashPassword(password));
  }
  return user;
}
```

Extra points: hashing is CPU and memory heavy and runs on the libuv thread pool, so a burst of logins can saturate it; rate limit login and size `UV_THREADPOOL_SIZE` if needed. A **pepper** (a secret key mixed in, stored outside the database) adds protection if only the database leaks. Compare secrets (API keys, tokens, HMACs) with `crypto.timingSafeEqual`, never `===`. Use `crypto.randomBytes` or `crypto.randomUUID` for anything security related, never `Math.random`.

5.9 Server sessions done right

The cookie carries only a random, unguessable session id; the data lives on the server. Cookie settings are the security:

Attribute	Setting	Why
`HttpOnly`	on	JavaScript cannot read it, so XSS cannot steal it
`Secure`	on	Sent over HTTPS only
`SameSite`	`Lax` (default choice) or `Strict`	Blocks the cookie on most cross-site requests (CSRF defence)
`Path=/` and name prefix `__Host-`	on	The `__Host-` prefix forces Secure, no Domain, Path=/, which prevents subdomain tricks
`Max-Age`	bounded	Pair with a server-side **idle** timeout and an **absolute** lifetime

```
import { randomBytes, createHash } from 'node:crypto';
const sha256 = (s: string) => createHash('sha256').update(s).digest('hex');

export async function createSession(res: express.Response, userId: string) {
  const sid = randomBytes(32).toString('base64url');                  // 256 bits of
entropy
  await redis.set(`sess:${sha256(sid)}`, JSON.stringify({ userId, createdAt:
Date.now() }), 'EX', 7 * 24 * 3600);
  res.cookie('__Host-sid', sid, { httpOnly: true, secure: true, sameSite: 'lax',
path: '/', maxAge: 7 * 24 * 3600 * 1000 });
}
export async function destroySession(sid: string) { await
redis.del(`sess:${sha256(sid)}`); }
```

The session id is stored **hashed**, so a leaked Redis dump does not yield usable cookies. Always issue a **new** session id at login (never reuse an anonymous one: that is session fixation), delete the session on logout, and delete all of a user's sessions on password change. Cookie-based auth needs CSRF protection for state-changing requests beyond `SameSite`: a CSRF token for forms, or require a custom header that cross-site forms cannot send.

5.10 JWT in depth

A JWT is `header.payload.signature`, each base64url encoded. The signature proves the contents were not changed; it does **not** hide them. Anyone can read the payload.

Algorithm family	Key	Use when
HS256 (HMAC)	One shared secret signs and verifies	One service issues and verifies
RS256, ES256, EdDSA (asymmetric)	Private key signs, public key verifies	Many services verify tokens; publish public keys as a **JWKS** and rotate by `kid`

```
import { SignJWT, jwtVerify, createRemoteJWKSet } from 'jose';
const secret = new TextEncoder().encode(config.JWT_SECRET);

export const signAccess = (userId: string, role: string) =>
  new SignJWT({ role }).setProtectedHeader({ alg: 'HS256' })
    .setSubject(userId).setIssuer('api.example.com').setAudience('web')
    .setIssuedAt().setExpirationTime('15m').setJti(crypto.randomUUID()).sign(secret);

export async function verifyAccess(token: string) {
  const { payload } = await jwtVerify(token, secret, {
    issuer: 'api.example.com', audience: 'web',
    algorithms: ['HS256'],            // pin it: this blocks algorithm-confusion and
'none' attacks
    clockTolerance: '5s',
  });
  return payload;
}
// Verifying tokens from a provider (Auth0, Keycloak, Cognito): public keys fetched
and cached from its JWKS URL
const JWKS = createRemoteJWKSet(new URL('https://idp.example.com/.well-
known/jwks.json'));
// await jwtVerify(token, JWKS, { issuer: 'https://idp.example.com/', audience: 'my-
api' });
```

Standard claims: `iss` (issuer), `sub` (subject, the user id), `aud` (audience: who the token is for), `exp`, `nbf`, `iat`, `jti` (unique id, useful for revocation). **Always check `iss` and `aud`**: a valid token from the same provider but issued for a different API must be rejected. Never use a "decode" function without verifying.

Revocation is the cost of stateless tokens. Options: short lifetimes (about 15 minutes) so stale permissions expire fast; a deny-list of `jti` values in Redis with a TTL equal to the token's remaining life; a `tokenVersion` column on the user that is embedded in the token and checked on each request (bumping it logs the user out everywhere, at the cost of a lookup).

Where the browser keeps tokens. Keeping access tokens in `localStorage` exposes them to any XSS. A common compromise: access token in memory only, refresh token in an `httpOnly`, `Secure`, `SameSite` cookie scoped to the refresh path. For a same-site web app, plain server sessions avoid the whole question.

Refresh token rotation with reuse detection (the flow drawn in 5.2):

```
CREATE TABLE refresh_tokens (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  family_id uuid NOT NULL,                  -- all tokens descended from one login
  token_hash text NOT NULL UNIQUE,          -- sha256 of the opaque token, never the
token
  expires_at timestamptz NOT NULL,
  used_at timestamptz,
  revoked_at timestamptz
);
```

```
export async function rotate(oldToken: string) {
  return withTx(pool, async (c) => {
    const { rows } = await c.query('SELECT * FROM refresh_tokens WHERE token_hash =
$1 FOR UPDATE', [sha256(oldToken)]);
    const t = rows[0];
    if (!t || t.expires_at < new Date() || t.revoked_at) throw new AppError(401,
'INVALID_REFRESH', 'Log in again');
    if (t.used_at) {                                  // an already-used token came
back: it was stolen or replayed
      await c.query('UPDATE refresh_tokens SET revoked_at = now() WHERE family_id =
$1', [t.family_id]);
      throw new AppError(401, 'REFRESH_REUSE', 'Session revoked');
    }
    await c.query('UPDATE refresh_tokens SET used_at = now() WHERE id = $1', [t.id]);
    const next = randomBytes(32).toString('base64url');
    await c.query('INSERT INTO refresh_tokens (user_id, family_id, token_hash,
expires_at) VALUES ($1,$2,$3, now() + interval \'30 days\')',
      [t.user_id, t.family_id, sha256(next)]);
    return { refreshToken: next, accessToken: await signAccess(t.user_id, 'user') };
  }, 1);
}
```

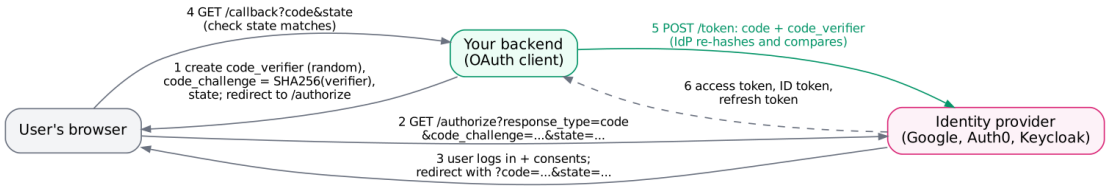
(A wrinkle: the revocation `UPDATE` above is rolled back if you throw inside the same transaction. In real code commit the revocation first, then throw. Card 5.F asks you to find and fix this.)

5.11 OAuth 2.0 and OpenID Connect in depth

Roles: the **resource owner** (the user), the **client** (your app), the **authorization server** / identity provider (Google, Auth0, Keycloak), and the **resource server** (an API). Pick the flow by the client type:

Flow	Use for
Authorization Code + **PKCE**	Web apps (including those with a backend), SPAs, mobile apps: the default

Flow	Use for
Client Credentials	Machine to machine, no user
Device Code	TVs and CLIs without a browser
Implicit and password grants	Obsolete; do not use



Security details that matter: **PKCE** (the verifier and its hash) stops an attacker who steals the authorization code from redeeming it; the `state` parameter ties the callback to the request that started it (CSRF defence); the OIDC `nonce` binds the ID token to the session (replay defence); the provider must match the **redirect URI exactly**; request the minimum **scopes**. The **ID token** tells *your client* who logged in; the **access token** is for calling an API. Do not send ID tokens to APIs as access tokens, and validate whichever you consume (signature, `iss`, `aud`, `exp`, and `nonce` for ID tokens).

```
import { randomBytes, createHash } from 'node:crypto';
const verifier = randomBytes(32).toString('base64url');                    // keep
in the user's session
const challenge = createHash('sha256').update(verifier).digest('base64url');   //
send in /authorize
const state = randomBytes(16).toString('base64url');                        //
also kept in the session
```

Use a maintained library for the rest (`openid-client` or Arctic for provider flows, `@fastify/oauth2`, Passport strategies) and read its current docs, because the API of these libraries changes between major versions. After a first social login, link it to an existing account **only if the email is verified by the provider**, otherwise attackers can take over accounts by registering a matching email elsewhere.

5.12 Authorization patterns in depth

RBAC schema: `users`, `roles`, `permissions`, `user_roles`, `role_permissions`. Check a **permission** (`note:update`), not a role name, in code, so roles can change without code changes. Roles alone cannot express "edit **your own** notes"; combine with ownership or attributes.

```
import { AbilityBuilder, createMongoAbility } from '@casl/ability';

export function abilityFor(user: { id: string; role: 'admin' | 'editor' | 'viewer' })
{
  const { can, build } = new AbilityBuilder(createMongoAbility);
  if (user.role === 'admin') can('manage', 'all');
  else {
    can('read', 'Note');
    if (user.role === 'editor') can(['update', 'delete'], 'Note', { ownerId: user.id
});  // condition on the record
  }
  return build();
```

```
}
// usage: if (!abilityFor(user).can('update', subject('Note', note))) throw new
AppError(403, 'FORBIDDEN', '...');
```

The IDOR / BOLA bug (insecure direct object reference, "broken object level authorization", number one in the OWASP API Security Top 10): `GET /orders/123` returns any order to any logged-in user. Fix at the data layer so you cannot forget it: `SELECT * FROM orders WHERE id = $1 AND user_id = $2`, and return `404` (not `403`) so you do not reveal that the id exists.

PostgreSQL row-level security enforces the rule in the database itself, a safety net for multi-tenant apps:

```
ALTER TABLE notes ENABLE ROW LEVEL SECURITY;
ALTER TABLE notes FORCE ROW LEVEL SECURITY;      -- applies to the table owner too
CREATE POLICY notes_owner ON notes
  USING (owner_id = current_setting('app.user_id')::uuid);
-- per request, inside the transaction: SELECT set_config('app.user_id', $1, true);
```

Connect as a role that is **not** a superuser and does not own the tables, or the policies are bypassed. Also guard **function-level** access (a normal user calling an admin endpoint) and **property-level** access (a user updating their own `role` field: whitelist writable fields).

5.13 CORS, headers, and CSRF without confusion

- **CORS** is a browser rule, not server security: it decides whether a page from origin A may *read* a response from origin B. A non-browser client ignores it entirely. For a request with custom headers or methods other than simple ones, the browser first sends a **preflight** `OPTIONS` request; your server must answer with `Access-Control-Allow-Origin` (the exact origin from an allow-list, never reflected blindly), `-Methods`, `-Headers`, and `-Credentials: true` only when you use cookies (and then `*` is not allowed).
- **CSRF** abuses cookies the browser attaches automatically. Defences: `SameSite` cookies, CSRF tokens, checking `Origin`/`Referer`, requiring a custom header. APIs that authenticate with an `Authorization` header (not cookies) are not vulnerable to classic CSRF.
- **helmet** sets sensible headers: `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, frame protections, and a starter `Content-Security-Policy`. A JSON-only API mostly needs HSTS, nosniff, and correct `Content-Type`; the CSP matters when you serve HTML.

5.14 Brute force, resets, verification, and MFA

- **Rate-limit in layers:** per IP (blunts bots), per account (blunts targeted guessing), and per IP+account. Pure account lockout lets an attacker lock out victims, so prefer progressive delays plus CAPTCHA or MFA step-up after repeated failures. Credential stuffing (known leaked passwords tried at scale) shows up as many accounts failing from many IPs: alert on failure *rate*.
- **Password reset:** generate a random token, store only its **hash** with a short expiry (15 to 60 minutes) and single use, always answer "if that email exists we sent a link" (no enumeration), and after a successful reset **revoke all sessions and refresh tokens**.
- **Email verification:** the same token pattern; do not trust an email until verified, especially before account linking.
- **TOTP MFA** (authenticator apps): generate a secret, show it as a QR code (`otpauth://` URI), verify the first code before enabling, accept a small window for clock drift, store the secret

encrypted, issue single-use **backup codes** (stored hashed), and rate-limit code attempts. `otplib` implements it.

- **Passkeys / WebAuthn** replace passwords with public-key credentials the user's device protects; they resist phishing. `@simplewebauthn/server` is the usual Node library.

5.15 Encrypting data and handling secrets

Operation	Purpose	Use
Hashing (one way)	Verify without storing the original	Passwords: argon2id. Integrity checks: SHA-256
Encryption (two way)	Store data you must read back (OAuth tokens, TOTP secrets)	AES-256-GCM with a random nonce per message; keys from a KMS
Signing / MAC	Prove origin and integrity	JWT, HMAC webhooks (`createHmac('sha256', secret)`)

```
import { randomBytes, createCipheriv, createDecipheriv } from 'node:crypto';

export function encrypt(plain: string, key: Buffer): string {      // key: exactly 32
bytes
  const iv = randomBytes(12);                                         // unique per
message, never reused with a key
  const c = createCipheriv('aes-256-gcm', key, iv);
  const enc = Buffer.concat([c.update(plain, 'utf8'), c.final()]);
  return Buffer.concat([iv, c.getAuthTag(), enc]).toString('base64');
}
export function decrypt(b64: string, key: Buffer): string {
  const buf = Buffer.from(b64, 'base64');
  const d = createDecipheriv('aes-256-gcm', key, buf.subarray(0, 12));
  d.setAuthTag(buf.subarray(12, 28));                                 // tampering
makes final() throw
  return Buffer.concat([d.update(buf.subarray(28)), d.final()]).toString('utf8');
}
```

Do not invent algorithms or modes. Keep the key in a KMS or secrets manager (envelope encryption: a KMS-held master key encrypts per-record data keys), plan **key rotation**, and never put keys in the repository or logs. **Supply chain:** commit the lockfile and install with `npm ci` / `pnpm install --frozen-lockfile`, review new dependencies, run `npm audit` and an update bot (Renovate or Dependabot), and consider `--ignore-scripts` plus an allow-list for install scripts.

5.16 More build cards

Card 5.E: timing-safe login

- **Aim:** remove the account-enumeration side channel.
- **Input:** the login code from 5.8, with and without the dummy-hash trick.
- **Output:** measured median response time for an existing email versus a non-existent one over 200 attempts each; without the trick they differ clearly, with it they match closely.
- **Check:** a script that times both cases and prints the two medians.

Card 5.F: refresh-reuse detection that actually revokes

- **Aim:** find a subtle transaction bug.
- **Input:** the `rotate` function above.
- **Output:** a test that presents a used token and asserts the whole family is revoked **afterwards** (including the newest token). It fails on the code as written; fix it (commit the revocation before throwing) so it passes.
- **Check:** the test goes from red to green.

Card 5.G: password reset flow

- **Aim:** a safe, complete reset.
- **Input:** `POST /auth/forgot {email}` and `POST /auth/reset {token, password}`.
- **Output:** identical responses for known and unknown emails; token stored hashed, expires, works once; all sessions revoked after reset.
- **Check:** use the token twice (second fails), use an expired token (fails), compare response bodies for known versus unknown email.

Card 5.H: ownership checks and row-level security

- **Aim:** make cross-user access impossible by construction.
- **Input:** `notes(owner_id)`, two users, endpoints from Part 2.
- **Output:** user B receives `404` for user A's note on read, update, and delete; then add RLS and prove a deliberately forgotten `WHERE owner_id` clause still returns nothing.
- **Check:** a test matrix of (user, action, owner) with expected status codes.

Card 5.I: JWT attack tests

- **Aim:** prove your verifier rejects bad tokens.
- **Input:** tokens with `alg: none`, a wrong `aud`, a wrong `iss`, an expired `exp`, a tampered payload, and a token signed with a different key.
- **Output:** every one returns `401`; only the correct token passes.
- **Check:** a test file with seven cases.

Card 5.J: TOTP enrollment

- **Aim:** add MFA.
- **Input:** `otplib` (or an equivalent), a QR code of the `otpauth://` URI, encrypted secret storage from 5.15.
- **Output:** enabling requires a valid first code; login then requires the code; 10 single-use backup codes work once each.
- **Check:** an automated test using the library to generate valid codes.

5.17 Answer key for the Part 5 self-checks

- **Why is SHA-256 wrong for passwords?** It is designed to be fast, so an attacker can test billions of guesses per second on a GPU; argon2id is deliberately slow and memory-hard and uses a per-password salt.

- **Why rotate refresh tokens?** A stolen refresh token is otherwise valid for weeks. With rotation each token works once, so a second use signals theft and lets you revoke the whole family.
- **SQL that stops user A reading user B's order:** `SELECT * FROM orders WHERE id = $1 AND user_id = $2`, with `$2` taken from the authenticated session, never from the request body or URL.

5.8 Deep dive: passwords and the account lifecycle

Hashing details. argon2id is *memory-hard*: it needs a lot of RAM and time per guess, which makes GPU and ASIC cracking expensive. Its three knobs are memory cost, time cost (iterations), and parallelism. Tune them so one hash takes roughly 100 to 500 ms on your production hardware, and follow the current OWASP Password Storage Cheat Sheet for minimum values. The salt is generated and stored inside the encoded hash string automatically. An optional **pepper** (a secret kept outside the database, for example in the secrets manager) adds protection if only the database leaks. Because hashing is deliberately slow, put login behind a rate limit (otherwise it is a cheap way to burn your CPU).

```
import argon2 from 'argon2';

const hash = await argon2.hash(password, { type: argon2.argon2id });   // salt is
embedded
const ok = await argon2.verify(hash, password);
if (ok && argon2.needsRehash(hash)) {
  // cost parameters were raised since this hash was made: store a fresh hash now
  await users.updateHash(user.id, await argon2.hash(password, { type: argon2.argon2id
}));
}
```

One-time tokens (email verification, password reset, magic links): generate with `crypto.randomBytes(32).toString('base64url')`, store only a **SHA-256 hash** of the token with `user_id`, `purpose`, `expires_at`, and `used_at`, email the raw token once, and accept it once. Hashing the stored value means a database leak does not leak usable links.

Password reset flow, safely

1. `POST /auth/forgot {email}` always answers `202` with the same text whether or not the email exists (prevents account enumeration). Do the real work in a background job so response time does not reveal it either.
2. Email a link with the raw token; it expires in 15 to 60 minutes and works once.
3. `POST /auth/reset {token, newPassword}` validates the password policy, stores the new hash, marks the token used, and **revokes all sessions and refresh tokens** of that user.
4. Notify the user by email that the password changed.

Other lifecycle rules: verify email before enabling sensitive actions; require the current password (or a fresh login) to change email or password; apply a delay or lockout after repeated failures (per account and per IP, so an attacker cannot lock a victim out forever); send security notifications (new device, password change). **MFA with TOTP** (`otplib`): generate a secret, show it as a QR code, store the secret encrypted, verify a 6-digit code with a small time window, and issue one-time **backup codes** (stored hashed). Passkeys (WebAuthn, `@simplewebauthn/server`) are the strongest and increasingly the default for new products.

5.9 Deep dive: cookie sessions done properly

A session is a random, unguessable id (at least 128 bits) in a cookie that maps to server-side data. The cookie carries no information of its own.

```
import session from 'express-session';
import { RedisStore } from 'connect-redis';

app.set('trust proxy', 1);                        // behind one proxy, so Secure
cookies work
app.use(session({
  store: new RedisStore({ client: redis, prefix: 'sess:' }),
  secret: config.SESSION_SECRET,
  name: '__Host-sid',
  resave: false,
  saveUninitialized: false,
  rolling: true,
  cookie: { httpOnly: true, secure: true, sameSite: 'lax', maxAge: 1000 * 60 * 60 * 8
},
}));

// after verifying credentials: issue a NEW session id (prevents session fixation)
req.session.regenerate((err) => { req.session.userId = user.id; req.session.save(()
=> res.sendStatus(204)); });
```

- **Session fixation:** always regenerate the id at login and at privilege change.
- **Lifetimes:** an *idle* timeout (rolling) plus an *absolute* maximum lifetime.
- **Logout everywhere:** keep an index `user:{id}:sessions` in Redis, or store sessions in PostgreSQL, so you can delete all of a user's sessions.
- **CSRF:** `SameSite=Lax` already blocks most cross-site POSTs. For extra safety use a CSRF token (synchroniser or double-submit pattern) on state-changing requests, and check the `Origin` header. APIs that use `Authorization: Bearer` headers (not cookies) are not CSRF-prone but are exposed to XSS token theft instead.
- **CORS with cookies:** the exact origin (not `*`), `credentials: true`, and the front-end must send `credentials: 'include'`.
- **Cookie prefix** `__Host-` forces `Secure`, `Path=/`, and no `Domain`, which blocks several cookie injection tricks.

5.10 Deep dive: JWT mechanics, signing keys, and refresh rotation

A JWT is three base64url parts: **header** (algorithm, key id), **payload** (claims), **signature**. Standard claims: `sub` (subject, the user id), `iss` (issuer), `aud` (audience), `exp`, `iat`, `nbf`, and `jti` (unique id). Anyone can read the payload; the signature only proves it was not altered.

Algorithm	Type	Use when
HS256	Shared secret (HMAC)	One service both signs and verifies
RS256, ES256, EdDSA	Public/private key pair	Many services verify, only the auth service holds the private key; publish public keys at a **JWKS** URL and rotate with a `kid`

Algorithm	Type	Use when
		header

```
import { SignJWT, jwtVerify, createRemoteJWKSet } from 'jose';

const secret = new TextEncoder().encode(config.JWT_SECRET);
export const signAccess = (user: { id: string; role: string }) =>
  new SignJWT({ role: user.role })
    .setProtectedHeader({ alg: 'HS256' })
    .setSubject(user.id).setIssuer('my-api').setAudience('my-web')
    .setIssuedAt().setExpirationTime('15m')
    .sign(secret);

export async function verifyAccess(token: string) {
  const { payload } = await jwtVerify(token, secret, {
    issuer: 'my-api', audience: 'my-web', algorithms: ['HS256'],   // pin the
algorithm
  });
  return payload;
}

// verifying tokens issued by an external identity provider
const jwks = createRemoteJWKSet(new URL('https://idp.example.com/.well-
known/jwks.json'));
// const { payload } = await jwtVerify(token, jwks, { issuer: '...', audience: '...'
});
```

Refresh token table and rotation

```
CREATE TABLE refresh_tokens (
  id uuid PRIMARY KEY,
  user_id uuid NOT NULL REFERENCES users(id),
  family_id uuid NOT NULL,            -- all tokens descended from one login
  token_hash bytea NOT NULL UNIQUE,   -- SHA-256 of the opaque token
  expires_at timestamptz NOT NULL,
  used_at timestamptz,                -- set when exchanged for a new token
  revoked_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now()
);
```

On `/refresh`: find the row by hash; if `used_at` or `revoked_at` is set, someone is replaying an old token, so revoke the entire `family_id` and force login; otherwise mark `used_at`, insert a new row in the same family, and return a new pair. Do all of it in one transaction.

Where the client keeps tokens: an `httpOnly` cookie is safe from XSS theft but needs CSRF care; memory-only storage is safe from persistence attacks but lost on reload (pair it with a refresh cookie); `localStorage` is readable by any injected script, so avoid it for long-lived tokens.

Revocation: short access-token lifetimes are the main control. For instant revocation keep a small deny-list of `jti` values in Redis until their natural expiry, or include a per-user `token_version` claim and compare to the database on sensitive routes.

5.11 Deep dive: OAuth 2.0 and OpenID Connect

Roles: the *resource owner* (the user), the *client* (your app), the *authorization server* (Google, Auth0, Keycloak), and the *resource server* (an API that accepts access tokens).

Authorization Code flow with PKCE (browser, mobile, and server apps)

1. Your app creates a random `code_verifier`, derives `code_challenge = base64url(sha256(verifier))`, and a random `state` (and `nonce` for OIDC); stores them in the session.
2. Redirect the user to the authorization endpoint with `client_id`, `redirect_uri`, `response_type=code`, `scope=openid email profile`, `state`, `code_challenge`, `code_challenge_method=S256`.
3. The user signs in and consents; the server redirects back with `code` and `state`.
4. Your backend **checks `state` equals the stored one**, then POSTs to the token endpoint with the `code` and `code_verifier` (plus the client secret for confidential clients).
5. You receive an access token, optionally a refresh token, and an **ID token** (a JWT about the user). Validate the ID token: signature through the provider's JWKS, `iss`, `aud`, `exp`, and `nonce`.
6. Create your own session for the user, linking the provider's `sub` to your user row.

```
import crypto from 'node:crypto';
const verifier = crypto.randomBytes(32).toString('base64url');
const challenge = crypto.createHash('sha256').update(verifier).digest('base64url');
const state = crypto.randomBytes(16).toString('base64url');
```

Common mistakes: loose `redirect_uri` matching (register exact URIs), skipping `state` (login CSRF), trusting the email from a provider without checking it is verified, using the implicit flow (deprecated), and treating the access token as proof of identity (that is the ID token's job). **Client credentials** flow is for machine-to-machine calls (no user). **Device code** flow suits TVs and CLIs. Prefer a maintained library (`openid-client`, Arctic, Passport strategies, or your framework's auth package) over writing the protocol yourself.

Protecting your own API with an external IdP: the client sends `Authorization: Bearer <access token>`; your API verifies signature (JWKS), `iss`, `aud`, `exp`, and required **scopes**, then maps the `sub` to a local user. Cache the JWKS and handle key rotation.

5.12 Deep dive: authorization models and enforcement

- **RBAC data model:** `users`, `roles`, `permissions`, `user_roles`, `role_permissions`. Check permissions (`orders:refund`), not role names, in code, so roles can change without redeploys.
- **Central policy function:** `can(user, 'update', note)` in one module, called by every controller. Scattered `if (user.role === 'admin')` checks become inconsistent. CASL gives a structured version of this.
- **Resource ownership:** pass the user id into the query (`WHERE id = $1 AND owner_id = $2`) so the database cannot return what the caller may not see. Return `404` rather than `403` when existence should stay hidden. Testing this is called **IDOR** (insecure direct object reference) testing: log in as user A and request user B's ids.
- **Row-level security (PostgreSQL)** enforces tenant isolation inside the database as a safety net:

```
ALTER TABLE notes ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON notes
  USING (tenant_id = current_setting('app.tenant_id')::uuid);

-- in each request transaction:
SELECT set_config('app.tenant_id', $1, true);   -- true = local to this transaction
```

The application's database role must **not** be the table owner or a superuser, otherwise it bypasses RLS (unless you use `FORCE ROW LEVEL SECURITY` for owners). With transaction-level pooling, set the variable inside the transaction as shown.

- **Field-level rules:** some fields editable only by admins; enforce with separate input schemas per role.
- **Admin and support tools** need audit logs: who did what, when, to which record.

5.13 Deep dive: hardening checklist with how-tos

Area	What to do
Security headers	`helmet` defaults, plus a Content-Security-Policy tuned to your front end (start in report-only mode), `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `Permissions-Policy`; an API serving only JSON can use a very strict CSP
CORS	Explicit origin allow-list from config; no wildcard with credentials; restrict methods and headers
Rate limits	Layered: a global per-IP limit, a tight limit on `/login` and `/forgot` keyed by IP **and** email, a per-API-key limit for clients; respond `429` with `Retry-After`
Input	Validate types and sizes everywhere; limit body size; reject unexpected fields; cap array lengths and nesting depth to avoid resource exhaustion
NoSQL injection	Reject object values where strings are expected (`{ "$ne": null }` in a Mongo login filter bypasses checks); schema validation prevents this
SSRF	When fetching user-supplied URLs: allow `https` only, resolve DNS and block private, loopback, and link-local ranges (including cloud metadata addresses), disable or re-check redirects, set short timeouts and size caps
Timing	Compare secrets and signatures with `crypto.timingSafeEqual` (lengths must match first)
Webhooks	Verify HMAC over the raw body, check a timestamp to defeat replays, accept each event id once

Area	What to do
Prototype pollution	Do not deep-merge untrusted objects into your own; validate input; use `Object.create(null)` for dictionaries keyed by user input
ReDoS	Avoid nested quantifiers in regexes applied to user input; use schema validators or linear-time matching
Uploads	Size limits, content sniffing, random names, private storage, virus scan, never serve user files from your main domain
Supply chain	Commit the lock file and use `npm ci` / `pnpm install --frozen-lockfile`; run `npm audit` in CI; enable Dependabot or Renovate; review new dependencies; consider blocking install scripts of unknown packages; scan for leaked secrets (gitleaks, GitHub secret scanning)
Containers	Non-root user, read-only filesystem where possible, minimal base image, scan images (Trivy)
Observability for security	Log auth events (login, failures, resets, role changes) with request id and IP; alert on spikes of 401, 403, and 429

Threat modelling in 10 minutes: list assets (user data, payment tokens), entry points (endpoints, webhooks, queues, admin UI), and for each ask the STRIDE questions: can someone **S**poof identity, **T**amper with data, **R**epudiate an action, **D**isclose information, cause **D**enial of service, or **E**levate privilege? Write down the mitigation next to each answer.

5.14 More build cards

Card 5.E: password reset end to end

- **Aim:** implement the safe flow in 5.8.
- **Input:** `POST /auth/forgot`, `POST /auth/reset`, a `one_time_tokens` table, an email job that prints the link to the console in development.
- **Output:** identical responses for known and unknown emails; a token works once and expires; a reset revokes existing sessions.
- **Check:** tests for reuse of a token (fails), an expired token (fails), and timing of both `forgot` cases (similar).

Card 5.F: TOTP multi-factor login

- **Aim:** second factor.
- **Input:** enrol (`otplib` secret plus QR), verify code, backup codes.
- **Output:** login returns an "mfa required" state until a valid code is posted; each backup code works once.
- **Check:** an authenticator app accepts your QR; replaying the same code twice within its window is rejected by your replay guard.

Card 5.G: Sign in with GitHub or Google using PKCE

- **Aim:** implement the authorization code flow with a library.
- **Input:** `GET /auth/github`, `GET /auth/github/callback`.
- **Output:** first login creates a local user linked by provider `sub`; a tampered `state` is rejected with `400`.
- **Check:** try the callback with a wrong state and with a reused code; both fail.

Card 5.H: tenant isolation with RLS

- **Aim:** a database-level safety net.
- **Input:** a `notes` table with `tenant_id`, the policy in 5.12, an app role without ownership, middleware that sets `app.tenant_id` per transaction.
- **Output:** a query that forgets the `WHERE tenant_id` still returns only the current tenant's rows.
- **Check:** a test connects as tenant A, runs `SELECT * FROM notes`, and asserts no tenant B rows.

Card 5.I: IDOR test suite

- **Aim:** catch broken access control automatically.
- **Input:** two users, each with a note, order, and file; a table of every id-based route.
- **Output:** a generated test that requests each of user B's resources as user A and expects `404` or `403`.
- **Check:** deliberately remove an ownership check and watch exactly that route fail.

Card 5.J: supply chain gate in CI

- **Aim:** automate dependency hygiene.
- **Input:** a GitHub Actions job running `npm ci`, `npm audit --omit=dev --audit-level=high`, secret scanning, and image scanning.
- **Output:** the build fails on a known high severity vulnerability or a committed secret.
- **Check:** add a fake secret in a branch and confirm the job fails.

Part 6: API styles and real-time communication

How clients talk to your server is a design decision with real consequences. This part compares the options and then covers real-time transports.

6.1 Theory: choosing an API style

Style	Idea	Strengths	Weaknesses	Use when
REST over JSON	Resources and HTTP verbs	Universal, cacheable, easy to debug, tooling everywhere	Over- and under-fetching, many round trips for complex screens	Public APIs, most CRUD apps
GraphQL	One endpoint, client asks for exact fields	No over-fetching, one request per screen, typed	Caching and rate limiting are harder, N+1 risk, query-cost	Many clients with different data needs, complex

Style	Idea	Strengths	Weaknesses	Use when
		schema	attacks	nested data
tRPC	Call server functions from a TypeScript client with inferred types	Zero schema duplication, very fast iteration	TypeScript only, tight coupling, not for public APIs	Full-stack TypeScript monorepo (for example Next.js + Node)
gRPC	Binary Protobuf contracts over HTTP/2	Fast, strict contracts, streaming, code generation in many languages	Not browser-friendly without a proxy, harder to debug	Service-to-service calls, polyglot microservices
WebSocket	Persistent two-way connection	Low latency both ways	Stateful connections complicate scaling	Chat, games, live collaboration
Server-Sent Events (SSE)	One-way stream from server over HTTP	Simple, auto-reconnect, works through proxies	Server to client only	Notifications, live feeds, streaming AI output
Webhooks	Server calls the client's URL on events	Decoupled integration	Needs retries, signatures, idempotency	Payments, third-party integrations

6.2 Theory: designing good REST

- **Status codes and shapes are part of the contract.** Pick one envelope and keep it.
- **Idempotency keys.** For `POST` that must not run twice (payments), accept an `Idempotency-Key` header, store the key with the result, and return the stored result on a retry.
- **Pagination.** Offset (`?page=3&limit=20`) is simple; cursor (`?after=abc&limit=20`) is stable and fast. Return a `nextCursor`.
- **Filtering and sorting.** `?status=paid&sort=-createdAt`, with an allow-list of sortable fields.
- **Versioning.** `/v1` in the path is the most common. Add fields freely; remove or rename only in a new version.
- **Caching.** `ETag` + `If-None-Match` returns `304` and saves bandwidth. `Cache-Control` for public data.
- **Rate-limit headers.** `RateLimit-*` or `Retry-After` on `429`.
- **Documentation.** Publish an OpenAPI (Swagger) document generated from your schemas. Clients, mocks, and tests can be generated from it.

6.3 Theory: real-time

WebSocket upgrades an HTTP connection into a persistent two-way channel. Each connection holds memory on one server, so with several servers you need a way to broadcast across them (Redis pub/sub, covered in Part 9). **Socket.IO** adds rooms, acknowledgements, automatic reconnection, and

fallbacks on top of WebSocket. **SSE** is often enough: if only the server pushes, choose SSE for its simplicity.

Heartbeats (ping/pong) detect dead connections; authenticate during the handshake; limit message size and rate per connection.

6.4 Libraries and alternatives

Need	Pick	Alternatives
OpenAPI from code	`@fastify/swagger`, `@nestjs/swagger`	`zod-to-openapi`, `tsoa`, `ts-rest`, hand-written spec with Redocly
GraphQL server	GraphQL Yoga	Apollo Server, Mercurius (Fastify), `@nestjs/graphql`
GraphQL schema building	Pothos (code-first)	TypeGraphQL, Nexus, schema-first SDL
Batching to fix N+1	`dataloader`	Prisma/Drizzle joins
tRPC	`@trpc/server` + `@trpc/client`	`ts-rest`, oRPC, Hono RPC
gRPC	`@grpc/grpc-js` + `@grpc/proto-loader`	`nice-grpc`, Connect (`@connectrpc/connect`), `@nestjs/microservices`
WebSocket	`ws`	`uWebSockets.js` (very fast), `@fastify/websocket`, `@nestjs/websockets`
Rooms, reconnects	Socket.IO + `@socket.io/redis-adapter`	Ably, Pusher, Centrifugo (hosted or standalone)
SSE	Plain `text/event-stream` response	`better-sse`
Webhook verification	HMAC signature check with `crypto.timingSafeEqual`	Provider SDK helpers

6.5 Build cards

Card 6.A: production-style REST resource

- **Aim:** apply the REST checklist to the Notes API.
- **Input:** `GET /v1/notes?limit=20&after=<cursor>&sort=-createdAt`, `ETag` support, OpenAPI document served at `/docs`.
- **Output:** list responses contain `data` and `nextCursor`; a repeated request with `If-None-Match` returns `304`.
- **Check:** `curl -i` twice and compare; open `/docs` and call an endpoint from the UI.

Card 6.B: idempotent POST

- **Aim:** make a retry safe.
- **Input:** `POST /v1/payments` with header `Idempotency-Key: abc`.
- **Output:** first call creates the payment (`201`); second call with the same key returns the same body and does not create another row.
- **Check:** send the same request 10 times in parallel; the table has one row (use a unique index on the key to settle races).

Card 6.C: GraphQL version of Notes

- **Aim:** feel the difference from REST.
- **Input:** a schema with `Note`, `User`, query `notes(first, after)`, mutation `createNote`.
- **Output:** one query fetches notes and their authors; a DataLoader makes it a constant number of DB queries.
- **Check:** log SQL statements while fetching 50 notes with authors.

Card 6.D: live notifications with SSE

- **Aim:** server push without WebSocket.
- **Input:** `GET /events` with `Content-Type: text/event-stream`; a `POST /notes` that emits an event.
- **Output:** an open `curl -N localhost:3000/events` terminal prints a line each time you create a note in another terminal.
- **Check:** kill and restart the server: the browser `EventSource` reconnects by itself.

Card 6.E: WebSocket chat room

- **Aim:** two-way real time.
- **Input:** Socket.IO server with `join(room)` and `message(room, text)` events and a token check in the handshake.
- **Output:** two browser tabs in the same room see each other's messages; a tab with a bad token is refused.
- **Check:** open three tabs, two in room A and one in room B; the third receives nothing from room A.

Self-check questions: Why can GraphQL be harder to cache than REST? When is SSE better than WebSocket? How does an idempotency key protect a payment retry?

6.6 Deep dive: designing and evolving REST APIs

Contract first or code first. *Contract first*: write the OpenAPI document by hand, review it with clients, generate server stubs and types. *Code first*: define schemas in code (Zod, TypeBox, Nest decorators) and generate the OpenAPI document. Code first keeps validation and docs from drifting; contract first suits public APIs with many consumers. Either way, **the spec is tested**, not decorative (Card 3.J).

Versioning and change rules

Change	Safe in the same version?

Change	Safe in the same version?
Add an optional request field, add a response field, add an endpoint, add an enum value on output (warn clients to tolerate unknown values)	Yes
Remove or rename a field, change a type, make an optional field required, change meaning, tighten validation	No: new version
Change a status code or error code that clients branch on	No

Version with a path prefix (`/v1`), or with a date header (the Stripe approach: `Api-Version: 2026-10-01`) if you want fine-grained evolution. Run old and new versions side by side, announce deprecations with the `Deprecation` and `Sunset` headers plus a changelog, and log which clients still call old versions before turning them off. Be conservative in what you send and tolerant in what you accept.

Listing, filtering, and search conventions. `?filter[status]=paid&filter[createdAfter]=...`, `?sort=-createdAt,id`, `?fields=id,title` (sparse fields), `?include=author` (embed relations on request). Always add a unique tiebreaker (`id`) to every sort used with cursor pagination or pages will skip or repeat rows. Encode a cursor as an opaque base64 string holding the last sort values, and sign or validate it so clients cannot craft arbitrary ones.

Long-running operations. Do not hold an HTTP request open for minutes. `POST /reports` returns `202 Accepted` with `Location: /reports/abc` and a status resource (`pending`, `running`, `succeeded`, `failed`, with `result` or `error`). Clients poll with `Retry-After` hints, or you also call a webhook or push an SSE event when done.

HTTP caching for APIs. `Cache-Control: private, max-age=0, must-revalidate` plus an `ETag` for user-specific data (conditional GETs return `304`); `public, max-age=60, stale-while-revalidate=300` for shared data; always send `Vary` for headers that change the response (`Accept-Encoding`, `Authorization`, `Accept-Language`). A cache that ignores `Vary` can serve one user's response to another.

Compression and payload size. Gzip or brotli at the proxy for JSON, but do not compress already compressed files. Return only fields clients need, and use `HEAD` and conditional requests for big resources.

SDKs and clients. Generate typed clients from OpenAPI (`openapi-typescript` with `openapi-fetch`, `orval`, `@hey-api/openapi-ts`, or Kiota for many languages) instead of hand-writing them, and publish them from CI so server and client types change together.

API gateway responsibilities. Authentication offload, rate limiting, request/response logging, routing, TLS, and canary routing. Options: Kong, Traefik, Envoy-based gateways, AWS API Gateway, Cloudflare. Keep business logic out of the gateway.

6.7 Deep dive: GraphQL in practice

- **Schema design.** Design around the domain, not the database. Use specific mutation inputs and payloads (`createNote(input: CreateNoteInput!): CreateNotePayload!`) where the

payload carries both the object and typed `userErrors` for expected failures; reserve the top-level `errors` array for unexpected failures. Use non-null (`!`) deliberately: a failing non-null field nulls out its parent.

- **Pagination.** Relay-style connections (`edges { node cursor } pageInfo { hasNextPage endCursor }`) are the common standard; they pair naturally with keyset pagination in the database.
- **N+1 and DataLoader.** A resolver for `Post.author` runs once per post. A `DataLoader` collects all `author` ids requested in the same tick and loads them with one `WHERE id = ANY($1)` query, and caches per request. Create loaders **per request** (never global) so caches do not leak between users.
- **Authorization** belongs in the resolvers or, better, the service layer they call, not only at the top-level field; a nested field can otherwise expose data (`user { invoices { ... } }`).
- **Query cost control.** Public GraphQL endpoints must limit **depth**, **complexity** (assign a cost per field and a maximum per query), and **timeouts**; disable introspection in production if the API is private, and prefer **persisted queries** (clients send an id of a pre-approved query, which also saves bandwidth and allows allow-listing).
- **Caching.** POST requests are not cacheable by default, so use persisted queries over GET with CDN caching, or cache at the resolver and DataLoader level. Normalised client caches (Apollo, urql, Relay) need stable ids on objects.
- **Subscriptions** run over WebSocket (`graphql-ws`) or SSE; they carry the same scaling issues as any real-time channel (6.9).
- **Federation.** Apollo Federation, GraphQL Mesh, or WunderGraph compose several GraphQL services behind one graph; adopt only when separate teams own parts of the schema.
- **Typing.** `GraphQL Code Generator` produces types for resolvers and clients, so schema changes break compilation rather than production.
- **When REST is better:** simple CRUD, public APIs that need CDN caching and easy rate limiting, file transfer, and teams without GraphQL experience.

6.8 Deep dive: tRPC and gRPC

tRPC. Server procedures (`query` and `mutation`) are plain TypeScript functions with a Zod input schema; the client imports only the *type* of the router, so calling `trpc.note.byId.query({ id })` is fully typed with no code generation. Middleware gives you auth and logging; adapters exist for Express, Fastify, Next.js, and others. Limits: both ends must be TypeScript and share types (monorepo), the wire format is internal and unversioned, and it is not meant for third-party consumers. Alternatives with the same idea: `ts-rest` (contract plus real REST routes, OpenAPI output), `oRPC`, Hono RPC, `zodios`.

gRPC. You define services in `.proto` files; tooling generates clients and servers for many languages; messages are compact Protobuf over HTTP/2.

Call type	Shape	Example
Unary	One request, one response	`GetUser`
Server streaming	One request, a stream of responses	Progress updates, log tail
Client streaming	A stream of requests, one response	Upload chunks

Call type	Shape	Example
Bidirectional streaming	Both sides stream	Chat, live sync

Key practices: always set a **deadline** on every call (gRPC propagates it so downstream work is cancelled); use the standard **status codes** (`NOT_FOUND`, `INVALID_ARGUMENT`, `UNAVAILABLE`, `DEADLINE_EXCEEDED`, `ALREADY_EXISTS`) rather than inventing your own; evolve schemas by only adding fields with new numbers and never reusing or renumbering old ones (use `reserved`); lint and break-check schemas in CI with **Buf**. Browsers cannot speak native gRPC, so use gRPC-Web with a proxy or the **Connect** protocol (`@connectrpc/connect`), which speaks gRPC, gRPC-Web, and plain HTTP/JSON from one handler. Load balancing needs care: a long-lived HTTP/2 connection pins all calls to one backend, so use a gRPC-aware balancer (Envoy, Linkerd) or client-side balancing.

6.9 Deep dive: real-time transports in depth

Transport	Direction	Reconnect and resume	Works through proxies	Best for
Short polling	Client asks repeatedly	Trivial	Always	Low-frequency status checks
Long polling	Client asks, server holds until data	Trivial	Usually	Fallback, simple notifications
SSE	Server to client	Built in (`Last-Event-ID`)	Mostly (disable proxy buffering)	Feeds, progress, streaming text
WebSocket	Both	You implement it	Mostly (needs upgrade support)	Chat, collaboration, games
WebTransport / HTTP/3	Both, multiple streams	Emerging	Limited	Low-latency media, future use
MQTT over WebSocket	Pub/sub	Built in, QoS levels	Mostly	IoT, many small devices

SSE specifics. Send `Content-Type: text/event-stream`, `Cache-Control: no-cache`, and `X-Accel-Buffering: no` (for Nginx). Each event is `id: 42\nevent: note\ndata: {...}\n\n`. After a disconnect the browser reconnects with `Last-Event-ID`, so keep a short replay buffer (Redis stream) to resend missed events. Send a comment line (`: ping`) every 15 to 30 seconds to keep intermediaries from closing the connection. Browsers limit about 6 connections per host over HTTP/1.1, so use HTTP/2.

WebSocket specifics

- **Authenticate during the handshake** (a cookie, or a short-lived ticket in the query string obtained from an authenticated HTTP call; avoid long-lived tokens in URLs because they end up in logs). Re-check permissions on every message, not only at connect.

- **Heartbeats.** Send ping frames and close sockets that miss a few pongs; mobile networks drop connections silently.
- **Backpressure.** `ws.send()` queues in memory; check `socket.bufferedAmount` and drop, slow, or disconnect clients that cannot keep up, or one slow client can exhaust server memory.
- **Message protocol.** Use a typed envelope `{ type, id, payload }`, validate every incoming message with a schema, limit `maxPayload`, and rate limit per connection. Add acknowledgements and per-message ids if you need delivery guarantees.
- **Reconnection.** Clients reconnect with exponential backoff and jitter (otherwise a server restart triggers a thundering herd), then **resume** by sending the last event id or timestamp they saw so the server replays the gap from storage.
- **Scaling out.** Connections are stateful, so you need (a) a pub/sub backbone so any instance can reach any user (Redis pub/sub or Streams, NATS, Kafka), (b) a way to find which instance holds which user if you must target them (a Redis hash of user to instance), (c) graceful deploys (send a `reconnect` message and drain), and (d) a per-instance connection limit with autoscaling on connection count. One Node process can hold tens of thousands of idle sockets, limited by memory, file descriptors (`ulimit -n`), and your message rate.
- **Rooms and presence.** Rooms are just sets of connection ids per channel; presence is a TTL key per user refreshed by heartbeats.

Webhooks (you send them). Sign each request (`HMAC-SHA256` of timestamp plus body, sent in a header with the timestamp so receivers can reject replays older than a few minutes), include a unique event id, retry with exponential backoff for up to a few days, treat only `2xx` as success with a short timeout (5 to 10 seconds), disable endpoints that fail for days, offer a delivery log and a manual "resend" button, and warn that events can arrive duplicated and out of order. Services such as Svix or Hookdeck provide this if you do not want to build it.

6.10 More build cards

Card 6.F: OpenAPI-first with generated types

- **Aim:** one source of truth for contracts.
- **Input:** an `openapi.yaml` describing `/notes` endpoints.
- **Output:** generated TypeScript types used by both the server handlers and a typed client (`openapi-typescript` + `openapi-fetch`); a request with a wrong field type fails at compile time.
- **Check:** change a field in the YAML, regenerate, and see compile errors in both server and client.

Card 6.G: cursor pagination with a stable sort

- **Aim:** no skipped or repeated rows.
- **Input:** 10,000 notes with many identical `createdAt` values; `?limit=50&after=<cursor>&sort=-createdAt`.
- **Output:** an opaque signed cursor that encodes `(createdAt, id)`; paging through everything returns each row exactly once even while new rows are inserted.
- **Check:** a script that walks all pages while another process inserts rows and verifies there are no duplicates and no gaps among the originally existing rows.

Card 6.H: long-running job resource

- **Aim:** the `202 Accepted` pattern.
- **Input:** `POST /exports`, `GET /exports/:id`.
- **Output:** immediate `202` with `Location`; the status moves `pending` to `running` to `succeeded` with a download URL; a failure shows an `error` object.
- **Check:** poll in a loop honouring `Retry-After`; kill the worker mid-job and confirm the job is retried and finishes.

Card 6.I: DataLoader batching

- **Aim:** prove N+1 is gone.
- **Input:** the GraphQL schema from Card 6.C, with posts, authors, and comments.
- **Output:** a query for 50 posts with authors and comment counts runs 3 SQL statements total, shown by query logs.
- **Check:** compare statement counts with and without DataLoader; add a depth limit of 6 and confirm a deeper query is rejected.

Card 6.J: gRPC service with deadlines

- **Aim:** typed service-to-service calls.
- **Input:** a `.proto` with `GetNote` and a server-streaming `WatchNotes`; a client that sets a 500 ms deadline.
- **Output:** the server handles `DEADLINE_EXCEEDED` by cancelling its database query; invalid ids return `INVALID_ARGUMENT`.
- **Check:** add a 2 second sleep on the server and confirm the client fails at 500 ms and the server logs the cancellation.

Card 6.K: SSE with replay

- **Aim:** lossless server push.
- **Input:** a Redis stream of events, `GET /events` that reads `Last-Event-ID` and replays missed entries before streaming live.
- **Output:** stop the server for 10 seconds while events are produced, restart, and the browser receives the missed events in order.
- **Check:** a browser `EventSource` page that logs event ids; verify there are no gaps.

Card 6.L: WebSocket hardening

- **Aim:** survive slow and hostile clients.
- **Input:** a `ws` server with ticket-based auth, 64 KB `maxPayload`, a rate limit of 20 messages per second, ping/pong every 20 seconds, and a `bufferedAmount` guard.
- **Output:** a client that never reads is disconnected before server memory grows past a limit; a client sending 1,000 messages per second is throttled then closed with code 1008.
- **Check:** a test client for each behaviour and a memory graph during the slow-client case.

Card 6.M: signed webhooks, both directions

- **Aim:** build the sender and the receiver.

- **Input:** a sender that posts events with `X-Signature: t=...,v1=...` and retries on non-`2xx`; a receiver that verifies the HMAC over the raw body and rejects timestamps older than 5 minutes.
- **Output:** valid events are processed once (deduplicate on event id); a replayed old request and a tampered body are rejected.
- **Check:** simulate receiver downtime and watch retries with backoff; then restore and confirm exactly one effect per event.

Deep-dive self-check: (1) Which API changes are breaking? (2) Why must DataLoader instances be created per request? (3) What does a gRPC deadline do that a client-side timeout alone does not? (4) How does SSE resume after a disconnect? (5) What happens to a Node server if it keeps calling `ws.send()` to a client that does not read? (6) Why sign and timestamp webhooks?

Part 6 deep dive

6.6 REST details that production APIs need

Modelling actions. Not everything is CRUD. Model an action as a sub-resource or a state change: `POST /orders/42/cancel`, `POST /users/7/password-reset`, or `PATCH /orders/42 { status: 'cancelled' }` with the server enforcing allowed transitions. Pick one convention and apply it everywhere.

HTTP caching headers.

Header	Meaning
`Cache-Control: public, max-age=300`	Any cache may reuse for 5 minutes
`Cache-Control: private`	Only the user's browser, never a shared CDN (use for per-user data)
`Cache-Control: no-store`	Never store (sensitive responses)
`Cache-Control: no-cache`	Store, but revalidate before each use
`ETag` + `If-None-Match`	Revalidation: server answers `304 Not Modified` with no body when unchanged
`Vary: Accept-Encoding, Authorization`	The response differs by these request headers; caches must key on them

Get `Vary` wrong and a CDN will serve one user's response to another. Express generates weak ETags for responses by default; Fastify has `@fastify/etag`; for database rows, derive the ETag from `version` or `updated_at` so a `304` is decided **before** doing the expensive work.

Keyset pagination with a composite cursor. A cursor over a non-unique column such as `created_at` repeats or skips rows unless you add a unique tie-breaker:

```
SELECT id, title, created_at FROM notes
WHERE ($1::timestamptz IS NULL OR (created_at, id) < ($1, $2))   -- row comparison,
uses the composite index
ORDER BY created_at DESC, id DESC
```

```
LIMIT $3;                                                    -- ask for limit +
1 to know if there is a next page
```

```
const enc = (c: { t: string; id: string }) =>
Buffer.from(JSON.stringify(c)).toString('base64url');
const dec = (s: string) => JSON.parse(Buffer.from(s, 'base64url').toString());

export async function listNotes(limit: number, after?: string) {
  const cur = after ? dec(after) : null;                    // validate the decoded
shape with Zod!
  const { rows } = await pool.query(SQL, [cur?.t ?? null, cur?.id ?? null, limit +
1]);
  const page = rows.slice(0, limit);
  const last = page.at(-1);
  return { data: page, nextCursor: rows.length > limit && last ? enc({ t:
last.created_at.toISOString(), id: last.id }) : null };
}
```

Support the index with `(created_at DESC, id DESC)`. Avoid returning an exact `total` for big tables (`count(*)` scans); offer an estimate or omit it.

Idempotency keys implemented. A table makes retries safe:

```
CREATE TABLE idempotency_keys (
  user_id uuid NOT NULL, key text NOT NULL, request_hash text NOT NULL,
  response_status int, response_body jsonb, created_at timestamptz NOT NULL DEFAULT
now(),
  PRIMARY KEY (user_id, key)
);
```

```
export async function withIdempotency(req: Req, handler: () => Promise<{ status:
number; body: unknown }>) {
  const key = req.get('Idempotency-Key');
  if (!key) return handler();
  const hash = sha256(JSON.stringify(req.body));
  const ins = await pool.query(
    'INSERT INTO idempotency_keys (user_id, key, request_hash) VALUES ($1,$2,$3) ON
CONFLICT DO NOTHING', [req.user.id, key, hash]);
  if (ins.rowCount === 0) {                                          // seen before
    const { rows: [row] } = await pool.query('SELECT * FROM idempotency_keys WHERE
user_id=$1 AND key=$2', [req.user.id, key]);
    if (row.request_hash !== hash) throw new AppError(422, 'IDEMPOTENCY_KEY_REUSED',
'Same key, different request');
    if (row.response_status === null) throw new AppError(409, 'REQUEST_IN_PROGRESS',
'Retry shortly');
    return { status: row.response_status, body: row.response_body };     // replay
the stored response
  }
  const result = await handler();
  await pool.query('UPDATE idempotency_keys SET response_status=$3, response_body=$4
WHERE user_id=$1 AND key=$2',
    [req.user.id, key, result.status, result.body]);
  return result;
}
```

Edge case: a crash between the handler and the update leaves the key "in progress". Add a lock timestamp and let a retry take over after a timeout, and where possible write the business change and the key's response in the **same database transaction**.

6.7 OpenAPI as the contract

Two workflows: **code-first** (schemas in code generate the spec: `@fastify/swagger`, `@nestjs/swagger`, `zod-to-openapi`) keeps spec and code in sync automatically; **spec-first** (write `openapi.yaml`, then generate types and server stubs) suits teams that agree on the contract before building. Tools around the spec:

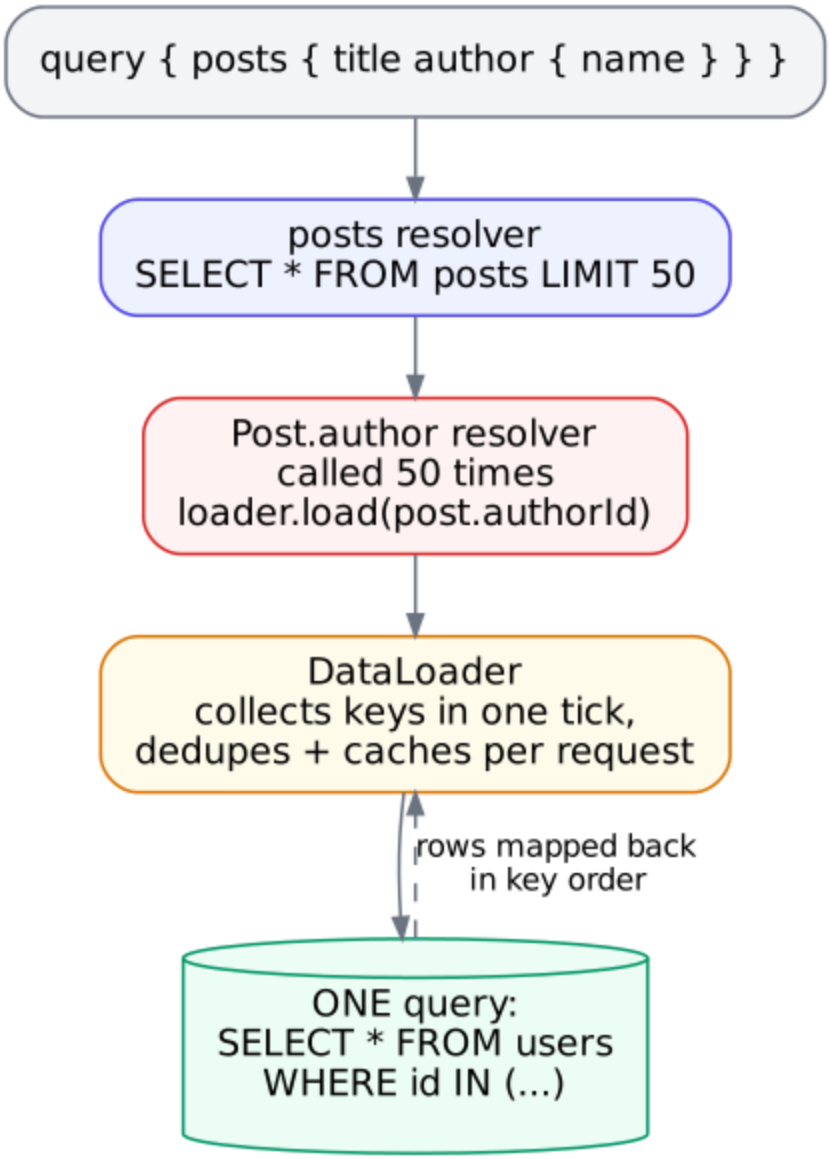
Job	Tools
TypeScript types or typed client from a spec	`openapi-typescript`, `orval`, `openapi-generator`
Documentation UI	Swagger UI, Redoc, Scalar
Mock server for frontend work	Prism, MSW
Lint the spec for style problems	Spectral, Redocly CLI
Detect breaking changes in CI	`oasdiff`, Optic
Validate real traffic against the spec	`express-openapi-validator`, Fastify schemas

Breaking changes include removing a field, making an optional field required, narrowing an enum, and changing a type. Run a breaking-change check in CI so a pull request cannot silently break clients.

6.8 GraphQL in depth

Schema design. Fields are nullable by default (so one failing field does not null the whole response); mark `!` only when you can guarantee it. Use **input types** for mutations and return a payload type (`createNote(input: CreateNoteInput!): CreateNotePayload!`) that can carry both the result and domain errors (as a union such as `CreateNoteResult = Note | ValidationError`), instead of relying only on the top-level `errors` array. Paginate lists with the **connection** pattern (`edges { node cursor } pageInfo { hasNextPage endCursor }`).

N+1 and DataLoader. A resolver runs per field per item, so `posts { author { name } }` over 50 posts calls the author resolver 50 times. DataLoader collects the keys requested in one tick, issues **one** batched query, and caches per request:




```
import DataLoader from 'dataloader';

export const makeLoaders = () => ({
  userById: new DataLoader<string, User>(async (ids) => {
    const rows = await db.select().from(users).where(inArray(users.id, [...ids]));
    const byId = new Map(rows.map((u) => [u.id, u]));
    return ids.map((id) => byId.get(id) ?? new Error(`user ${id} not found`));   //
MUST return results in key order
  }),
});
// Create loaders PER REQUEST in the GraphQL context, never globally:
// a global loader caches across users and leaks data between them.
```

```
// resolver: Post: { author: (post, _args, ctx) =>
ctx.loaders.userById.load(post.authorId) }
```

Abuse protection. A client can write an expensive query (deeply nested, or asking for huge lists). Defend with **depth limits**, **complexity (cost) analysis**, pagination caps, **persisted queries** (only pre-registered query ids allowed, which also shrinks requests), timeouts, and per-user rate limits by cost. Libraries: `graphql-armor`, `graphql-depth-limit`, `graphql-query-complexity`. Decide deliberately whether to expose introspection in production. **Subscriptions** use the `graphql-ws` protocol over WebSocket. **Federation** (Apollo Federation, GraphQL Mesh) composes several services into one graph; adopt it only when multiple teams own parts of the schema. Caching is harder than REST because everything is a `POST` to one URL: use persisted queries with `GET`, response caching keyed by query + variables + user, and per-field caching in resolvers.

6.9 tRPC in depth

tRPC exports your router **type** to the client, so calling `trpc.noteById.query({ id })` is checked end to end with no code generation.

```
import { initTRPC, TRPCError } from '@trpc/server';
import { z } from 'zod';

const t = initTRPC.context<{ user: AuthUser | null; notes: NotesService
}>().create();
const authed = t.procedure.use(({ ctx, next }) => {
  if (!ctx.user) throw new TRPCError({ code: 'UNAUTHORIZED' });
  return next({ ctx: { ...ctx, user: ctx.user } });           // narrows ctx.user to
non-null downstream
});

export const appRouter = t.router({
  noteById: t.procedure.input(z.object({ id: z.string().uuid() })).query(({ input,
ctx }) => ctx.notes.get(input.id)),
  createNote: authed.input(createNote).mutation(({ input, ctx }) =>
ctx.notes.create(ctx.user.id, input)),
});
export type AppRouter = typeof appRouter;       // the ONLY thing the client imports
```

It shines in a TypeScript monorepo (Next.js + a Node server). Limits: only TypeScript clients benefit, the contract is your internal types (so it is not a stable public API unless you version carefully), and you give up HTTP-level conventions like cache headers and REST tooling. Alternatives with the same idea but an OpenAPI output: `ts-rest`, oRPC, Hono RPC.

6.10 gRPC in depth

A `.proto` file defines messages and services; code is generated (or loaded dynamically) for any language. Four call shapes: **unary** (request, response), **server streaming**, **client streaming**, and **bidirectional streaming**.

```
syntax = "proto3";
package notes;
service Notes {
  rpc GetNote (GetNoteRequest) returns (Note);
  rpc WatchNotes (WatchRequest) returns (stream Note);       // server streaming
}
```

```
message GetNoteRequest { string id = 1; }
message WatchRequest {}
message Note { string id = 1; string title = 2; }
```

```
import * as grpc from '@grpc/grpc-js';
import * as protoLoader from '@grpc/proto-loader';

const def = protoLoader.loadSync('notes.proto', { keepCase: true, longs: String,
defaults: true });
const pkg = grpc.loadPackageDefinition(def) as any;
const server = new grpc.Server();
server.addService(pkg.notes.Notes.service, {
  GetNote: (call: any, cb: any) => cb(null, { id: call.request.id, title: 'hello' }),
});
server.bindAsync('0.0.0.0:50051', grpc.ServerCredentials.createInsecure(), (err) => {
if (err) throw err; });
```

Things to master: always set a **deadline** on the client call (a deadline propagates to downstream calls); use gRPC **status codes** (`NOT_FOUND`, `INVALID_ARGUMENT`, `UNAVAILABLE`, `DEADLINE_EXCEEDED`) rather than HTTP ones; **metadata** is the header equivalent (auth tokens, trace ids); **interceptors** are the middleware; **never reuse or renumber a field number** (add new fields, mark old ones `reserved`) to stay wire compatible. Because gRPC holds long-lived HTTP/2 connections, a layer-4 load balancer pins all of a client's calls to one backend; use an HTTP/2 aware proxy (Envoy) or client-side load balancing. Browsers cannot speak native gRPC: use **Connect** (`@connectrpc/connect`) or gRPC-Web with a proxy. The standard gRPC health-checking protocol plugs into Kubernetes probes.

6.11 Webhooks you send

Outgoing webhooks need a security and reliability design of their own:

- **Sign** each delivery so receivers can verify it came from you and was not altered: HMAC-SHA256 over `timestamp + '.' + rawBody` with a per-endpoint secret, sent in a header such as `X-Signature: t=1760000000,v1=<hex>`. Receivers reject timestamps older than about 5 minutes (replay protection).
- **Retry** with exponential backoff and jitter for non-2xx or timeouts (for example 1 min, 5 min, 30 min, 2 h, then give up and mark the endpoint unhealthy); keep a **delivery log** with attempts, status, and response snippet so customers can debug and manually replay.
- Include a stable **event id** so receivers can deduplicate. Do not promise ordering; include a timestamp or sequence number instead.
- Timeout the call (a few seconds), run deliveries from a **queue** (Part 8), never inline in the request that caused the event, and protect yourself from **SSRF** by validating destination URLs (no private IP ranges, no redirects).

```
import { createHmac, timingSafeEqual } from 'node:crypto';

export const sign = (secret: string, ts: number, rawBody: string) =>
  `t=${ts},v1=${createHmac('sha256',
secret).update(`${ts}.${rawBody}`).digest('hex')}`;

export function verify(secret: string, header: string, rawBody: string, toleranceSec
= 300) {
```

```
  const parts = Object.fromEntries(header.split(',').map((p) => p.split('=') as
[string, string]));
  if (Math.abs(Date.now() / 1000 - Number(parts.t)) > toleranceSec) return false;
// too old or from the future
  const expected = createHmac('sha256',
secret).update(`${parts.t}.${rawBody}`).digest();
  const given = Buffer.from(parts.v1 ?? '', 'hex');
  return given.length === expected.length && timingSafeEqual(given, expected);
// constant-time compare
}
```

6.12 WebSocket in depth

- **Authenticate the handshake.** Browsers cannot set an `Authorization` header on `new WebSocket(url)`. Avoid putting a long-lived token in the URL query string (it ends up in logs). Use a cookie, or request a **short-lived one-time ticket** over normal HTTPS and send it as the first message or query parameter, then validate it and close with a code if invalid. Re-check authorization for each room join or action, not only at connect.
- **Heartbeats.** Dead connections (laptop lid closed, mobile network dropped) look open to the server. Ping periodically and terminate sockets that did not answer.
- **Backpressure.** `ws.send` buffers in memory for slow clients; check `ws.bufferedAmount` and drop or disconnect clients that fall too far behind.
- **Limits.** Set `maxPayload`, a per-connection message rate limit, and a cap on connections per user.
- **Protocol design.** Wrap messages in an envelope `{ type, id, payload }` so you can add types, correlate request/response with `id`, and version.
- **Delivery semantics.** A socket gives **at-most-once** delivery: a message sent while the client is reconnecting is lost. For anything important, persist events with increasing ids and let the client resume with the last id it saw.
- **Reconnection (client).** Exponential backoff **with jitter** (so 10 000 clients do not reconnect in the same instant after a deploy).

```
import { WebSocketServer, WebSocket } from 'ws';
const wss = new WebSocketServer({ server, maxPayload: 64 * 1024 });
const alive = new WeakSet<WebSocket>();

wss.on('connection', (ws, req) => {
  // validate ticket from req.url or a cookie here; ws.close(4401, 'unauthorized') on
failure
  alive.add(ws);
  ws.on('pong', () => alive.add(ws));
  ws.on('message', (raw) => {
    let msg: { type: string; id?: string; payload?: unknown };
    try { msg = JSON.parse(raw.toString()); } catch { return ws.close(1003, 'bad
json'); }
    // validate msg with Zod, then dispatch by msg.type
  });
});

setInterval(() => {
  for (const ws of wss.clients) {
    if (!alive.has(ws)) { ws.terminate(); continue; }
    alive.delete(ws);          // must answer the next ping to stay
```

```
    ws.ping();
  }
}, 30_000).unref();
```

6.13 Server-Sent Events in depth

SSE is plain HTTP with `Content-Type: text/event-stream`. Each message is lines of `id:`, `event:`, `data:` ended by a blank line; `retry:` sets the reconnect delay. The browser's `EventSource` reconnects automatically and sends the `Last-Event-ID` header so you can **resume** from where it left off (a feature you would otherwise build yourself over WebSocket).

```
app.get('/events', requireAuth, async (req, res) => {
  res.writeHead(200, {
    'Content-Type': 'text/event-stream', 'Cache-Control': 'no-cache, no-transform',
    Connection: 'keep-alive', 'X-Accel-Buffering': 'no',          // tell Nginx not
to buffer the stream
  });
  const lastId = Number(req.get('Last-Event-ID') ?? 0);
  for (const e of await events.since(req.user.id, lastId)) res.write(`id:
${e.id}\nevent: ${e.type}\ndata: ${JSON.stringify(e.data)}\n\n`);

  const unsubscribe = bus.subscribe(req.user.id, (e) => res.write(`id:
${e.id}\nevent: ${e.type}\ndata: ${JSON.stringify(e.data)}\n\n`));
  const ping = setInterval(() => res.write(': ping\n\n'), 25_000);       // comment
line keeps proxies from closing the idle stream
  req.on('close', () => { clearInterval(ping); unsubscribe(); });
});
```

Notes: over HTTP/1.1 browsers allow only about six connections per host, which limits several tabs; HTTP/2 removes that. Compression middleware can buffer a stream, so exclude this route. SSE only flows server to client; send commands with normal `POST` requests. For streaming AI-style responses with a `POST` body, use `fetch` with a readable stream (`response.body.getReader()`) because `EventSource` only does `GET`.

6.14 More build cards

Card 6.F: composite-cursor pagination

- **Aim:** stable pages under concurrent inserts.
- **Input:** 10 000 notes, many with identical `created_at` values (seed them on purpose).
- **Output:** paging with the (created_at, id) cursor returns every row exactly once; a naive `created_at`-only cursor visibly skips or repeats rows.
- **Check:** a script that pages to the end and compares the set of ids with the table.

Card 6.G: idempotency table

- **Aim:** implement 6.6 completely.
- **Input:** `POST /v1/payments` with `Idempotency-Key`.
- **Output:** 10 parallel identical requests create one payment and return the same body; a changed body with the same key gets 422; a crash mid-request is recoverable after the lock timeout you add.
- **Check:** parallel `curl` or a small script; kill the process in the middle once.

Card 6.H: DataLoader proof

- **Aim:** measure the N+1 fix.
- **Input:** GraphQL `posts { title author { name } }` on 50 posts.
- **Output:** a query counter shows 51 queries without DataLoader and 2 with it; a second request does not reuse the first request's cache.
- **Check:** log SQL statements per request and assert the counts in a test.

Card 6.I: signed webhook sender and receiver

- **Aim:** secure outgoing events.
- **Input:** the `sign`/`verify` helpers above, a BullMQ delivery job with backoff, and a receiver endpoint.
- **Output:** a tampered body, a wrong secret, and a 10-minute-old timestamp are all rejected; a receiver that returns 500 twice then 200 receives the event on the third attempt.
- **Check:** unit tests for the three rejections and an integration test for the retry.

Card 6.J: resilient WebSocket client

- **Aim:** survive server restarts.
- **Input:** a browser or Node client with reconnect (exponential backoff plus jitter) and a `lastEventId` sent on reconnect.
- **Output:** restart the server during a stream; the client reconnects and receives the events it missed exactly once.
- **Check:** run a counter producer, restart the server, assert there are no gaps or duplicates.

Card 6.K: SSE with resume

- **Aim:** use `Last-Event-ID`.
- **Input:** the SSE handler above and an `events` table with an increasing id.
- **Output:** kill and restart the server; `curl -N -H 'Last-Event-ID: 41'` replays events 42 onward.
- **Check:** compare the ids received before and after a forced disconnect.

6.15 Answer key for the Part 6 self-checks

- **Why can GraphQL be harder to cache than REST?** Requests are usually `POST`s to one URL with varying bodies, so HTTP and CDN caching by URL does not apply, and one query mixes data with different freshness and permissions; you cache with persisted queries over `GET` and per-field or per-entity caches instead.
- **When is SSE better than WebSocket?** When only the server pushes, you want automatic reconnect with resume, and you want to stay on plain HTTP (works through proxies, simpler auth and scaling).
- **How does an idempotency key protect a payment retry?** The server stores the first result under the key; a retry with the same key and the same request returns the stored result instead of charging again, and a retry with a different request is rejected.

6.6 Deep dive: REST details that separate good APIs from fragile ones

HTTP caching and conditional requests. `Cache-Control: public, max-age=60` lets browsers and CDNs reuse a response; `private` limits it to the user's browser; `no-store` forbids storing (use for

sensitive data); `stale-while-revalidate=30` serves stale data while refreshing in the background. An `ETag` (a fingerprint of the representation) lets clients revalidate cheaply: the client sends `If-None-Match`, the server answers `304 Not Modified` with no body. The same `ETag` also powers **optimistic concurrency** on writes via `If-Match` (reply `412 Precondition Failed` when it no longer matches). Add `Vary: Accept-Encoding` (and `Origin` when CORS differs) so caches do not mix variants.

Pagination styles compared

Style	Request	Pros	Cons
Offset	`?page=3&limit=20`	Easy, allows jumping to a page	Slow on deep pages; items shift when rows are inserted
Keyset (cursor)	`?after=<opaque>&limit=20`	Fast and stable at any depth	No random page jump; sort key must be unique (add `id` as tie-breaker)
Time window	`?since=2026-10-01T00:00:00Z`	Natural for sync and feeds	Needs a reliable timestamp index

Make cursors **opaque** (base64 of the last sort values) so you can change the internals. Optionally also send a `Link: <...>; rel="next"` header. Never return an unbounded list: enforce a maximum `limit`.

Filtering, sorting, and field selection. Use an allow-list of filterable and sortable fields (never pass user input as a column name), simple operators in the query (`?price[gte]=10` or `?price_gte=10`), and optional sparse fields (`?fields=id,title`) to cut payload size. Document every parameter in OpenAPI.

Idempotency keys, implemented. The `Idempotency-Key` header makes a `POST` safe to retry. Algorithm:

1. Hash the request (method, path, body) and try to insert `(key, user_id, request_hash, status='in_progress')` into an `idempotency_keys` table with a **unique constraint** on `(user_id, key)` and `ON CONFLICT DO NOTHING`.
2. **Inserted:** run the operation, then store the response status and body and set `status='done'` (ideally in the same transaction as the business change).
3. **Conflict, same hash, `done`:** replay the stored response and add a header such as `Idempotent-Replayed: true`.
4. **Conflict, same hash, `in_progress`:** answer `409` (or wait briefly) because the first request is still running.
5. **Conflict, different hash:** answer `422`; the key was reused for a different request.
6. Delete old keys after 24 hours.

Long-running operations. Do not hold a request open for minutes. Return `202 Accepted` with a `Location: /v1/exports/abc` header; the client polls that resource (`status: queued, running, succeeded, failed`) or receives a webhook or SSE event when finished. Pair with a queue (Part 8).

Versioning strategies

Strategy	Example	Note

Strategy	Example	Note
Path	`/v1/notes`	Most common, simple to route and cache
Header	`Accept: application/vnd.myapp.v2+json`	Clean URLs, harder to test in a browser
Date-based	`Stripe-Version: 2026-10-01`	Fine-grained, needs per-version transformers
No versions, only additive changes		Works for internal APIs with disciplined rules

Backward-compatible changes: new optional fields, new endpoints, new enum values (only if clients tolerate unknown values). Breaking: removing or renaming fields, changing types or meaning, tightening validation. Announce deprecation with `Deprecation` and `Sunset` headers and a changelog, and log which clients still call old versions before removal.

OpenAPI generated from your schemas (Fastify with Zod; check the current docs for exact option names):

```
import swagger from '@fastify/swagger';
import swaggerUi from '@fastify/swagger-ui';
import { serializerCompiler, validatorCompiler, jsonSchemaTransform } from 'fastify-type-provider-zod';

app.setValidatorCompiler(validatorCompiler);
app.setSerializerCompiler(serializerCompiler);
await app.register(swagger, { openapi: { info: { title: 'Notes API', version: '1.0.0' } }, transform: jsonSchemaTransform });
await app.register(swaggerUi, { routePrefix: '/docs' });
```

Schema-first (write the OpenAPI file, generate server stubs and clients with `openapi-typescript`, `orval`, or `openapi-generator`) and code-first (derive the spec from code, as above) both work. Whichever you choose, add a CI check (`oasdiff`, Spectral) that flags breaking changes and lint violations.

6.7 Deep dive: GraphQL in production

Schema design rules. Model the domain, not your tables. Use `input` types for mutations and return a payload type (`createNote(input: CreateNoteInput!): CreateNotePayload!`) so you can add fields later. Use **connections** (`edges`, `node`, `pageInfo`, cursors, as in the Relay specification) for lists. Make fields nullable unless you can always guarantee them (a failing field nulls its nearest nullable parent). Model expected failures as union results (`CreateNoteResult = Note | ValidationError`) and keep the top-level `errors` array for unexpected failures.

The N+1 problem and DataLoader. A resolver for `Note.author` runs once per note. DataLoader batches all `load(id)` calls made in the same tick into one query. **Create loaders per request** (in the context) so caches never cross users.

```
import DataLoader from 'dataloader';

export const makeLoaders = (db: Db) => ({
```

```
  userById: new DataLoader<string, User>(async (ids) => {
    const rows = await db.select().from(users).where(inArray(users.id, [...ids]));
    const byId = new Map(rows.map((u) => [u.id, u]));
    return ids.map((id) => byId.get(id) ?? new Error('user not found'));   // same
order as ids
  }),
});

const resolvers = {
  Note: { author: (note: Note, _args: unknown, ctx: Ctx) =>
ctx.loaders.userById.load(note.authorId) },
};
```

Protecting a GraphQL endpoint. Because clients write the query, one request can ask for something enormous. Defences: **depth limit**, **query cost / complexity analysis**, request size and timeout limits, **persisted queries** (the server only runs pre-registered queries, which also enables GET caching), pagination limits on every list, rate limiting by cost not by request count, authentication in the context and **authorization in resolvers or a schema directive** (never only in the UI), and disabling introspection or GraphiQL for untrusted clients in production if the API is not public.

Other facts: subscriptions use WebSocket or SSE (`graphql-ws`, GraphQL Yoga's SSE support); HTTP caching is limited because most calls are `POST` (use persisted queries with GET, response caching, or a gateway cache); file upload is outside the core spec (use presigned URLs); for many teams and services, **federation** (Apollo Federation, or schema stitching and gateways like GraphQL Mesh) composes several GraphQL services into one graph. Reach for GraphQL when client needs genuinely vary; for a single web app with a stable set of screens, REST plus well-designed endpoints is simpler.

6.8 Deep dive: tRPC and end-to-end typing

tRPC lets the client import the **type** of the server's router, so calling `trpc.note.create.mutate({ title: 'x' })` is fully typed with no code generation. Procedures declare an input schema (Zod), run middleware (auth), and return data.

```
import { initTRPC, TRPCError } from '@trpc/server';
import { z } from 'zod';

const t = initTRPC.context<{ user?: { id: string } }>().create();
const authed = t.procedure.use(({ ctx, next }) => {
  if (!ctx.user) throw new TRPCError({ code: 'UNAUTHORIZED' });
  return next({ ctx: { user: ctx.user } });
});

export const appRouter = t.router({
  note: t.router({
    list: authed.input(z.object({ limit: z.number().max(100).default(20) })).query(({
ctx, input }) => notes.list(ctx.user.id, input.limit)),
    create: authed.input(z.object({ title: z.string().min(1) })).mutation(({ ctx,
input }) => notes.create(ctx.user.id, input)),
  }),
});
export type AppRouter = typeof appRouter;   // the client imports ONLY this type
```

Adapters exist for Express, Fastify, and standalone Node. **Limits:** both ends must be TypeScript in one repository (or a published types package); it is not a public API contract; versioning and non-TypeScript

clients are awkward. **Alternatives that keep OpenAPI:** `ts-rest`, `oRPC`, Hono RPC, or Fastify with Zod type providers plus a generated client.

6.9 Deep dive: gRPC between services

gRPC defines the contract in a `.proto` file; code is generated for each language; calls travel as binary Protobuf over HTTP/2.

```
syntax = "proto3";
package inventory.v1;

service Inventory {
  rpc Reserve (ReserveRequest) returns (ReserveReply);          // unary
  rpc Watch (WatchRequest) returns (stream StockEvent);         // server streaming
}
message ReserveRequest { string sku = 1; int32 qty = 2; }
message ReserveReply   { bool ok = 1; string reservation_id = 2; }
```

- **Four call types:** unary, server streaming, client streaming, bidirectional streaming.
- **Deadlines:** always set one per call (`deadline` in the client options). A call without a deadline can wait forever. Deadlines propagate through chained calls.
- **Errors:** a status code set (`OK`, `INVALID_ARGUMENT`, `NOT_FOUND`, `DEADLINE_EXCEEDED`, `UNAVAILABLE`, ...) plus details; retry only idempotent calls and only on `UNAVAILABLE`.
- **Metadata** is the equivalent of headers (auth token, request id).
- **Schema evolution rules:** never reuse or renumber a field number; add new fields with new numbers; remove by marking `reserved`. Old and new clients keep working.
- **Load balancing:** HTTP/2 keeps one long connection, so a plain L4 balancer sends all calls from a client to one server. Use an L7 proxy that understands gRPC (Envoy, Nginx with gRPC, a service mesh) or client-side balancing.
- **Browsers** cannot speak raw gRPC; use gRPC-Web or Connect (`@connectrpc/connect`), which also supports plain JSON over HTTP and is pleasant in TypeScript.
- **Health checking** uses the standard `grpc.health.v1` service; Kubernetes can probe it.
- **Tooling:** `buf` for linting and breaking-change detection of proto files, `@grpc/grpc-js` with `@grpc/proto-loader` (dynamic) or generated TypeScript via `ts-proto` or Connect-ES.

6.10 Deep dive: WebSocket and SSE in production

WebSocket basics. The client sends an HTTP request with `Upgrade: websocket`; after the `101 Switching Protocols` response both sides exchange framed messages. Control frames `ping` and `pong` detect dead peers; send a ping every 20 to 30 seconds and close connections that do not answer, because proxies and mobile networks silently drop idle connections.

Design rules

- **Message envelope:** `{ type, id, payload }` with a client-generated `id` so you can send acknowledgements and deduplicate retries.
- **Authenticate at connect.** Use a short-lived ticket (a one-time token fetched over authenticated HTTP and passed on connect) or the session cookie; avoid long-lived tokens in the URL because URLs end up in logs. Authorise every room join and every message, not only the connection.

- **Backpressure:** check `socket.bufferedAmount` (browser) or the `ws` send callback before sending more to a slow client; drop or disconnect clients that fall too far behind.
- **Limit** message size, message rate per socket, and rooms per user.
- **Reconnect with exponential backoff and jitter** on the client, and on reconnect ask for missed messages using the last seen sequence number. WebSocket gives at-most-once delivery by default; reliability is your protocol's job (ids, acks, replay from a store such as a database or Redis Streams).
- **Capacity:** each connection costs memory and a file descriptor. Raise the OS limit (`ulimit -n`), measure memory per connection, and test with a tool such as `artillery` or `k6` WebSocket scenarios. A single Node process can hold tens of thousands of idle connections if handlers stay light.
- **Scaling out:** sticky sessions when using Socket.IO polling fallback, and a pub/sub backplane (Redis adapter, NATS, Kafka) so rooms span instances (Part 9).

```
// Socket.IO with handshake auth and rooms
io.use(async (socket, next) => {
  try { socket.data.user = await verifyAccess(String(socket.handshake.auth.token));
next(); }
  catch { next(new Error('unauthorized')); }
});
io.on('connection', (socket) => {
  socket.on('join', async (roomId: string, ack) => {
    if (!(await canJoin(socket.data.user, roomId))) return ack?.({ ok: false });
    await socket.join(roomId);
    ack?.({ ok: true });
  });
  socket.on('message', async (msg, ack) => {
    const saved = await messages.save(socket.data.user.sub, msg);   // persist first
    io.to(msg.roomId).emit('message', saved);
    ack?.({ ok: true, id: saved.id });
  });
});
```

SSE with replay. `EventSource` reconnects by itself and sends the `Last-Event-ID` header, which lets the server resume without losing events.

```
app.get('/events', (req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/event-stream', 'Cache-Control': 'no-
cache', Connection: 'keep-alive', 'X-Accel-Buffering': 'no' });
  const lastId = Number(req.headers['last-event-id'] ?? 0);
  const send = (e: { id: number }) => res.write(`id: ${e.id}\ndata:
${JSON.stringify(e)}\n\n`);
  for (const e of eventsSince(lastId)) send(e);          // replay what the client
missed
  bus.on('note', send);
  const hb = setInterval(() => res.write(': ping\n\n'), 25_000);   // keeps proxies
from closing the stream
  req.on('close', () => { clearInterval(hb); bus.off('note', send); });
});
```

Proxy settings matter: disable response buffering for the SSE route (`proxy_buffering off` in Nginx, or the `X-Accel-Buffering: no` header) and raise read timeouts.

6.11 Deep dive: sending webhooks to your own customers

If your product notifies customers' servers, build it like a small reliable messaging system:

- Each event has a unique id, type, timestamp, and JSON body; sign `timestamp + '.' + rawBody` with HMAC-SHA256 and a per-endpoint secret; send the signature and timestamp in headers so receivers can verify and reject replays.
- Deliver from a **queue** with a short timeout (5 to 10 seconds) and retries with exponential backoff over hours; stop and disable endpoints that fail for days; notify the customer.
- Store every attempt (status, response code, duration) in a **delivery log** and offer a manual "resend" action.
- Document that delivery is **at least once**, ordering is not guaranteed, and receivers must deduplicate by event id.
- Protect yourself: SSRF rules for customer URLs (Part 5), concurrency limits per endpoint, and a circuit breaker for slow receivers.

6.12 More build cards

Card 6.F: conditional GET and optimistic update

- **Aim:** ETag in practice.
- **Input:** `GET /v1/notes/:id` returns an `ETag`; `PATCH` requires `If-Match`.
- **Output:** second `GET` with `If-None-Match` returns `304`; a `PATCH` with a stale `ETag` returns `412`.
- **Check:** a test performs two overlapping edits and asserts one `200` and one `412`.

Card 6.G: full idempotency middleware

- **Aim:** implement the six-step algorithm in 6.6.
- **Input:** the `idempotency_keys` table and a `POST /v1/payments` handler.
- **Output:** replays return the stored response; changed bodies return `422`; concurrent duplicates return `409` or wait.
- **Check:** 20 parallel identical requests create exactly one payment row and 19 replay or conflict responses.

Card 6.H: GraphQL guard rails

- **Aim:** make the endpoint safe.
- **Input:** a GraphQL Yoga or Apollo server with depth limit, complexity limit, and DataLoader.
- **Output:** a query nested 12 levels deep is rejected; a query asking 100 notes with authors runs 2 SQL statements.
- **Check:** log SQL and assert statement counts in a test.

Card 6.I: gRPC service with a deadline

- **Aim:** service-to-service call done right.
- **Input:** the `Inventory` proto above, a server, and a client with a 500 ms deadline plus one retry on `UNAVAILABLE`.
- **Output:** killing the server mid-call yields a clean error, not a hang.
- **Check:** make the server sleep 2 seconds and see `DEADLINE_EXCEEDED` after 500 ms.

Card 6.J: reliable real-time feed

- **Aim:** no missed events after a reconnect.
- **Input:** SSE (or Socket.IO) feed where each event has a monotonically increasing id stored in PostgreSQL or Redis Streams.
- **Output:** disconnect the client for 10 seconds while 20 events happen; on reconnect it receives all 20 in order, once each.
- **Check:** a script asserts the received ids are consecutive with no duplicates.

Card 6.K: outgoing webhooks

- **Aim:** build the sender side.
- **Input:** `webhook_endpoints` and `webhook_deliveries` tables, a BullMQ delivery worker, HMAC signatures.
- **Output:** a failing receiver is retried with growing delays and then disabled; a verifier script accepts valid signatures and rejects tampered or stale ones.
- **Check:** run a local receiver that returns `500` twice then `200` and watch the delivery log.

Part 7: Caching, performance, and scaling

The rule of performance work: **measure first, change one thing, measure again.** Guessing wastes time.

7.1 Theory: caching

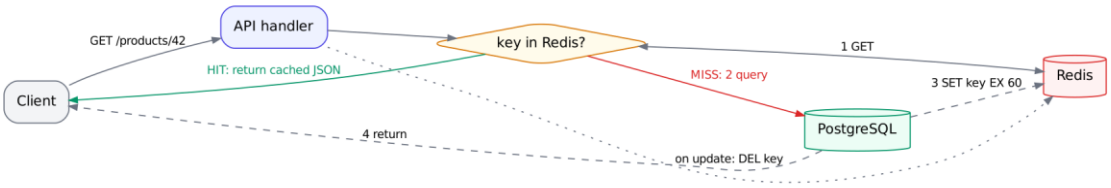
A cache stores the result of expensive work so repeated requests are cheap. The price is **staleness**: the cache may be out of date. Decide for each piece of data how stale is acceptable.

Layer	Example	Good for
Browser	`Cache-Control: max-age=3600`	Static assets, public data
CDN	Cloudflare, CloudFront, Fastly	Static files, cacheable public GET responses
Reverse proxy	Nginx, Varnish	Whole-response caching
In-process memory	`lru-cache`	Tiny hot data; lost on restart; not shared between instances
Shared cache	Redis, Memcached	Query results, sessions, computed values shared by all instances
Database	Materialised views, query cache	Heavy reports

Strategies

- **Cache-aside (lazy loading)**: app checks the cache, on a miss reads the database and fills the cache. Most common (diagram below).

- **Write-through**: write to cache and database together; reads are always warm; writes are slower.
- **Write-behind**: write to cache, flush to the database later; fast but you can lose data.
- **Read-through**: the cache library itself loads missing values.



Hard problems

- **Invalidation.** Delete or update the key whenever the source changes, or accept a short TTL. Prefer short TTLs plus explicit deletes for important data.
- **Cache stampede.** A hot key expires and 1,000 requests hit the database at once. Fixes: single-flight (one request rebuilds, the rest wait), add random jitter to TTLs, or refresh before expiry.
- **Cache penetration.** Repeated requests for missing keys bypass the cache. Cache a short-lived "not found" marker.
- **Consistency.** A cached copy can disagree with the database for up to the TTL. Never cache data whose staleness causes harm (account balances in a payment decision).

7.2 Theory: making one instance fast

- **Keep handlers non-blocking** (Part 1). Offload CPU work to `worker_threads` with a pool (Piscina), or to a queue.
- **Database first.** Most slow endpoints are slow queries or N+1 patterns. Add indexes, select only needed columns, batch queries, use connection pooling.
- **Parallelise independent I/O** with `Promise.all`; add timeouts and limits so one slow dependency cannot hold every request.
- **Compress** responses (gzip or brotli) at the reverse proxy; **stream** large downloads instead of buffering.
- **Reduce payloads**: pagination, field selection, no giant nested arrays.
- **Keep-alive** connections to databases and upstream APIs (use an HTTP agent such as `undici`).
- **Profile** with the Node inspector, `clinic.js`, `0x` flamegraphs; look at event-loop delay (`perf_hooks.monitorEventLoopDelay`).

7.3 Theory: scaling out

Vertical scaling = a bigger machine. **Horizontal scaling** = more copies behind a load balancer. For horizontal scaling your app must be **stateless**: no user data in process memory, no files on local disk, no in-memory sessions. Put state in PostgreSQL, Redis, or object storage, and every instance becomes interchangeable.

- **Load balancer** distributes requests (round robin, least connections). Nginx, HAProxy, cloud load balancers.

- **Health checks**: `/healthz` (process alive) and `/readyz` (can reach DB and Redis). The balancer removes unhealthy instances.
- **Graceful shutdown**: on `SIGTERM` stop accepting new requests, finish in-flight ones, close DB pools, then exit.
- **Database scaling order**: indexes and query fixes, then caching, then read replicas, then partitioning or sharding. Sharding is the last resort.
- **Rate limiting algorithms**: fixed window (simple, bursty at edges), sliding window (smoother), **token bucket** (allows bursts, enforces average rate), leaky bucket (smooths output). Store counters in Redis so all instances share them.

7.4 Libraries and alternatives

Need	Pick	Alternatives
In-memory cache	`lru-cache`	`node-cache`, `quick-lru`, `@isaacs/ttlcache`
Shared cache	Redis with `ioredis`	Memcached (`memjs`), Valkey (Redis fork), Dragonfly, KeyDB
Cache helpers	`cacheable`, `@nestjs/cache-manager`	Hand-written `getOrSet` helper
Single-flight / locks	`async-mutex` (in-process), Redis `SET NX PX` or `redlock` (distributed)	Postgres advisory locks
Rate limiting	`rate-limiter-flexible` (Redis-backed)	`express-rate-limit` + `rate-limit-redis`, `@fastify/rate-limit`
Worker threads	`piscina`	Raw `worker_threads`, `workerpool`
Multi-process	PM2, Node `cluster`	Containers per core on Kubernetes (preferred today)
Load testing	`autocannon`, k6	Artillery, wrk, Locust
Profiling	`clinic`, `0x`, Chrome DevTools via `--inspect`	Datadog or Pyroscope continuous profiling
HTTP client	`undici` (also behind global `fetch`)	`got`, `axios`, `ky`
CDN / edge	Cloudflare, CloudFront	Fastly, Bunny

7.5 Build cards

Card 7.A: cache-aside helper

- **Aim:** one reusable `getOrSet(key, ttlSeconds, loader)`.
- **Input:** a slow `loadProduct(id)` (simulate with a 300 ms delay or a heavy query).

- **Output:** first call 300 ms, later calls under 5 ms until the TTL ends; updating a product deletes its key.
- **Check:** log HIT or MISS; call 5 times and watch 1 MISS then 4 HITs; update then call again for a MISS.

Card 7.B: stampede protection

- **Aim:** survive a hot key expiring.
- **Input:** 200 parallel requests for the same expired key.
- **Output:** the loader runs once; all 200 receive the value.
- **Check:** a counter inside the loader prints 1.

Card 7.C: Redis rate limiter

- **Aim:** a limit shared by all instances.
- **Input:** limit 10 requests per minute per API key; run two server processes on different ports.
- **Output:** the 11th request within a minute gets `429` with `Retry-After`, whichever process receives it.
- **Check:** alternate requests between the two ports with a script.

Card 7.D: load test and graceful shutdown

- **Aim:** get a baseline and prove safe restarts.
- **Input:** `autocannon -c 100 -d 30` against `/notes`; handle `SIGTERM`.
- **Output:** a table of requests per second and p99 latency before and after caching; during a restart under load there are zero failed requests.
- **Check:** run autocannon, send `kill -TERM <pid>` mid-run, compare error counts.

Self-check questions: Which data would you never cache, and why? What makes an app stateless? Why is a token bucket friendlier to users than a fixed window?

7.6 Deep dive: cache design in detail

Key design. A cache key must contain every input that changes the value: `product:v3:42:en:INR`. Include a **schema version** (`v3`) so a deploy that changes the cached shape just starts using new keys instead of serving old shapes; include tenant and locale; never put raw user input in a key without normalising it. Keep a naming convention (`service:entity:version:id`).

What to cache, and what not. Cache data that is expensive to compute or fetch, read far more often than written, and tolerant of staleness. Do not cache per-user secrets in a shared key, results of authorization decisions for long, or anything whose staleness causes money or safety errors. Measure the **hit ratio** (hits / (hits + misses)); below about 80 percent the cache may cost more than it saves, and always compare latency with and without it.

Two-level caches. An in-process cache (L1, `lru-cache`, microseconds) in front of Redis (L2, about a millisecond) in front of the database. The danger is that each instance's L1 can be stale. Fix with a very short L1 TTL (1 to 5 seconds), or publish invalidation messages over Redis pub/sub so every instance drops the key, or use client-side caching features of the Redis client (RESP3 tracking).

Stampede and expiry strategies

Technique	How it works	Trade-off
Single-flight (request coalescing)	Concurrent misses for a key share one in-flight loader promise	Per process only; combine with a Redis lock for cross-instance
TTL jitter	`ttl * (0.9 + Math.random() * 0.2)`	Spreads expiries; trivial
Stale-while-revalidate	Serve the expired value immediately, refresh in the background	Slightly stale data, very smooth latency
Probabilistic early refresh (XFetch)	Each reader may refresh before expiry, with rising probability as expiry nears	No lock needed; needs the recompute time stored
Refresh-ahead	A job refreshes hot keys on a schedule	Wasted work on keys nobody reads
Negative caching	Cache "not found" for a few seconds	Stops repeated misses for missing ids

```
const inflight = new Map<string, Promise<unknown>>();
export async function getOrSet<T>(key: string, ttlSec: number, load: () =>
Promise<T>): Promise<T> {
  const hit = await redis.get(key);
  if (hit) return JSON.parse(hit) as T;
  let p = inflight.get(key) as Promise<T> | undefined;
  if (!p) {
    p = load().then(async (v) => {
      await redis.set(key, JSON.stringify(v), 'EX', Math.round(ttlSec * (0.9 +
Math.random() * 0.2)));
      return v;
    }).finally(() => inflight.delete(key));
    inflight.set(key, p);
  }
  return p;
}
```

Invalidation choices. (1) TTL only: simplest, accept staleness. (2) Delete on write: delete the key after the database commit (delete, do not update, to avoid races where an old value overwrites a new one). Race to know: a reader can miss, read the old row, the writer updates and deletes, and then the reader sets the old value; a short TTL bounds the damage. (3) Event-driven: publish a change event (outbox, Part 8) and let a consumer invalidate or rebuild caches. (4) Version tags: bump a per-entity version number and put it in the key, so old keys simply age out.

HTTP and CDN caching. For public GET endpoints send `Cache-Control: public, max-age=60, s-maxage=300, stale-while-revalidate=600`. `s-maxage` applies to shared caches (CDN) only. Purge by URL or **surrogate key/tag** when data changes (Cloudflare cache tags, Fastly surrogate keys). Never let a CDN cache responses that depend on cookies or `Authorization` unless you have set `Vary` or a custom cache key correctly.

Serialisation cost. Caching a large object costs CPU to `JSON.stringify` and `parse`. Cache the final response bytes when possible, or use a faster format (MessagePack via `@msgpack/msgpack`) and compression (`zlib` brotli or `lz4`) only for big values.

7.7 Deep dive: measuring performance correctly

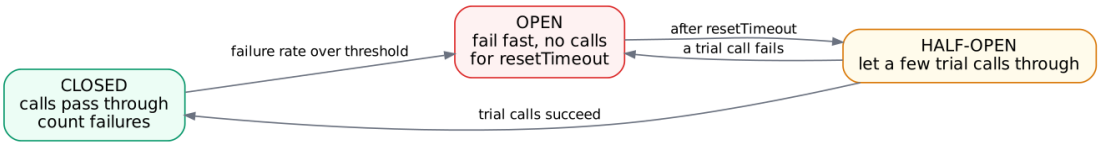
- **Use percentiles, not averages.** An average of 80 ms can hide a p99 of 3 seconds. Track p50, p95, p99 (and max) per route. A user making 20 requests per page sees the p99 of the system often.
- **Little's Law.** Requests in flight `L = arrival rate x time in system`. At 500 requests per second and 200 ms each, about 100 requests are in flight at once; that tells you how many database connections or sockets you need.
- **The utilisation knee.** Queues grow explosively near full utilisation: in a simple model waiting time grows like `rho / (1 - rho)`, so at 90 percent busy the wait is about 9 times the service time, and at 50 percent it equals it. Plan for 50 to 70 percent peak utilisation.
- **USE and RED.** For resources (CPU, memory, pool, queue) track **U**tilisation, **S**aturation, **E**rrors; for services track **R**ate, **E**rrors, **D**uration.
- **Load testing method.** Test a production-like environment with production-like data size. Use an **open model** (a fixed arrival rate, as real users behave, with tools like k6 `constant-arrival-rate` or `wrk2`) rather than only a closed loop of N virtual users, otherwise slow responses reduce the offered load and hide problems (**coordinated omission**). Warm up first, run long enough to see GC and pool effects, step the load up to find the knee, then test spikes and a soak run of hours for leaks.
- **Find the bottleneck, then change one thing.** Order of suspicion: database queries and locks, external calls without timeouts, N+1 loops, serialisation of large payloads, event-loop blocking, GC pressure, then framework overhead.
- **Profiling tools.** `node --cpu-prof` writes a profile you can open in Chrome DevTools; `clinic doctor` and `clinic flame` automate diagnosis and flame graphs; `0x` for flame graphs; `--heap-prof` for allocations; `perf_hooks` `performance.eventLoopUtilization()` shows how busy the loop is (near 1.0 means CPU bound). In production use continuous profilers (Pyroscope, Datadog, Grafana Cloud Profiles) at low overhead.
- **Database-side measurement.** `pg_stat_statements` (total time per query), `auto_explain`, and slow query logs (`log_min_duration_statement`); correlate with your traces.

7.8 Deep dive: protecting the system under load

Timeout budgets. A request that must finish in 2 seconds cannot give each of 3 downstream calls 2 seconds. Pass a deadline down (a propagated `AbortSignal` or a deadline timestamp) and give each call a share. Always set timeouts shorter at the inner layers than the outer ones, or the outer layer gives up while the inner work continues uselessly.

Concurrency limits and bulkheads. Limit parallel calls to each dependency (`p-limit`, `bottleneck`) and use separate pools for separate jobs, so a slow reporting query cannot use all connections that checkout needs.

Circuit breaker. When a dependency is failing, stop hammering it: fail fast, give it time to recover, then probe carefully.



Libraries: `opossum` (breaker with fallback and stats), `cockatiel` (retry, breaker, timeout, bulkhead as composable policies). Pair with a **fallback** (cached value, default, degraded feature) and expose breaker state as a metric.

Retries without making things worse. Retries multiply load: if three layers each retry three times, one failure becomes 27 calls (a **retry storm**). Retry at one layer only, use exponential backoff with jitter, cap attempts, and use a **retry budget** (for example, retries may be at most 10 percent of normal traffic).

Load shedding. When overloaded, reject early and cheaply instead of failing slowly for everyone: return `503` with `Retry-After` when event-loop delay or queue depth passes a threshold (`@fastify/under-pressure`, or your own middleware reading `monitorEventLoopDelay`). Prioritise (health checks, paying customers, writes over reports) and drop the rest.

Graceful degradation. Decide in advance what turns off first: recommendations, analytics, exports. Feature flags make this a switch rather than a deploy.

Idempotent, bounded work. Cap request size, query size, result size, and concurrency per user; unbounded anything becomes an outage.

7.9 Deep dive: scaling out in practice

Stateless checklist. No session data or uploaded files in process memory or local disk; no in-process schedulers that must run once (use a queue or a lock); no instance-specific caches that must agree (or accept short staleness); no sticky routing needed unless you use WebSockets; configuration only from environment; logs to stdout; clean shutdown on `SIGTERM`.

Load balancing algorithms

Algorithm	Idea	Use
Round robin	Rotate through instances	Equal instances, similar requests
Least connections	Send to the instance with fewest open requests	Uneven request durations
Power of two choices	Pick two at random, send to the less loaded	Large fleets; cheap and good
IP hash / consistent hash	Same client or key goes to the same instance	Sticky sessions, cache locality, WebSockets
Weighted	More traffic to bigger instances	Mixed hardware, canary releases

Layer 4 balancers (TCP) are fast and simple; layer 7 (HTTP) balancers can route by path, header, and cookie, do TLS, and retry idempotent requests. Always configure **health checks** and **connection draining** so deploys do not drop requests.

Zero-downtime deploys. *Rolling*: replace instances a few at a time. *Blue/green*: run a full new copy and switch traffic. *Canary*: send 1 to 5 percent to the new version, compare error rate and latency, then continue or roll back (diagram in Part 11). All need backward-compatible database migrations (Part 4) and readiness checks (`/readyz` returns 200 only when the app can serve).

Autoscaling signals. CPU percent is the default, but for I/O-bound Node services better signals are requests in flight per instance, event-loop delay, queue depth or oldest job age (for workers), and p95 latency. Scale out early, scale in slowly, and set minimum instances for availability. Remember downstream limits: ten more API instances can overload the database; scale the bottleneck, not just the front.

Nginx as reverse proxy (sketch)

```
upstream api { least_conn; server 10.0.0.11:3000; server 10.0.0.12:3000; keepalive
64; }
server {
  listen 443 ssl http2;
  location / {
    proxy_pass http://api;
    proxy_http_version 1.1;
    proxy_set_header Connection "";
    proxy_set_header X-Request-Id $request_id;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_read_timeout 30s;
    client_max_body_size 1m;
  }
}
```

Rate limiting algorithms, concretely

Algorithm	Behaviour	Weakness
Fixed window	Count per minute window, reset at the boundary	A client can send 2x the limit across a boundary
Sliding window log	Store each request time in a sorted set	Memory per user
Sliding window counter	Weighted sum of current and previous window	Approximate
Token bucket	Tokens refill at a rate up to a capacity; each request spends one	Needs atomic read-modify-write
Leaky bucket	Requests drain at a fixed rate	No bursts

The read-modify-write must be atomic across instances, so do it in a Redis Lua script (or use `rate-limiter-flexible`, which implements these):

```
-- KEYS[1]=key  ARGV: ratePerSec, capacity, nowSeconds, cost
local d = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(d[1]) or tonumber(ARGV[2])
local ts = tonumber(d[2]) or tonumber(ARGV[3])
tokens = math.min(tonumber(ARGV[2]), tokens + (tonumber(ARGV[3]) - ts) *
tonumber(ARGV[1]))
local ok = 0
if tokens >= tonumber(ARGV[4]) then tokens = tokens - tonumber(ARGV[4]); ok = 1 end
redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', ARGV[3])
redis.call('PEXPIRE', KEYS[1], math.ceil(tonumber(ARGV[2]) / tonumber(ARGV[1]) *
2000))
return ok
```

Take the time from Redis (`TIME`) or accept small clock skew between app servers; decide what to do if Redis is down (**fail open** for general traffic, **fail closed** for login and payment limits).

Capacity planning arithmetic. Estimate peak requests per second (daily users x requests per user / 86,400 x peak factor, often 3 to 10), multiply by service time to get concurrency (Little's Law), divide by what one instance sustains at 60 percent utilisation (from your load test), add redundancy (N+1 or N+2), and check every shared dependency (database connections, Redis ops per second, network bandwidth, third-party rate limits) against the same peak.

7.10 More build cards

Card 7.E: XFetch or stale-while-revalidate

- **Aim:** smooth expiry without locks.
- **Input:** your `getOrSet` helper and a loader that takes 300 ms.
- **Output:** under constant load, no request ever waits the full 300 ms after the first fill, because an expired value is served while one background refresh runs.
- **Check:** autocannon at 200 requests per second for 60 seconds with a 10 second TTL; the p99 stays near the cache latency, and a counter shows about one refresh per TTL.

Card 7.F: two-level cache with invalidation

- **Aim:** L1 in memory plus L2 in Redis, kept in sync.
- **Input:** `lru-cache` in each of two server instances, Redis behind, and a Redis pub/sub channel for invalidations.
- **Output:** updating a product on instance A makes instance B return the new value within one second.
- **Check:** a script alternates reads between ports after a write and logs the first read that sees the new value.

Card 7.G: open-model load test with k6

- **Aim:** find the knee honestly.
- **Input:** a k6 script using `constant-arrival-rate` stepping from 50 to 800 requests per second against `/notes/:id`.
- **Output:** a table of rate versus p50, p95, p99, and error rate; the rate where p95 doubles is your knee.

- **Check:** plot the result and write one sentence naming the resource that saturated first (use pool waiting count and event-loop delay metrics).

Card 7.H: circuit breaker and fallback

- **Aim:** survive a failing dependency.
- **Input:** a flaky upstream service (returns errors 70 percent of the time between minute 1 and 2); `opossum` around the call; a cached fallback.
- **Output:** during the failure window the breaker opens, latency stays low, responses come from the fallback; it closes again after recovery.
- **Check:** graph breaker state and error rate; confirm the upstream receives far fewer calls while open.

Card 7.I: load shedding

- **Aim:** stay alive when overwhelmed.
- **Input:** event-loop-delay middleware that returns `503` plus `Retry-After` above 100 ms, exempting `/health`.
- **Output:** at 3x capacity the server answers some requests quickly with `503` while accepted requests keep a low latency, instead of every request timing out.
- **Check:** compare error and latency distributions with and without shedding under the same overload.

Card 7.J: Redis token bucket

- **Aim:** a correct distributed limiter.
- **Input:** the Lua script above, run through `ioredis` `defineCommand`, limit 5 requests per second with burst 10.
- **Output:** 10 immediate requests succeed, then about 5 per second; two server instances share the limit.
- **Check:** a script firing 100 requests across both instances reports the number allowed over 10 seconds (expect about 10 + 5 x 10).

Card 7.K: zero-downtime restart

- **Aim:** prove deploys do not drop requests.
- **Input:** two instances behind Nginx with health checks and draining; a rolling restart script.
- **Output:** during a restart under autocannon load, the error count is 0.
- **Check:** run it with and without graceful shutdown (Card 1.H) and compare error counts.

Deep-dive self-check: (1) Why include a version in a cache key? (2) Why delete, rather than update, a cache key after a write? (3) What is coordinated omission and how does an open-model test avoid it? (4) Why can retries at every layer cause a retry storm? (5) Why is 90 percent CPU utilisation a bad steady state for latency? (6) When should a rate limiter fail open and when closed?

Part 7 deep dive

7.6 Cache design in depth

Key design. A key must contain everything the value depends on: `product:v3:42:en:USD` (entity, schema version, id, locale, currency). Include a **version prefix** you can bump to invalidate everything after a format change. Normalise query parameters (sort them, drop irrelevant ones) so equivalent requests share a key, and include the tenant or user id for anything personalised, or you will serve one user's data to another.

A production-grade `getOrSet` with negative caching, TTL jitter, and single-flight (concurrent misses in one process share one load):

```
const inflight = new Map<string, Promise<unknown>>();
const NULL = '__null__';                                  // marker so "not found"
is cached too

export async function getOrSet<T>(key: string, ttlSec: number, load: () => Promise<T
| null>): Promise<T | null> {
  const hit = await redis.get(key);
  if (hit !== null) return hit === NULL ? null : (JSON.parse(hit) as T);

  const pending = inflight.get(key);
  if (pending) return pending as Promise<T | null>;       // someone in this process
is already loading

  const p = (async () => {
    try {
      const value = await load();
      const jitter = Math.floor(ttlSec * 0.1 * Math.random());   // spread expiries
out
      await redis.set(key, value === null ? NULL : JSON.stringify(value), 'EX', value
=== null ? 30 : ttlSec + jitter);
      return value;
    } finally {
      inflight.delete(key);
    }
  })();
  inflight.set(key, p);
  return p;
}
```

This protects each *instance*. To protect the database from many instances at once, add a short Redis lock (`SET lock:key token NX PX 5000`) so only one instance loads while others briefly wait or serve slightly stale data.

More techniques

- **Stale-while-revalidate:** keep serving the expired value while one background refresh runs. Users never wait on a miss for popular keys. HTTP has the same idea in `Cache-Control: stale-while-revalidate=60`.
- **Probabilistic early expiration:** each reader may refresh *slightly before* expiry with a probability that rises as expiry nears, so refreshes happen one at a time instead of all at the deadline.

- **Two levels:** a tiny in-process LRU (L1, microseconds) in front of Redis (L2, about a millisecond over the network). Invalidate L1 on every instance by publishing the key on a Redis pub/sub channel. Keep L1 TTLs very short (seconds) as a safety net.
- **Tag-based invalidation:** store which keys belong to a tag (`category:7` -> a set of product keys) so one event clears a group.
- **Write-through vs cache-aside:** write-through keeps hot data warm and consistent but couples writes to the cache; cache-aside is simpler and the default.
- **Measure the hit ratio** per cache (hits / (hits + misses)); a cache below roughly 80 to 90 percent on a hot path often has bad keys or too-short TTLs. Also watch Redis memory, evictions, and the latency of cache calls themselves.
- **Serialise cheaply.** `JSON.stringify` of large objects costs CPU on the single thread; cache the smallest useful shape.

Never cache data where staleness causes harm (balances used for a payment decision, permissions you must revoke instantly), or per-user data under a shared key. Cache stampedes, penetration, and invalidation are the three failure modes to design for from day one.

7.7 Profiling and performance engineering

Work scientifically: define the target (for example p95 under 300 ms at 200 requests/second), reproduce with a load test, **profile to find the real bottleneck**, change one thing, measure again.

Symptom	Likely cause	Tool
High CPU, slow responses	Hot JavaScript (JSON, regex, loops, sync crypto)	`node --cpu-prof`, clinic flame, 0x, Chrome DevTools profiler
Low CPU but slow responses	Waiting on database, network, or the pool	Tracing (OpenTelemetry), `pg_stat_statements`, pool metrics
Latency spikes, periodic freezes	Event loop blocked, or GC pauses	Event-loop-delay histogram, `--trace-gc`, clinic doctor
Memory grows forever	Leak (Part 1.13)	Heap snapshots, `--heapsnapshot-signal`
Fine alone, slow under load	Contention: pool exhaustion, locks, thread pool queue	Pool wait metrics, `pg_locks`, `UV_THREADPOOL_SIZE`

Measure event loop health in production, not only CPU:

```
import { monitorEventLoopDelay, performance } from 'node:perf_hooks';
const h = monitorEventLoopDelay({ resolution: 20 });
h.enable();
let last = performance.eventLoopUtilization();
setInterval(() => {
  const now = performance.eventLoopUtilization();
  const util = performance.eventLoopUtilization(now, last).utilization;      // 0..1:
how busy the loop is
  last = now;
  logger.info({ elDelayP99Ms: h.percentile(99) / 1e6, elDelayMaxMs: h.max / 1e6,
```

```
elUtil: util }, 'event loop');
  h.reset();
}, 10_000).unref();
```

Common JavaScript hot spots: `JSON.stringify/parse` on multi-megabyte objects; **regular expressions with catastrophic backtracking** (ReDoS: a pattern like `(a+)+$` on crafted input pegs the CPU: validate untrusted input with bounded, simple patterns or a linear-time engine such as RE2 via `re2`); synchronous `crypto`/`zlib`/`fs` calls; building huge strings; sorting large arrays per request; serial `await` in a loop where the calls are independent; unbounded `Promise.all` over thousands of items (hammers the database and exhausts memory). Bound concurrency instead:

```
import pLimit from 'p-limit';
const limit = pLimit(10);                                          // at most 10 in
flight
const results = await Promise.all(ids.map((id) => limit(() => fetchUser(id))));
```

7.8 Load testing properly

Most bad load tests use a **closed model** (N virtual users loop as fast as they can). When the server slows down, the test automatically sends fewer requests, hiding the problem (called coordinated omission). Real traffic is an **open model**: users arrive at a rate regardless of how slow you are. k6 supports this with arrival-rate executors:

```
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  scenarios: {
    steady: { executor: 'constant-arrival-rate', rate: 200, timeUnit: '1s', duration:
'2m',
              preAllocatedVUs: 50, maxVUs: 400 },
  },
  thresholds: { http_req_failed: ['rate<0.01'], http_req_duration: ['p(95)<300',
'p(99)<800'] },
};
export default function () {
  const r = http.get(`${__ENV.BASE}/v1/notes?limit=20`);
  check(r, { 'status is 200': (x) => x.status === 200 });
}
```

Rules: test a production-like environment and dataset size (an empty database lies), include a warm-up, run the load generator on a **different machine**, test the realistic mix of endpoints, look at **percentiles** (p95, p99) not averages, increase load in steps to find the **knee** where latency climbs sharply, and also test failure (kill a replica, slow a dependency). Averages hide the slow tail that real users feel; one page that triggers 20 backend calls hits the p99 of each.

Little's law makes capacity concrete: concurrent requests in flight = arrival rate x average latency. At 200 requests/second and 100 ms average latency, about 20 requests are in flight; if latency rises to 1 s the same traffic means 200 in flight, which explains why one slow dependency can exhaust a pool or memory almost instantly.

Operation	Rough order of magnitude
In-process memory access	nanoseconds to microseconds

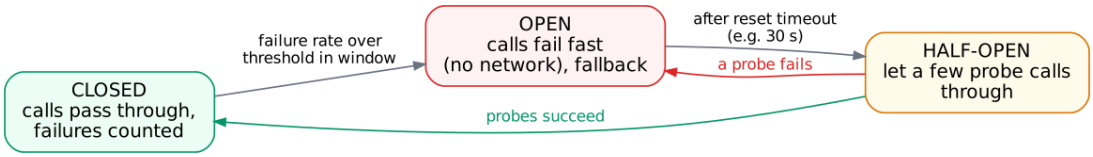
Operation	Rough order of magnitude
Redis call in the same data centre	about 0.2 to 1 ms
Indexed PostgreSQL query (warm)	about 0.5 to 5 ms
Unindexed scan of a large table	tens of ms to seconds
Another service in the same region	a few ms to tens of ms
Cross-region or cross-continent call	roughly 50 to 300 ms

(These are approximate rules of thumb for reasoning, not benchmarks; measure your own system.)

7.9 Resilience: timeouts, retries, breakers, load shedding

A slow dependency is more dangerous than a dead one, because requests pile up waiting. Defend in layers:

7. **Timeouts everywhere**, nested sensibly: the client's timeout is longer than your server's, which is longer than each downstream call's, so you fail with a clean error before the caller gives up.
8. **Retry only safe operations** (idempotent, or protected by idempotency keys), only on transient errors (timeouts, `503`, connection reset, not `400`), with **exponential backoff and full jitter**, and a **retry budget** so a struggling service is not hit with triple load: `delay = random(0, min(cap, base * 2^attempt))`.
9. **Circuit breaker:** after enough failures, stop calling the dependency for a while and fail fast (or use a fallback), then probe.
10. **Bulkheads:** separate connection pools or concurrency limits per dependency, so one slow partner cannot consume every connection.
11. **Load shedding:** when you are overloaded, reject early with `503` and `Retry-After` instead of accepting work you cannot finish. A bounded queue plus a concurrency limit is the simplest form; `@fastify/under-pressure` returns 503 when event-loop delay, heap, or RSS cross thresholds.




```
import CircuitBreaker from 'opossum';

const breaker = new CircuitBreaker(callPartner, {
  timeout: 3000,                    // count a call slower than 3 s as a failure
  errorThresholdPercentage: 50,     // open when half the calls in the window fail
  volumeThreshold: 10,              // ...but only after at least 10 calls
  resetTimeout: 30_000,             // try a probe after 30 s (half-open)
});
breaker.fallback(() => ({ source: 'cache', data: lastKnownGood() }));
breaker.on('open', () => logger.warn('partner breaker OPEN'));
```

```
const result = await breaker.fire(request);

export const backoff = (attempt: number, baseMs = 100, capMs = 10_000) =>
  Math.random() * Math.min(capMs, baseMs * 2 ** attempt);          // full jitter
```

`cockatiel` bundles retry, timeout, bulkhead, and breaker policies in one composable library, and `p-retry` handles simple retry loops.

7.10 Scaling in depth

Autoscaling signals. CPU alone is a poor signal for I/O-bound Node services. Better: requests per second per instance, p95 latency, event-loop utilisation, or **queue depth and oldest-job age** for workers (KEDA scales Kubernetes deployments on queue length). Scale out early enough that new instances are ready (startup and warm-up take time) and scale in slowly.

Zero-downtime deploys on Kubernetes. On shutdown Kubernetes marks the pod terminating and sends `SIGTERM` while the service may still route a few requests to it. The robust sequence: make the **readiness** probe fail immediately on `SIGTERM`, wait a few seconds for load balancers to stop sending traffic (a `preStop` sleep), stop accepting new connections, finish in-flight requests, close pools, exit, all within `terminationGracePeriodSeconds`. Rolling updates replace pods gradually; **canary** releases send a small percentage of traffic to the new version and roll back automatically on error-rate or latency regression; **blue/green** keeps two full environments and flips traffic.

Sticky sessions pin a client to one instance; avoid them by being stateless, and use them only where a protocol requires it (Socket.IO long-polling fallback).

The database scaling ladder (climb only as far as measurements force you):

12. Fix slow queries and missing indexes; add connection pooling.
13. Cache hot reads.
14. Vertical scaling (a bigger instance is cheap compared with engineering time).
15. Read replicas for read-heavy load (mind replication lag).
16. **Partitioning** big tables by time or tenant (declarative partitioning in PostgreSQL), which speeds pruning and makes retention a cheap `DROP PARTITION`.
17. **Sharding** across servers (Citus for PostgreSQL, Vitess for MySQL, or application-level) with a shard key chosen so most queries touch one shard. This is the hardest step: cross-shard joins and transactions become difficult.

Consistent hashing assigns keys to nodes so adding or removing a cache node moves only a small fraction of keys (used by distributed caches and sharded systems). **CAP and PACELC** in one line: during a network partition a distributed store must choose consistency or availability, and even without partitions it trades latency against consistency; know which your datastore chose.

Token bucket in Redis (shared by all instances; allows bursts, enforces an average rate):

```
-- KEYS[1] = bucket key; ARGV: capacity, refill_per_sec, cost
local t = redis.call('TIME')
local now = t[1] * 1000 + math.floor(t[2] / 1000)
local cap, rate, cost = tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3])
local d = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(d[1]) or cap
```

```
local ts = tonumber(d[2]) or now
tokens = math.min(cap, tokens + math.max(0, now - ts) / 1000 * rate)
local allowed = 0
if tokens >= cost then tokens = tokens - cost; allowed = 1 end
redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', KEYS[1], math.ceil(cap / rate * 1000) * 2)
return { allowed, math.floor(tokens) }
```

Call it with `redis.eval(script, 1, key, capacity, refillPerSec, 1)`, and answer `429` with `Retry-After` when `allowed` is 0. Run the whole script atomically (Redis executes a script without interleaving), which is exactly why Lua is used.

7.11 More build cards

Card 7.E: two-level cache with invalidation

- **Aim:** combine an in-process LRU and Redis.
- **Input:** `lru-cache` as L1 (TTL 5 s, max 1000 entries), Redis as L2, a pub/sub channel `invalidate`.
- **Output:** updating a product on instance A removes it from the L1 of instance B within a second; L1 hits are visibly faster than L2 hits.
- **Check:** run two instances; update on one, read on the other, print HIT-L1, HIT-L2, MISS labels.

Card 7.F: find the blocker

- **Aim:** use a flamegraph.
- **Input:** an endpoint secretly doing a synchronous 80 ms regex or `JSON.stringify` of a large object, plus the event-loop monitor from 7.7.
- **Output:** the monitor shows p99 delay spikes under load; a flamegraph (`clinic flame` or `0x`) points to the exact function; after the fix the delay drops.
- **Check:** screenshot the flamegraph before and after, with the p99 numbers.

Card 7.G: open-model load test

- **Aim:** a test that cannot hide slowness.
- **Input:** the k6 script above against `/v1/notes`.
- **Output:** find the arrival rate at which p95 first exceeds 300 ms (the knee); then compare with a closed-model run at the same concurrency and explain the difference.
- **Check:** a short table of rate, p95, p99, error rate for 100, 200, 400, 800 requests/second.

Card 7.H: breaker, timeout, and shedding

- **Aim:** survive a slow dependency.
- **Input:** a fake partner endpoint that you can set to respond in 10 s; `opossum`, a 1 s timeout, and `@fastify/under-pressure` (or your own concurrency limit).
- **Output:** with the partner slow, your API returns the fallback within about 1 s, then fails fast once the breaker opens, and its own latency stays flat; the breaker closes again after the partner recovers.
- **Check:** run autocannon through the whole story and plot latency over time.

Card 7.I: token bucket

- **Aim:** a precise shared rate limiter.

- **Input:** the Lua script above; two server processes; capacity 10, refill 1 per second.
- **Output:** 10 immediate requests pass, the 11th returns `429` with a correct `Retry-After`, then one request per second passes, regardless of which process receives it.
- **Check:** a script alternating between the two processes and logging timestamps and statuses.

Card 7.J: Little's law exercise

- **Aim:** reason about capacity on paper, then confirm.
- **Input:** an endpoint with measured average latency; a target of 500 requests/second.
- **Output:** your computed in-flight concurrency, the DB pool size it implies, and the instance count; then a load test that confirms or corrects the estimate.
- **Check:** write the calculation and the measured numbers side by side.

7.12 Answer key for the Part 7 self-checks

- **Which data would you never cache, and why?** Data where staleness is harmful (balances used for decisions, instantly revocable permissions) or per-user data under a shared key, because a stale or wrong-user answer is worse than a slow one.
- **What makes an app stateless?** No request-specific or user data in process memory or local disk between requests: sessions, uploads, caches that must be shared, and job state live in Redis, the database, or object storage, so any instance can serve any request.
- **Why is a token bucket friendlier than a fixed window?** It permits short bursts up to the bucket size while enforcing the long-run average, and it avoids the fixed window's edge effect where a client can send double the limit across a window boundary.

7.6 Deep dive: a production-grade cache layer

A good cache helper is small but handles the hard cases: serialisation, TTL jitter, single-flight (no stampede), negative caching, and failure tolerance (if Redis is down, fall back to the source instead of failing every request).

```
import Redis from 'ioredis';

const inflight = new Map<string, Promise<unknown>>();     // single-flight inside
this process

export async function getOrSet<T>(
  redis: Redis, key: string, ttlSec: number, loader: () => Promise<T | null>,
): Promise<T | null> {
  try {
    const hit = await redis.get(key);
    if (hit !== null) return hit === '__null__' ? null : (JSON.parse(hit) as T);
  } catch { /* cache down: fall through to the loader */ }

  const existing = inflight.get(key) as Promise<T | null> | undefined;
  if (existing) return existing;                            // join the request
already loading this key

  const p = (async () => {
    const value = await loader();
    const jitter = Math.floor(Math.random() * ttlSec * 0.1);   // spread expiry times
```

```
by up to 10%
    const ttl = value === null ? 30 : ttlSec + jitter;         // short negative-
cache TTL for "not found"
    redis.set(key, value === null ? '__null__' : JSON.stringify(value), 'EX',
ttl).catch(() => {});
    return value;
  })().finally(() => inflight.delete(key));

  inflight.set(key, p);
  return p;
}
```

This single-flight map protects each *process*. With many instances the database can still see one rebuild per instance, which is usually fine. If a rebuild is very expensive, add a **distributed lock** (`SET lock:key token NX PX 10000`): the winner rebuilds, the losers wait briefly and re-read the cache, or serve a slightly stale copy.

Stale-while-revalidate. Store two TTLs: a soft expiry (after which the value is considered stale) and a hard expiry. When stale, return the old value immediately and refresh in the background. Users never wait for a rebuild of popular keys.

Key design. `v1:product:42` (a version prefix lets you invalidate everything by bumping `v1` to `v2`), include every input that changes the result (`v1:feed:user:7:page:2:locale:en`), never put secrets or raw user input in keys, and keep keys short. For per-tenant data always include the tenant id, otherwise one customer can read another's cached data.

Invalidation strategies

Strategy	How	Trade-off
TTL only	Let entries expire	Simplest; staleness up to the TTL
Delete on write	After a successful commit, `DEL key`	Race: a concurrent reader may re-cache the old value just before the commit; keep the TTL short too
Update on write (write-through)	Write the new value to the cache	Needs the same serialisation everywhere
Tag or group keys	Maintain a Redis set of keys per tag (`tag:product:42`) and delete them together	Extra bookkeeping
Event-driven	Publish `ProductChanged`; each service evicts its own cache	Decoupled; relies on reliable events (outbox, Part 8)
Versioned namespace	`v{n}` stored in Redis; increment to invalidate a whole group instantly	Old keys age out by TTL

A safe order for delete-on-write is: update the database, then delete the key, and keep a modest TTL as the backstop. For read-after-write consistency for the writing user, also return the fresh data from the write response.

What to cache, and what not to. Good: expensive aggregate queries, reference data (countries, plans), rendered fragments, results of third-party API calls (respecting their terms), permission lookups with short TTL. Avoid caching: per-user data under shared keys, anything where staleness has financial or safety consequences, and objects too large to deserialise cheaply (JSON parsing of a 5 MB blob can block the event loop).

Measure the cache. Track hit ratio (hits divided by hits plus misses), latency of hits and misses, evictions, memory used, and key count. A cache with a 20 percent hit ratio often costs more than it saves.

7.7 Deep dive: HTTP and CDN caching for APIs

- Public, rarely changing responses: `Cache-Control: public, max-age=300, stale-while-revalidate=60` so a CDN absorbs most traffic.
- Personalised responses: `Cache-Control: private` (or `no-store` if sensitive) and `Vary: Authorization` or `Cookie` when a shared cache could otherwise mix users.
- **Cache keys at the CDN** include the URL and the headers named in `Vary`; normalise query-string order and strip tracking parameters so equivalent requests share one entry.
- **Purging:** CDNs let you purge by URL or by surrogate key (a tag you set in a response header such as `Surrogate-Key: product-42`); call the purge API after writes.
- Static assets with content hashes in file names can use `max-age=31536000, immutable`.
- Browsers revalidate with `ETag`; make your framework generate weak ETags cheaply or compute them from a version or `updated_at` instead of hashing the full body.
- **Request coalescing** at the proxy (Nginx `proxy_cache_lock on`, Varnish) is the proxy-level version of single-flight.

7.8 Deep dive: profiling a slow Node service step by step

Work like a scientist: reproduce, measure, form a hypothesis, change one thing, measure again.

18. **Reproduce with a load tool** that matches reality (`autocannon` for quick runs, `k6` for scenarios with ramp-up and thresholds). Keep the data volume realistic; a fast query on 100 rows says nothing about 10 million.
19. **Find where time goes.** Look at p50, p95, p99 latency per route and at the error rate. Tail latency (p99) is what users complain about.
20. **Check the event loop.** If loop delay is high, you are CPU-bound or blocking; if loop delay is low but latency is high, you are waiting on something (database, network, lock).
21. **CPU profile** (`node --cpu-prof app.js`, or `clinic flame`, or `0x`): open the flamegraph and look for wide boxes. Typical culprits: JSON parse and stringify of large bodies, synchronous crypto or bcrypt-style hashing without the thread pool, regex backtracking, huge `Array.prototype` chains, template rendering.
22. **Database profile.** Turn on `pg_stat_statements`, sort by total time, and read `EXPLAIN (ANALYZE, BUFFERS)` for the top queries (Part 4).
23. **Trace** a single slow request across services with OpenTelemetry to see which span dominates.
24. **Fix the biggest cost first**, then re-run the *same* test and compare. Keep a table of before and after numbers.

```
import { monitorEventLoopDelay } from 'node:perf_hooks';
const h = monitorEventLoopDelay({ resolution: 20 });
```

```
h.enable();
setInterval(() => {
  logger.info({ p50: h.percentile(50) / 1e6, p99: h.percentile(99) / 1e6, max: h.max
/ 1e6 }, 'event loop delay (ms)');
  h.reset();
}, 10_000).unref();
```

A healthy API usually shows event-loop p99 under 50 ms. Export the numbers as a metric and alert on sustained growth.

Typical wins, roughly in order of payoff: fix N+1 and add missing indexes; cache hot reads; cut payload size and add pagination; parallelise independent calls; reuse connections (keep-alive, pooled clients); move CPU work to workers; compress at the proxy; upgrade the Node.js and library versions (V8 improvements are free). Micro-optimising JavaScript expressions is almost never the answer.

7.9 Deep dive: resilience patterns for dependencies

A slow dependency is more dangerous than a dead one: requests pile up, memory grows, and everything times out. Protect your service with these layers, in this order of importance:

25. **Timeouts** on every outbound call (database statements, HTTP, Redis, queues). Shorter than the caller's own timeout so failures surface where they can be handled.
26. **Retries with exponential backoff and jitter**, only for idempotent operations or those protected by idempotency keys, and with a retry budget (for example at most 10 percent of calls) so retries cannot create a storm.
27. **Circuit breaker:** after enough failures the breaker opens and calls fail immediately (or use a fallback) for a cooldown period, then lets a few trial calls through (half-open).
28. **Bulkheads:** separate pools or concurrency limits per dependency so one slow service cannot consume every connection.
29. **Load shedding:** when overloaded, reject early with `503` and `Retry-After` instead of accepting everything and failing slowly. Prioritise health checks and critical routes.
30. **Fallbacks and graceful degradation:** serve cached or default data, hide a non-essential widget, queue the work for later.

```
import CircuitBreaker from 'opossum';

const breaker = new CircuitBreaker(callPaymentProvider, {
  timeout: 3000,                  // treat calls over 3 s as failures
  errorThresholdPercentage: 50,   // open when half of recent calls fail
  resetTimeout: 15_000,           // try again after 15 s
});
breaker.fallback(() => ({ status: 'pending', note: 'queued for retry' }));
breaker.on('open', () => logger.warn('payment breaker open'));

async function withRetry<T>(fn: () => Promise<T>, tries = 3, baseMs = 200):
Promise<T> {
  for (let i = 0; ; i++) {
    try { return await fn(); }
    catch (e) {
      if (i + 1 >= tries) throw e;
      const delay = Math.random() * baseMs * 2 ** i;       // full jitter
      await new Promise((r) => setTimeout(r, delay));
```

```
    }
  }
}
```

`cockatiel` bundles retry, timeout, circuit breaker, and bulkhead policies in one composable library; `p-limit` and `bottleneck` cap concurrency. Test failure on purpose: stop Redis, slow the database with a network delay tool such as `toxiproxy`, and watch the service degrade instead of collapse.

7.10 Deep dive: rate limiting algorithms and a Redis implementation

Algorithm	How it works	Behaviour
Fixed window	Counter per `key + minute`, reset each minute	Simple; allows a double burst across a window boundary
Sliding window log	Store a timestamp per request in a sorted set; count those in the last N seconds	Precise; memory per request
Sliding window counter	Weighted sum of the current and previous window counters	Good accuracy, low memory
Token bucket	A bucket refills at a steady rate up to a maximum; each request takes a token	Allows bursts but enforces a long-run average; the usual choice
Leaky bucket	Requests queue and drain at a constant rate	Smooths traffic; adds latency
Concurrency limit	Maximum simultaneous in-flight requests per key	Protects expensive endpoints

The check-and-update must be **atomic**, or two instances can both allow the last token. Do it in a Redis Lua script (or a single `INCR` for fixed windows).

```
// Token bucket in Redis (Lua runs atomically on the server)
const script = `
local key, rate, cap, now = KEYS[1], tonumber(ARGV[1]), tonumber(ARGV[2]),
tonumber(ARGV[3])
local d = redis.call('HMGET', key, 'tokens', 'ts')
local tokens = tonumber(d[1]) or cap
local ts = tonumber(d[2]) or now
tokens = math.min(cap, tokens + (now - ts) * rate)
local allowed = 0
if tokens >= 1 then tokens = tokens - 1; allowed = 1 end
redis.call('HSET', key, 'tokens', tokens, 'ts', now)
redis.call('PEXPIRE', key, math.ceil(cap / rate) + 1000)
return allowed
`;
const allowed = await redis.eval(script, 1, `rl:${apiKey}`, 5 / 1000 /* 5 tokens per
second */, 20 /* burst of 20 */, Date.now());
```

Choose keys wisely: per user id when authenticated, per IP when anonymous (remember the `trust proxy` setting), per API key for integrations, and combine (login attempts per IP **and** per email). Return `429` with `Retry-After` and the `RateLimit-*` headers. Decide the failure mode: if Redis is down, **fail open** (allow) for general traffic and **fail closed** for sensitive endpoints such as login. `rate-limiter-flexible` implements most of these algorithms with several stores, so you rarely hand-write the script.

7.11 Deep dive: scaling out in practice

- **Statelessness checklist:** no sessions in memory, no uploaded files on local disk, no in-process schedulers that must run exactly once, no sticky assumptions about which instance handles a request, and config only from the environment.
- **Health checks:** `/healthz` (liveness) returns `200` if the process is running and the event loop is responsive; `/readyz` (readiness) also checks it can serve traffic (database reachable, migrations applied, warm-up done). Do **not** make liveness depend on external services, or a database outage will restart every instance in a loop.
- **Zero-downtime deploys:** rolling update with readiness gates; on `SIGTERM` the instance first fails readiness (so the balancer stops sending traffic), waits a few seconds, then drains in-flight requests and exits. Set the platform's termination grace period longer than your drain time.
- **Autoscaling signals:** CPU is crude for I/O-bound Node services; also consider request concurrency, queue depth for workers, and event-loop delay. Scale workers on queue backlog age.
- **Capacity planning:** from load tests, find requests per second per instance at your latency target; divide expected peak by that figure and add headroom (commonly 30 to 50 percent), and ensure the **database** can take the combined connections and load, because it, not Node, is usually the real limit.
- **Multi-region** is a major step: data replication, conflict handling, and latency. Start with one region, multiple availability zones, a CDN for static and cacheable content, and backups in another region.
- **Cost awareness:** measure cost per request; caching, right-sized instances, and efficient queries usually beat adding servers.

```
let shuttingDown = false;
app.get('/readyz', async (_req, res) => {
  if (shuttingDown) return res.status(503).json({ ready: false });
  try { await db.query('SELECT 1'); await redis.ping(); res.json({ ready: true }); }
  catch { res.status(503).json({ ready: false }); }
});
app.get('/healthz', (_req, res) => res.json({ ok: true }));

process.on('SIGTERM', () => {
  shuttingDown = true;                       // readiness fails immediately
  setTimeout(async () => {                   // give the balancer time to notice
    server.close(async () => { await db.end(); redis.disconnect(); process.exit(0);
});
    setTimeout(() => process.exit(1), 25_000).unref();   // hard stop if draining
hangs
  }, 5_000);
});
```

7.12 More build cards

Card 7.E: production `getOrSet`

- **Aim:** implement the helper in 7.6.
- **Input:** the loader of Card 7.A, Redis, a counter of loader calls.
- **Output:** 200 parallel misses call the loader once per process; a `null` result is cached briefly; stopping Redis does not break the endpoint.
- **Check:** tests for each of those three behaviours.

Card 7.F: stale-while-revalidate

- **Aim:** hide rebuild latency.
- **Input:** a value with a soft TTL of 10 s and a hard TTL of 60 s.
- **Output:** after 10 s the first request returns the old value instantly and triggers one background refresh; the next request sees the new value.
- **Check:** log latency per request; none exceeds 10 ms while a 300 ms rebuild happens in the background.

Card 7.G: token bucket limiter with Lua

- **Aim:** a correct distributed limiter.
- **Input:** the script in 7.10, two server processes, a load script.
- **Output:** a steady 5 requests per second is allowed with bursts to 20; the long-run average never exceeds the rate across both processes.
- **Check:** send 1,000 requests over 60 seconds from the script and count `200` versus `429`.

Card 7.H: circuit breaker drill

- **Aim:** watch the breaker change states.
- **Input:** a flaky fake payment service (fails 70 percent of the time) and an `opossum` breaker.
- **Output:** logs show closed, open, half-open, closed; while open the endpoint answers in under 5 ms with the fallback.
- **Check:** a graph or log timeline of breaker state versus error rate.

Card 7.I: profile and fix a deliberately slow endpoint

- **Aim:** run the full profiling loop.
- **Input:** an endpoint with an N+1, a missing index, and a synchronous `JSON.stringify` of a 20 MB object.
- **Output:** a before and after table of p50, p95, p99, and requests per second for each fix applied one at a time.
- **Check:** the flamegraph before and after, plus the `EXPLAIN` output of the slow query.

Card 7.J: graceful rollout

- **Aim:** zero failed requests during a deploy.
- **Input:** two containers behind Nginx or Traefik, health and readiness endpoints, the shutdown code in 7.11.
- **Output:** restart each instance in turn during a 60 second load test with no errors.

- **Check:** autocannon reports 0 non-2xx responses; remove the readiness-fail step and observe errors appear.

Part 8: Asynchronous processing (queues, jobs, events, outbox, sagas)

Not every task belongs inside the request. Sending an email, resizing an image, generating a report, or calling a slow partner API should happen **after** you answer the user. This part is the heart of scalable backend design.

8.1 Theory: why queues

- **Fast responses**: the API replies `202 Accepted` in milliseconds.
- **Reliability**: if the worker crashes, the job is not lost and is retried.
- **Smoothing spikes**: a burst of 10,000 requests becomes a backlog that workers drain at a steady rate.
- **Isolation**: heavy CPU work runs in separate worker processes and cannot freeze the API.
- **Decoupling**: the producer does not know who consumes the message.

[image: Job queue with workers, retries, and dead-letter set]

Vocabulary

Term	Meaning
Producer / consumer (worker)	Adds a message / processes it
Work queue	Each message is processed by exactly one worker
Pub/Sub (topic)	Each subscriber group gets its own copy
Acknowledgement (ack)	Worker confirms success; without it the message returns to the queue
Retry with backoff	Try again after growing delays (1 s, 5 s, 30 s)
Dead-letter queue (DLQ)	Where messages go after the retries run out, for inspection
Delayed / scheduled job	Run at a future time or on a cron schedule
Concurrency	How many jobs one worker runs in parallel

Delivery guarantees. *At-most-once* can lose messages. *At-least-once* never loses but may deliver twice (the realistic default). *Exactly-once* is mostly a myth across systems; you get **effectively-once** by making consumers **idempotent**: processing the same message twice has the same effect as once (store processed message ids with a unique constraint, or write operations as "set" instead of "add").

8.2 Theory: events and the outbox pattern

In an **event-driven** design, a service announces facts (`OrderPlaced`) and others react. A classic bug: you save an order to the database, then publish an event, and the process crashes in between, so the event is lost. You cannot make a database and a broker commit together. The **transactional outbox** solves it:

write the business row and an `outbox` row in **one database transaction**; a relay process reads unsent outbox rows and publishes them, marking them sent. Delivery is at-least-once, so consumers must be idempotent.

[image: Transactional outbox pattern]

8.3 Theory: sagas for multi-service workflows

A business action spanning services (reserve stock, charge card, create shipment) cannot use one database transaction. A **saga** is a sequence of local transactions, each with a **compensating action** that undoes it if a later step fails (refund the card, release the stock).

- **Choreography**: services react to each other's events. Simple for 2 or 3 steps; hard to follow as it grows.
- **Orchestration**: one coordinator tells each service what to do and handles failures. Easier to reason about; a workflow engine (Temporal) makes this durable.

8.4 Theory: choosing a message system

System	Model	Strengths	Weaknesses	Pick when
BullMQ (Redis)	Job queue	Delays, retries, priorities, repeatable jobs, dashboard (Bull Board), great Node API	Redis durability and memory limits	Background jobs in a Node app (the default choice)
pg-boss / Graphile Worker (PostgreSQL)	Job queue in your DB	No extra infrastructure, transactional enqueue with your data	Lower throughput than Redis	Moderate volume, want the outbox-like guarantee for free
RabbitMQ	Broker with exchanges and queues	Flexible routing, acks, mature	Operate a cluster; not a log	Complex routing, task distribution between services
Kafka (and Redpanda)	Distributed append-only log with partitions and consumer groups	Huge throughput, replay, ordering within a partition, event sourcing	Operational and conceptual weight	Event streaming, analytics pipelines, many consumers per event
AWS SQS + SNS, Google Pub/Sub	Managed queue / topics	No servers to run, scales automatically	Vendor lock-in; fewer features	You are on that cloud already
NATS (JetStream)	Lightweight messaging	Fast, simple	Smaller Node ecosystem	Low-latency service messaging

System	Model	Strengths	Weaknesses	Pick when
Temporal, Inngest, Trigger.dev	Durable workflow engines	Long-running, retried, observable workflows; sagas become code	Extra platform	Multi-step processes with waiting and human steps

8.5 Libraries

Need	Pick	Alternatives
Redis job queue	`bullmq` (+ `@nestjs/bullmq`)	`bee-queue`, `bull` (older)
Queue dashboard	Bull Board, Taskforce	Custom admin page
Postgres queue	`pg-boss`	`graphile-worker`
RabbitMQ	`amqplib`, `amqp-connection-manager`	`rascal`, `@nestjs/microservices` RMQ transport
Kafka	`kafkajs`, `@confluentinc/kafka-javascript`	`@nestjs/microservices` Kafka transport
SQS	`@aws-sdk/client-sqs`	`sqs-consumer`
Cron	`croner`, `node-cron`	`@nestjs/schedule`, BullMQ repeatable jobs, system cron / Kubernetes CronJob
In-process events	`node:events`, `eventemitter3`	`@nestjs/event-emitter`
Workflows	`@temporalio/client` and worker SDK	Inngest, Trigger.dev, `workflow` libraries
Schema contracts for events	Zod / JSON Schema, Avro or Protobuf with a registry	CloudEvents format

8.6 Build cards

Card 8.A: email job with BullMQ

- **Aim:** move slow work out of the request.
- **Input:** `POST /users` creates a user then enqueues `send-welcome-email` (simulate sending with a 3 second delay).
- **Output:** the endpoint answers in under 50 ms; the worker (a separate process) logs "email sent" 3 seconds later.
- **Check:** stop the worker, create users, restart it: all queued emails are processed.

Card 8.B: retries and dead-letter

- **Aim:** failure handling.
- **Input:** a job that fails the first two times (throw on `attempt < 3`), `attempts: 5`, exponential backoff.
- **Output:** logs show attempts at growing intervals and success on the third; a job that always fails ends up in the failed set.
- **Check:** open Bull Board and see the failed job with its error.

Card 8.C: idempotent consumer

- **Aim:** survive duplicate delivery.
- **Input:** a `processed_messages(message_id PRIMARY KEY)` table; a handler that inserts the id and the business effect in one transaction.
- **Output:** delivering the same message 5 times changes the data once.
- **Check:** enqueue the same job id 5 times and compare row counts.

Card 8.D: transactional outbox

- **Aim:** never lose an event.
- **Input:** `POST /orders` writes `orders` and `outbox` in one transaction; a relay loop uses `SELECT ... FOR UPDATE SKIP LOCKED LIMIT 50` and publishes to a queue.
- **Output:** kill the app right after the transaction commits: after restart the relay still publishes the event.
- **Check:** add a `process.exit(1)` between commit and publish in a test, then restart.

Self-check questions: Why can a broker not give you true exactly-once delivery? What is the difference between a work queue and pub/sub? When would you choose Kafka over BullMQ?

8.7 Deep dive: BullMQ and job queue mechanics

Job states. `waiting` (ready), `prioritized`, `delayed` (scheduled for later or waiting between retries), `active` (a worker holds it under a lock), `completed`, `failed`, `waiting-children` (a parent waits for child jobs). A paused queue holds jobs without handing them out.

The options you will use constantly

Option	Effect	Advice
`attempts` + `backoff` (`{ type: 'exponential', delay: 1000 }`)	Retry failed jobs	Set per job type; transient errors retry, permanent errors should not
`delay`	Run later	Reminders, delayed emails
`priority`	Lower number runs first	Use sparingly; priorities cost more in Redis
`jobId`	Custom id; adding a job with an existing id is	A cheap way to dedupe (`send-welcome:user-42`); newer versions also offer dedicated deduplication

Option	Effect	Advice
	ignored	options, check the docs
`removeOnComplete` / `removeOnFail`	Keep only the last N or the last N hours	Without this Redis fills up; keep failures longer than successes
Worker `concurrency`	Parallel jobs per worker	I/O-bound jobs can use 10 to 50; CPU-bound should be 1 per core or use sandboxed processors
Limiter (`{ max, duration }`)	Global rate limit for a queue	Respect third-party API limits (for example 10 emails per second)
Flows (parent and children)	Parent runs after all children finish	Fan out work (resize 5 images) then a final step
Repeatable jobs / job schedulers	Cron-style recurrence stored in Redis	One definition shared by all instances, so it runs once per tick

Locks and stalled jobs. An active job is protected by a lock that the worker renews periodically. If the worker dies or its event loop is blocked so long that it cannot renew, the job is marked **stalled** and given to another worker. So **a job can run twice**; every handler must be idempotent, and CPU-heavy work should run in a sandboxed processor (a separate process or thread) so renewal keeps working.

Operational rules for the Redis behind it. Use `maxmemory-policy noeviction` (an evicted queue key means lost jobs), enable AOF persistence if jobs must survive a Redis restart, give BullMQ its own Redis (or at least its own logical setup) away from your cache, and monitor memory. Keep job payloads small: store ids and fetch data inside the job, not 5 MB blobs.

Worker lifecycle. On `SIGTERM` call `await worker.close()` so it finishes the current jobs and stops taking new ones; deploys that kill workers mid-job create stalled jobs. Listen to `failed`, `error`, and `stalled` events and send them to logs and metrics. Use `QueueEvents` or Bull Board for visibility.

```
const emailQueue = new Queue('email', { connection, defaultJobOptions: {
  attempts: 5, backoff: { type: 'exponential', delay: 2000 },
  removeOnComplete: { age: 3600, count: 1000 }, removeOnFail: { age: 7 * 24 * 3600 },
}});
await emailQueue.add('welcome', { userId }, { jobId: `welcome:${userId}` });

const worker = new Worker('email', async (job) => {
  const user = await users.findById(job.data.userId);       // fetch fresh data
  await sendOnce(`welcome:${user.id}`, () => mailer.send(user)); // idempotent send
}, { connection, concurrency: 10, limiter: { max: 10, duration: 1000 } });
worker.on('failed', (job, err) => log.error({ jobId: job?.id, err }, 'job failed'));
process.on('SIGTERM', async () => { await worker.close(); process.exit(0); });
```

Error classes in workers. Distinguish *transient* (network, 503, deadlock: retry), *permanent* (validation, user deleted: do not retry; in BullMQ throw `UnrecoverableError`), and *poison* messages (always crash a worker; they must reach the failed set after a few attempts rather than loop forever). Add a **dead-letter policy**: alert on failed-set growth, give operators a tool to inspect, fix, and replay jobs, and make replay safe through idempotency.

Ordering and fairness. Plain queues do not guarantee order once concurrency exceeds 1 or retries happen. If order matters per entity (all events for one order), either process that entity's events through a single partition or key (Kafka), use groups (BullMQ Pro) or a queue-per-key pattern, or design handlers to accept out-of-order events by carrying a version or timestamp and ignoring stale ones. To keep one heavy tenant from starving others, use per-tenant rate limits, separate queues for big customers, or weighted scheduling.

Idempotency patterns, concretely

31. **Natural idempotency:** operations written as *set*, not *add* (`status = 'shipped'`, upsert by key).
32. **Idempotency key table:** insert (`message_id`) with a unique constraint in the same transaction as the business change; a duplicate insert fails and the handler exits quietly.
33. **Conditional update:** `UPDATE orders SET status='paid' WHERE id=$1 AND status='pending'`; if zero rows change, it already happened.
34. **External calls:** pass your own idempotency key to the provider (Stripe supports this) so a retried call does not charge twice.
35. **Inbox table** (consumer-side outbox): record received message ids and processing state, so processing and deduplication are transactional.

8.8 Deep dive: Kafka and RabbitMQ concepts

Kafka mental model. A **topic** is split into **partitions**; each partition is an append-only log with ordered **offsets**. Producers choose the partition by **key** (the same key always goes to the same partition, so per-key ordering holds). Consumers join a **consumer group**: every partition is read by exactly one consumer in the group, so the number of partitions caps parallelism, and different groups each get every message independently. Consumers track progress by **committing offsets**. Messages are kept for a retention period (days) or compacted by key, so new consumers can **replay** history, which queues cannot do.

- **At-least-once** is the default pattern: process, then commit the offset; a crash between the two means reprocessing. Auto-commit before processing risks loss.
- **Rebalances** happen when consumers join or leave; during one, processing pauses and duplicates can occur. Use cooperative rebalancing and keep processing time per batch short.
- **Producer settings:** `acks=all`, idempotent producer (`enable.idempotence`) to avoid duplicates on retry, sensible `linger.ms` and batching, and compression (`lz4`, `zstd`). Transactions give exactly-once only for read-process-write *within Kafka*; effects outside Kafka (a database, an email) still need idempotency.
- **Schemas.** Use a schema registry with Avro or Protobuf (or JSON Schema) and compatibility rules, so producers cannot break consumers.
- **Lag** (latest offset minus committed offset) is the key health metric; alert on it growing. A **dead-letter topic** holds messages that failed repeatedly.
- **Node clients:** `kafkajs` is widely used (check its maintenance status before choosing), and `@confluentinc/kafka-javascript` wraps librdkafka for higher throughput and features. Managed options: Confluent Cloud, AWS MSK, Redpanda (Kafka-compatible and simpler to run), Upstash.
- **When it is overkill:** a few thousand jobs per hour, no replay need, a small team. BullMQ or SQS will serve you with far less to run.

RabbitMQ mental model. Producers publish to **exchanges**; **bindings** route messages to **queues**; consumers read from queues and **ack** each message.

Exchange type	Routing	Example
Direct	Exact routing key	`invoice.created` to the billing queue
Topic	Pattern (`order.*`, `*.created`)	Several services subscribe to different event families
Fanout	All bound queues	Broadcast a cache-invalidation message
Headers	Header matching	Rare

Key practices: set **prefetch** (`channel.prefetch(10)`) so one worker does not hoard messages; ack only after the work is done (`nack` with `requeue=false` sends to a **dead-letter exchange**); declare queues as **durable**, messages as persistent, and prefer **quorum queues** over classic mirrored queues for replicated safety; put per-queue and per-message TTLs and a retry topology (a delay queue that dead-letters back to the main queue) for backoff. Reconnect logic is essential (`amqp-connection-manager`).

SQS and similar cloud queues. A received message becomes invisible for a **visibility timeout**; if you do not delete it in time it reappears, so set the timeout longer than processing time or extend it. Standard queues are at-least-once and unordered; FIFO queues give ordering per message group and deduplication within a window. Dead-letter queues and `maxReceiveCount` are built in. Pair SNS (fan-out) with SQS queues per consumer.

Choosing, in one table

Need	Choose
Background jobs, delays, retries, one Node app	BullMQ
Jobs and outbox with no new infrastructure	`pg-boss` or `graphile-worker`
Flexible routing between services, acknowledgements	RabbitMQ
Replayable event log, high volume, many independent consumers	Kafka or Redpanda
Managed, simple, on AWS	SQS + SNS (or EventBridge for routing and schedules)
Long workflows with timers and human steps	Temporal, Inngest, Trigger.dev, AWS Step Functions

8.9 Deep dive: event design and the outbox in practice

Commands versus events. A *command* asks one service to do something and may be rejected (`ReserveStock`). An *event* states a fact that already happened and cannot be refused (`StockReserved`). Name events in the past tense, and publish facts, not instructions.

Event envelope. Every event carries: `id` (unique), `type`, `version`, `occurredAt`, `source`, `subject` or aggregate id, `traceId`, and the `data`. The CloudEvents specification standardises this shape.

Thin versus fat events. A *notification* event holds only ids (`OrderPlaced { orderId }`); consumers call back for details (always fresh, but couples services and adds load). *Event-carried state transfer* includes the data consumers need (decoupled, but you must version the payload and avoid leaking private fields). Prefer fat events for stable, public facts and thin events for volatile or sensitive data.

Evolving events. Only add optional fields; never change meaning; keep old consumers working; support two versions during migrations; test with consumer-driven contracts (Pact) and a schema registry. Events stored for long periods may need **upcasters** that convert old versions when read.

Outbox relay options

Option	How	Trade-off
Polling relay	A worker selects unsent rows with `FOR UPDATE SKIP LOCKED` and publishes	Simple; adds a little latency and DB load; easy to run in Node
Log-based CDC	Debezium reads the PostgreSQL write-ahead log and publishes outbox rows to Kafka	Low latency, no polling load; needs Kafka Connect and operations
LISTEN/NOTIFY wake-up + polling	`NOTIFY` tells the relay to poll now	Lower latency than pure polling; NOTIFY is not durable, so keep the poll as a backstop

Whichever you choose, number events per aggregate, publish in commit order when ordering matters, mark rows sent only after the broker confirms, purge old rows, and make consumers idempotent.

8.10 Deep dive: sagas and workflow engines

[image: A saga with compensating steps]

Designing compensations. A compensation is a new business action, not a database rollback: *refund* the card, *release* the stock, *send a cancellation email*. Some steps cannot be undone (an email already sent); order your saga so the risky and irreversible steps come last, and make every step and compensation idempotent and retryable. Record the saga's state (`started`, `stock_reserved`, `charged`, `completed`, `compensating`, `cancelled`) in a table so a crash resumes where it stopped. Handle timeouts (a payment that never answers) as failures that trigger compensation, and plan for the **semantic lock** problem: during the saga the order is in a pending state that other operations must respect.

Choreography versus orchestration, in practice. With choreography, `orders` emits `OrderPlaced`, `inventory` reacts and emits `StockReserved`, `payments` reacts and emits `PaymentSucceeded`: no central owner, but the flow lives nowhere and cycles are easy to create. With orchestration, one component (or a workflow engine) calls each step and decides what next, so the process is readable in one place. Start with orchestration for anything beyond three steps.

Temporal in a nutshell. You write the workflow as ordinary TypeScript; Temporal records every step in a durable history and replays it after crashes, so the code appears to run exactly once even across restarts and deploys. Rules: workflow code must be **deterministic** (no `Date.now`, `Math.random`, or direct network calls; those go in **activities**), activities are retried by policy, and **signals** and **timers** let a workflow wait days for an event.

```
// workflow.ts (deterministic; activities do the side effects)
const { reserveStock, chargeCard, createShipment, releaseStock, refundCard } =
  proxyActivities<typeof activities>({ startToCloseTimeout: '30s', retry: {
maximumAttempts: 5 } });

export async function placeOrder(order: Order) {
  const undo: Array<() => Promise<void>> = [];
  try {
    await reserveStock(order); undo.push(() => releaseStock(order));
    await chargeCard(order);   undo.push(() => refundCard(order));
    await createShipment(order);
    return 'confirmed';
  } catch (err) {
    for (const fn of undo.reverse()) await fn();
    return 'cancelled';
  }
}
```

Alternatives: Inngest and Trigger.dev (function-based durable execution, simpler to adopt), AWS Step Functions (state machines as JSON), Restate, DBOS, and BullMQ flows for small cases. A hand-rolled saga in a table plus a queue works for two or three steps; beyond that, an engine pays for itself.

8.11 More build cards

Card 8.E: BullMQ with a rate limit and clean shutdown

- **Aim:** a well-behaved worker.
- **Input:** a queue `email` with a limiter of 5 per second, `attempts: 5` with exponential backoff, and `SIGTERM` handling.
- **Output:** 100 queued jobs finish at about 5 per second; killing the worker with `SIGTERM` mid-run lets the active jobs finish and leaves the rest waiting; restart continues with no duplicates.
- **Check:** timestamps in the logs, plus Bull Board counts before and after the restart.

Card 8.F: provoke a stalled job

- **Aim:** see why idempotency is non-negotiable.
- **Input:** a job handler that blocks the event loop for 60 seconds with a busy loop (lock cannot renew).
- **Output:** the job is reported stalled and processed again by another worker; with a business-effect counter you see the effect applied twice.
- **Check:** then fix it with an idempotency key table and move the CPU work into a sandboxed processor; the effect now applies once.

Card 8.G: poison message handling and replay

- **Aim:** operate a dead-letter set.

- **Input:** a job type that always throws for payloads missing a field.
- **Output:** after 3 attempts the job lands in the failed set and raises an alert; a small CLI lists failed jobs, lets you edit the payload, and re-adds them.
- **Check:** replay the fixed jobs twice and confirm the business effect happens once per job.

Card 8.H: ordering per key

- **Aim:** keep events for one order in order.
- **Input:** 5 event types per order across 1,000 orders, produced out of order by 4 concurrent producers; handlers receive them from a queue with concurrency 10.
- **Output:** a version check (`WHERE version = $expected`, or ignore if `incoming.version <= stored.version`) makes the final state correct regardless of arrival order.
- **Check:** a property test that shuffles events and asserts the final state is always identical.

Card 8.I: Kafka or Redpanda consumer group

- **Aim:** feel partitions and offsets.
- **Input:** a Redpanda container, topic `orders` with 3 partitions, keyed producer (`orderId`), and 2 consumers in one group.
- **Output:** each consumer gets a distinct set of partitions; all events for one `orderId` go to one consumer in order; stopping one consumer triggers a rebalance and the other takes over.
- **Check:** log `(partition, offset, key)` per message and verify per-key order and no loss across the rebalance.

Card 8.J: RabbitMQ retry topology

- **Aim:** backoff with dead-lettering.
- **Input:** main queue with a dead-letter exchange to a `retry-10s` queue (TTL 10 seconds) that dead-letters back to the main queue; give up after 3 tries to a final `dead` queue.
- **Output:** a failing message reappears after about 10 seconds up to three times, then sits in `dead`.
- **Check:** read the `x-death` header to count attempts and print the timeline.

Card 8.K: orchestrated saga with compensation

- **Aim:** all-or-nothing across three steps.
- **Input:** the order saga (reserve, charge, ship) as a state table plus a worker, or as a Temporal workflow; each step can be made to fail by a flag.
- **Output:** failing `ship` triggers `refundCard` then `releaseStock` in reverse order and the order ends `cancelled`; killing the orchestrator mid-saga and restarting resumes and still finishes correctly.
- **Check:** run the saga with a failure injected at each step and in each compensation, and assert the final inventory and payments tables are consistent.

Card 8.L: outbox with a polling relay and a crash

- **Aim:** extend Card 8.D with ordering and metrics.
- **Input:** relay publishing in `id` order per aggregate, marking rows sent only after broker confirmation, plus a gauge for `oldest unsent age`.

- **Output:** killing the relay mid-batch leads to duplicates at most (never loss), consumers deduplicate, and the gauge returns to near zero after restart.
- **Check:** chaos script: kill the relay at random times during 10,000 orders and compare produced orders with processed events.

Deep-dive self-check: (1) Why can a BullMQ job run twice even without a bug in your code? (2) What limits consumer parallelism in Kafka and why? (3) When is a message acked in a safe consumer and what failure does that choice allow? (4) What does an idempotency key table protect you from? (5) Why must Temporal workflow code be deterministic? (6) How do thin and fat events differ in coupling?

Part 8 deep dive

8.7 BullMQ in depth

BullMQ stores jobs in Redis. You run two kinds of process from one code base: the **API** (adds jobs) and **workers** (process them), deployed and scaled separately.

```
import { Queue, Worker, UnrecoverableError } from 'bullmq';
import IORedis from 'ioredis';

const connection = new IORedis(config.REDIS_URL!, { maxRetriesPerRequest: null });
// required for workers

export const emailQueue = new Queue('email', {
  connection,
  defaultJobOptions: {
    attempts: 5,
    backoff: { type: 'exponential', delay: 2000 },            // 2 s, 4 s, 8 s ...
    removeOnComplete: { age: 3600, count: 1000 },             // keep Redis small
    removeOnFail: { age: 7 * 24 * 3600 },                     // keep failures a week
for inspection
  },
});

// Producer: a deterministic jobId de-duplicates (adding the same id again is ignored
while it exists)
await emailQueue.add('welcome', { userId }, { jobId: `welcome:${userId}` });
await emailQueue.add('reminder', { userId }, { delay: 24 * 3600 * 1000, priority: 5
});

// Worker process
const worker = new Worker('email', async (job) => {
  const user = await users.find(job.data.userId);
  if (!user) throw new UnrecoverableError('user deleted');   // do NOT retry what can
never succeed
  await mailer.send(user.email, render(job.name, user));       // must be idempotent:
may run twice
}, { connection, concurrency: 10, limiter: { max: 50, duration: 1000 } });  // at
most 50 jobs/second

worker.on('failed', (job, err) => logger.error({ jobId: job?.id, attempts:
job?.attemptsMade, err }, 'job failed'));
```

```
process.on('SIGTERM', async () => { await worker.close(); process.exit(0); });   // finish current jobs first
```
Things that surprise people:

- **Stalled jobs.** A worker holds a lock on its job and renews it. If the process dies or blocks the event loop so long the lock lapses, BullMQ considers the job stalled and gives it to another worker, so a job can run **twice**. This is exactly why handlers must be idempotent and why you must not block the loop inside a worker (move CPU work into a sandboxed processor or `worker_threads`).
- **Redis eviction.** Set `maxmemory-policy noeviction` on the Redis that holds queues; an eviction policy can silently delete job data.
- **Retries versus permanent failures.** Retry transient errors (timeouts, 429, 503); throw `UnrecoverableError` for bad input and missing records so they go straight to failed.
- **Fairness.** One tenant enqueuing 100 000 jobs starves the others in a shared queue. Use separate queues or priorities per tier, a per-tenant rate limit, or BullMQ's group features (check your version's docs).
- **Flows.** `FlowProducer` models parent and child jobs (run these three, then that one).
- **Repeating work.** Newer BullMQ versions use *job schedulers* (`upsertJobScheduler` with a cron `pattern`); older versions used the `repeat` option. Check the docs for your version, and make the scheduled job idempotent.
- **Observe it.** Export queue depth, oldest waiting job age, active, failed, and retry counts as metrics; **age of the oldest job** is the best alert signal ("emails are 20 minutes late"). Bull Board gives a quick UI.

Job design rules: put **ids, not data**, in the payload (re-read current state in the worker, so a retry sees fresh data and the payload stays small); make every job idempotent; give long jobs a timeout and report progress; version the payload shape (`{ v: 2, ... }`) because jobs outlive deploys; and have a documented process for the failed set (inspect, fix, replay).

8.8 A Postgres-backed queue from scratch

When you already have PostgreSQL and moderate volume, a jobs table avoids a new moving part and lets you **enqueue in the same transaction** as your business write (the main reason to choose it).

```
CREATE TABLE jobs (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  queue text NOT NULL, payload jsonb NOT NULL,
  status text NOT NULL DEFAULT 'queued',       -- queued | running | done | failed
  run_at timestamptz NOT NULL DEFAULT now(),
  attempts int NOT NULL DEFAULT 0, max_attempts int NOT NULL DEFAULT 5,
  locked_at timestamptz, last_error text
);
CREATE INDEX jobs_ready_idx ON jobs (queue, run_at) WHERE status = 'queued';   -- partial index: only waiting rows

-- Claim up to 10 jobs; SKIP LOCKED lets many workers pull different rows without blocking each other
WITH next AS (
  SELECT id FROM jobs
  WHERE queue = $1 AND status = 'queued' AND run_at <= now()
```

```
  ORDER BY run_at, id FOR UPDATE SKIP LOCKED LIMIT 10
)
UPDATE jobs SET status = 'running', locked_at = now(), attempts = attempts + 1
FROM next WHERE jobs.id = next.id RETURNING jobs.*;
```

The rest: on success `UPDATE ... SET status='done'`; on failure set `status='queued'` with `run_at = now() + backoff` (or `failed` when `attempts >= max_attempts`); a **reaper** query returns `running` jobs whose `locked_at` is older than a timeout to `queued` (a visibility timeout, like SQS). Poll every second or two, or wake workers with `LISTEN/NOTIFY`. Libraries that implement all of this well: `pg-boss`, `graphile-worker`.

8.9 Idempotent consumers and the inbox table

Because delivery is at-least-once, record which messages you have handled **in the same transaction** as the effect, so a crash cannot separate the two:

```
export async function handleOnce(msg: { id: string; type: string; data: any }) {
  await withTx(pool, async (c) => {
    const r = await c.query('INSERT INTO processed_messages (message_id) VALUES ($1)
ON CONFLICT DO NOTHING', [msg.id]);
    if (r.rowCount === 0) return;                        // duplicate delivery: do
nothing
    await applyBusinessEffect(c, msg);                   // same transaction: both
commit or neither does
  });
}
```

Some effects are naturally idempotent (`SET status = 'paid'`, `INSERT ... ON CONFLICT DO NOTHING`); others are not (`balance = balance + 10`, sending an email). For non-idempotent external effects, pass the message id as the provider's **idempotency key** so the *provider* deduplicates. Clean the table by age, keeping it longer than your broker's maximum redelivery window.

8.10 The outbox relay in code

```
CREATE TABLE outbox (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  aggregate_id text NOT NULL, topic text NOT NULL, payload jsonb NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(), sent_at timestamptz
);
CREATE INDEX outbox_unsent_idx ON outbox (id) WHERE sent_at IS NULL;
```

```
// In the business transaction: insert the order AND the event row together.
await c.query('INSERT INTO outbox (aggregate_id, topic, payload) VALUES ($1, $2,
$3)',
  [order.id, 'orders.v1', { id: crypto.randomUUID(), type: 'OrderPlaced', occurredAt:
new Date(), data: { orderId: order.id } }]);

// Relay loop (run in a worker; several relays can run safely thanks to SKIP LOCKED)
export async function relayBatch(): Promise<number> {
  const client = await pool.connect();
  try {
    await client.query('BEGIN');
    const { rows } = await client.query(
      'SELECT id, topic, payload FROM outbox WHERE sent_at IS NULL ORDER BY id FOR
```

```
UPDATE SKIP LOCKED LIMIT 50');
    for (const r of rows) await broker.publish(r.topic, r.payload, { key:
r.payload.data.orderId });
    if (rows.length) await client.query('UPDATE outbox SET sent_at = now() WHERE id =
ANY($1)', [rows.map((r) => r.id)]);
    await client.query('COMMIT');
    return rows.length;
  } catch (e) { await client.query('ROLLBACK'); throw e; } finally {
client.release(); }
}
```

If the process crashes after publishing but before commit, the rows are published again on the next run: **at-least-once**, which is why consumers use the inbox. Practical notes: use the aggregate id as the broker **partition key** so events about one order stay ordered; delete or archive old sent rows; poll every few hundred milliseconds or use `LISTEN/NOTIFY` to wake the relay; for very high volume, **change data capture** (Debezium reading the PostgreSQL write-ahead log) replaces polling and publishes with lower latency and no extra query load.

8.11 Kafka concepts you need before using it

- A **topic** is split into **partitions**, each an ordered, append-only log. Ordering is guaranteed **only within a partition**; the message **key** decides the partition (same key, same partition, so all events for one order stay in order).
- A **consumer group** shares a topic's partitions among its members: each partition is read by exactly one member of the group, so parallelism is capped by the partition count. Different groups each get every message independently (the pub/sub behaviour).
- Consumers track an **offset** per partition. **Commit the offset only after processing** (at-least-once); committing before gives at-most-once and can lose messages.
- Messages are **retained** for a period (days or forever) regardless of consumption, so you can **replay** or add a new consumer that reads history. **Log compaction** keeps only the latest value per key (a changelog or table).
- **Rebalances** (a member joins or leaves) pause consumption briefly; keep handlers fast and heartbeats alive.
- Producers can be made **idempotent** (no duplicates on retry) and Kafka supports transactions for exactly-once *within Kafka*; end-to-end exactly-once with an outside database still needs an idempotent consumer.
- Use a **schema registry** (Avro or Protobuf) so producers and consumers evolve schemas safely.

```
import { Kafka } from 'kafkajs';
const kafka = new Kafka({ clientId: 'orders-api', brokers:
config.KAFKA_BROKERS.split(',') });

const producer = kafka.producer({ idempotent: true });
await producer.connect();
await producer.send({ topic: 'orders.v1', messages: [{ key: order.id, value:
JSON.stringify(evt) }] });

const consumer = kafka.consumer({ groupId: 'email-service' });
await consumer.connect();
await consumer.subscribe({ topic: 'orders.v1', fromBeginning: false });
await consumer.run({
```

```
  eachMessage: async ({ message }) => { await
handleOnce(JSON.parse(message.value!.toString())); },
});   // offset is committed after eachMessage resolves
```

Check the maintenance status and docs of the client library you choose (`kafkajs` is widely used; `@confluentinc/kafka-javascript` is Confluent's client). Kafka is powerful but operationally heavy: consider a managed service (Confluent Cloud, MSK, Redpanda Cloud) unless you have a clear need for its throughput, replay, or many independent consumers.

8.12 RabbitMQ and managed queues

RabbitMQ routes messages: a producer publishes to an **exchange**, which routes to **queues** by **bindings** (`direct` by exact key, `topic` by patterns like `orders.*.created`, `fanout` to all, `headers`). Consumers **ack** after processing (an unacked message is redelivered if the connection drops) and set a **prefetch** count to limit in-flight messages per consumer. Retries are modelled with a **dead-letter exchange** plus per-queue TTL (a delay queue). Use **quorum queues** for replicated durability. Messages are removed when acked: there is no replay.

	BullMQ	RabbitMQ	Kafka	SQS (+SNS)
Model	Redis job queue	Smart broker, routing	Distributed log	Managed queue / topic
Replay old messages	No	No	Yes	No
Ordering	Per queue (best effort with concurrency)	Per queue	Per partition	FIFO queues only
Delays, retries, priorities	Built in	Via TTL and DLX plugins	Build it yourself	Delay and DLQ built in
Ops burden	Run Redis	Run a cluster	Highest	None
Choose when	Node app background jobs	Complex routing between services	Event streaming, many consumers	You are on AWS and want no servers

8.13 Event and message design

- **Command or event?** A *command* asks one service to do something (`ChargeCard`, imperative, can be rejected). An *event* states a fact that already happened (`PaymentSucceeded`, past tense, cannot be rejected). Events decouple producers from consumers; do not disguise commands as events.
- **Use an envelope** with a unique `id`, a `type`, `occurredAt`, a payload `version`, and a `correlationId` that follows the whole business flow:

```
export const OrderPlaced = z.object({
  id: z.string().uuid(), type: z.literal('OrderPlaced'), version: z.literal(1),
```

```
  occurredAt: z.coerce.date(), correlationId: z.string(),
  data: z.object({ orderId: z.string(), userId: z.string(), totalCents:
z.number().int() }),
});
```

- **Thin versus fat events.** A thin event carries ids and consumers call back for details (always fresh, but couples services and adds load). A fat event carries the data consumers need (**event-carried state transfer**: no callbacks, but the schema becomes a contract and data may be stale).
- **Evolve compatibly:** add optional fields, never repurpose or remove fields, and introduce `version: 2` side by side. Consumers must **ignore unknown fields**.
- **Poison messages** (always fail) must not block a queue: limit retries, then park them in a dead-letter destination with the error, and alert.
- Always assume **duplicates and reordering**; carry a version or timestamp so a consumer can ignore an older update that arrives after a newer one.

8.14 Sagas in depth

[image: Order saga with compensations]

The saga above is an **orchestration**: a coordinator owns the state machine and decides what happens next. Rules for making it correct:

- Each step is a **local transaction** in one service. Each step that can be undone has a **compensation** (`refund`, `release stock`); a compensation is *semantic*, not a rollback (a refund is a new transaction, and an email already sent cannot be unsent).
- **Compensations and steps must be idempotent and retryable**, because the coordinator itself may crash and resume.
- Order steps so the **hardest-to-undo step (the pivot) comes last**: reserve stock and validate first, charge the card, and only then do irreversible things like shipping labels.
- Persist the saga **state** (`started`, `stock_reserved`, `paid`, `failed`, ...) and a deadline for each step; a timeout is a failure.
- Provide a **manual resolution path** for stuck sagas; monitoring stuck instances is part of the design.

A workflow engine makes the coordinator durable (it persists progress and retries activities, resuming after crashes). Simplified Temporal-style code:

```
import { proxyActivities } from '@temporalio/workflow';
const a = proxyActivities<typeof activities>({ startToCloseTimeout: '30 seconds',
retry: { maximumAttempts: 5 } });

export async function orderSaga(order: Order): Promise<void> {
  const compensations: Array<() => Promise<void>> = [];
  try {
    await a.reserveStock(order);   compensations.unshift(() =>
a.releaseStock(order));
    await a.chargeCard(order);     compensations.unshift(() => a.refundCard(order));
    await a.createShipment(order);
  } catch (err) {
    for (const undo of compensations) await undo();   // newest first
    throw err;
```

```
  }
}
```

Choreography (services react to each other's events) needs no coordinator and is great for two or three steps, but the flow is implicit: nobody can point to one place that shows the whole process, and cyclic dependencies sneak in. Move to orchestration when you cannot draw the flow from memory.

8.15 Scheduled work without double-running

With several instances, a naive `setInterval` or `node-cron` runs the job on every instance. Options: a **single scheduler** (a Kubernetes `CronJob`, or BullMQ's scheduler that enqueues one job per tick for any worker to take), or a **lock**. For a Postgres advisory lock, remember it belongs to a *connection*, so hold one dedicated connection (not a pooled one released between statements):

```
const client = await pool.connect();                    // keep this connection for the lifetime of the leader role
const { rows: [r] } = await client.query('SELECT pg_try_advisory_lock($1) AS got', [4242]);
if (r.got) { /* I am the leader: run the scheduler loop here */ }
// the lock is released automatically if this connection or process dies
```

Decide up front: what happens to **missed runs** after downtime (skip, or catch up), how to handle **overlap** (a job still running when the next tick fires), and **time zones and daylight saving** (schedule in UTC unless the business rule is local time). Make every run idempotent and record each run's start, end, and outcome.

8.16 More build cards

Card 8.E: production-grade worker

- **Aim:** a worker that behaves well under failure.
- **Input:** the BullMQ setup in 8.7 with `concurrency: 5` and an email job that sometimes throws a transient error and sometimes `UnrecoverableError`.
- **Output:** transient errors retry with backoff; unrecoverable ones go straight to failed; `kill -TERM` during processing lets the in-flight job finish; a job that blocks the loop for 40 s is detected as stalled and re-run (demonstrate why idempotency matters).
- **Check:** logs and Bull Board screenshots of each case.

Card 8.F: Postgres SKIP LOCKED queue

- **Aim:** understand queues by building one.
- **Input:** the `jobs` table and claim query above; three worker processes; 1 000 jobs.
- **Output:** every job runs exactly once under normal operation; killing a worker mid-job leaves a `running` row that the reaper re-queues; total run is faster with three workers than with one.
- **Check:** a counter table incremented once per job and compared to 1 000.

Card 8.G: outbox end to end with a duplicate-tolerant consumer

- **Aim:** prove at-least-once plus idempotency equals correct.
- **Input:** `orders` and `outbox` tables, the relay in 8.10, a consumer using `handleOnce`, and a fault injector that randomly throws after publishing.

- **Output:** despite injected crashes and duplicate deliveries, each order produces exactly one downstream effect.
- **Check:** reconcile order count against effect-row count after 1 000 orders.

Card 8.H: Kafka ordering experiment

- **Aim:** see partition ordering.
- **Input:** a local Kafka (or Redpanda) in Docker, a topic with 3 partitions, a producer sending 1 000 events for 10 order ids.
- **Output:** with the order id as key, each order's events are consumed in order; with no key, you observe reordering across partitions; with two consumers in one group, partitions split between them, and a third consumer sits idle if you have only 2 partitions.
- **Check:** log `partition` and `offset` per message and analyse the output.

Card 8.I: saga with failure injection

- **Aim:** verify compensations.
- **Input:** the three-step order saga with each step able to fail on demand.
- **Output:** a table of (failing step) to (final state and compensations run): failing shipment refunds and releases stock; failing payment only releases stock; compensations that themselves fail are retried.
- **Check:** automated test over all failure points, asserting stock and money are always consistent at the end.

Card 8.J: dead-letter inspection and replay

- **Aim:** operate failures, not just cause them.
- **Input:** a queue with a poison message and a bug that makes 5 percent of jobs fail.
- **Output:** a small admin script or endpoint that lists failed jobs with errors, and replays selected ones after the bug is fixed, without duplicating side effects.
- **Check:** replay the same failed set twice and confirm no duplicate effects.

8.17 Answer key for the Part 8 self-checks

- **Why can a broker not give true exactly-once delivery?** A message and its acknowledgement can each be lost, so after a failure the broker cannot know whether the consumer finished; to never lose messages it must redeliver, which can duplicate. You get *effectively once* by making the consumer idempotent.
- **Work queue versus pub/sub?** In a work queue each message goes to exactly one of many competing workers (sharing load); in pub/sub each subscriber (or consumer group) receives its own copy of every message (broadcasting a fact).
- **When Kafka over BullMQ?** When you need a durable, replayable log, very high throughput, per-key ordering at scale, or many independent consumers of the same events; for ordinary background jobs inside one Node application BullMQ is simpler and sufficient.

Part 9: System design scenarios (what to use, and the alternatives)

Interviews and real projects both ask the same thing: *given this requirement, which pieces do you pick and why?* Use this method every time:

36. **Clarify requirements**: functional (what it does) and non-functional (users, requests per second, latency, availability, data size, consistency needs).
37. **Estimate**: requests per second, storage per year, read/write ratio.
38. **Sketch the simplest design** that could work, then find the bottleneck.
39. **Name trade-offs** and alternatives out loud.

9.1 Scenario: URL shortener (read-heavy)

Requirements: create short links, redirect fast, optional expiry, click counts. Reads outnumber writes about 100 to 1.

[image: URL shortener architecture]

- **ID generation:** auto-increment id encoded in base62 (`nanoid` or `base-x`), or random 7-character codes with a unique constraint and retry on collision. A counter service or Snowflake-style ids when you have several writers.
- **Storage:** PostgreSQL is plenty to billions of rows with an index on `code`. DynamoDB or Cassandra when you truly need massive scale.
- **Speed:** cache `code -> url` in Redis; return `301` (cacheable, fewer hits) or `302` (every click reaches you, better for analytics).
- **Analytics:** never write click counts in the redirect path. Emit an event to a queue (BullMQ or Kafka) and aggregate in a worker.
- **Abuse:** rate limit creation, validate URLs (no `javascript:`), block private IP targets.

Choice	Pick	Alternative
Framework	Fastify	Hono, Express
Database	PostgreSQL via Drizzle or Prisma	DynamoDB, MongoDB
Cache	Redis (`ioredis`)	In-process `lru-cache` per instance for the hottest codes
Analytics	BullMQ then a counters table	Kafka + ClickHouse

9.2 Scenario: file uploads and media

Never stream big files through your API servers into your own disk. Use **presigned URLs**: the API authorises the upload and returns a short-lived URL; the client uploads directly to object storage (S3, Cloudflare R2, Google Cloud Storage, MinIO); the API stores only metadata. Then a worker generates thumbnails.

- Libraries: `@aws-sdk/client-s3` + `@aws-sdk/s3-request-presigner`; `multer` or `@fastify/multipart` or `busboy` when you must accept multipart on the server; `sharp` for

image resizing; `file-type` to check real content type; `tus` (resumable uploads) or S3 multipart for files over 100 MB; `clamav` for virus scanning.
- Checks: size limit, allowed MIME types, random object keys (never trust user filenames), private buckets with signed download URLs.

9.3 Scenario: real-time chat or notifications (stateful connections)

One server cannot hold every user, and Alice and Bob may be connected to different servers. Use **Redis pub/sub** (the Socket.IO Redis adapter) so a message published on instance 1 reaches the socket on instance 2. The load balancer needs sticky sessions for Socket.IO's HTTP long-polling fallback, or you force the WebSocket transport.

[image: Scaling WebSockets with Redis pub/sub]

- **Persist** messages in PostgreSQL (or Cassandra/ScyllaDB at huge scale); push is a delivery optimisation, not the source of truth.
- **Presence** (online/offline) in Redis with TTL keys refreshed by heartbeats.
- **Offline users:** store notifications, send push via FCM/APNs, or email through a queue.
- **Alternatives:** managed services (Ably, Pusher, Supabase Realtime), Centrifugo, NATS, Redis Streams or Kafka when you need replay and ordering.

9.4 Scenario: search

- **Small to medium:** PostgreSQL full-text search (`tsvector`, GIN index) or trigram (`pg_trgm`) for typo tolerance.
- **Product-grade search** (typos, facets, ranking, autocomplete): Meilisearch or Typesense (simple, fast), Elasticsearch or OpenSearch (powerful, heavier), Algolia (hosted).
- **Sync strategy:** the database is the source of truth. Update the search index through events or an outbox worker, and design a re-index job from scratch for recovery.
- **Client libraries:** `meilisearch`, `typesense`, `@elastic/elasticsearch`, `@opensearch-project/opensearch`.

9.5 Scenario: payments and webhooks

- Use a provider (Stripe, Razorpay, Adyen) and **never touch raw card numbers**.
- Store your own `payments` table with a status machine (`pending`, `succeeded`, `failed`, `refunded`); only forward transitions.
- **Webhooks:** verify the signature using the raw request body (so keep the raw bytes, not the parsed JSON), respond `200` quickly, push the work to a queue, and handle duplicates by event id.
- **Idempotency keys** on every create call (Part 6). Reconcile with the provider daily because webhooks can be missed.
- Money: store integers in the smallest unit (paise or cents) plus currency; never floating point.

9.6 Scenario: scheduled and long-running jobs

- Cron-like tasks: BullMQ repeatable jobs or a Kubernetes CronJob. Ensure only **one** instance runs the task (a distributed lock or a queue guarantees it).

- Long workflows with waits and retries (onboarding emails over 14 days, order fulfilment): Temporal or Inngest instead of hand-written timers.
- Reports and exports: enqueue a job, write the file to object storage, notify the user with a download link.

9.7 Scenario: multi-tenant SaaS

Model	How	Pros	Cons
Shared tables with `tenant_id`	One schema; every query filters by tenant	Cheapest, simplest	One missed filter leaks data; use PostgreSQL **row-level security** as a safety net
Schema per tenant	Separate PostgreSQL schema	Better isolation	Migrations multiply
Database per tenant	Separate DB	Strongest isolation, per-tenant backup	Costly, connection management

Resolve the tenant once (subdomain, header, or token claim), carry it in `AsyncLocalStorage`, and add tests that try to read another tenant's data.

9.8 Scenario: high-traffic read API (feed, catalogue)

CDN for public GETs, Redis for hot objects, read replicas for the database, pagination with cursors, precomputed feeds written by background workers ("fan-out on write") for users with few followers and computed at read time ("fan-out on read") for celebrities. Measure with real traffic shapes before choosing.

9.9 Quick pattern-to-library map

Technique	Typical Node library	Other options
Retry with backoff	`p-retry`, `async-retry`	BullMQ built-in for jobs
Circuit breaker (stop calling a failing service)	`opossum`	`cockatiel` (also retry, timeout, bulkhead)
Timeout / cancellation	`AbortSignal.timeout(ms)`	`p-timeout`
Concurrency limit	`p-limit`, `p-queue`	`bottleneck`
Distributed lock	`redlock`	Postgres advisory locks
Feature flags	OpenFeature, Unleash, GrowthBook	Config table
Request context	`AsyncLocalStorage`	`nestjs-cls`
Dependency injection outside Nest	`awilix`, `tsyringe`, `inversify`	Manual wiring (often enough)

Technique	Typical Node library	Other options
Domain events	`eventemitter3`	`@nestjs/event-emitter`
Date and money maths	`date-fns`, `dinero.js` or integers	`big.js`, `decimal.js`
Email	`nodemailer` + provider (SES, Resend, Postmark, SendGrid)	Templating with `mjml`, `react-email`
PDF / reports	`pdfkit`, Puppeteer or Playwright render	`pdf-lib`
Web scraping / HTTP	`undici`, `cheerio`	Playwright
Geospatial	PostGIS	`h3-js`, Redis GEO

9.10 Build cards (design practice)

Card 9.A: build the URL shortener end to end

- **Aim:** apply Parts 2 to 8 in one project.
- **Input:** `POST /shorten {url}`, `GET /:code`, `GET /stats/:code`.
- **Output:** redirects served from Redis after the first hit; click events flow through a queue; creation limited to 10 per minute per IP.
- **Check:** load test the redirect with autocannon; record requests per second with and without Redis.

Card 9.B: presigned upload

- **Aim:** keep file bytes off your server.
- **Input:** `POST /uploads {filename, contentType, size}` and a local MinIO container (S3 compatible).
- **Output:** the API returns a presigned `PUT` URL; `curl --upload-file` to that URL succeeds; a worker creates a 200 px thumbnail with `sharp`.
- **Check:** reject a 50 MB file and a `.exe` content type at the API step.

Card 9.C: chat across two instances

- **Aim:** prove cross-server broadcast.
- **Input:** run two Socket.IO servers on ports 3001 and 3002 with the Redis adapter.
- **Output:** a client on 3001 and a client on 3002 in the same room receive each other's messages.
- **Check:** stop Redis and observe what breaks; restart and see recovery.

Card 9.D: Stripe-style webhook receiver

- **Aim:** secure and idempotent webhooks.
- **Input:** a `POST /webhooks/pay` route that reads the raw body and an `X-Signature` HMAC header.
- **Output:** bad signature gives `400`; duplicate event id is ignored; valid event enqueues a job and returns `200` in under 100 ms.

- **Check:** replay the same payload twice with a script and compare database rows.

Self-check questions: Why presigned URLs instead of uploading through the API? Why is the database, not the search engine, the source of truth? What can go wrong with shared-table multi-tenancy, and what is the safety net?

9.11 Deep dive: estimation and the design template

Back-of-envelope numbers worth memorising. One day is about 86,400 seconds (round to 100,000); one month about 2.6 million seconds. A single well-tuned Node instance serves roughly 1,000 to 10,000 simple requests per second, but only 100 to 1,000 when each request makes database calls (your load test decides). PostgreSQL on one decent machine handles thousands of simple indexed reads per second and a few thousand writes per second; Redis handles tens to hundreds of thousands of operations per second. A memory read is about 100 ns, a Redis call within a data centre about 0.5 to 1 ms, a database indexed read 1 to 5 ms, a cross-region round trip 50 to 150 ms.

Worked example: the URL shortener. Assume 100 million new links per month and a 100:1 read ratio.

Quantity	Calculation	Result
Writes per second (average)	100,000,000 / 2,600,000	about 40 per second
Reads per second (average)	40 x 100	about 4,000 per second
Peak reads (x5)	4,000 x 5	about 20,000 per second
Storage per month	100,000,000 x about 500 bytes	about 50 GB; about 3 TB over 5 years
Code space	62^7	about 3.5 trillion codes, far above 6 billion links in 5 years
Cache	20 percent of links give most traffic	About 20 percent of recent links in Redis fits in a few GB

Conclusions you can defend: one PostgreSQL primary is enough for writes; reads at 20,000 per second need Redis in front (or replicas); the redirect service is stateless, so run 3 to 5 instances for redundancy; analytics must be asynchronous.

The design template (use it for any scenario, in an interview or a design document):

40. **Requirements.** Functional (user stories), non-functional (scale, latency, availability target, durability, consistency, security, cost), and explicit non-goals.
41. **API.** The main endpoints or events with request and response shapes.
42. **Data model.** Entities, keys, relationships, access patterns (the queries that must be fast), size and growth.
43. **High-level design.** A box-and-arrow diagram of components and data flow; the simplest design that meets the requirements.

44. **Deep dives.** The two or three hardest parts (hot spots, consistency, failure handling).
45. **Scale and failure.** What breaks first at 10x; replication, caching, queues, partitioning; what happens when each component fails; degradation plan.
46. **Operations.** Metrics and alerts, deployment and rollback, migrations, security, cost.
47. **Trade-offs and alternatives.** What you chose, what you rejected, and what would make you change your mind.

Consistency models you should be able to name. *Strong* (every read sees the latest write; a single primary database), *read-your-writes* (a user sees their own changes; route that user to the primary for a short time), *monotonic reads* (time never goes backwards for a user; pin them to one replica), *eventual* (replicas converge later; fine for counters, feeds). The CAP theorem says that during a network partition you choose consistency or availability; PACELC adds that even without partitions you trade latency for consistency. Decide per feature: a bank balance needs strong consistency, a like count does not.

Unique id generation

Approach	Properties	Caveats
Database sequence / identity	Compact, ordered, simple	Single writer; guessable; leaks volume
UUID v4	No coordination	Random, so poor index locality on large tables
UUID v7 / ULID	Time-ordered, no coordination	Reveals creation time
Snowflake style (timestamp + machine id + sequence)	64-bit, sortable, high rate	Needs unique machine ids and clock care
Hash of content	Deduplicates	Collisions to consider; not secret
Random short code (nanoid)	Short, unguessable	Check collisions with a unique index

9.12 Deep dive: more scenarios

Notification system (email, SMS, push, in-app). Components: an API that accepts a *notification request* (user, template, data, priority, idempotency key); a **preferences** service (channels per user and topic, quiet hours, unsubscribes, which law requires you to honour); a **templating** step (`mjml`, `handlebars`, localisation); one **queue per channel** so a slow SMS provider cannot block email; **workers** that call providers with rate limits and retries; a **delivery log** table (queued, sent, delivered, bounced, failed) updated by provider webhooks; and **provider failover** (SES then Postmark; Twilio then another SMS vendor) behind a circuit breaker. Design points: deduplicate with the idempotency key; set a TTL (an OTP older than 10 minutes must not be sent); batch low-priority messages into a **digest**; protect from your own bugs with a per-user rate cap (a loop must never send 1,000 emails); suppress addresses that bounced or complained; sign unsubscribe links. Libraries: `nodemailer`, `@aws-sdk/client-ses`, `web-push`, `firebase-admin` (FCM), `twilio`, `bullmq`.

News feed or timeline. The central choice is *when* to build each user's feed. **Fan-out on write**: when someone posts, push the post id into the feed list of every follower (a Redis sorted set per user, capped at a few hundred). Reads are instant; writes are expensive for accounts with millions of followers. **Fan-out on read**: build the feed at request time by merging recent posts of everyone followed. Writes are cheap; reads are expensive. The standard answer is a **hybrid**: fan out on write for ordinary accounts, and for celebrity accounts merge their recent posts at read time. Store posts once in the database; feeds hold ids; hydrate ids to objects through a cache. Rank with a score (time, engagement) in a separate service or by sorted-set score. Paginate by cursor.

E-commerce checkout and inventory. Prevent overselling with an atomic conditional update, not read-then-write: `UPDATE stock SET qty = qty - $1 WHERE sku = $2 AND qty >= $1` and check that one row changed. For carts that hold stock, create a **reservation** with an expiry (a row or Redis key with TTL) and release it with a delayed job when it expires; confirm it when payment succeeds. Use the saga from Part 8 for payment, stock, and shipping; use idempotency keys at every step; reprice on the server (never trust a client price); keep an immutable **order snapshot** (price, tax, address at purchase time) rather than joining to mutable product data.

Booking and reservations (no double booking). Let the database enforce the rule. With PostgreSQL range types and an exclusion constraint, two overlapping bookings for the same resource simply cannot be inserted:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE TABLE bookings (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  room_id bigint NOT NULL,
  during tstzrange NOT NULL,
  EXCLUDE USING gist (room_id WITH =, during WITH &&)
);
-- a second overlapping insert fails with SQLSTATE 23P01 (exclusion violation) ->
return 409
```

Add a short **hold** (pending booking with expiry) during checkout, and handle time zones by storing instants in UTC plus the resource's zone.

Geo and location features. PostGIS (`geography` type, GiST index, `ST_DWithin` for "within 5 km") is the default and handles most needs. Redis `GEOADD`/`GEOSEARCH` suits frequently updated points (drivers). For uniform grid bucketing and analytics use H3 (`h3-js`) or geohash cells. Do not compute distances in Node by loading every row.

Leaderboards and counters. Redis sorted sets for ranks; periodically flush to the database for durability. For huge counters (views, likes) buffer increments in Redis or memory and write batched updates (`UPDATE ... SET n = n + $delta`) to avoid hot rows; accept approximate counts where fine (HyperLogLog for unique counts).

Analytics and event ingestion. Clients send batches of events; an ingestion API validates, stamps server time, and appends to a queue or Kafka topic quickly (return `202`); consumers write in batches to a columnar store (ClickHouse, BigQuery, Snowflake, DuckDB for small scale) or to partitioned PostgreSQL tables (TimescaleDB). Deduplicate on event id; handle late and out-of-order events by using event time and windows; keep raw events in object storage (Parquet) for reprocessing; pre-aggregate common

rollups. Protect the API with per-key rate limits and size caps; sample or drop under overload rather than failing the product.

Streaming responses from an LLM or a slow generator. Use SSE or a streamed fetch response and write chunks as they arrive. Key practices: stop upstream work when the client disconnects (`req.on('close', () => controller.abort())` with an `AbortController` passed to the provider call); cap concurrent generations per user and per instance (`p-limit`, a semaphore); enforce token and time budgets; cache deterministic results by a hash of the prompt and parameters; retry with a fallback model or provider on `429` and `5xx` using backoff; put long jobs on a queue and stream progress; store usage per request for billing; never forward provider keys to clients; and treat model output as untrusted input (validate structured output with a schema before using it).

Resume and job matching pipeline (relevant to the capstone). Upload a resume to object storage through a presigned URL; a queue job extracts text (`pdf-parse`, `mammoth` for docx, or an OCR fallback), normalises sections, and extracts skills; a second job computes embeddings and stores them in PostgreSQL with `pgvector` (an HNSW index for nearest-neighbour search) next to structured fields; ranking combines hard filters (location, years of experience), keyword or BM25 scores (PostgreSQL full-text search or Meilisearch), and vector similarity, then a re-ranking step; results and the explanation (which skills matched) are cached per job description hash; every stage is idempotent and its version is recorded so you can **reprocess** when the model changes. Keep scoring logic in a pure, unit-tested module with fixed fixtures so you can compare ranking versions offline before releasing.

Data structures that help at scale. A **Bloom filter** answers "definitely not present" cheaply (check before a database lookup for ids that mostly do not exist; Redis Stack provides one, or use `bloom-filters`). **Consistent hashing** (`hashring`) spreads keys over cache nodes so adding a node moves only a small share of keys. A **trie** or a prefix index serves autocomplete (in practice PostgreSQL `pg_trgm`, Redis sorted sets with lexicographic ranges, or a search engine). **HyperLogLog** counts unique items in kilobytes. **Count-min sketch** approximates top-N frequencies.

9.13 More build cards

Card 9.E: back-of-envelope sheet

- **Aim:** make estimates a habit.
- **Input:** the capstone's expected usage (for example 50,000 active users, 20 requests per user per day, 5 resumes uploaded per recruiter per day).
- **Output:** a one-page table with requests per second (average and peak), storage per year, queue throughput, Redis memory, and database connection budget, each with its formula.
- **Check:** compare the result with a load test of your prototype and note which estimate was off by more than 2x and why.

Card 9.F: notification pipeline

- **Aim:** channel isolation and safety.
- **Input:** `POST /notifications {userId, template, data, idempotencyKey}`; two queues (`email`, `sms`) with fake providers; a preferences table; a delivery log.

- **Output:** opted-out users get nothing; the same idempotency key creates one delivery; an expired OTP is dropped; a failing provider trips a breaker and the message goes to the backup provider.
- **Check:** a scenario table with six cases and the expected log rows for each.

Card 9.G: hybrid feed

- **Aim:** feel the fan-out trade-off.
- **Input:** 10,000 users, a follower graph with a few accounts of 5,000 followers, Redis sorted sets for feeds.
- **Output:** writing a post by an ordinary account fans out to its followers' feeds; a celebrity post is merged at read time; reading a feed returns hydrated posts in under 20 ms.
- **Check:** time the write path for an ordinary and a celebrity account with pure fan-out versus hybrid, and tabulate the difference.

Card 9.H: no overselling, no double booking

- **Aim:** let the database enforce invariants.
- **Input:** a stock table with 5 units and 200 parallel purchase attempts; a bookings table with the exclusion constraint above and 200 parallel overlapping reservations.
- **Output:** exactly 5 purchases succeed and stock is never negative; exactly one overlapping booking per slot succeeds, the rest return `409`.
- **Check:** assert the final database state after each run and repeat 20 times.

Card 9.I: event ingestion with batching

- **Aim:** sustain a high write rate cheaply.
- **Input:** `POST /events` accepting arrays of up to 500 events; a queue; a consumer that inserts in batches of 1,000 with `COPY` or a multi-row insert into a partitioned table.
- **Output:** 10,000 events per second accepted by one instance in a local test; duplicates by event id are dropped.
- **Check:** autocannon with batch payloads, row counts, and a duplicate-injection test.

Card 9.J: streaming with cancellation

- **Aim:** stop wasted work.
- **Input:** a fake generator endpoint streaming 1 token every 50 ms for 60 seconds over SSE, with per-user concurrency limit 2.
- **Output:** closing the browser tab stops the generator within a second (log a message); a third concurrent stream from one user gets `429`.
- **Check:** count active generators through a gauge metric before and after disconnects.

Card 9.K: resume matching vertical slice

- **Aim:** a small but real pipeline.
- **Input:** 200 sample resumes and 10 job descriptions; PostgreSQL with `pgvector`; BullMQ jobs for parse and embed (use any embedding library or API you prefer).

- **Output:** `GET /jobs/:id/candidates?limit=20` returns ranked candidates with a score breakdown (filters passed, keyword score, vector score); re-running a job version does not create duplicates.
- **Check:** a fixed evaluation set with known good matches and a script that prints precision at 10 for each scoring version.

Deep-dive self-check: (1) Why does the exclusion constraint beat a check-then-insert in application code? (2) Why fan out on write for most users but not for celebrities? (3) What does consistent hashing buy when you add a cache node? (4) Why must an OTP notification have a TTL? (5) How do you stop an LLM generation when the client disconnects? (6) Which consistency model is enough for a like counter, and which for an account balance?

Part 9 deep dive

9.11 Back-of-envelope estimation, worked example

Estimates decide whether you need one PostgreSQL instance or a sharded cluster, so do them before drawing boxes. For the URL shortener with **100 million new links per month** and a **100:1 read-to-write ratio**:

Quantity	Calculation	Result
Seconds per month	30 x 86 400	about 2.6 million
Writes per second (average)	100 million / 2.6 million	about 40 per second
Reads per second (average)	40 x 100	about 4 000 per second
Peak (assume 5 times average)	4 000 x 5	about 20 000 reads per second
Bytes per row	code, long URL, timestamps, indexes	about 500 bytes
Storage after 5 years	100 million x 12 x 5 x 500 bytes	about 3 TB
Hot data (20 percent of links get 80 percent of reads)	cache the hot 20 percent of one day's reads	a few GB of Redis

Conclusions you can defend: 40 writes per second is trivial for one PostgreSQL primary; 20 000 reads per second at the peak is well served by Redis in front of a replica or two; 3 TB fits one modern database server, so **partition by time or code range** for maintenance, and **shard only if growth proves you wrong**. State your assumptions and recompute when they change; the exercise matters more than the exact digits.

9.12 URL shortener: the details that make it good

Code generation strategies

Strategy	How	Pros	Cons

Strategy	How	Pros	Cons
Counter + base62	Database sequence encoded in base 62	Short, unique, no collision handling	Sequential codes are guessable and leak volume
Random code + unique constraint	7 random base62 characters, retry on conflict	Unguessable	One extra insert attempt on collision; at billions of rows collisions become common
Hash of the URL, truncated	`sha256(url)` first characters	Same URL gives the same code	Collisions need a salt-and-retry; hashes reveal nothing but need handling
Range allocation	Each app instance leases a block of 10 000 ids from the database and hands them out locally	No database call per create; scales to many writers	Gaps when an instance dies; still sequential unless scrambled
Snowflake-style ids	Timestamp + machine id + sequence in 64 bits	Distributed, time-sortable	Needs unique machine ids; longer codes

Seven base-62 characters give 62^7, about 3.5 trillion, codes. To make sequential ids unguessable, pass the counter through a **bijective scramble** (multiply by a large odd number modulo 62^7, or a small Feistel network) before encoding; it stays collision free and reversible.

```
const ALPHABET = '0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz';
export function toBase62(n: bigint): string {
  if (n === 0n) return '0';
  let s = '';
  while (n > 0n) { s = ALPHABET[Number(n % 62n)] + s; n /= 62n; }
  return s;
}
```

Other decisions: **301 versus 302** (301 is cached by browsers and CDNs, so clicks never reach you again; 302 or 307 keeps analytics and lets you change the target); **custom aliases** need a uniqueness check and a reserved-word list; **expiry** is a `expires_at` column plus a cleanup job, and the redirect checks it; **shard key**, if ever needed, is the code (hash it for even spread); **abuse** (phishing links) needs URL scanning against a safe-browsing list, per-user quotas, and a takedown path; keep redirects **highly available**: serve from cache even if the primary database is down, and cache negative results briefly.

9.13 File uploads end to end

The flow, with a state machine so nothing is trusted until verified:

48. Client calls `POST /uploads { filename, contentType, size }`. The API validates type and size limits, inserts an `uploads` row (`status = 'pending'`, random object key), and returns a **presigned URL**.
49. Client sends the bytes **directly to object storage** (S3, R2, GCS, MinIO).
50. Storage notifies you (an S3 event to a queue) or the client calls `POST /uploads/:id/complete`.

51. A worker **verifies**: `HEAD` the object for the real size, read the first bytes and sniff the true type with `file-type` (never trust the declared `Content-Type` or the extension), optionally virus-scan (ClamAV), then set `status = 'ready'` and enqueue derivatives (thumbnails).
52. Reads go through a CDN with **signed URLs** (or signed cookies) for private files.
53. A lifecycle rule or cleanup job deletes `pending` uploads older than 24 hours.

```
import { S3Client, PutObjectCommand } from '@aws-sdk/client-s3';
import { getSignedUrl } from '@aws-sdk/s3-request-presigner';
const s3 = new S3Client({ region: config.AWS_REGION });     // for MinIO or R2 also
set endpoint and forcePathStyle as needed

export async function createUpload(userId: string, contentType: string, size: number)
{
  if (!ALLOWED_TYPES.has(contentType) || size > 10 * 1024 * 1024) throw new
AppError(400, 'UPLOAD_REJECTED', 'Type or size not allowed');
  const key = `uploads/${userId}/${crypto.randomUUID()}`;        // never derive the
key from the user's filename
  const url = await getSignedUrl(s3, new PutObjectCommand({
    Bucket: config.BUCKET, Key: key, ContentType: contentType, ContentLength: size,
// signed: the client cannot change them
  }), { expiresIn: 300 });
  await uploads.insert({ userId, key, contentType, size, status: 'pending' });
  return { url, key };
}
```

A presigned `PUT` with a signed `ContentLength` limits the size to exactly what you signed. If you need a **range** ("up to 10 MB"), use a **presigned POST** with a policy condition (`content-length-range`), available through `@aws-sdk/s3-presigned-post`. For files over about 100 MB use **multipart upload** (parts uploaded in parallel and resumable) or the **tus** protocol. Image processing with `sharp` is fast (libvips), but resize with explicit limits (`limitInputPixels`) to avoid decompression bombs, **strip EXIF metadata** (it contains GPS location), and re-encode to a known format so a malicious file is not served as-is. Serve user uploads from a **separate domain** (or at least with `Content-Disposition: attachment` and `X-Content-Type-Options: nosniff`) so an uploaded HTML or SVG cannot run scripts on your origin.

9.14 Real-time chat, implemented properly

Data model first, because transports come and go: `conversations`, `memberships(user_id, conversation_id, last_read_message_id)`, `messages(id bigint identity, conversation_id, sender_id, client_msg_id uuid, body, created_at)` with an index on `(conversation_id, id DESC)` and a unique constraint on `(sender_id, client_msg_id)`. The **client-generated `client_msg_id`** makes sends idempotent: a retried message after a flaky connection does not duplicate.

The rule: **persist first, then broadcast, then ack.** If you broadcast before the insert succeeds, users see messages that vanish after a refresh.

```
import { Server } from 'socket.io';
import { createAdapter } from '@socket.io/redis-adapter';
import Redis from 'ioredis';

const pub = new Redis(config.REDIS_URL!); const sub = pub.duplicate();
const io = new Server(httpServer, { cors: { origin: config.WEB_ORIGIN, credentials:
```

```
true } });
io.adapter(createAdapter(pub, sub));                                    // cross-
instance rooms via Redis pub/sub

io.use(async (socket, next) => {                                        //
authenticate the handshake
  const user = await verifyTicket(socket.handshake.auth.ticket);        // short-
lived one-time ticket from your HTTPS API
  if (!user) return next(new Error('unauthorized'));
  socket.data.user = user; next();
});

io.on('connection', (socket) => {
  const user = socket.data.user;
  socket.join(`user:${user.id}`);                                        // personal
room: notifications to all of a user's devices
  socket.on('room:join', async (conversationId: string, ack) => {
    if (!(await memberships.isMember(user.id, conversationId))) return ack({ ok:
false });   // authorise every join
    await socket.join(`conv:${conversationId}`);
    ack({ ok: true });
  });
  socket.on('message:send', async (m: { conversationId: string; clientMsgId: string;
body: string }, ack) => {
    const saved = await messages.insertIdempotent(user.id, m);           // unique
(sender_id, client_msg_id)
    io.to(`conv:${m.conversationId}`).emit('message:new', saved);        // broadcast
to everyone, on any instance
    ack({ ok: true, id: saved.id });
  });
});
```

Routines that make it robust: on every reconnect the client calls `GET /conversations/:id/messages?after=<last id>` to **catch up on anything missed** (the socket alone is at-most-once); typing indicators and presence are **ephemeral** (not persisted, short TTL in Redis); the default Redis pub/sub adapter is fire-and-forget (a message published while an instance is briefly disconnected from Redis is lost), so rely on the catch-up fetch, or use the Redis Streams adapter for stronger delivery; offline users get **push notifications** (FCM or APNs) triggered by a queue job, not by the socket code; each connection costs file descriptors and tens of kilobytes of memory, so measure connections per instance with a load test and scale horizontally; **ordering** within a conversation comes from the database id, not from arrival time at the server.

9.15 Search in depth

PostgreSQL full-text search, with ranking and typo tolerance:

```
ALTER TABLE jobs ADD COLUMN search tsvector GENERATED ALWAYS AS (
  setweight(to_tsvector('english', title), 'A') ||
  setweight(to_tsvector('english', coalesce(description, '')), 'B')) STORED;
CREATE INDEX jobs_search_idx ON jobs USING GIN (search);

SELECT id, title, ts_rank(search, q) AS rank
FROM jobs, websearch_to_tsquery('english', $1) AS q      -- understands quotes, OR,
and -exclusions like a web search box
```

```
WHERE search @@ q AND tenant_id = $2                        -- ALWAYS filter by
permission in the query itself
ORDER BY rank DESC LIMIT 20;

-- typo tolerance with trigrams
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX jobs_title_trgm ON jobs USING GIN (title gin_trgm_ops);
SELECT id, title, similarity(title, $1) AS s FROM jobs WHERE title % $1 ORDER BY s
DESC LIMIT 10;   -- 'enginer' finds 'engineer'
```

Semantic (meaning-based) search embeds text as vectors and finds nearest neighbours. With the `pgvector` extension you can keep this in PostgreSQL:

```
CREATE EXTENSION IF NOT EXISTS vector;
ALTER TABLE jobs ADD COLUMN embedding vector(384);          -- dimension depends on
your embedding model
CREATE INDEX jobs_embedding_idx ON jobs USING hnsw (embedding vector_cosine_ops);
SELECT id, title FROM jobs ORDER BY embedding <=> $1 LIMIT 20;       -- <=> is cosine
distance
```

Hybrid search combines keyword rank and vector similarity (for example reciprocal rank fusion: score = sum of 1 / (60 + rank) from each list) and usually beats either alone: keywords catch exact terms and ids, vectors catch paraphrases.

When you outgrow PostgreSQL (facets over millions of documents, language analysis, instant autocomplete), add a search engine as a **secondary copy**: Meilisearch and Typesense are simple and fast; Elasticsearch and OpenSearch are the most powerful and heaviest. Keep it in sync through the **outbox** (Part 8): an indexer worker consumes `JobUpdated` events and upserts documents. For a schema or analyzer change, build a **new index**, backfill it, then atomically switch an alias (zero downtime), and keep a re-index job ready for disaster recovery. Search is eventually consistent, so show the user's own just-created item from the database if needed. Enforce **permissions and tenant filters inside the engine query**, never by filtering results afterwards (it breaks pagination and leaks counts).

9.16 Payments done right

The provider (Stripe, Razorpay, Adyen) holds the card data; you hold **intent and state**. A reliable checkout flow:

54. Your API creates an order (`pending`) and a provider **checkout session or payment intent**, sending your order id as metadata and an **idempotency key**.
55. The customer pays on the provider's hosted page or element.
56. The provider sends a **webhook**; your handler verifies the signature on the **raw body**, stores the event **by id** (so duplicates are ignored), enqueues fulfilment, and returns `200` fast.
57. A worker marks the order `paid` and fulfils it, idempotently.
58. A **reconciliation job** compares your orders with the provider's records daily and fixes mismatches (missed webhooks happen).

```
// Webhooks need the exact raw bytes for signature checking: register this route
BEFORE express.json()
app.post('/webhooks/stripe', express.raw({ type: 'application/json' }), async (req,
res) => {
  let event;
  try { event = stripe.webhooks.constructEvent(req.body, req.get('stripe-
```

```
signature')!, config.STRIPE_WEBHOOK_SECRET); }
  catch { return res.sendStatus(400); }
  const ins = await pool.query(
    'INSERT INTO webhook_events (id, type, payload) VALUES ($1,$2,$3) ON CONFLICT DO
NOTHING', [event.id, event.type, event]);
  if (ins.rowCount) await fulfilmentQueue.add(event.type, { eventId: event.id }, {
jobId: event.id });
  res.sendStatus(200);
});
```

Order state machine: `pending -> paid -> fulfilled -> refunded`, plus `failed` and `cancelled`; only forward transitions allowed, enforced in one function and with a conditional update (`UPDATE ... WHERE status = 'pending'`). Webhooks can arrive out of order or more than once, so a handler must be safe if it sees `paid` after `refunded`. **Money:** integer minor units plus an ISO currency code; never floats; decide rounding rules once (for tax and splits). For balances, wallets, and marketplaces use a **double-entry ledger**: an append-only `ledger_entries(id, transaction_id, account_id, amount, direction)` table where every transaction's entries sum to zero, never update or delete rows (post a reversing entry), and derive balances by summing (with periodic snapshots). It makes audits and bug hunts tractable.

9.17 Multi-tenancy in depth

Resolve the tenant **once** at the edge (subdomain, a claim in the token, or a header you verify against the user's memberships), then enforce isolation in layers: application filters, **row-level security** in the database, and tests.

```
export async function withTenant<T>(tenantId: string, fn: (c: PoolClient) =>
Promise<T>): Promise<T> {
  const c = await pool.connect();
  try {
    await c.query('BEGIN');
    await c.query("SELECT set_config('app.tenant_id', $1, true)", [tenantId]);   //
'true' = local to this transaction
    const result = await fn(c);
    await c.query('COMMIT');
    return result;
  } catch (e) { await c.query('ROLLBACK').catch(() => {}); throw e; } finally {
c.release(); }
}
```

```
ALTER TABLE notes ENABLE ROW LEVEL SECURITY;
ALTER TABLE notes FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON notes
  USING (tenant_id = current_setting('app.tenant_id')::uuid)
  WITH CHECK (tenant_id = current_setting('app.tenant_id')::uuid);     -- also blocks
writing into another tenant
```

Because the setting is transaction-local, it is safe with pooled connections and PgBouncer transaction mode; forgetting `withTenant` returns zero rows (fail closed) instead of everyone's rows. Other concerns: **noisy neighbours** (one tenant's heavy usage slows others: per-tenant rate limits, quotas, and separate queues); **schema-per-tenant** needs `search_path` switching and migrations that run N times; **per-tenant encryption keys** for strict customers; **tenant lifecycle** (export, suspend, delete, backup and restore of

one tenant); **usage metering** for billing; and a test suite that logs in as tenant A and tries every endpoint with tenant B's ids, asserting `404`.

9.18 Feeds, counters, and read-heavy design

Approach	How	Best when	Risk
Fan-out on **read**	Query everyone you follow at request time, merge, sort	Few follows, low volume, simple	Slow for users following thousands
Fan-out on **write**	When someone posts, push the post id into each follower's feed list	Read-heavy feeds	A celebrity with 10 million followers makes one post cost 10 million writes
Hybrid	Fan-out on write for normal accounts, on read for celebrities, merge at read time	Real social products	More moving parts

```
// Redis sorted-set timeline: score = timestamp, member = post id
await redis.zadd(`feed:${followerId}`, Date.now(), postId);
await redis.zremrangebyrank(`feed:${followerId}`, 0, -1001);              // keep
only the newest 1000
const ids = await redis.zrevrangebyscore(`feed:${userId}`, cursor ?? '+inf', '-inf',
'LIMIT', 0, 20);
const posts = await getPostsByIds(ids);                                  // batch
fetch from cache/DB (never one by one)
```

Counters (likes, views): updating one database row per click creates a **hot row** with lock contention. Use Redis `INCR` and flush totals to the database periodically in a worker, or store increments as events and aggregate. Accept that displayed counts are slightly stale. Cache the rendered feed for a few seconds with stale-while-revalidate, use a CDN for public pages, and precompute expensive aggregates (materialised views refreshed on a schedule).

9.19 Notifications and other cross-cutting systems

A notification service is a pipeline: **event** (OrderShipped) -> **rules** (does this user want it? which channels?) -> **template** per channel and language -> **per-channel queue** (email, push, SMS, in-app) -> **provider** with retries and fallback -> **delivery log**. Design points: user **preferences** and legally required **unsubscribe** handling; **deduplication and digests** (do not send 30 emails for 30 comments); **rate limits per user and per provider**; **quiet hours** and time zones; idempotent sends keyed by (event id, channel, recipient); **bounces and complaints** feeding a suppression list (essential for email reputation); and template tests with realistic data. Keep it a separate worker service even inside a modular monolith, so a provider outage cannot slow your API.

9.20 More build cards

Card 9.E: obfuscated short codes

- **Aim:** unguessable codes from a sequence.
- **Input:** `toBase62` and a bijective scramble of a database sequence.

- **Output:** 1 million consecutive ids map to 1 million distinct codes, none sequential-looking, decodable back to the id.
- **Check:** a test asserting uniqueness and round-trip, and a printed sample of 20 codes.

Card 9.F: full upload pipeline

- **Aim:** untrusted file in, safe derivative out.
- **Input:** MinIO in Docker, `createUpload`, a `complete` endpoint, and a worker with `file-type` and `sharp`.
- **Output:** uploading a PNG renamed `.jpg` is detected by sniffing; an oversized file is rejected before upload; the thumbnail has EXIF removed; abandoned pending uploads are cleaned up.
- **Check:** four test files (valid image, fake image, huge file, image with GPS EXIF) with expected outcomes.

Card 9.G: chat that survives reconnects

- **Aim:** no lost or duplicate messages.
- **Input:** the Socket.IO server above on two instances with the Redis adapter, a client with `clientMsgId` and catch-up fetch.
- **Output:** while a client is disconnected, 20 messages are sent by another; on reconnect it shows all 20 exactly once, in order; sending the same `clientMsgId` twice stores one message.
- **Check:** a scripted two-client test with a forced disconnect in the middle.

Card 9.H: hybrid search

- **Aim:** compare keyword, fuzzy, and semantic retrieval.
- **Input:** 5 000 job postings, `tsvector` + GIN, `pg_trgm`, and `pgvector` with any embedding model you can run locally.
- **Output:** for 10 test queries (including a typo and a paraphrase) a table of top-3 results from each method and from reciprocal rank fusion, with your judgement of which wins when.
- **Check:** write down one query where keyword search fails and one where vector search fails.

Card 9.I: checkout with webhooks and a ledger

- **Aim:** a correct payment flow against a fake provider.
- **Input:** a mock provider that sends webhooks late, twice, and out of order; your order state machine, event table, and a two-entry ledger.
- **Output:** after replaying 1 000 randomised webhook sequences, every order ends in the right state, no order is fulfilled twice, and ledger entries sum to zero per transaction.
- **Check:** a property-style test over shuffled and duplicated event streams.

Card 9.J: tenant isolation test suite

- **Aim:** prove isolation instead of hoping.
- **Input:** the `withTenant` helper, RLS policies, and endpoints for notes and files.
- **Output:** an automated matrix: for every endpoint, tenant B using tenant A's resource ids always gets `404`; a query run without `withTenant` returns zero rows.
- **Check:** the matrix is generated from your route list so new endpoints cannot be forgotten.

Card 9.K: Redis feed

- **Aim:** fan-out on write with a celebrity exception.
- **Input:** 1 000 users, a follow graph with 3 celebrities of 500 followers, sorted-set feeds.
- **Output:** posting by a normal user updates its followers' feeds; celebrity posts are merged at read time; the merged feed is correctly ordered and paginated by score.
- **Check:** compare write counts per post for both strategies and the read latency of a merged feed.

9.21 Answer key for the Part 9 self-checks

- **Why presigned URLs instead of uploading through the API?** The bytes bypass your servers entirely, so large uploads do not consume your bandwidth, memory, or worker time and scale with the storage service; the API only authorises and records metadata.
- **Why is the database, not the search engine, the source of truth?** Search engines are optimised for retrieval, can lag, lose data in a bad reindex, and lack transactions; if the index is wrong you rebuild it from the database, but not the other way round.
- **What can go wrong with shared-table multi-tenancy, and what is the safety net?** One query missing the `tenant_id` filter leaks another tenant's data; row-level security in the database (with a transaction-local tenant setting) makes the omission return nothing, and an isolation test suite catches regressions.

Part 10: Architecture patterns

Architecture is how you split code so that change stays cheap. The goal is not purity; it is to keep business rules independent of frameworks and databases so each can change without rewriting the rest.

10.1 Layered and clean (hexagonal) architecture

The dependency rule: **inner layers never import outer layers.** Business rules do not know about Express, Prisma, or Redis. They know interfaces (**ports**); the outer layer supplies **adapters** that implement them.

[image: Layers and the dependency direction]

A folder layout that works for both Fastify and Nest, organised **by feature** (not by technical type):

```
src/
  modules/
    orders/
      orders.controller.ts     # transport: HTTP in/out
      orders.service.ts        # use cases, transactions
      orders.repository.ts     # interface + Prisma implementation
      orders.schemas.ts        # Zod schemas / DTOs
      orders.events.ts         # domain events
      orders.test.ts
    users/ ...
  shared/ (errors, config, logger, db, queue)
  main.ts (compose everything)
```

Benefits: you can test services with a fake repository (no database), swap Prisma for Drizzle by writing a new adapter, and understand one feature by opening one folder.

Domain-Driven Design, the useful parts. Talk to domain experts and use their words (the *ubiquitous language*). Group code into **bounded contexts** (Billing and Shipping may both have a "Customer" that means different things). Inside, model **entities** (have identity), **value objects** (compared by value, like `Money`), and **aggregates** (a cluster changed as one unit through a root; one transaction per aggregate). You do not need the whole DDD vocabulary for CRUD apps; borrow ideas as complexity grows.

10.2 Monolith, modular monolith, microservices

	Monolith	Modular monolith	Microservices
Deployment	One unit	One unit, strict module boundaries	Many independent units
Data	One database	One database, tables owned per module	Database per service
Calls between parts	Function calls	Calls through module public APIs or in-process events	Network (HTTP, gRPC, events)
Team scaling	Hard past a point	Good	Best for many teams
Operational cost	Lowest	Low	High: tracing, deploys, versioning, network failures, data consistency
Start here?	For tiny apps	**Yes, for most products**	Only when a real need appears (team size, independent scaling, fault isolation)

Honest advice: begin with a modular monolith. Extract a service only when a boundary is stable and you have a concrete reason. Microservices trade code complexity for operational and consistency complexity; many teams regret splitting too early. NestJS modules map cleanly onto this path.

[image: Microservices with gateway, per-service databases, and events]

What microservices require

- **API gateway / BFF** for auth, rate limiting, routing, and aggregation (Kong, Traefik, Nginx, AWS API Gateway, or a small Fastify/Express service).
- **Service-to-service calls** with timeouts, retries, circuit breakers (`opossum`, `cockatiel`); prefer async events when the caller does not need an answer now.
- **Data ownership**: no shared tables; use events and sagas (Part 8) for cross-service consistency.
- **Contracts**: OpenAPI, Protobuf, or event schemas with versioning; consumer-driven contract tests (Pact).
- **Observability**: distributed tracing with a correlation id (Part 11).
- **Service discovery and config**: Kubernetes DNS and ConfigMaps, or Consul.

10.3 CQRS and event sourcing (advanced, use sparingly)

CQRS separates the model used to change data (commands) from the model used to read it (queries), often with different stores: write to PostgreSQL, read from a denormalised view or search index kept up to date by events. It helps when reads and writes have very different shapes or loads. **Event sourcing** stores the *sequence of events* as the truth and rebuilds state by replaying them; it gives audit and time travel but makes queries, migrations, and mistakes harder. Both add real complexity; adopt them for a specific pain, not for style. NestJS has `@nestjs/cqrs`; EventStoreDB and Kafka are common event stores.

10.4 Other patterns worth knowing

Pattern	Idea	Use
Repository	Hide persistence behind an interface	Testability, swapping databases
Unit of work	Group changes into one transaction	Consistent aggregates
Dependency injection	Receive dependencies instead of creating them	Tests, flexibility (Nest, awilix, or manual wiring)
Strangler fig	Route traffic piece by piece from an old system to a new one	Migrations without a rewrite
Bulkhead and circuit breaker	Isolate failures; stop calling failing dependencies	Resilience
Backend for Frontend	One API tailored per client type	Web versus mobile needs
Feature flags	Deploy code dark, release by config	Safe rollouts
Twelve-factor app	Config in env, stateless processes, logs to stdout, disposable processes	Cloud-ready services

10.5 Build cards

Card 10.A: refactor Notes into layers

- **Aim:** feel the dependency rule.
- **Input:** the Part 4 Notes API with SQL inside route handlers.
- **Output:** `NotesController` (HTTP only) calls `NotesService` (rules) which depends on a `NotesRepository` interface; `PrismaNotesRepository` implements it.
- **Check:** write a unit test for `NotesService` using an in-memory fake repository; it runs in milliseconds with no database.

Card 10.B: two modules, one boundary

- **Aim:** a modular monolith.
- **Input:** `orders` and `inventory` modules; placing an order must reserve stock.

- **Output:** `orders` calls only `inventory`'s exported interface (`reserve(sku, qty)`), never its tables; a lint rule (`eslint-plugin-boundaries` or `dependency-cruiser`) forbids deep imports.
- **Check:** try a forbidden import and confirm the linter fails the build.

Card 10.C: first extraction

- **Aim:** experience a service boundary.
- **Input:** move `inventory` to its own process with its own database, exposed over HTTP or gRPC.
- **Output:** `orders` calls it with a 1 second timeout, one retry, and a circuit breaker; stop the inventory service and `orders` returns a clean `503`, not a hang.
- **Check:** kill the inventory container during a load test and read the error rate.

Self-check questions: What does the dependency rule forbid? Give two reasons to stay a modular monolith. What problem does a saga solve that a database transaction cannot?

10.6 Deep dive: layered and hexagonal architecture in real TypeScript

The diagram in 10.1 becomes concrete with four kinds of files. Read this example top to bottom: the domain knows nothing, the service knows only interfaces, the adapter knows the database, and the composition root connects them.

```
// domain/order.ts: pure rules, no imports from frameworks or drivers
export class DomainError extends Error { constructor(public code: string) {
super(code); } }
export class Order {
  private events: DomainEvent[] = [];
  private constructor(readonly id: string, private items: Item[], private status:
'draft' | 'placed' | 'cancelled') {}
  static draft(id: string) { return new Order(id, [], 'draft'); }
  addItem(sku: string, qty: number, unitPrice: Money) {
    if (this.status !== 'draft') throw new DomainError('ORDER_LOCKED');
    if (!Number.isInteger(qty) || qty <= 0) throw new DomainError('BAD_QUANTITY');
    this.items.push({ sku, qty, unitPrice });
  }
  place() {
    if (this.items.length === 0) throw new DomainError('EMPTY_ORDER');
    this.status = 'placed';
    this.events.push({ type: 'OrderPlaced', orderId: this.id });
  }
  pullEvents() { const e = this.events; this.events = []; return e; }
}

// application/ports.ts: what the service needs, as interfaces
export interface OrderRepository { findById(id: string): Promise<Order | null>;
save(o: Order): Promise<void>; }
export interface EventPublisher { publish(events: DomainEvent[]): Promise<void>; }

// application/place-order.ts: one use case, one class
export class PlaceOrder {
  constructor(private orders: OrderRepository, private tx: TransactionRunner) {}
  async execute(input: { orderId: string }) {
    return this.tx.run(async () => {                 // one transaction per use case
      const order = await this.orders.findById(input.orderId);
```

```
      if (!order) throw new NotFoundError('ORDER');
      order.place();                                  // rules live in the domain
      await this.orders.save(order);                  // saves state AND outbox rows
for pullEvents()
    });
  }
}

// infrastructure/pg-order-repository.ts: adapter
export class PgOrderRepository implements OrderRepository { /* SQL via db(), maps
rows to Order */ }

// main.ts: composition root, the only place that knows every concrete class
const orders = new PgOrderRepository();
const placeOrder = new PlaceOrder(orders, new PgTransactionRunner(pool));
app.post('/orders/:id/place', async (req, reply) => { await placeOrder.execute({
orderId: req.params.id }); reply.code(204); });
```

Transactions across repositories without passing a connection around. Keep the active transaction client in `AsyncLocalStorage`; every repository calls `db()` and automatically joins the transaction of the current use case.

```
const store = new AsyncLocalStorage<PoolClient>();
export const db = () => store.getStore() ?? pool;      // repositories use
db().query(...)
export async function withTx<T>(fn: () => Promise<T>): Promise<T> {
  const c = await pool.connect();
  try { await c.query('BEGIN'); const r = await store.run(c, fn); await
c.query('COMMIT'); return r; }
  catch (e) { await c.query('ROLLBACK'); throw e; }
  finally { c.release(); }
}
```

Where each kind of logic lives

Kind of logic	Layer	Example
Shape and format of input	Transport (schema)	Zod validation, parsing dates
Authentication and coarse access	Transport / middleware	Verify token, set `user`
Business invariants	Domain	An order cannot be placed empty
Orchestration, transactions, calling ports	Application (use case)	Load, change, save, publish
Fine-grained permissions	Application (policy function)	Can this user cancel this order
SQL, HTTP clients, queues	Infrastructure	Repositories, mail adapters
Mapping to the response shape	Transport (mapper)	Domain order to API JSON

Mapping and DTOs. Keep three shapes distinct: the *API shape* (what clients see), the *domain object* (rules), and the *persistence shape* (rows). Map between them at the boundaries. It looks like duplication but prevents a database column rename from changing your public API.

Avoid the common failures. *Anemic domain model*: entities are bags of fields and all logic sits in services, so rules are scattered; move invariants into the entity. *Over-abstraction*: an interface for everything, with one implementation each and layers that only forward calls; add an interface where you actually have, or need, two implementations or a test fake. *God service*: `OrderService` with 40 methods; split by use case. *Leaky repository*: returning query builders or ORM entities to callers. *Smart controllers*: logic in route handlers. The test of a good boundary is that a test of the use case needs no HTTP server and no database.

How much structure does your project deserve? A 10-endpoint CRUD service does fine with route handlers calling a small service and an ORM; apply hexagonal structure to the parts with real rules (pricing, booking, ranking) and keep the rest simple. Let the design grow when pain appears (duplicated rules, slow tests, painful changes).

10.7 Deep dive: domain-driven design that pays off

- **Ubiquitous language.** Name classes, tables, events, and endpoints with the business's own words ("shortlist a candidate", not "update status to 3"). Keep a glossary; mismatched words are the earliest sign of a modelling problem.
- **Bounded contexts.** Each context has its own model and vocabulary. "Candidate" in sourcing (a lead with contact data) differs from "Candidate" in interviewing (a person with a schedule). Do not force one shared class; translate at the border.
- **Aggregates.** A consistency boundary: load and save the whole aggregate, change it only through its root, keep it small, and reference other aggregates **by id**. One use case normally changes one aggregate per transaction; coordinate several with events (eventual consistency). Large aggregates cause lock contention and slow loads.
- **Value objects.** Immutable and compared by value (`Money`, `EmailAddress`, `DateRange`); validate in the constructor so invalid values cannot exist.
- **Domain events.** Facts raised by aggregates (`OrderPlaced`); collected on the entity and written to the outbox in the same transaction (Part 8).
- **Context map patterns** (how two contexts relate)

Pattern	Meaning	Use when
Shared kernel	Two teams share a small common model	Tight collaboration, accept coupling
Customer-supplier	Upstream serves downstream's needs	Clear owner and negotiation
Conformist	Downstream accepts upstream's model as is	No leverage (a third-party API)
Anti-corruption layer (ACL)	A translation layer protects your model from a foreign one	Legacy or messy external systems
Open host service + published language	Upstream offers a stable, documented API/events	Many consumers

Event storming is a quick workshop method: put domain events on sticky notes on a timeline, then add commands, actors, aggregates, and read models; clusters of notes suggest bounded contexts. Even a solo session on a whiteboard finds missing rules.

10.8 Deep dive: modular monolith, enforced

A modular monolith only works if boundaries are checked by tools, not by good intentions.

- **One public entry per module** (`modules/orders/index.ts` exports the use cases and types others may use); everything else is private. Ban deep imports with `eslint-plugin-boundaries`, `dependency-cruiser`, Nx module boundary rules, or TypeScript project references.
- **Data ownership.** Each module owns its tables (a separate PostgreSQL **schema** per module makes this visible and lets you grant permissions per module). No joins across module tables; ask the other module through its public API or keep a local read copy fed by events.
- **Communication.** Synchronous calls through the public interface for queries that need an answer now; in-process events (an event bus or `EventEmitter`, backed by the outbox if you need reliability) for reactions. Choose events when the caller should not know about the reaction (placing an order should not import the email module).
- **Dependency direction between modules** should form a graph with no cycles (draw it; a cycle means a missing module or a wrong boundary). Shared code goes in a small `shared` package (ids, errors, logger types) that depends on nothing else.
- **Monorepo tooling.** `pnpm` workspaces with Turborepo or Nx give you per-package builds, caching, and affected-only tests. Package-per-module makes boundaries physical (a package must declare its dependencies in `package.json`).
- **Nest.** Map one Nest module per bounded context; export only a facade provider; use `@nestjs/event-emitter` or CQRS events for reactions; ban importing another module's repository.
- **Testing boundaries.** Contract-style tests per module's public API, plus an architecture test that fails the build on a forbidden import.

10.9 Deep dive: from monolith to services

Good reasons to extract a service: independent scaling needs (image processing), different runtime or language needs (ML in Python), a team that must deploy on its own schedule, fault isolation for a risky component, regulatory isolation of sensitive data. **Weak reasons:** it feels modern; the codebase is messy (fix the modularity first); "for performance" without a measurement.

Extraction recipe (the strangler fig)

59. Make the module's boundary clean inside the monolith; all callers go through its public API.
60. Put a facade in front that can call either the local module or a remote service, switchable by a flag.
61. Build the service with its own database; migrate data with a backfill plus change capture (outbox or CDC) so both stay in sync for a while.
62. Route a small share of traffic (flag, canary) and compare results (shadow reads).
63. Switch writes, then remove the old code and tables in a later release.

Communication styles

Style	Pros	Cons	Use
Synchronous request/response (HTTP, gRPC)	Simple mental model, immediate answer	Temporal coupling; failures cascade; latency adds up	Queries needing an answer; keep chains short (depth 2 or 3)
Asynchronous events	Loose coupling, resilience, easy fan-out	Eventual consistency, harder debugging	State changes others react to
API composition / BFF	One call for a screen	Aggregator becomes a bottleneck	UI needs data from several services
Shared database	None worth having	Hidden coupling	Avoid

The distributed monolith trap. Symptoms: services that must be deployed together, share a database, call each other in long synchronous chains, or break when one is down. You carry all the costs of microservices with none of the benefits. If a change routinely touches three services, the boundaries are wrong.

What to have in place before the second service: per-service CI/CD, centralised logs with a correlation id, distributed tracing, health checks and timeouts and retries with budgets, contract tests, service discovery and config, a way to run the whole system locally (Docker Compose, Tilt), runbooks and on-call, and an ADR explaining why.

Service mesh and platform. Istio or Linkerd handle mutual TLS, retries, and traffic splitting at the network level; useful at scale, heavy for small teams. A simpler path is a managed container platform plus a gateway.

10.10 Deep dive: CQRS and event sourcing, concretely

[image: CQRS with a separate read model kept up to date by events]

CQRS in four steps. (1) Commands change state through the domain model and return little or nothing. (2) The write model commits and records an event (outbox). (3) A projector consumes events and updates one or more **read models** shaped for screens (a denormalised table, a Redis hash, an Elasticsearch index). (4) Queries read only from read models. Benefits: each side is optimised separately, read models can be rebuilt from events, and heavy reads do not slow writes. Costs: eventual consistency (the read model lags by milliseconds to seconds; show the user their own change from the command result or wait for a version marker), more moving parts, and projector idempotency and ordering to manage. Start with **CQRS-lite**: separate command and query *code paths* on the same database (queries use plain SQL or views), and add separate stores only when measured pain appears.

Event sourcing. Instead of storing current state, store the full ordered list of events per aggregate (`stream = order-42`: `OrderCreated`, `ItemAdded`, `OrderPlaced`) and compute state by replaying them. To save a change, append events with an **expected version** (optimistic concurrency); a conflict means someone else wrote first. **Snapshots** store computed state every N events so loading stays fast.

Projections build read models and can be rebuilt by replaying from the start, which gives you audit history and new views from old data. Hard parts: event schema evolution (upcasting), eventual consistency everywhere, querying across aggregates, mistakes are fixed by compensating events rather than edits, and **personal data erasure** (you cannot delete from an immutable log; encrypt personal fields with a per-subject key and delete the key, a technique called crypto-shredding). Tools: EventStoreDB (now Kurrent), PostgreSQL as an event store (a simple events table with `(stream_id, version)` unique), Marten-like libraries in other languages, and in Node `@nestjs/cqrs`, `emmett`, or hand-written code. Use event sourcing only where the history itself is a business requirement (ledgers, audits, complex workflows), not as a default.

10.11 Architecture decisions and fitness

Architecture Decision Records (ADRs). One short file per significant decision, kept in the repository (`docs/adr/0007-use-bullmq-for-jobs.md`):

```
# 7. Use BullMQ for background jobs
Status: accepted (2026-10-20)
Context: We need retries, delays and rate limits for email and parsing. Volume is
about 50k jobs per day. The team knows Redis.
Decision: Use BullMQ on a dedicated Redis instance.
Alternatives considered: pg-boss (fewer dependencies, lower throughput), Kafka
(replay not needed), SQS (cloud lock-in).
Consequences: Need Redis persistence and monitoring; jobs must be idempotent; revisit
if volume exceeds 5M per day.
```

Fitness functions are automated checks that protect architectural properties: a test that fails on forbidden imports, a CI check that the bundle or image size stays under a limit, a latency budget test, a query count assertion for key endpoints (catches N+1 regressions), dependency graph no-cycle checks, and a check that every endpoint has an authorisation test.

Evolutionary approach. Make the cheapest decision that keeps options open, record why, and set a trigger for revisiting (a metric or a team size). Most architecture regret comes from deciding too early without data, not from deciding late.

10.12 More build cards

Card 10.D: the Order aggregate

- **Aim:** put invariants in the domain.
- **Input:** the `Order` class sketch above with items, status, and `Money` as a value object.
- **Output:** invalid operations (placing an empty order, adding to a placed order, a negative quantity) throw `DomainError` with specific codes; 15 unit tests run in under a second with no database.
- **Check:** mutation-test mentally: delete one `if` from the class and confirm a test fails.

Card 10.E: transaction context with AsyncLocalStorage

- **Aim:** one transaction per use case without threading a connection.
- **Input:** the `withTx` and `db()` helpers; two repositories (`orders`, `outbox`).
- **Output:** a use case that saves an order and an outbox row; throwing after the first save rolls both back.

- **Check:** a test that injects a failure between the two writes and asserts neither row exists.

Card 10.F: boundary enforcement

- **Aim:** make module rules fail the build.
- **Input:** modules `orders`, `inventory`, `notifications`, each with `index.ts` as its public API; `dependency-cruiser` or `eslint-plugin-boundaries` config.
- **Output:** an import of `inventory/internal/stock.repository` from `orders` fails CI; the module graph has no cycles (render it with `depcruise --output-type dot | dot -Tsvg`).
- **Check:** commit one forbidden import on a branch and confirm CI is red.

Card 10.G: replace an adapter

- **Aim:** prove the ports idea.
- **Input:** your `NotesRepository` interface with a Prisma adapter.
- **Output:** a second adapter (Drizzle, raw SQL, or in-memory) passes the same contract test suite; switching is one line in the composition root.
- **Check:** a parameterised test suite that runs the same 20 cases against both adapters.

Card 10.H: CQRS-lite read model

- **Aim:** a fast read side fed by events.
- **Input:** orders written through the aggregate; an `order_summary` table updated by a projector consuming outbox events; `GET /orders/summary`.
- **Output:** the summary endpoint reads one table with no joins; deleting the table and replaying all events rebuilds it identically.
- **Check:** compare summary totals with a direct SQL aggregation over the write tables; they must match after replay.

Card 10.I: strangler extraction rehearsal

- **Aim:** practise the five steps safely.
- **Input:** the `inventory` module and a facade interface `InventoryApi` with a local and an HTTP implementation.
- **Output:** a flag switches 10 percent of calls to the HTTP service; shadow reads compare results and log mismatches; killing the service makes the facade fall back or fail fast according to policy.
- **Check:** run both modes under the same load test and compare latency and error rates.

Card 10.J: write three ADRs

- **Aim:** make decisions explicit.
- **Input:** your capstone's choices for framework, queue, and search.
- **Output:** three ADRs with context, decision, alternatives, consequences, and a revisit trigger each.
- **Check:** ask a peer (or yourself in a week) whether each ADR answers "why not the alternative?" without extra explanation.

Deep-dive self-check: (1) What belongs in the domain layer and what must never be imported there? (2) Why reference other aggregates by id? (3) What makes a modular monolith more than a folder

convention? (4) Name three symptoms of a distributed monolith. (5) How does CQRS change what "read your own writes" means? (6) Why is crypto-shredding used with event sourcing?

Part 10 deep dive

10.6 Clean architecture in real TypeScript

The layers from the diagram become four kinds of file. Read the example top to bottom and notice that nothing in the first three blocks imports Express, Prisma, or Redis.

```
// ---- domain/money.ts: a value object (immutable, compared by value, always valid)
export class DomainError extends Error {}
export class Money {
  private constructor(readonly cents: number, readonly currency: string) {}
  static of(cents: number, currency: string) {
    if (!Number.isInteger(cents) || cents < 0) throw new DomainError('money must be a
non-negative integer of minor units');
    return new Money(cents, currency);
  }
  add(o: Money) { this.assertSame(o); return Money.of(this.cents + o.cents,
this.currency); }
  times(n: number) { return Money.of(this.cents * n, this.currency); }
  private assertSame(o: Money) { if (o.currency !== this.currency) throw new
DomainError('currency mismatch'); }
}

// ---- domain/order.ts: an aggregate root that protects its own rules
export class Order {
  private lines: { sku: string; qty: number; price: Money }[] = [];
  private _status: 'draft' | 'placed' | 'cancelled' = 'draft';
  readonly events: { type: string; data: unknown }[] = [];            // domain
events collected, dispatched after commit
  constructor(readonly id: string, readonly customerId: string) {}

  addLine(sku: string, qty: number, price: Money) {
    if (this._status !== 'draft') throw new DomainError('cannot change a placed
order');
    if (!Number.isInteger(qty) || qty <= 0) throw new DomainError('quantity must be a
positive integer');
    this.lines.push({ sku, qty, price });
  }
  get total(): Money { return this.lines.reduce((sum, l) =>
sum.add(l.price.times(l.qty)), Money.of(0, 'INR')); }
  place() {
    if (this.lines.length === 0) throw new DomainError('an order needs at least one
line');
    this._status = 'placed';
    this.events.push({ type: 'OrderPlaced', data: { orderId: this.id, totalCents:
this.total.cents } });
  }
  get status() { return this._status; }
}

// ---- application/place-order.ts: a use case; depends on ports (interfaces), never
on adapters
```

```
export interface OrderRepository { save(o: Order): Promise<void>; byId(id: string):
Promise<Order | null>; }
export interface UnitOfWork { run<T>(fn: () => Promise<T>): Promise<T>; }      //
wraps a database transaction
export interface PriceCatalog { priceOf(sku: string): Promise<Money>; }

export class PlaceOrder {
  constructor(private orders: OrderRepository, private prices: PriceCatalog, private
uow: UnitOfWork) {}
  async execute(cmd: { orderId: string; customerId: string; items: { sku: string;
qty: number }[] }) {
    const order = new Order(cmd.orderId, cmd.customerId);
    for (const i of cmd.items) order.addLine(i.sku, i.qty, await
this.prices.priceOf(i.sku));
    order.place();
    await this.uow.run(() => this.orders.save(order));          // the repository
also writes the outbox rows from order.events
    return { id: order.id, total: order.total.cents };
  }
}

// ---- infrastructure/prisma-order-repository.ts: an adapter implements the port
// export class PrismaOrderRepository implements OrderRepository { ...maps Order <->
rows... }

// ---- main.ts: the composition root is the ONLY place that knows concrete classes
// const placeOrder = new PlaceOrder(new PrismaOrderRepository(prisma), new
HttpPriceCatalog(cfg), new PrismaUnitOfWork(prisma));
// app.post('/orders', validate(schema), async (req, res) =>
res.status(201).json(await placeOrder.execute(req.valid.body)));
```

Where each kind of rule lives. *Shape and format* (is `qty` a number, is the email well formed) belong to **edge validation** (Zod). *Business invariants* (an order needs a line, cannot be edited after placing) belong in the **domain**, so they hold no matter which entry point (HTTP, queue, CLI, test) calls it. *Orchestration and permissions* (load, authorise, call several things, commit) belong to the **use case**. *Mapping* between layers is explicit: HTTP DTO to command, database row to entity, entity to response DTO. Never return ORM entities or domain objects straight from a controller.

Is it overkill? For simple CRUD, a thin service plus repository is enough; the domain layer pays off when rules are numerous and change often (pricing, eligibility, workflows). Start simple and extract a rich domain model where the logic gets hairy, as a refactoring, not as a ritual.

Testing by layer. Domain: plain unit tests, no mocks. Use cases: unit tests with in-memory fakes of the ports (milliseconds). Adapters: integration tests against the real database. Run the **same contract test suite** against both the fake and the real repository so the fake cannot drift from reality:

```
export function orderRepositoryContract(name: string, make: () =>
Promise<OrderRepository>) {
  describe(`${name} satisfies OrderRepository`, () => {
    it('returns what was saved', async () => {
      const repo = await make();
      const o = new Order('o1', 'c1'); o.addLine('sku', 2, Money.of(500, 'INR'));
o.place();
      await repo.save(o);
```

```
      expect((await repo.byId('o1'))?.total.cents).toBe(1000);
    });
    it('returns null for unknown ids', async () => { expect(await (await
make()).byId('nope')).toBeNull(); });
  });
}
// orderRepositoryContract('InMemory', async () => new InMemoryOrderRepository());
// orderRepositoryContract('Postgres', async () => new PrismaOrderRepository(await
freshTestDb()));
```

Hexagonal vocabulary. *Driving* (primary) ports are how the outside calls you (use cases); *driven* (secondary) ports are what you need from the outside (repositories, mailers, clocks, payment gateways). Adapters implement or call them. The same idea as "ports and adapters" and "onion architecture", under different names.

10.7 Dependency injection without a framework

You do not need a container. A **composition root** (one file) builds the object graph by hand and passes dependencies through constructors; it is explicit, type-checked, and trivially debuggable. Reach for a container (`awilix`, `tsyringe`, `inversify`, or Nest's built-in one) when the graph is large or you need scopes (per-request instances). Two rules: prefer **constructor injection**, and avoid the **service locator** anti-pattern (code asking a global container for what it needs), which hides dependencies and brings back the coupling DI was meant to remove. Inject the **clock** and the **id generator** too (`{ now(): Date }`, `{ next(): string }`) so tests can be deterministic.

10.8 The modular monolith, enforced

A modular monolith is only modular if boundaries are **enforced by tooling**, because convention erodes under deadline pressure. The rules:

- Each module exposes a **public API** (a facade or a few functions and types exported from its `index.ts`); everything else is private.
- A module **owns its tables**; no other module reads or writes them or joins across them. Cross-module data comes through the public API or from events.
- Modules communicate by **calling the facade** (synchronous, same transaction possible) or by **in-process events** (decoupled; write them through the outbox so they survive crashes).
- No circular dependencies between modules (dependency direction forms a graph without cycles).

```
// .dependency-cruiser.cjs: fail the build on deep imports into another module
module.exports = {
  forbidden: [
    { name: 'modules-only-via-index', severity: 'error',
      from: { path: '^src/modules/([^/]+)/' },
      to: { path: '^src/modules/([^/]+)/',
            pathNot: ['^src/modules/$1/', '^src/modules/[^/]+/index\\.ts$'] } },   //
own files, or another module's index only
    { name: 'no-circular', severity: 'error', from: {}, to: { circular: true } },
  ],
};
```

Run `depcruise src` in CI. `eslint-plugin-boundaries` and Nx module-boundary rules do the same job. In **NestJS**, a module that does not export a provider keeps it private, and `exports` is your facade; in **Fastify**, plugin encapsulation plays a similar role.

Extraction path. Because modules already own their data and talk through narrow interfaces, extracting one into a service later means replacing an in-process call with a network call (and a shared-database join with an event or an API call), which is a contained change instead of a rewrite.

10.9 Microservices: criteria, failure modes, and migration

Split when several of these are true: separate teams need to deploy independently; one part has very different scaling or availability needs; a boundary is stable and maps to a business capability (a bounded context); a failure in one part must not take down others; different technology is genuinely required. **Do not split** because it feels modern or to "organise code": modules do that more cheaply.

The distributed monolith is the common failure: services that share a database, call each other synchronously in long chains, and must be deployed in lockstep. You get every cost of distribution and none of the benefits. Signs: a change usually touches several services; you cannot deploy one alone; one service being down breaks most requests.

Availability maths. If a request needs five services in a synchronous chain and each is 99.9 percent available, the chain is about 0.999^5 = 99.5 percent available: roughly 3.6 hours of downtime a month instead of 43 minutes. Every synchronous hop multiplies risk and adds latency; prefer asynchronous events where the caller does not need the answer now, and cache or replicate the read data you need.

Concern	Typical answer
Query data owned by another service	API composition in the gateway/BFF, or a local read model built from events (CQRS)
Transaction across services	Saga (Part 8), not distributed two-phase commit
Shared code	Small, stable libraries only (clients, schemas); beware shared domain libraries, which couple deploys
Contract changes	Versioned APIs and events, consumer-driven contract tests (Pact), additive changes
Debugging a request	Distributed tracing with a propagated trace id (Part 11)
Local development	Docker Compose or a dev cluster, contract mocks for services you are not running
Cost	Per-service pipelines, monitoring, on-call; budget for a platform team as the count grows

Organisation matters as much as code (Conway's law: system structure mirrors team communication structure). Give each service one owning team.

Strangler fig migration (moving off a legacy system or monolith without a big-bang rewrite): put a routing layer (gateway or reverse proxy) in front; build the new capability behind it; route a small slice of traffic (one endpoint, or one customer group) to the new code; compare results (shadow traffic is a safe first step); grow the slice; finally retire the old path. Each step is reversible by changing a route.

10.10 CQRS and event sourcing, with working shapes

[image: CQRS: separate write model and read model joined by events]

Use CQRS when reads and writes have very different shapes or scale (a normalised write model versus a denormalised, search-friendly read model) or when you need several tailored read views. The cost is **eventual consistency**: after a command succeeds, the read model lags briefly. Handle it in the UX and API: return the created resource (or its id and version) from the command so the client can show it optimistically, let the client poll or subscribe, or offer **read-your-writes** by letting a query wait until the read model has reached a version token. **Projections** (the code building read views from events) must be **idempotent and rebuildable**: you should be able to drop a read model and reconstruct it by replaying the events.

Event sourcing stores the facts instead of the current state; state is derived by folding the events:

```
CREATE TABLE events (
  stream_id uuid NOT NULL, version int NOT NULL,                -- per aggregate,
starting at 1
  type text NOT NULL, data jsonb NOT NULL, occurred_at timestamptz NOT NULL DEFAULT
now(),
  PRIMARY KEY (stream_id, version)                              -- the primary key
IS the concurrency control
);
```

```
type AccountEvent =
  | { type: 'Opened'; data: { owner: string } }
  | { type: 'Deposited'; data: { cents: number } }
  | { type: 'Withdrawn'; data: { cents: number } };
type Account = { owner: string; balance: number; version: number };

const apply = (s: Account, e: AccountEvent): Account => {
  switch (e.type) {
    case 'Opened':    return { owner: e.data.owner, balance: 0, version: s.version +
1 };
    case 'Deposited': return { ...s, balance: s.balance + e.data.cents, version:
s.version + 1 };
    case 'Withdrawn': return { ...s, balance: s.balance - e.data.cents, version:
s.version + 1 };
  }
};
const load = async (id: string) => (await readStream(id)).reduce(apply, { owner: '',
balance: 0, version: 0 });

export async function withdraw(id: string, cents: number) {
  const acct = await load(id);
  if (acct.balance < cents) throw new DomainError('insufficient funds');
  // optimistic concurrency: inserting version N+1 fails with a unique violation if
```

```
someone else appended first -> reload and retry
  await appendEvent(id, acct.version + 1, { type: 'Withdrawn', data: { cents } });
}
```

Practical issues: replaying long streams is slow, so take periodic **snapshots**; events live forever, so you need **event versioning** (an *upcaster* converts old event shapes to the current one when reading); events are immutable, so **deleting personal data** (privacy law) needs a strategy such as keeping personal fields outside the log or encrypting them per person and discarding the key (**crypto-shredding**); and ad hoc queries need projections. Event sourcing buys a perfect audit trail, temporal queries (state as of last Tuesday), and natural integration events; it costs complexity, so choose it for domains where history *is* the product (ledgers, bookkeeping, inventory movements, compliance), not by default.

10.11 Cross-cutting concerns: where each one belongs

Concern	Layer	Mechanism
Authentication (who)	Transport	Middleware, guard, or hook; produces an `AuthUser` object passed inward
Authorization (may they)	Use case (and database as backup)	Policy check using the user and the loaded resource
Input shape validation	Transport	Schema at the edge
Business invariants	Domain	Methods on entities and value objects that throw domain errors
Transactions	Use case, via a unit-of-work port	One transaction per use case; never inside controllers or entities
Domain events	Domain collects, use case dispatches after commit (or writes to the outbox in the transaction)	Avoid publishing before commit
Error translation	Transport	One handler maps `DomainError` and `NotFound` to HTTP codes
Logging, tracing, metrics	Everywhere, but by middleware and wrappers	Context via `AsyncLocalStorage`
Caching	Adapters (decorate a repository)	`CachedProductRepository` wraps `PostgresProductRepository` behind the same port

The decorator pattern for caching shows why ports are worth it: callers cannot tell that the repository now consults Redis first.

10.12 Keeping architecture honest

- **Architecture Decision Records (ADRs):** one short file per significant decision: *Context* (the forces), *Decision*, *Consequences* (good and bad), *Alternatives considered*, *Status*. Store them in the repository. Six months later they answer "why did we do it this way?".
- **Fitness functions:** automated tests for architectural properties: no import cycles, modules only via public API (the `depcruise` rule above), no controller imports a repository directly, p95 latency budgets in CI load tests, bundle or image size limits.
- **Evolve incrementally.** Prefer reversible decisions; delay irreversible ones (a datastore choice, a public API shape) until you know more; and reassess at clear triggers (team size, p95, incident patterns) rather than on fashion.

10.13 More build cards

Card 10.D: a rich domain model, tested without a database

- **Aim:** put rules where they belong.
- **Input:** the `Money` and `Order` classes above, plus a `cancel()` rule (only placed orders, not shipped ones) and a `Discount` value object.
- **Output:** 15 or more fast unit tests covering every invariant and the emitted events, running in under a second with no I/O.
- **Check:** run the tests with the network disabled.

Card 10.E: repository contract tests

- **Aim:** a fake that cannot lie.
- **Input:** the contract suite above, an `InMemoryOrderRepository`, and a PostgreSQL implementation.
- **Output:** the same suite passes for both; deliberately break the fake and watch the contract fail.
- **Check:** both test runs in CI output.

Card 10.F: enforce the boundary

- **Aim:** make the architecture executable.
- **Input:** `depcruise` (or `eslint-plugin-boundaries`) configured with the rules in 10.8 on your capstone modules.
- **Output:** a deliberate violation (importing `inventory/internal/stock.ts` from `orders`) fails CI with a readable message; a dependency cycle between two modules also fails.
- **Check:** screenshots of the failing and the passing run.

Card 10.G: a rebuildable projection

- **Aim:** CQRS with proof it can recover.
- **Input:** an `OrderPlaced` event stream and a projector that maintains `customer_order_totals` (a denormalised table).
- **Output:** a command to **truncate the read model and rebuild it by replaying events**, producing identical totals; replaying the same events twice leaves the totals unchanged (idempotent projector).
- **Check:** compare table checksums before the wipe and after the rebuild.

Card 10.H: an event-sourced account

- **Aim:** understand optimistic concurrency in an event store.
- **Input:** the `events` table and `withdraw` above.
- **Output:** 20 concurrent withdrawals of 10 from a balance of 100 never overdraw: the unique key rejects conflicting appends and the retry reloads fresh state; the balance ends at exactly 0 with 10 successes and 10 insufficient-funds failures.
- **Check:** an automated concurrent test run 50 times.

Card 10.I: availability arithmetic

- **Aim:** feel the cost of synchronous chains.
- **Input:** a request path touching gateway, auth, orders, inventory, and payments, each with a target of 99.9 percent.
- **Output:** the composed availability, then a redesign where inventory and payments are reached asynchronously or with a cached fallback, and the new number with your assumptions written down.
- **Check:** a one-page note with both calculations and the diagram.

Card 10.J: write an ADR

- **Aim:** practise recording decisions.
- **Input:** the real choice "modular monolith versus microservices for the capstone".
- **Output:** a one-page ADR with context, decision, consequences, rejected alternatives, and the explicit triggers that would make you revisit it.
- **Check:** a teammate (or a fresh reading a week later) can explain the reasoning from the page alone.

10.14 Answer key for the Part 10 self-checks

- **What does the dependency rule forbid?** Code in an inner layer (domain, use cases) importing from an outer layer (frameworks, ORM, HTTP); dependencies point only inward, with the inner layer owning the interfaces the outer layer implements.
- **Two reasons to stay a modular monolith:** one deployable unit and one database keep operations, transactions, and debugging simple; module boundaries already give most of the organisational benefit, and a later extraction stays cheap if boundaries are enforced.
- **What does a saga solve that a database transaction cannot?** Consistency across several independent services and databases: a saga sequences local transactions and undoes completed steps with compensating actions when a later step fails, since no single transaction can span them.

Part 11: Testing, observability, and deployment

Code is finished only when it is tested, visible in production, and shippable by a pipeline.

11.1 Theory and tools: testing

The test pyramid: many fast **unit tests** (pure logic, services with fake repositories), a moderate number of **integration tests** (HTTP routes with a real database), and few **end-to-end tests** (the whole system). Also add **contract tests** between services and **load tests** before launches.

Need	Pick	Alternatives
Test runner	Vitest	Jest, `node:test` (built in), Mocha
HTTP tests (Express)	`supertest`	`light-my-request` (Fastify `app.inject`), `@nestjs/testing` + supertest
Real dependencies	Testcontainers (PostgreSQL, Redis in Docker)	docker-compose in CI, `pg-mem` (fake, less faithful)
Mocking outbound HTTP	`msw`, `nock`, undici `MockAgent`	Hand-written fakes
Test data	`@faker-js/faker`, factory functions	Fishery
Contract tests	Pact	OpenAPI-based validators
Load tests	k6, autocannon	Artillery
Coverage	v8 coverage (built in to Vitest/Jest)	c8, nyc

Good habits: each test creates its own data and cleans up (transaction rollback or truncate); test behaviour, not implementation; do not mock the database in integration tests (it hides real SQL bugs); keep the suite under a few minutes.

11.2 Theory and tools: observability

When production misbehaves you need three signals:

- **Logs**: what happened (structured JSON, with request id).
- **Metrics**: numbers over time. Track the **RED** set per endpoint: **R**ate, **E**rrors, **D**uration (p50, p95, p99). Also CPU, memory, event-loop delay, DB pool usage, queue depth.
- **Traces**: the path of one request across services, with timing per step.

[image: Observability pipeline]

Need	Pick	Alternatives
Logger	`pino`	`winston`
Metrics	`prom-client` + Prometheus + Grafana	Datadog, New Relic, OpenTelemetry metrics
Tracing and	OpenTelemetry (`@opentelemetry/sdk-node`	Datadog APM, Elastic APM

Need	Pick	Alternatives
instrumentation	with auto-instrumentations)	
Trace backend	Jaeger, Grafana Tempo	Honeycomb, Datadog
Error tracking	Sentry	Rollbar, Bugsnag
Log storage	Grafana Loki, Elasticsearch/OpenSearch (ELK)	CloudWatch, Datadog Logs
Uptime / alerts	Grafana alerts, Alertmanager, PagerDuty, UptimeRobot	Cloud provider alarms

Alert on **symptoms users feel** (error rate, latency, queue backlog age), not on every CPU blip. Define a service level objective (for example 99.9 percent of requests succeed under 500 ms) and alert when you burn its budget too fast.

11.3 Theory and tools: deployment

- **Containers.** Package the app in a Docker image so it runs identically everywhere. Use a multi-stage build, a slim base image, a non-root user, and copy only what you need.
- **Config and secrets** come from environment variables injected at run time (never baked into the image).
- **Health endpoints:** `/healthz` and `/readyz`, plus graceful shutdown on `SIGTERM` (Part 7).
- **CI/CD:** on every pull request run lint, type check, tests, build the image; on merge deploy automatically, with a way to roll back.
- **Database migrations** run as a separate release step before the new code starts, and must be backward compatible with the old code (Part 4).
- **Environments:** local, staging (production-like), production.

```
# syntax=docker/dockerfile:1
FROM node:22-slim AS build
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN corepack enable && pnpm install --frozen-lockfile
COPY . .
RUN pnpm build && pnpm prune --prod

FROM node:22-slim
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
USER node
EXPOSE 3000
CMD ["node", "dist/main.js"]
```

Where to run	Good for	Notes
Railway, Render, Fly.io	Fast start, small teams	Managed deploys, simple scaling

Where to run	Good for	Notes
AWS ECS/Fargate, Google Cloud Run, Azure Container Apps	Serious production without managing Kubernetes	Container in, autoscaling out
Kubernetes (EKS, GKE, AKS)	Many services, platform teams	Powerful, high operational cost
Serverless functions (AWS Lambda, Vercel, Cloudflare Workers)	Spiky or low traffic, event handlers	Cold starts, connection pooling and time limits need care
VPS with Docker Compose and Nginx	Cheap and understandable	You handle patching and backups

Other tools: GitHub Actions or GitLab CI for pipelines; Terraform or Pulumi for infrastructure as code; Nginx or Caddy as reverse proxy with automatic TLS; PM2 if you run without containers.

11.4 Build cards

Card 11.A: integration tests with a real database

- **Aim:** trustworthy tests.
- **Input:** the Notes API, Vitest, supertest, Testcontainers PostgreSQL.
- **Output:** tests for create, validation failure, not found, and pagination; each starts from a clean database.
- **Check:** `pnpm test` passes from a fresh clone with only Docker installed; break an SQL query on purpose and see a test fail.

Card 11.B: metrics and a dashboard

- **Aim:** see your service.
- **Input:** `prom-client` default metrics plus an `http_request_duration_seconds` histogram labelled by route and status; a `docker-compose` with Prometheus and Grafana.
- **Output:** a Grafana panel showing requests per second and p95 latency per route.
- **Check:** run autocannon and watch the graph move; slow one route on purpose and find it on the dashboard.

Card 11.C: tracing one request

- **Aim:** follow a request through API, queue, and worker.
- **Input:** OpenTelemetry auto-instrumentation and Jaeger in Docker.
- **Output:** one trace shows the HTTP span, the SQL span, and the job span with timings.
- **Check:** find the slowest span for a deliberately slow query.

Card 11.D: containerise and ship

- **Aim:** a pipeline from commit to running container.
- **Input:** the Dockerfile above and a GitHub Actions workflow with jobs `lint`, `test`, `build`, `deploy`.

- **Output:** a push to `main` builds an image, runs migrations, and deploys; the image runs as a non-root user and is under about 250 MB.
- **Check:** `docker run` with a missing env variable must fail fast with a clear message (Card 3.C).

Self-check questions: Why test against a real database? What do RED metrics tell you that logs alone do not? Why must migrations be backward compatible with the previous release?

11.5 Deep dive: testing strategy and techniques

Test doubles, named precisely. A *dummy* fills a parameter and is never used; a *stub* returns canned answers; a *spy* records calls; a *mock* asserts expected calls; a *fake* is a working lightweight implementation (an in-memory repository). Prefer **fakes and real dependencies** over mocks of your own code: mock-heavy tests break on every refactor and prove little. Mock only at the true edges (third-party HTTP, clock, randomness).

What each level should cover

Level	Scope	Speed	Typical tools	Examples
Unit	One function or class, no I/O	Milliseconds	Vitest, `node:test`	Pricing rules, domain aggregates, validators, ranking maths
Integration	Your code with a real database, Redis, or queue	Tens to hundreds of ms	Testcontainers, supertest, `app.inject`	Repositories, routes with auth, transactions, migrations
Contract	The agreement between a client and a server	Fast	Pact, OpenAPI validators	Consumer expectations on an API
End to end	Whole system through the public interface	Seconds	Playwright (API and browser), k6	Sign up, upload, search, pay
Non-functional	Load, security, resilience	Minutes	k6, ZAP, chaos tools	Knee point, injection scan, dependency failure

A useful rule: most tests at unit and integration level, a handful of end-to-end journeys, and nothing tested twice at several levels without reason.

Database tests that stay fast and isolated

- Run one real PostgreSQL in a Testcontainers container for the whole test run (`reuse` it locally), apply migrations once, then isolate each test: wrap the test in a transaction and roll back (fastest, but code that opens its own transactions needs savepoints), or truncate tables between tests (simple, slower), or clone a **template database** per worker (`CREATE DATABASE test_w3 TEMPLATE test_base`, very fast and good for parallel workers).

- Each test builds its own data with **factories** or builders (`aUser({ role: 'admin' })`) and never depends on rows another test created.
- Test migrations too: apply all migrations from empty, and apply the new migration on a copy of the previous release's schema with sample data.

Async, time, and randomness. Inject a clock (`now: () => Date`) or use fake timers (`vi.useFakeTimers()`) for expiry logic; seed random generators or inject an id generator; never use `setTimeout` sleeps to wait for async work, poll for a condition with a timeout (`vi.waitFor`) or await the event (for queues, `QueueEvents.waitUntilFinished`).

Flaky tests come from shared state, real clocks, ordering assumptions, network calls, race conditions, and port conflicts. Fix or delete them; a flaky suite teaches the team to ignore red. Quarantine with a ticket and a deadline, never retry silently forever.

```
// routes.test.ts (Fastify + Vitest + Testcontainers)
let app: FastifyInstance;
beforeAll(async () => { process.env.DATABASE_URL = await startPostgres(); app = await
buildApp(); await migrate(); });
afterAll(async () => { await app.close(); });
beforeEach(async () => { await truncateAll(); });

it('rejects a duplicate email with 409', async () => {
  await app.inject({ method: 'POST', url: '/v1/users', payload: { email: 'a@x.com',
password: 'correct horse battery' } });
  const res = await app.inject({ method: 'POST', url: '/v1/users', payload: { email:
'A@x.com', password: 'another long password' } });
  expect(res.statusCode).toBe(409);
  expect(res.json().type).toMatch(/conflict/);
});
```

Techniques that find more bugs per hour

- **Property-based testing** (`fast-check`): state a rule ("decoding an encoded cursor returns the original", "a refund never exceeds the payment") and let the tool try hundreds of random inputs and shrink failures to a minimal case. Great for parsers, ranking functions, state machines, and idempotency.
- **Mutation testing** (Stryker): it changes your code slightly and checks that tests fail; surviving mutants reveal weak tests that coverage numbers hide.
- **Snapshot tests** are cheap but get approved blindly; use them for stable structured output only, and review diffs carefully.
- **Table-driven tests** make authorisation matrices and error mappings readable (`it.each([...])`).
- **Test the unhappy paths:** timeouts, partial failures, duplicates, concurrent requests, and retries. Concurrency tests (`Promise.all` of 20 identical requests) catch race conditions that sequential tests never see.
- **Coverage** is a smoke alarm, not a goal. Look at uncovered branches in critical code, not the percentage.
- **Outbound HTTP:** intercept with `msw` or undici's `MockAgent` and also test timeouts, `500`s, slow responses, and malformed bodies.

- **Webhooks and queues:** build a helper that signs payloads and calls your endpoint, and run workers in the test process against a real Redis, waiting on job completion.
- **Security tests:** the authorisation matrix (Card 5.J), dependency audit in CI, and a scheduled OWASP ZAP baseline scan against a staging deployment.
- **Practice TDD where it fits:** write a failing test, make it pass, refactor. It works best for rules and algorithms, less for glue code.

11.6 Deep dive: logs, metrics, and traces in practice

Metrics with `prom-client`. Expose `GET /metrics` (on an internal port) with default Node metrics plus your own. Use a **histogram** for latency with buckets suited to your service (for example 5 ms to 10 s), a **counter** for events (requests, errors, jobs), and a **gauge** for current values (queue depth, pool waiting).

```
const httpDuration = new Histogram({ name: 'http_request_duration_seconds', help:
'HTTP latency',
  labelNames: ['method', 'route', 'status'], buckets: [0.005, 0.025, 0.1, 0.25, 0.5,
1, 2.5, 5, 10] });
app.addHook('onResponse', (req, reply, done) => {
  httpDuration.labels(req.method, req.routeOptions.url ?? 'unmatched',
String(reply.statusCode)).observe(reply.elapsedTime / 1000);
  done();
});
```

Cardinality is the budget. Every distinct label combination is a separate time series. Never label by user id, raw URL, email, or request id (millions of series will crash Prometheus or your bill). Use the **route template** and bounded values such as status class and method.

Node-specific signals to chart: event-loop delay (p99) and utilisation, GC pause time, `heapUsed` and `rss`, open handles, active requests, HTTP agent socket counts, database pool total, idle, and waiting, and per-dependency call latency and error rate.

Tracing with OpenTelemetry. Install the SDK **before** your application code is imported (for ESM use the loader or a separate `--import ./otel.js` file), enable auto-instrumentation for HTTP, Fastify or Express or Nest, `pg`, `ioredis`, and your queue library, and export through OTLP to a collector.

```
// otel.ts: run with  node --import ./otel.js dist/main.js
import { NodeSDK } from '@opentelemetry/sdk-node';
import { getNodeAutoInstrumentations } from '@opentelemetry/auto-instrumentations-
node';
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-otlp-http';
new NodeSDK({
  serviceName: 'jobs-api',
  traceExporter: new OTLPTraceExporter({ url: process.env.OTEL_EXPORTER_OTLP_ENDPOINT
+ '/v1/traces' }),
  instrumentations: [getNodeAutoInstrumentations()],
}).start();
```

Important ideas: a **trace** is a tree of **spans**; context travels between services in the W3C `traceparent` header (and inside queue messages: add it to the job payload and restore it in the worker so one trace covers HTTP, queue, and worker); add useful attributes to spans (`user.tier`, `job.type`, never secrets); **sample** to control cost (head sampling decides at the start, for example 10 percent; tail sampling in the collector keeps all slow and failed traces, which is far more useful); and put `trace_id` in every log line so

you can jump from a log to its trace and back. Check the current OpenTelemetry docs for exact package names and flags; the Node SDK changes quickly.

SLIs, SLOs, and error budgets. An **SLI** is a measured ratio (good requests / valid requests, where good means non-5xx and under 500 ms). An **SLO** is the target (99.9 percent over 30 days). The **error budget** is what is left (0.1 percent of requests, about 43 minutes of full outage in 30 days). Alert on **burn rate**: how fast the budget is being used. A common pair from the Google SRE workbook: page when the burn rate is above 14.4 over both 1 hour and 5 minutes (about 2 percent of the monthly budget gone in an hour); ticket for a burn of 6 over 6 hours and 30 minutes. When the budget is spent, freeze feature releases and work on reliability.

Alerting rules. Page a human only for symptoms that need action now and that users feel (SLO burn, error rate, latency, queue age, data pipeline stalled); route everything else to a ticket or a dashboard. Every alert links to a **runbook** (what it means, how to check, how to mitigate, who to call). Review noisy alerts weekly and delete or fix them. Add **synthetic checks** (a script that logs in and performs a key action every minute from outside) because internal metrics can be green while the login is broken.

Dashboards. One service overview per service: traffic, errors, latency percentiles, saturation (CPU, memory, event-loop delay, pool waiting), dependencies (database, cache, queue depth and oldest job age), deployments marked as annotations. Build dashboards from the questions you ask during an incident.

Error tracking (Sentry and similar). Initialise early, tag `release` (git SHA) and `environment`, upload source maps, scrub personal data (`beforeSend`), group by fingerprint, assign owners, and capture the request id. Treat each new error type after a deploy as a regression signal.

Incident handling. Declare an incident early, assign an incident commander, communicate status on a regular cadence, mitigate before diagnosing (roll back, flip a flag, scale, shed load), keep a timeline, and write a **blameless postmortem** within a few days: impact, timeline, contributing factors (not people), what went well, and action items with owners and dates.

11.7 Deep dive: containers, Kubernetes, and CI/CD

Docker habits for Node

- Use a multi-stage build and a slim or distroless base image; pin the Node major version and, for reproducibility, the image digest; add a `.dockerignore` (`node_modules`, `.git`, `.env`, tests).
- Install dependencies in a layer that depends only on the lock file so the cache survives code changes; install with `npm ci` or `pnpm install --frozen-lockfile`; ship only production dependencies.
- Run as the non-root `node` user, set `NODE_ENV=production`, and start with `node dist/main.js`, not `npm start` (npm does not forward `SIGTERM`, so graceful shutdown never runs). Use an init process (`docker run --init` or `tini`) so zombie processes are reaped and signals are forwarded.
- Scan images and dependencies in CI (Trivy, Grype, Docker Scout), generate an SBOM, and sign images (cosign) if you have compliance needs.
- Local development: Docker Compose with PostgreSQL, Redis, MinIO (S3), and Mailpit (email catcher), plus a healthcheck on each service so your app waits for readiness.

Kubernetes in one table

Object	Purpose	Notes for a Node API
Deployment	Runs N replicas, rolling updates	`maxUnavailable: 0`, `maxSurge: 25%` for zero-downtime
Service	Stable address and load balancing	ClusterIP behind an Ingress
Ingress / Gateway	HTTP routing, TLS	Nginx, Traefik, cloud controllers
ConfigMap, Secret	Configuration and secrets as env or files	Use an external secrets operator with a real secret manager
HorizontalPodAutoscaler	Scales replicas on metrics	CPU is the default; custom metrics (requests in flight, queue depth) are better
Probes	Startup, readiness, liveness	Readiness checks dependencies; liveness checks only the process (a database outage must not restart every pod)
PodDisruptionBudget	Limits simultaneous voluntary evictions	Keep at least N replicas during node drains
Requests and limits	Scheduling and caps	Set memory request equal to limit; set `--max-old-space-size` to about 75 percent of the memory limit; CPU limits can cause throttling, so measure before setting them

```
spec:
  terminationGracePeriodSeconds: 40
  containers:
    - name: api
      image: registry.example.com/jobs-api:3f9c2ab
      ports: [{ containerPort: 3000 }]
      resources: { requests: { cpu: 250m, memory: 512Mi }, limits: { memory: 512Mi } }
}
      env: [{ name: NODE_OPTIONS, value: "--max-old-space-size=384" }]
      readinessProbe: { httpGet: { path: /readyz, port: 3000 }, periodSeconds: 5 }
      livenessProbe:  { httpGet: { path: /healthz, port: 3000 }, periodSeconds: 10,
failureThreshold: 3 }
      lifecycle: { preStop: { exec: { command: ["sleep", "8"] } } }   # let the load
balancer stop sending traffic first
```

Graceful shutdown timeline. Pod marked terminating, so endpoints are removed (this takes a few seconds to propagate); `preStop` sleep gives that time; `SIGTERM` reaches Node; the app stops accepting connections, finishes in-flight requests, closes pools and workers; the pod exits before `terminationGracePeriodSeconds` or is killed with `SIGKILL`. Set the app's own deadline lower than the grace period.

Pipeline shape

[image: A CI/CD pipeline with canary and automatic rollback]

```
# .github/workflows/ci.yml (sketch)
on: { pull_request: {}, push: { branches: [main] } }
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4          # pin third-party actions to a commit SHA
in real projects
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with: { node-version-file: .nvmrc, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - run: pnpm lint && pnpm typecheck && pnpm test      # Testcontainers needs
Docker on the runner
  build:
    needs: test
    if: github.ref == 'refs/heads/main'
    permissions: { id-token: write, contents: read }       # OIDC: no long-lived
cloud keys in secrets
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t $REGISTRY/jobs-api:${{ github.sha }} .
      - run: docker push $REGISTRY/jobs-api:${{ github.sha }}
```

Delivery practices. Build the image **once** and promote the same artifact through staging to production; tag with the commit SHA (never `latest`); run migrations as a separate, backward-compatible step before rollout; deploy with a canary and automated checks on error rate and latency; keep rollback one command away (redeploy the previous SHA); use feature flags to decouple deploy from release; keep environments similar (same container, different config); store infrastructure as code (Terraform, OpenTofu, Pulumi, AWS CDK) and review it like application code; use short-lived OIDC credentials from CI to the cloud.

Backups and disaster recovery. Define **RPO** (how much data you can lose) and **RTO** (how long you can be down), then choose: point-in-time recovery for the database, cross-region copies of backups and object storage, infrastructure as code to rebuild, and a **practised** restore (schedule a game day twice a year). Multi-region active-active is expensive and complex; a warm standby region is the usual first step.

Cost awareness. The big levers: right-size instances using measured utilisation, autoscale down at night, cache and CDN to cut compute and egress, retention limits on logs and metrics, avoid chatty cross-zone traffic, and review the bill monthly with the team.

11.8 More build cards

Card 11.E: isolated database tests per worker

- **Aim:** fast, parallel, independent integration tests.
- **Input:** a Testcontainers PostgreSQL started once, a migrated template database, and a per-worker database cloned from it.

- **Output:** 100 integration tests run in parallel across 4 workers in under 20 seconds with no cross-test interference.
- **Check:** run the suite 10 times in random order; it must pass every time.

Card 11.F: property-based test for cursors

- **Aim:** find edge cases automatically.
- **Input:** `encodeCursor` and `decodeCursor` for `(createdAt, id)` and a `fast-check` arbitrary generating dates and ids.
- **Output:** the property "decode(encode(x)) equals x" holds for 1,000 random inputs and malformed cursors always return a clean `400`, never a crash.
- **Check:** introduce a bug (drop milliseconds) and confirm the framework shrinks to a minimal failing example.

Card 11.G: mutation test your pricing rules

- **Aim:** measure test quality.
- **Input:** a pricing module with discounts and tax and its unit tests; Stryker configured for that folder.
- **Output:** a mutation score above 80 percent; every surviving mutant either gets a new test or is documented as equivalent.
- **Check:** compare the mutation score with line coverage and note the gap.

Card 11.H: RED and runtime metrics with alerts

- **Aim:** observe the service from outside and inside.
- **Input:** the histogram hook above plus event-loop delay, pool waiting, and queue depth gauges; Prometheus alert rules.
- **Output:** a Grafana dashboard and two alerts (error ratio above 2 percent for 5 minutes; p95 above 800 ms for 10 minutes) that fire when you run a fault-injection script.
- **Check:** trigger each alert on purpose and confirm the notification and runbook link arrive.

Card 11.I: one trace across HTTP, queue, and worker

- **Aim:** propagate context through a queue.
- **Input:** OpenTelemetry SDK; inject `traceparent` into the job data when enqueuing and extract it in the worker.
- **Output:** a single trace shows the HTTP request span, the enqueue span, and the worker's processing span with its SQL spans.
- **Check:** screenshot the trace in Jaeger or Tempo and identify the slowest span.

Card 11.J: production-grade Dockerfile and probes

- **Aim:** shut down cleanly in a container.
- **Input:** the multi-stage Dockerfile, `--init`, and `/healthz` and `/readyz` endpoints.
- **Output:** `docker stop` finishes in-flight requests and exits with code 0 within the grace period; `/readyz` returns 503 once shutdown starts so traffic drains.
- **Check:** run autocannon, `docker stop` mid-test, and verify zero failed requests and the exit code.

Card 11.K: Kubernetes rolling update with zero errors

- **Aim:** validate the manifest.
- **Input:** the Deployment above on a local cluster (kind or minikube), two replicas, and a load test.
- **Output:** `kubectl rollout restart` under load produces no failed requests; removing `preStop` produces some, and you can explain why.
- **Check:** compare error counts with and without `preStop` and with a 1 second versus 8 second sleep.

Card 11.L: a blameless postmortem

- **Aim:** learn from failure.
- **Input:** break your staging system on purpose (exhaust the database pool, or fill Redis), observe, and recover.
- **Output:** a one-page postmortem with timeline, impact, contributing factors, detection time, and three action items with owners.
- **Check:** each action item should prevent, detect faster, or mitigate the failure, and be verifiable by a test or an alert.

Deep-dive self-check: (1) Why can a mock-heavy test suite hurt? (2) How do you isolate database tests in parallel? (3) Why is labelling a metric by user id dangerous? (4) What does a burn-rate alert tell you that a plain error-rate alert does not? (5) Why does `npm start` break graceful shutdown in a container? (6) Why should a liveness probe not check the database?

Part 11 deep dive

11.5 Testable application structure

Testing is easy when the app is built by a **factory** that receives its dependencies, instead of a module that connects to a database as a side effect of being imported.

```
// app.ts: no listen(), no process.env reads, no global singletons
export function buildApp(deps: { pool: Pool; redis: Redis; mailer: Mailer; clock: {
now(): Date } }) {
  const app = Fastify({ logger: false });
  app.register(notesRoutes, { prefix: '/v1/notes', deps });
  return app;
}
// server.ts (production only): const app = buildApp(realDeps); await app.listen({
port: config.PORT, host: '0.0.0.0' });
```

Real PostgreSQL in tests with Testcontainers, one container per test run, a clean slate before every test:

```
import { PostgreSqlContainer, StartedPostgreSqlContainer } from
'@testcontainers/postgresql';
import { Pool } from 'pg';
import { afterAll, beforeAll, beforeEach, describe, expect, it } from 'vitest';

let container: StartedPostgreSqlContainer; let pool: Pool; let app: ReturnType<typeof
buildApp>;

beforeAll(async () => {
  container = await new PostgreSqlContainer('postgres:16').start();
```

```
  pool = new Pool({ connectionString: container.getConnectionUri() });
  await runMigrations(pool);
  app = buildApp({ pool, redis: fakeRedis, mailer: fakeMailer, clock: { now: () =>
new Date('2026-01-01T00:00:00Z') } });
}, 120_000);
afterAll(async () => { await app.close(); await pool.end(); await container.stop();
});
beforeEach(async () => { await pool.query('TRUNCATE notes, users RESTART IDENTITY
CASCADE'); });

describe('POST /v1/notes', () => {
  it('creates a note and returns 201', async () => {
    const res = await app.inject({ method: 'POST', url: '/v1/notes', headers:
authFor('user-1'), payload: { title: 'hello' } });
    expect(res.statusCode).toBe(201);
    expect(res.json()).toMatchObject({ title: 'hello' });
    expect((await pool.query('SELECT count(*)::int AS n FROM
notes')).rows[0].n).toBe(1);
  });
  it('rejects an empty title with 400 and a stable error code', async () => {
    const res = await app.inject({ method: 'POST', url: '/v1/notes', headers:
authFor('user-1'), payload: { title: '' } });
    expect(res.statusCode).toBe(400);
    expect(res.json().code).toBe('VALIDATION');
  });
});
```

Patterns that keep suites fast and reliable:

- **Arrange, act, assert**, one behaviour per test, names that read as sentences (`rejects an empty title with 400`).
- **Parallelism needs isolation.** Vitest runs files in parallel workers; give each worker its own database (create `test_<workerId>` from a migrated template database) or its own schema, or run integration files serially.
- **Control time and randomness.** Inject a clock; use `vi.useFakeTimers()` for timers and retries; seed random generators.
- **Outbound HTTP:** intercept with `msw` or undici's `MockAgent` so tests never touch the network and can simulate 500s, slow responses, and malformed bodies.
- **Queues:** test the **handler function** directly with fakes for fast tests, plus a few integration tests with a real Redis container where the worker processes a real job end to end.
- **Auth:** a test helper that mints valid tokens for a given user and role keeps every test short.
- **Property-based testing** (`fast-check`) finds edge cases you did not think of: state a property and the library tries hundreds of generated inputs and shrinks failures to the smallest example.

```
import fc from 'fast-check';
it('base62 encoding round-trips', () =>
  fc.assert(fc.property(fc.bigInt({ min: 0n, max: 2n ** 53n }), (n) =>
fromBase62(toBase62(n)) === n)));
```

- **Flaky tests** have a short list of causes: shared state between tests, dependence on order, real time or randomness, real network, unawaited promises, and race conditions in the code itself. Fix the cause; never add retries to hide it. Quarantine and track any flaky test, because an ignored red build teaches the team to ignore red.

- **Coverage is a signal, not a goal.** 100 percent line coverage with weak assertions proves little. **Mutation testing** (Stryker) changes your code (flip a `>` to `>=`) and checks that a test fails, which measures whether tests actually assert behaviour.
- **Contract tests** (Pact) let a consumer and provider verify their agreement independently, which is how you deploy services separately without end-to-end test suites that take an hour.
- Keep **end-to-end tests** few and focused on critical journeys (sign up, pay), and run **smoke tests** after every deploy.

11.6 Metrics in depth

Metric types: a **counter** only goes up (requests, errors; you graph its *rate*); a **gauge** goes up and down (queue depth, active connections); a **histogram** buckets observations so you can compute percentiles across many instances (latency); a summary computes percentiles inside one process and cannot be aggregated across instances, so prefer histograms.

```
import client from 'prom-client';
client.collectDefaultMetrics();                                    // process CPU,
memory, event loop lag, GC, handles

const httpDuration = new client.Histogram({
  name: 'http_request_duration_seconds', help: 'HTTP request latency',
  labelNames: ['method', 'route', 'status'] as const,
  buckets: [0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5],
});
app.use((req, res, next) => {
  const end = httpDuration.startTimer();
  res.on('finish', () => end({ method: req.method, route: req.route?.path ??
'unmatched', status: String(res.statusCode) }));
  next();
});
app.get('/metrics', async (_req, res) => { res.set('Content-Type',
client.register.contentType); res.end(await client.register.metrics()); });
```

Label cardinality is the classic trap. Every distinct label combination is a separate time series stored in memory. Use the route **template** (`/notes/:id`), never the raw URL, and never user ids, emails, or request ids as labels, or you will crash your metrics backend. Pick buckets around your SLO (if the target is 300 ms, have buckets near 0.25 and 0.5).

Two checklists tell you what to measure. **RED** for request-driven services: Rate, Errors, Duration. **USE** for resources (CPU, memory, pool, disk): Utilisation, Saturation, Errors. Google's "four golden signals" (latency, traffic, errors, saturation) say the same. Useful PromQL:

```
# p95 latency per route over 5 minutes
histogram_quantile(0.95, sum by (le, route)
(rate(http_request_duration_seconds_bucket[5m])))

# 5xx error ratio
sum(rate(http_request_duration_seconds_count{status=~"5.."}[5m])) /
sum(rate(http_request_duration_seconds_count[5m]))

# event loop lag (default metric from prom-client)
nodejs_eventloop_lag_p99_seconds
```

Also export **business metrics** (orders placed, payments failed, queue age): technical health can look fine while the business is broken.

11.7 Distributed tracing in depth

A **trace** is the tree of **spans** (timed operations) for one request. A **trace id** is passed along every hop in the W3C `traceparent` header, so the gateway, API, database call, and worker all join the same trace. OpenTelemetry's Node SDK instruments HTTP, Express, Fastify, PostgreSQL, Redis, and more automatically, but it must be loaded **before** the libraries it patches:

```
// otel.ts: start with  node --import ./dist/otel.js dist/server.js
import { NodeSDK } from '@opentelemetry/sdk-node';
import { getNodeAutoInstrumentations } from '@opentelemetry/auto-instrumentations-
node';
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-otlp-http';

const sdk = new NodeSDK({
  serviceName: 'orders-api',
  traceExporter: new OTLPTraceExporter({ url:
`${process.env.OTEL_EXPORTER_OTLP_ENDPOINT}/v1/traces` }),
  instrumentations: [getNodeAutoInstrumentations()],
});
sdk.start();
process.on('SIGTERM', () => { sdk.shutdown().finally(() => process.exit(0)); });   //
flush spans on shutdown
```

```
import { trace, SpanStatusCode } from '@opentelemetry/api';
const tracer = trace.getTracer('orders');

export async function placeOrder(cmd: Cmd) {
  return tracer.startActiveSpan('place-order', async (span) => {
    try {
      span.setAttribute('order.id', cmd.orderId);           // attributes make traces
searchable (avoid personal data)
      return await doPlaceOrder(cmd);
    } catch (e) {
      span.recordException(e as Error);
      span.setStatus({ code: SpanStatusCode.ERROR });
      throw e;
    } finally { span.end(); }
  });
}
```

Three extras that make tracing truly useful: **propagate across queues** by copying the current `traceparent` into the job's data and starting the worker's span as a child (otherwise the trace ends at the queue); put the **trace id into every log line** (a pino `mixin` that reads the active span) so you can jump from a log line to its trace and back; and **sample** (record a fraction of traces to control cost): head sampling decides at the start (cheap, random), tail sampling in the Collector decides after seeing the whole trace so it can keep all errors and slow traces.

11.8 SLOs and alerting that people can live with

- An **SLI** is a measured indicator (fraction of requests that succeed in under 500 ms). An **SLO** is the target for it (99.9 percent over 30 days). The **error budget** is what remains (0.1 percent, about 43 minutes of full outage per 30 days).
- Alert on **budget burn rate**, not on single spikes. A common pattern pairs a fast window and a slow window (for example burning budget 14 times too fast over 5 minutes **and** 1 hour pages someone now; 6 times too fast over 30 minutes and 6 hours opens a ticket). It pages on real, sustained user impact and stays quiet on blips.
- Every page needs a **runbook**: what it means, how to confirm, the first three things to try, who to escalate to. If an alert has no action, delete it.
- **Symptoms over causes:** page on "checkout errors are up", not on "CPU is 85 percent". Keep cause-level signals on dashboards for diagnosis.
- Build **synthetic probes** that exercise a critical user journey every minute from outside your network, because internal metrics cannot see a broken DNS record or an expired certificate (also alert on certificate expiry).
- After an incident write a **blameless postmortem**: timeline, impact, root causes (plural), what went well, what did not, and tracked action items with owners.

Error tracking (Sentry): initialise it before your app code, attach the release version and environment, upload source maps, scrub personal data before sending, and group by fingerprint so one bug is one issue. Error tracking tells you *what threw*; metrics tell you *how much it hurts*.

11.9 Containers and runtime hardening

Improvements over the basic Dockerfile in 11.3:

```
# syntax=docker/dockerfile:1
FROM node:22-slim AS deps
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN corepack enable && pnpm install --frozen-lockfile

FROM deps AS build
COPY . .
RUN pnpm build

FROM node:22-slim AS prod-deps
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN corepack enable && pnpm install --frozen-lockfile --prod

FROM gcr.io/distroless/nodejs22-debian12 AS runtime
WORKDIR /app
ENV NODE_ENV=production
COPY --from=prod-deps --chown=nonroot:nonroot /app/node_modules ./node_modules
COPY --from=build --chown=nonroot:nonroot /app/dist ./dist
USER nonroot
EXPOSE 3000
CMD ["dist/server.js"]
```

Key points: **never start with `npm start` or `pnpm start`** in the container, because the package manager becomes PID 1 and does not forward `SIGTERM` to Node, so graceful shutdown never runs; run `node` directly (distroless images use `node` as the entrypoint). If you need an init process for zombie reaping, use `docker run --init` or `tini`. Order Dockerfile layers so dependencies are cached separately from source; keep a `.dockerignore` (`node_modules`, `.git`, `.env`, tests). **Alpine versus slim/distroless:** Alpine uses musl libc, which can break or slow native modules (`sharp`, `argon2`, `bcrypt`, Prisma engines) unless prebuilt binaries exist; Debian-slim or distroless (glibc) is the safer default, and distroless adds a smaller attack surface (no shell). Scan images in CI (Trivy, Grype), generate an SBOM, pin base images by digest for reproducibility, and rebuild regularly to pick up security patches. Run as a **non-root** user with a **read-only root filesystem** where possible, drop Linux capabilities, and never bake secrets into image layers.

Memory and CPU in containers. Set the V8 heap explicitly: `NODE_OPTIONS=--max-old-space-size=384` for a 512 MiB limit (leave headroom for buffers, native memory, and the thread pool); an OOM-kill is far harder to diagnose than a heap-limit crash with a stack trace. CPU limits cause throttling that appears as latency spikes (many teams set CPU requests but no CPU limits); the libuv thread pool and V8's GC threads also see the container's CPU count.

```
# Kubernetes Deployment (essentials)
apiVersion: apps/v1
kind: Deployment
metadata: { name: orders-api }
spec:
  replicas: 3
  strategy: { type: RollingUpdate, rollingUpdate: { maxUnavailable: 0, maxSurge: 1 }
}   # never drop below desired capacity
  selector: { matchLabels: { app: orders-api } }
  template:
    metadata: { labels: { app: orders-api } }
    spec:
      terminationGracePeriodSeconds: 30
      containers:
        - name: api
          image: registry.example.com/orders-api:GIT_SHA          # immutable tag =
the commit
          ports: [{ containerPort: 3000 }]
          envFrom: [{ secretRef: { name: orders-api-env } }]
          env: [{ name: NODE_OPTIONS, value: '--max-old-space-size=384' }]
          resources: { requests: { cpu: 250m, memory: 512Mi }, limits: { memory:
512Mi } }
          readinessProbe: { httpGet: { path: /readyz, port: 3000 }, periodSeconds: 5,
failureThreshold: 2 }
          livenessProbe:  { httpGet: { path: /healthz, port: 3000 }, periodSeconds:
10, failureThreshold: 3 }
          startupProbe:   { httpGet: { path: /healthz, port: 3000 }, periodSeconds:
2, failureThreshold: 30 }
          lifecycle: { preStop: { exec: { command: ['sleep', '5'] } } }       # let
the load balancer stop sending traffic first
```

Probe rules: *Liveness* answers "is this process stuck? restart it" and must **not** check the database or other dependencies (a database blip would restart every pod and make the outage worse). *Readiness* answers "should I receive traffic now?" and may check dependencies the service cannot work without; fail it on `SIGTERM` so the pod drains. *Startup* gives slow boots time before the other probes begin.

11.10 CI/CD in depth

[image: A CI/CD pipeline from pull request to canary and rollback]

```
# .github/workflows/ci.yml (pin third-party actions to a full commit SHA in security-
sensitive repos)
name: ci
on: { pull_request: {}, push: { branches: [main] } }

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - run: pnpm lint && pnpm typecheck
      - run: pnpm test            # Testcontainers works: GitHub-hosted runners
include Docker

  build-and-deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    permissions: { id-token: write, contents: read }     # OIDC: short-lived cloud
credentials, no stored keys
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t $REGISTRY/orders-api:${{ github.sha }} .
      # scan the image, push it, run migrations (expand step), deploy staging, smoke
test, then canary to prod
```

Principles: **build once, deploy the same artefact** to every environment (the image tagged with the commit), changing only configuration; use **OIDC federation** to your cloud instead of long-lived access keys stored as secrets; require passing checks and review before merge; run **migrations as a separate, earlier step** that is backward compatible with the running version (Part 4.11); gate production with an approval or an automated canary analysis; and make **rollback boring**: redeploy the previous image tag, with the database already compatible because migrations are expand-only.

Strategy	How	Trade-off
Rolling	Replace instances gradually	Simple default; two versions run side by side briefly
Blue/green	Two full environments, switch traffic	Instant rollback; double the capacity during the switch
Canary	Send 1 to 5 percent of traffic to the new version, watch metrics, then ramp	Best safety; needs good metrics and routing

Strategy	How	Trade-off
Feature flags	Deploy dark, enable per user or percentage	Decouples deploy from release; flags need cleanup

Infrastructure as code (Terraform, OpenTofu, Pulumi) keeps environments reproducible and reviewable; keep staging as close to production as budget allows (same database engine and major version, same container image, smaller sizes).

11.11 Backups and disaster recovery

Two numbers define the plan: **RPO** (how much data you can lose, in time) and **RTO** (how long you can be down). Daily backups mean an RPO of up to a day; **point-in-time recovery** (continuous WAL archiving, offered by managed PostgreSQL such as RDS, Cloud SQL, Neon, Supabase) brings it to minutes. Use multi-availability-zone deployments for hardware failure, keep backups in a **different account or region** so one compromise cannot delete both, protect them against deletion (object lock), and above all **practise restores**: a backup you have never restored is a hope, not a backup. Document the recovery procedure as a runbook and time it. Also back up what is not in the database: object storage (versioning and replication), secrets (in the secrets manager), and infrastructure definitions (in git).

11.12 More build cards

Card 11.E: app factory with isolated parallel tests

- **Aim:** fast and reliable integration tests.
- **Input:** the Notes API refactored into `buildApp(deps)`, Testcontainers PostgreSQL, and Vitest with 4 workers.
- **Output:** 30 or more tests run in parallel without interfering (one database or schema per worker); the suite finishes in under a minute.
- **Check:** run the suite 10 times in a row; any failure means hidden shared state to fix.

Card 11.F: property-based test and mutation score

- **Aim:** test deeper than examples.
- **Input:** your pagination cursor encoder or the base62 helpers, `fast-check`, and Stryker.
- **Output:** a round-trip property test; a mutation report before and after you add missing assertions, with the mutation score improving.
- **Check:** screenshot the two Stryker reports.

Card 11.G: RED dashboard and an alert

- **Aim:** observe a service you built.
- **Input:** the histogram middleware, Prometheus and Grafana in Docker Compose, and the PromQL queries above.
- **Output:** a dashboard with rate, error ratio, and p95 per route; an alert that fires when the 5xx ratio exceeds 2 percent for 5 minutes, tested by making a route fail on purpose.
- **Check:** export the dashboard JSON to git; show the alert firing and resolving.

Card 11.H: trace through a queue

- **Aim:** one trace across process boundaries.
- **Input:** the OpenTelemetry setup above, Jaeger in Docker, and a BullMQ job that carries `traceparent` in its data.
- **Output:** one trace shows the HTTP request, the SQL insert, the enqueue, and the worker's processing span as a child.
- **Check:** a screenshot of the trace tree, and a log line you can find by pasting its trace id into the log search.

Card 11.I: zero-downtime rollout drill

- **Aim:** prove deploys do not drop requests.
- **Input:** a local Kubernetes cluster (kind or minikube), the Deployment above, and `k6` at a constant 100 requests/second.
- **Output:** roll out a new image during the test with zero failed requests; then break readiness on purpose in the new version and watch the rollout stall without taking traffic; then roll back.
- **Check:** k6 summary with error rate 0 for the good rollout, and `kubectl rollout status` output for the bad one.

Card 11.J: restore drill

- **Aim:** trust your backups.
- **Input:** a PostgreSQL database with seeded data, `pg_dump` or WAL archiving, and a script.
- **Output:** restore into a fresh instance, run a verification query set (row counts, checksums on key tables), and record how long it took (your measured RTO).
- **Check:** a written runbook with the exact commands and the measured time.

11.13 Answer key for the Part 11 self-checks

- **Why test against a real database?** Mocks encode your assumptions about SQL, constraints, transactions, and concurrency, so they pass while real queries fail; a containerised database exercises the actual behaviour (unique violations, isolation, migrations) at modest cost.
- **What do RED metrics tell you that logs alone do not?** Logs describe individual events; RED metrics aggregate them into rates, error ratios, and latency percentiles over time, which show trends and let you alert and compare releases without reading millions of lines.
- **Why must migrations be backward compatible with the previous release?** During and after a deploy (and any rollback) the old and new code run against the same database, so a destructive or incompatible migration breaks whichever version is not the one the schema was written for.

Part 12: Capstone project, study plan, and cheat sheet

12.1 Capstone: a hiring platform API

One project that touches almost every part: companies post jobs, candidates apply with a resume, resumes are parsed in the background, and recruiters search and rank candidates.

Feature	Parts it exercises	Suggested stack
Register, login, roles (candidate,	5	Fastify or Nest, argon2, jose, Redis

Feature	Parts it exercises	Suggested stack
recruiter, admin), refresh rotation		sessions or JWT
Jobs and applications CRUD with validation and cursor pagination	2, 3, 4, 6	PostgreSQL, Drizzle or Prisma, Zod, OpenAPI
Resume upload with presigned URLs and size and type checks	9	S3-compatible storage (MinIO locally)
Background resume parsing and text extraction, retries, dead-letter	8	BullMQ workers, idempotent handlers
Search jobs and candidates, filters and ranking	9	PostgreSQL full-text first, then Meilisearch or Typesense
Multi-tenant companies with row-level security	9	Shared tables with `tenant_id` and PostgreSQL RLS
Email notifications and status changes through the outbox	8	Outbox table, relay, queue, nodemailer
Rate limits, caching of hot job listings	7	Redis, `rate-limiter-flexible`
Tests, metrics, tracing, Docker, CI	11	Vitest, Testcontainers, OpenTelemetry, GitHub Actions
Real-time status updates for recruiters	6	SSE first, Socket.IO if you need two-way

Milestones

64. Skeleton, config, logging, health checks, Docker Compose with PostgreSQL and Redis.
65. Auth and roles, then jobs and applications.
66. Uploads and the parsing worker.
67. Search, caching, rate limiting.
68. Outbox and notifications.
69. Tests, dashboards, CI, deploy. Write a one-page design document with the trade-offs you chose and the alternatives you rejected.

12.2 A 12-week plan

Week	Part	Deliverable
1	1	Cards 1.A to 1.C
2	2	The Notes API in all three frameworks
3	3	Validation, errors, config, logging added
4	4	PostgreSQL, relations, transaction, index measurements

Week	Part	Deliverable
5	5	Full auth with refresh rotation and hardening
6	6	OpenAPI, idempotent POST, SSE, one GraphQL or tRPC variant
7	7	Cache helper, stampede test, Redis rate limiter, load test
8	8	BullMQ jobs, idempotent consumer, outbox
9	9	URL shortener and presigned uploads
10	10	Layered refactor and module boundaries
11	11	Tests, metrics, tracing, Docker, CI
12	Capstone	Finish, deploy, write the design document

12.3 Decision cheat sheet

If you need...	Reach for...	Consider instead...
A quick, understandable API	Express 5	Hono
High throughput and schema-first design	Fastify	Hono on Node
Large team structure and DI	NestJS (on the Fastify adapter if speed matters)	Fastify with feature folders
Relational data	PostgreSQL	MySQL
Type-safe database access	Prisma or Drizzle	Kysely, raw `pg`
Flexible document data	MongoDB with Mongoose	PostgreSQL `jsonb`
Sessions, cache, counters, pub/sub	Redis	Valkey, Memcached, Dragonfly
Background jobs in Node	BullMQ	pg-boss, Temporal, Inngest
High-volume event streaming	Kafka	Redpanda, NATS JetStream, SQS/SNS
Login for a product	Hosted provider or Better Auth	Passport + your own sessions
Typed validation	Zod	Valibot, TypeBox, class-validator
Real-time two-way	Socket.IO + Redis adapter	`ws`, Ably, Centrifugo
One-way push	SSE	WebSocket
Service-to-service RPC	gRPC or Connect	HTTP + OpenAPI, tRPC (TypeScript

If you need...	Reach for...	Consider instead...
		only)
Search	PostgreSQL full-text, then Meilisearch	Elasticsearch, Typesense
File storage	S3-compatible object storage	Local disk (single server only)
Logging	pino	winston
Observability	OpenTelemetry + Grafana stack	Datadog, New Relic
Tests	Vitest + supertest + Testcontainers	Jest, `node:test`
Deployment	Containers on Cloud Run, ECS, Fly.io, or Render	Kubernetes, serverless

12.4 Common mistakes checklist

- Business logic inside controllers; SQL inside handlers.
- Trusting client input, or skipping ownership checks.
- `async` handlers without error handling (Express 4), unhandled promise rejections.
- Blocking the event loop with sync or CPU-heavy code.
- One connection per request instead of a pool; N+1 queries; missing indexes on foreign keys.
- Storing state in process memory or on local disk, then scaling horizontally.
- Retrying non-idempotent operations; publishing events outside the transaction that creates them.
- Caching without an invalidation plan or TTL.
- Secrets in the repository; wide-open CORS; no rate limits on login.
- Logging sensitive data; no request ids; no metrics until the first outage.
- Microservices before a modular monolith; a shared database between services.
- Migrations that break the running version.

12.5 Where to keep learning

The official documentation is the most reliable source and stays current: the Node.js docs and guides, the Express, Fastify, and NestJS documentation, the PostgreSQL manual (especially the chapters on indexes and concurrency control), the Redis documentation, and the OWASP Top 10 and Cheat Sheet Series for security.

Library names and capabilities in this report come from general knowledge, not from pages I opened while writing, so confirm current versions and option names in each project's documentation before you install. Good books to read alongside: *Designing Data-Intensive Applications* by Martin Kleppmann (the best single book on the ideas behind Parts 4, 7, 8, and 9) and *Node.js Design Patterns* by Mario Casciaro and Luciano Mammino.

12.6 Capstone specification: the hiring platform in detail

This is the full brief for the capstone from 12.1. Treat it like a ticket from a product manager: build it in the milestone order, and write an ADR (Part 10) whenever you make a real choice.

Users and stories

Persona	Stories
Recruiter	Create and publish a job; see ranked candidates for it with an explanation of the score; change an application's status; get notified when a candidate applies
Candidate	Register; upload a resume; apply to a job; see the status of my applications; delete my data
Tenant admin	Invite recruiters; assign roles; see an audit log
Platform operator	See health, queue depth, error rates; reprocess resumes after a parser or model change

Non-functional targets (invent numbers, then test against them)

Area	Target
Load	100 requests per second average, 500 peak; 2,000 resume uploads per day
Latency	p95 under 300 ms for reads, under 600 ms for writes; search under 500 ms
Availability	99.9 percent monthly for the API; resume processing may be delayed up to 10 minutes
Data	5 million candidates and 20 million applications after 3 years
Security	Tenant isolation proven by tests, passwords with argon2id, audit log of role changes and exports
Recovery	RPO 5 minutes, RTO 1 hour

Architecture. One API process (modular monolith with modules `auth`, `jobs`, `applications`, `search`, `notifications`), one worker process, PostgreSQL, Redis, and S3-compatible storage. The diagram shows who talks to whom.

[image: Capstone architecture]

Data model. Every tenant-owned table has `tenant_id` and a row-level security policy. `applications` has a unique pair `(job_id, candidate_id)` so a double click cannot apply twice. `score_breakdown` and `scoring_version` record why a candidate ranked where they did, so you can compare scoring versions and reprocess. `outbox` rows are written in the same transaction as the business change.

[image: Capstone data model]

API surface

Method and path	Who	Notes
`POST /v1/auth/register`, `POST /v1/auth/login`, `POST /v1/auth/refresh`, `POST /v1/auth/logout`	Anyone / user	Rate limited; same error for bad email or password
`GET/POST /v1/jobs`, `GET/PATCH /v1/jobs/:id`, `POST /v1/jobs/:id/publication`	Recruiter	Cursor pagination; `If-Match` on `PATCH`
`POST /v1/resumes/uploads`	Candidate	Returns a presigned URL and a resume id; client uploads directly
`POST /v1/resumes/:id/completion`	Candidate	Verifies the object exists, enqueues `parse-resume`, returns `202`
`POST /v1/jobs/:id/applications`	Candidate	Idempotent through the unique pair and an `Idempotency-Key`
`GET /v1/jobs/:id/candidates?limit&after&minScore`	Recruiter	Ranked list with `scoreBreakdown`; cached per job version
`PATCH /v1/applications/:id`	Recruiter	Status machine: `applied`, `screening`, `interview`, `offer`, `rejected`; forward moves only unless an admin
`GET /v1/events`	Recruiter	SSE stream of new applications
`GET /v1/audit-log`	Admin	Append-only, filterable
`GET /healthz`, `GET /readyz`, `GET /metrics`	Infrastructure	Internal port for metrics

Queues and jobs

Queue	Job	Trigger	Idempotency key	Retry policy
`resume`	`parse-resume`	Upload completed	`resume:{id}:{parserVersion}`	5 attempts, exponential; unrecoverable on a corrupt file
`resume`	`embed-resume`	Parse succeeded	`embed:{id}:{modelVersion}`	5 attempts
`scoring`	`rescore-job`	Job changed or scoring version bumped	`score:{jobId}:{version}`	3 attempts, low priority
`email`	`send-email`	Outbox event	Event id	5 attempts, 10 per second

Queue	Job	Trigger	Idempotency key	Retry policy
				limit
`outbox`	relay loop	Always on	Event id	Never gives up; alert on oldest unsent age

The ranking module (keep it pure). `rank(job, candidates, config) -> RankedCandidate[]` takes plain data and returns scores with a breakdown: *hard filters* (required work authorisation, minimum years, location), *keyword/BM25 score* from PostgreSQL full-text search over skills and titles, *vector similarity* between the job and resume embeddings, and a *weighted combination* with weights in config. No I/O inside, so you can unit test it with fixtures, run it on an offline evaluation set, and compare versions. Store `scoring_version` with every result.

Definition of done per milestone

Milestone	Done means
1 Skeleton	Typed config, structured logs with request id, health endpoints, Docker Compose, CI running lint, types, tests
2 Auth and jobs	Registration and login with argon2id, refresh rotation, roles, authorisation matrix test, jobs CRUD with validation and problem-details errors
3 Uploads and parsing	Presigned upload, parse and embed jobs, idempotent and retried, dead-letter alert, a poison-file test
4 Search and ranking	Ranked candidates with explanations, evaluation script with precision at 10, cache and invalidation, rate limits
5 Outbox and notifications	Applications emit events through the outbox; email worker; a crash test proves no lost events
6 Quality and operations	Integration tests on a real database, load test report, dashboards and two alerts, canary-style deploy, a restore drill, three ADRs

Test plan summary. Unit tests for ranking and domain rules; integration tests for every route (status codes, auth, validation, tenant isolation across two tenants); concurrency tests (double apply, double upload completion); queue tests (retry, stalled job, duplicate delivery); a migration test; one end-to-end journey (register, upload, apply, rank, notify); a load test with the knee point recorded.

Risks to think about before starting: tenant data leaks (mitigate with RLS and the cross-tenant test), resume files that are malicious or enormous (limits, type checks, scanning), personal data obligations (retention, deletion, export, consent), bias and fairness in ranking (keep the score explainable, avoid protected attributes, log versions, review outcomes), model or parser changes silently altering rankings (versioning and offline evaluation), and queue backlogs after an outage (autoscale workers, alert on oldest job age).

Variations to deepen the learning. Swap pieces and compare: Fastify versus NestJS for the API; Prisma versus Drizzle; BullMQ versus `pg-boss`; PostgreSQL full-text versus Meilisearch; sessions versus JWT with refresh rotation; polling relay versus Debezium; extract `search` into its own service using the strangler steps.

12.7 Skill ladder: where you stand

Use this as a self-assessment. Each row is something you can demonstrate with code you wrote, not something you read.

Level	You can	Evidence
Foundations	Explain the event loop, build a REST API with validation, errors, config, and logs in Express and Fastify	Cards 1.A to 3.J
Competent	Design a schema, write transactions and indexes, read query plans, implement login with refresh rotation, write integration tests, containerise and deploy	Cards 4.A to 5.D, 11.A, 11.D
Proficient	Add caching and rate limits correctly, run background jobs idempotently, use the outbox, handle failures with timeouts and breakers, trace a request end to end	Cards 7.A to 8.D, 11.B, 11.C
Advanced	Design systems from requirements with estimates and trade-offs, enforce module boundaries, run zero-downtime migrations and deploys, operate with SLOs and runbooks	Cards 9.A to 10.C, 11.D, capstone
Expert	Choose and justify architectures for new problems, mentor others, lead incident response, evolve systems over years, and know when not to add complexity	Design documents, postmortems, ADRs

12.8 Question bank for practice and interviews

Answer each in your own words, out loud, in about two minutes, then check your answer against the matching part of this report.

Runtime and language. How does the event loop order timers, promises, and `nextTick`? What blocks the loop and how do you find it? When do you use worker threads instead of a queue? How do streams handle backpressure? What causes memory leaks in Node?

Frameworks and design. Compare Express, Fastify, and NestJS honestly. What does Fastify encapsulation do? How does Nest dependency injection decide what to build? Where does business logic live and why? How do you keep a codebase testable?

APIs. What makes an endpoint idempotent and how do you make a `POST` safe to retry? Offset versus cursor pagination? How do you version an API without breaking clients? REST versus GraphQL versus gRPC versus tRPC: when each? How do you design a webhook system?

Data. What does an index do and when does it hurt? Explain transaction isolation levels and write skew. How do you avoid lost updates? How do you migrate a column name with zero downtime? When would you use Redis, MongoDB, or a search engine instead of PostgreSQL? What is replica lag and how does it bite?

Security. Sessions versus JWT: trade-offs and revocation. How do you store passwords? What is CSRF and which defences apply? What is SSRF? How do you prevent broken access control in a multi-tenant system? How does OAuth with PKCE work?

Scale and reliability. How do you handle a cache stampede? What is a retry storm and how do you prevent it? How do you find the bottleneck in a slow endpoint? Token bucket versus fixed window? How do you do a zero-downtime deploy? What do you do when a dependency is down?

Async and distributed systems. Why is exactly-once delivery a myth and what do you do instead? Explain the transactional outbox. Saga versus two-phase commit? How do Kafka consumer groups and partitions work? How do you keep ordering where it matters? When would you pick BullMQ, RabbitMQ, Kafka, or SQS?

Architecture. Monolith, modular monolith, or microservices for a new product and why? What is a bounded context? How do you enforce module boundaries? What is a distributed monolith? When is CQRS or event sourcing worth it?

Operations. What metrics would you put on a dashboard for an API? What is an SLO and a burn-rate alert? How do you propagate a trace through a queue? How would you run a post-incident review?

Design prompts. Design a URL shortener, a notification service, a news feed, a rate limiter, a file-sharing service, a chat system, a ticket booking system, a job queue, an analytics pipeline, and a resume ranking service. Use the design template from 9.11 each time.

12.9 Final build cards (capstone milestones)

Card 12.A: milestone 1, the skeleton

- **Aim:** start every project from a solid base.
- **Input:** the 12.6 brief; a new repository with `pnpm`, TypeScript strict mode, Fastify (or Nest), pino, Zod config, Docker Compose with PostgreSQL, Redis, and MinIO.
- **Output:** `docker compose up` followed by `pnpm dev` serves `/healthz` and `/readyz`; CI runs lint, typecheck, and one test; a request id appears in every log line.
- **Check:** a fresh clone reaches a green CI run and a working `/readyz` in under 10 minutes by following your README.

Card 12.B: milestone 2, auth, tenancy, and jobs

- **Aim:** a secure core.
- **Input:** the data model for tenants, users, and jobs with RLS; argon2id; refresh rotation; roles.

- **Output:** the authorisation matrix test passes; a recruiter in tenant A cannot read tenant B's jobs even with a guessed id; jobs support cursor pagination and `If-Match` updates.
- **Check:** the matrix runs in CI and fails if any repository filter or RLS policy is removed.

Card 12.C: milestone 3, resume pipeline

- **Aim:** reliable background processing.
- **Input:** presigned upload, completion endpoint, `parse-resume` and `embed-resume` workers.
- **Output:** a PDF and a DOCX resume are parsed and embedded; a corrupt file lands in the failed set with an alert; processing the same upload twice changes nothing.
- **Check:** kill the worker mid-job and see the job resume; a metric shows queue depth and oldest job age.

Card 12.D: milestone 4, ranking

- **Aim:** explainable, versioned ranking.
- **Input:** the pure `rank` function, an evaluation set of 100 resumes with labelled good matches for 5 jobs, full-text and vector search.
- **Output:** `GET /v1/jobs/:id/candidates` returns scores with breakdown; `pnpm eval` prints precision at 10 and recall for each scoring version.
- **Check:** change a weight and see precision move; the result is cached and invalidated when the job changes.

Card 12.E: milestone 5, events and notifications

- **Aim:** no lost events.
- **Input:** the outbox table, relay, and email worker; SSE endpoint for recruiters.
- **Output:** applying to a job produces an email to the recruiter and an SSE event exactly once from the user's point of view, even when the relay crashes or restarts.
- **Check:** the chaos script from Card 8.L passes against the capstone.

Card 12.F: milestone 6, operate it

- **Aim:** prove it works under pressure.
- **Input:** load test with k6 open model, dashboards, two alerts, canary deploy, a database restore.
- **Output:** a two-page report with the knee point, the first saturated resource, the fix you applied, before and after numbers, and the measured restore time against the 1-hour target.
- **Check:** a teammate follows your runbook to trigger and resolve one alert without your help.

Card 12.G: the design document

- **Aim:** communicate decisions.
- **Input:** everything above.
- **Output:** a design document using the template in 9.11 with diagrams, ADRs, a risk table, and the list of alternatives you rejected.
- **Check:** a reviewer can answer "why this and not the alternative?" for five choices from the document alone.

12.10 Reading list and how to keep learning

- **Primary sources:** the Node.js documentation and API reference, the Fastify, Express, and NestJS documentation, the PostgreSQL manual, the Redis and BullMQ documentation, the OpenTelemetry documentation, and the OWASP cheat sheets. Library details change often, so verify exact option names there.
- **Books:** *Designing Data-Intensive Applications* (Kleppmann), *Node.js Design Patterns* (Casciaro and Mammino), *Release It!* (Nygard, on stability patterns), *Domain-Driven Design Distilled* (Vernon), *Building Microservices* (Newman), *Site Reliability Engineering* and *The Site Reliability Workbook* (Google, freely available online), and *Web Application Security* (Hoffman). Check for current editions.
- **Practice:** rebuild a known system (a URL shortener, a job queue) from scratch; read the source of small libraries you use (`p-limit`, `lru-cache`, `pino`); read postmortems from public companies; contribute a bug fix or doc improvement to an open-source project; keep a short engineering journal of decisions and mistakes.
- **Habit loop:** pick one part per week, read the theory, build the cards, break your own code with a failure test, and write down what surprised you. Use the self-check questions as spaced repetition; revisit each part after a month.
- **Staying current:** follow release notes for Node.js (LTS schedule), Fastify, NestJS, your ORM, and PostgreSQL; run dependency updates weekly through automation; and treat every "best practice" article, including this report, as a starting point to verify against current documentation and your own measurements.

Deep-dive self-check: (1) Which two capstone tables enforce "no double apply" and "no lost event", and how? (2) Why must the `rank` function do no I/O? (3) What would you measure to decide whether a scoring-version change is an improvement? (4) Which risk in 12.6 would hurt most in production and what is your control for it? (5) Pick one variation (Fastify versus Nest, BullMQ versus `pg-boss`) and state the trade-off in two sentences.

Want to be quizzed? Run `/teach` on this session and I will quiz you part by part, tracking what you have mastered until everything is confirmed. Library names, options, and versions here come from general knowledge, so confirm exact details in the current official documentation before relying on them.